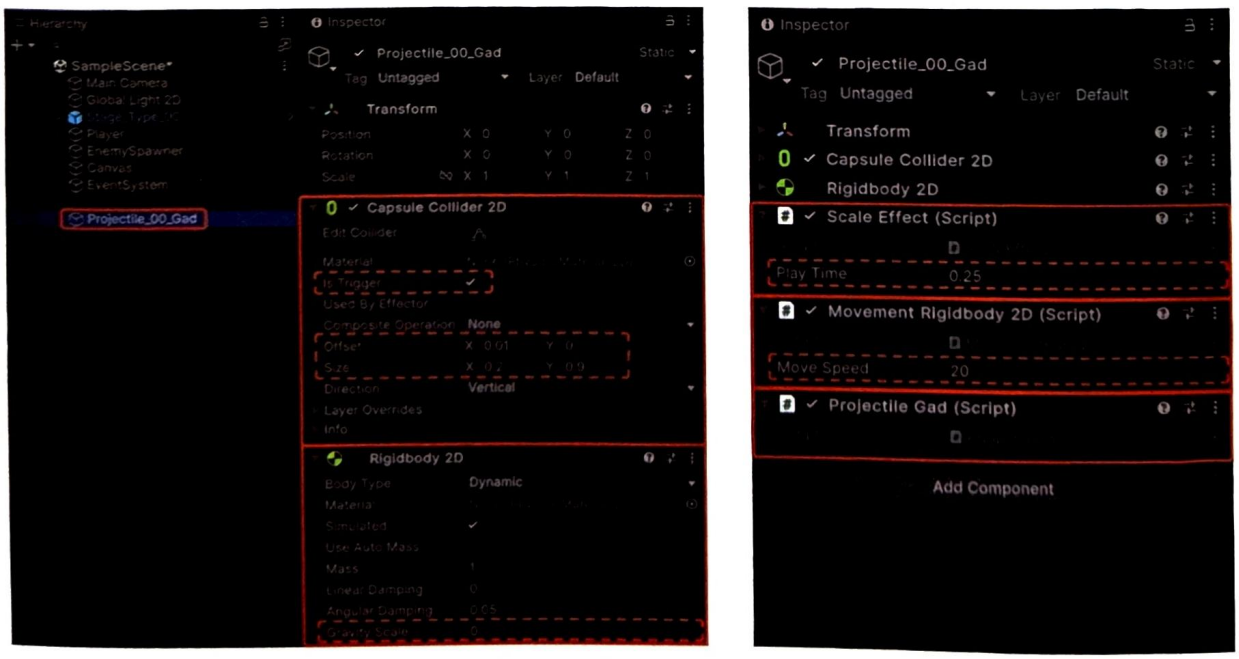
Enemy 태그 생성하고 설정하기

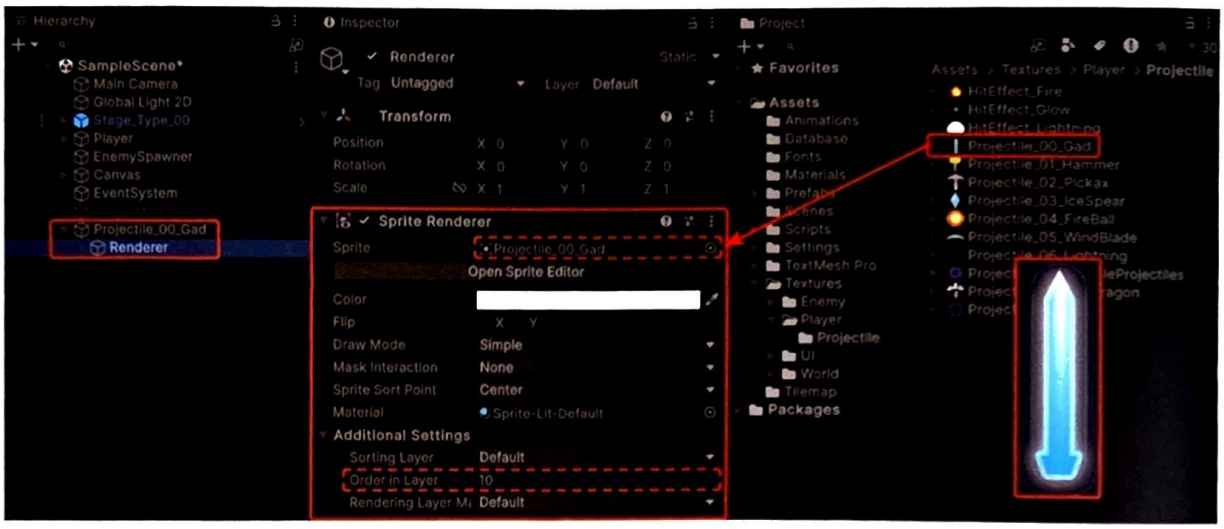
03 [GameObject → Create Empty] 메뉴로 기본 공격 발사체를 제어하는 빈 오브젝트를 생성하고 이름을 'Projectile_00_Gad'로 변경합니다. 발사체 제어와 충돌 처리, 크기 변화, 이동을 위해 Capsule Collider 2D, Rigidbody 2D, Scale Effect, Movement Rigidbody 2D, Projectile Gad 컴포넌트를 추가합니다.

Capsule Collider 2D 컴포넌트의 Is Trigger를 true로 설정하고 Offset은 (0.01, 0), Size는 (0.2, 0.9)로 설정합니다. Rigidbody 2D 컴포넌트의 Gravity Scale은 '0'으로 설정합니다. 마지막으로 Scale Effect 컴포넌트의 Play Time은 '0.25'로, Movement Rigidbody 2D 컴포넌트의 Move Speed를 '20'으로 설정합니다.



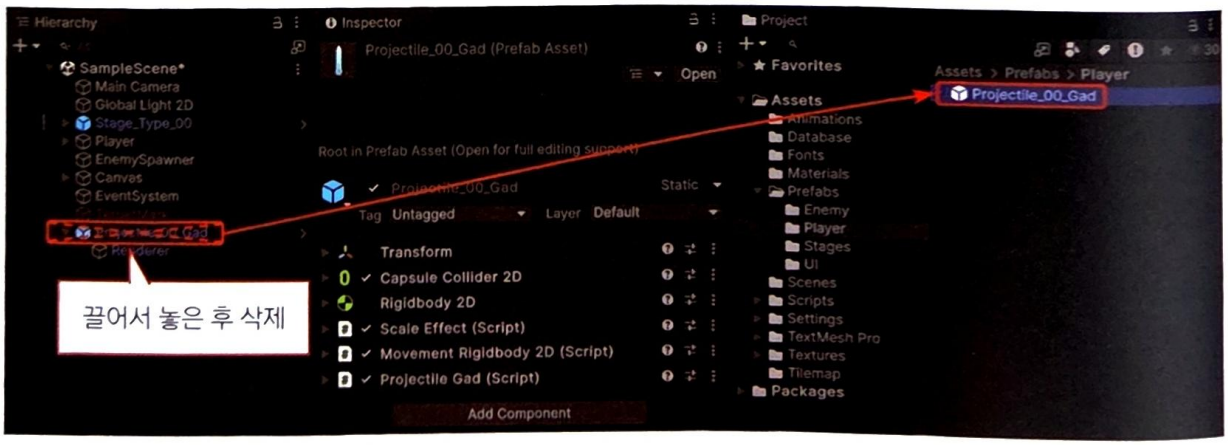
기본 공격 발사체를 제어하는 빈 오브젝트 생성하고 설정하기

04 [GameObject → 2D Object → Sprites → Square] 메뉴로 기본 공격 발사체를 화면에 표시하는 오브젝트를 생성하고 이름을 'Renderer'로 변경한 후 Projectile_00_Gad 오브젝트의 자식으로 배치합니다. Sprite Renderer 컴포넌트의 Sprite를 Projectile_00_Gad.png로 설정하고 Order in Layer를 '10'으로 설정합니다.



기본 공격 발사체를 출력하는 오브젝트 생성하고 설정하기

05 기본 공격 발사체(Projectile_00_Gad)를 Project View의 Assets 〉 Prefabs 〉 Player 폴더로 끌어서 놓아 프리팹을 생성하고 Hierarchy View에 있는 Projectile_00_Gad 오브젝트는 삭제합니다.



Projectile_00_Gad 오브젝트 프리팹 생성하기

Do it! 실습 기본 공격 스킬 제어하기

01 Assets 〉 Scripts 〉 Skill 폴더에 기본 공격 스킬을 제어하는 SkillGad.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/SkillGad.cs

코드 05-08 기본 공격 스킬 제어

```
01: using UnityEngine;
02:
03: public class SkillGad : MonoBehaviour
04: {
05:     [SerializeField]
06:     private ProjectileGad projectile;
07:
08:     private float currentCooldownTime;
09:     private Transform spawnPoint;
10:     private PlayerBase owner;
11:
12:     public void Setup(PlayerBase owner, Transform spawnPoint)
13:     {
14:         this.owner = owner;
15:         this.spawnPoint = spawnPoint;
16:     }
17:
18:     public void OnSkill()
19:     {
20:         // 스킬이 사용할 수 있는 상태인지 검사(쿨타임)
21:         if ( Time.time - currentCooldownTime >
                owner.Stats.GetStat(StatType.CooldownTime).Value )
22:         {
23:             var result = CalculateDamage();
24:             ProjectileGad gad = Instantiate(projectile, spawnPoint.position,
                    Quaternion.identity);
25:             gad.Setup(owner.Target, result);
26:
27:             currentCooldownTime = Time.time;
28:         }
29:     }
30:
31:     private float CalculateDamage()
32:     {
33:         bool isCriticalHit = Random.value <
                owner.Stats.GetStat(StatType.CriticalChance).Value;
```

```
34:         float damage = owner.Stats.GetStat(StatType.Damage).Value;
35:
36:         if ( isCriticalHit )
37:             return damage * owner.Stats.GetStat(StatType.CriticalMultiplier).Value;
38:         else
39:             return damage;
40:     }
41: }
```

18~29줄 기본 공격 스킬을 사용해서 적을 공격할 때 호출하는 **OnSkill()** 메서드는 현재 시간(**Time.time**)에서 **currentCooldownTime**을 뺀 값이 플레이어의 **CooldownTime**값보다 크면 스킬을 사용합니다. 즉, 플레이어의 **CooldownTime** 스탯에 설정된 시간마다 **CalculateDamage()** 메서드를 호출해서 공격력을 계산하고 발사체 오브젝트를 생성합니다.

31~40줄 **CalculateDamage()** 메서드에서 크리티컬 공격 여부를 판단합니다. 플레이어의 크리티컬 확률 스탯은 0.0~1.0 사이 정규화한 값으로 설정하고 화면에 확률을 출력할 때는 100을 곱해 0~100%로 나타냅니다. **Random.value**는 0.0~1.0 사이 임의의 값을 반환하는 속성으로, 크리티컬 확률이 이 값보다 크면 크리티컬이 발동한 것으로 간주합니다. 크리티컬 공격이라면 기본 공격력에 크리티컬 공격력을 곱한 수를, 일반 공격이라면 기본 공격력을 반환합니다.

02 같은 폴더에 플레이어의 모든 스킬을 제어하는 SkillSystem.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/SkillSystem.cs

코드 05-09 플레이어의 모든 스킬 제어

```
01: using UnityEngine;
02:
03: public class SkillSystem : MonoBehaviour
04: {
05:     [SerializeField]
06:     private SkillGad skillGad;
07:     [SerializeField]
08:     private Transform skillSpawnPoint;
09:
10:     private PlayerBase owner;
11:
```

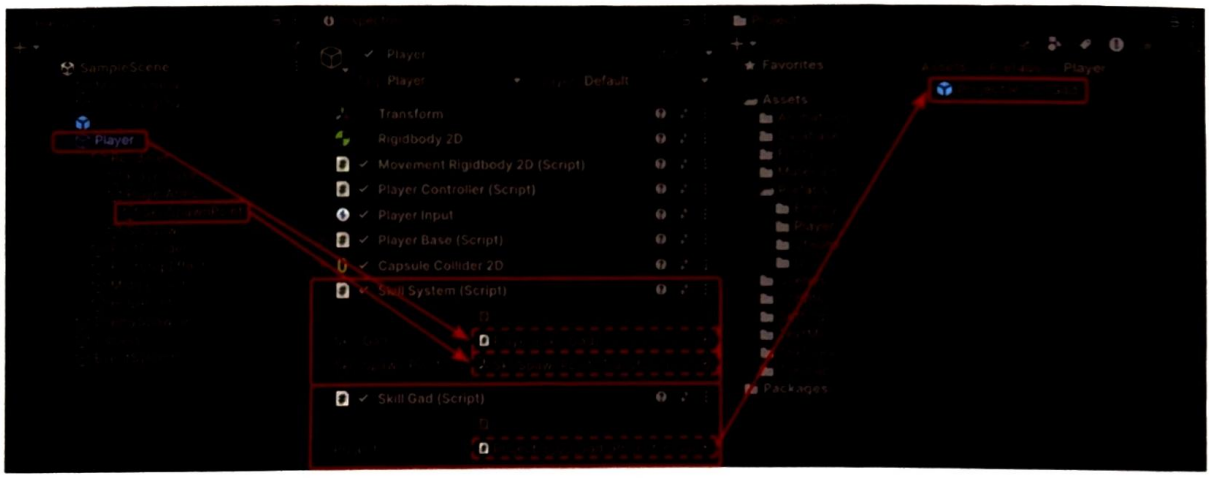
```
12:     private void Awake()
13:     {
14:         owner = GetComponent<PlayerBase>();
15:         skillGad.Setup(owner, skillSpawnPoint);
16:     }
17:
18:     private void Update()
19:     {
20:         // 플레이어의 목표가 없거나 이동 중이면 모든 스킬 사용 불가
21:         if ( owner.Target == null || owner.IsMoved == true ) return;
22:
23:         // 기본 공격 스킬 업데이트
24:         skillGad.OnSkill();
25:     }
26: }
```

03 [GameObject → Create Empty] 메뉴로 스킬 발사체의 생성 위치로 사용할 빈 오브젝트를 생성하고 이름을 'SkillSpawnPoint'로 변경합니다. 무기는 회전하므로 SkillSpawnPoint를 PlayerArms 오브젝트의 자식으로 배치합니다.

SkillSpawnPoint 오브젝트 Transform 컴포넌트의 Position을 (0.34, 0.055, 0)으로 설정하고 Select Icon 버튼 을 클릭해서 아이콘을 선택합니다. 여기서는 빨간색 원 아이콘을 지정했습니다.

스킬 발사체 생성 위치로 사용할 오브젝트 생성하고 설정하기

04 Player 오브젝트에 Skill System과 Skill Gad 컴포넌트를 추가합니다. Skill System 컴포넌트의 Skill Gad 변수에는 Player 오브젝트를, Skill Spawn Point 변수에는 SkillSpawnPoint 오브젝트를 등록합니다. Skill Gad 컴포넌트의 Projectile 변수에는 Projectile_00_Gad 프리팹을 등록합니다.



Player 오브젝트에 Skill System과 Skill Gad 컴포넌트 추가하고 설정하기

05 플레이 버튼 ▶을 눌러 게임을 실행하면 현재 플레이어의 쿨타임 시간인 2.5초마다 기본 공격 발사체를 생성해 목표를 향해 발사하고 발사체와 충돌한 적의 체력은 기본 공격력인 25씩 감소합니다.

Do it! 실습 기본 공격 피격 효과 표현하기

01 Assets 〉 Scripts 〉 Common 폴더에 파티클 오브젝트를 제어하는 ParticleAutoDestroyer.cs 스크립트를 생성합니다.

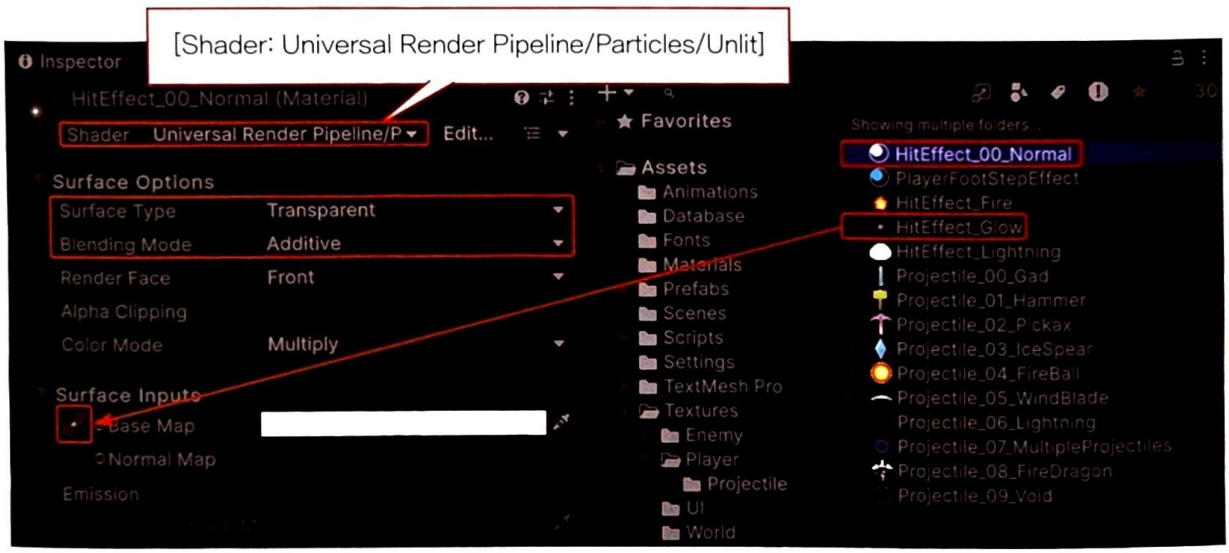
Assets/Scripts/Common/ParticleAutoDestroyer.cs

코드 05-10 파티클 오브젝트 재생 완료 후 삭제

```
01: using UnityEngine;
02:
03: public class ParticleAutoDestroyer : MonoBehaviour
04: {
05:     private ParticleSystem particle;
06:
07:     private void Awake()
08:     {
```

```
09:         particle = GetComponent<ParticleSystem>();
10:     }
11:
12:     private void Update()
13:     {
14:         if ( particle.isPlaying == false )
15:         {
16:             Destroy(gameObject);
17:         }
18:     }
19: }
```

02 Project View의 ＋▾ 버튼을 클릭해 [Material] 메뉴로 Assets 〉 Materials 폴더에 기본 공격 피격 효과에 적용할 재질 Material을 생성하고 이름을 'HitEffect_00_Normal'로 변경합니다. Shader를 [Universal Render Pipeline/Particles/Unlit]로 설정하고 Surface Type은 [Transparent]로, Blending Mode는 [Additive]로 설정합니다. 끝으로 Base Map에 HitEffect_Glow.png 이미지를 등록합니다.

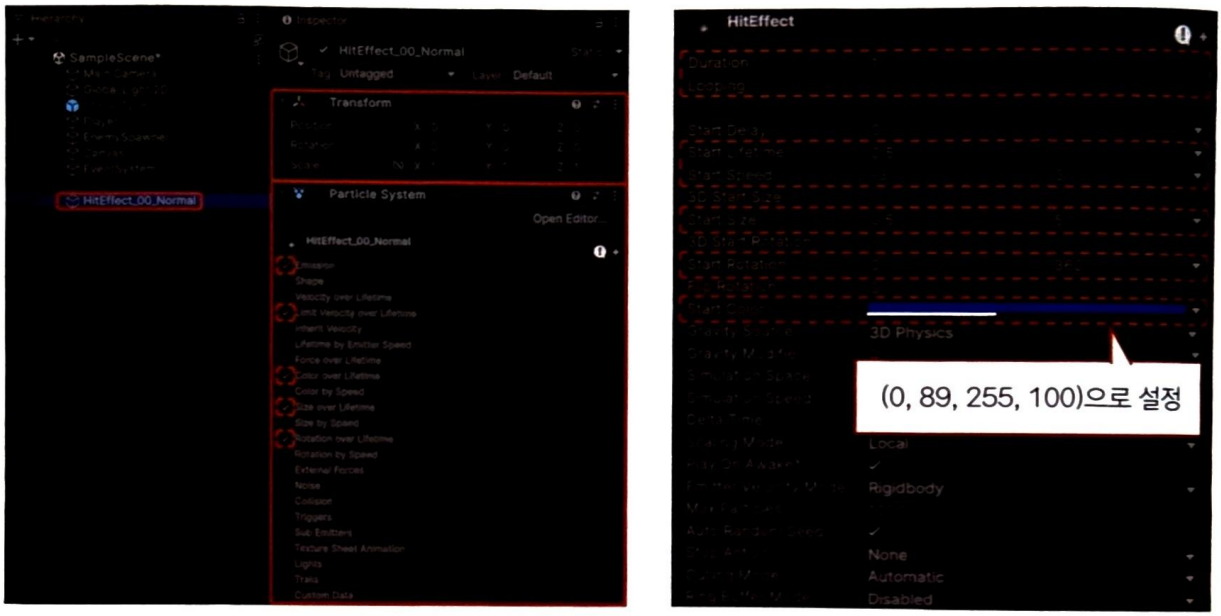


기본 공격 피격 효과에 적용할 재질 생성하고 설정하기

03 [GameObject → Effects → Particle System] 메뉴로 제자리에서 빛나는 기본 공격 피격 효과를 생성하고 이름을 'HitEffect_00_Normal'로 변경합니다. 다음 표를 참고해서 Particle System 컴포넌트 Main 모듈의 재생 시간 Duration, 반복 여부 Looping, 시작 수명 Start Lifetime, 속도 Start Speed, 시작 크기 Start Size, 회전 Start Rotation, 시작 색상 Start Color 변수를 설정합니다.

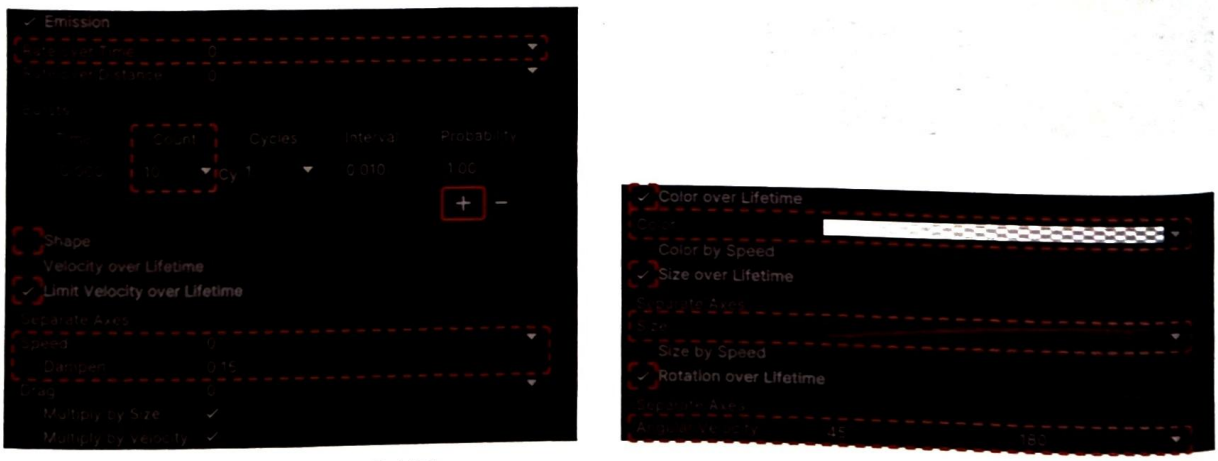
Particle System 컴포넌트 Main 모듈 설정값

Duration	Looping	Start Lifetime	Start Speed	Start Size	Start Rotation	Start Color
1	false	0.5	-3, 3	0.5, 5	0, 360	(0, 89, 255, 100)

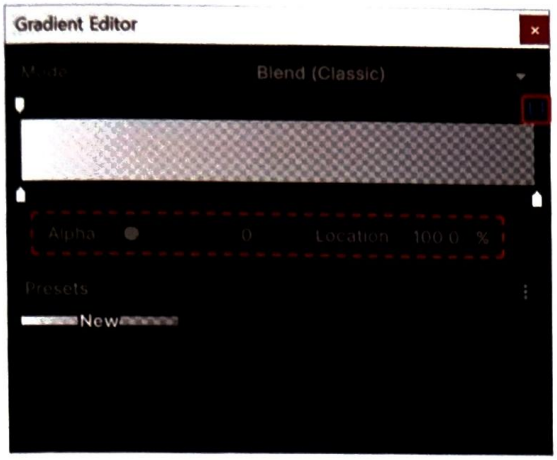


Particle System 컴포넌트 Main 모듈 설정

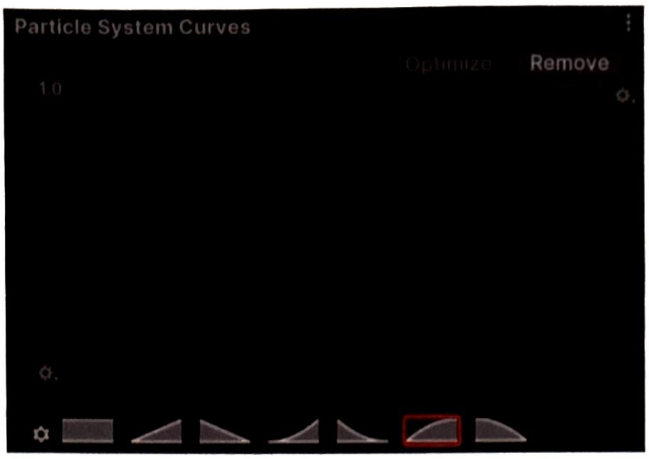
04 Emission 모듈의 Rate over Time을 '0'으로 설정하고 Bursts 리스트에 요소를 추가한 후 Count를 '10'으로 설정합니다. Shape 모듈은 비활성화합니다. Limit Velocity over Lifetime 모듈을 활성화하고 Speed는 '0', Dampen은 '0.15'로 설정합니다. Color over Lifetime 모듈 활성화하고 위치인 Location을 '100.0%'로, 투명도인 Alpha를 '0'으로 설정합니다.
Size over Lifetime 모듈을 활성화하고 Size 그래프를 위로 꺾인 곡선으로 설정합니다. Rotation over Lifetime 모듈을 활성화하고 Angular Velocity를 '45', '180'으로 설정합니다.



Particle System 컴포넌트 기타 모듈 설정

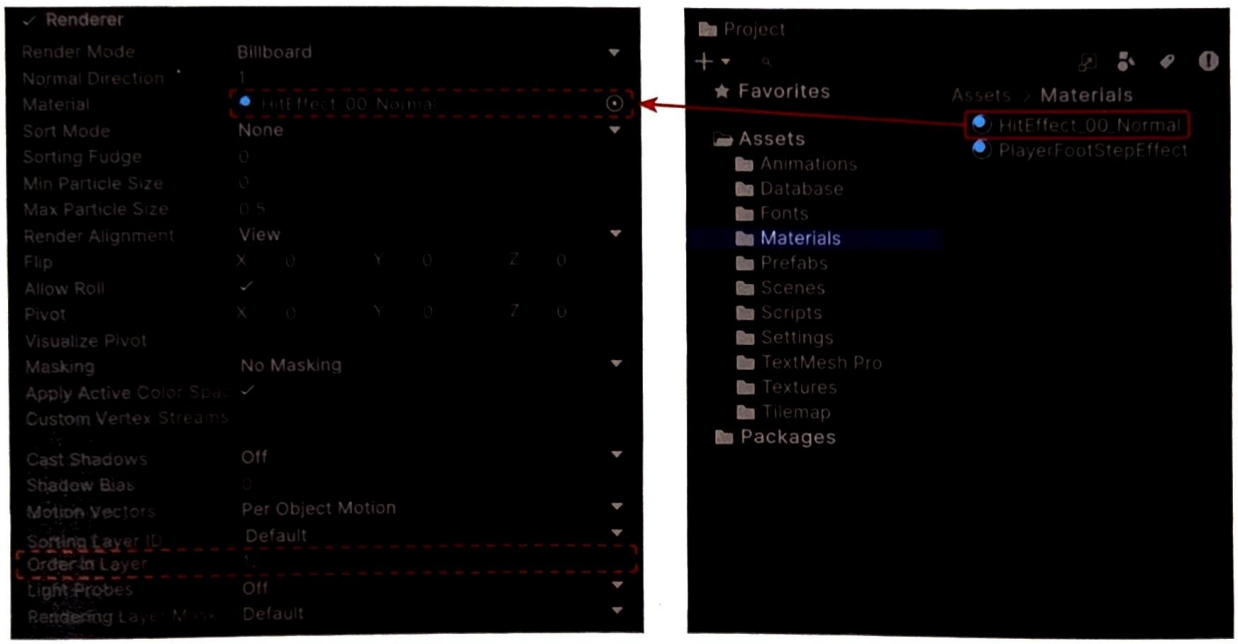


Color over Lifetime 모듈 설정



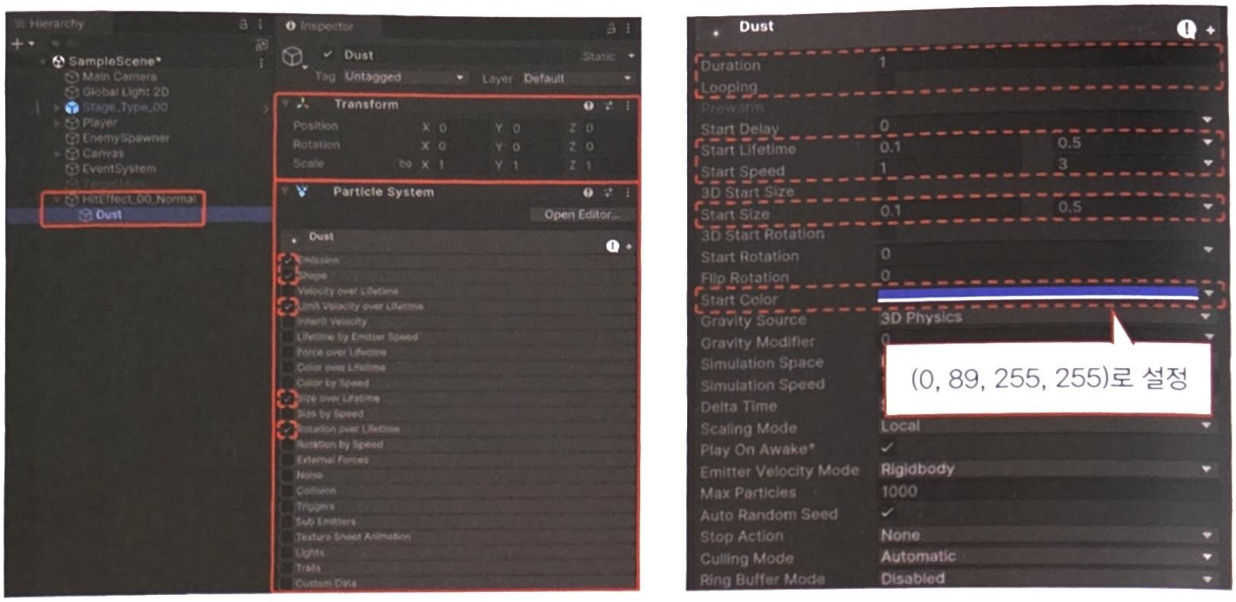
Size over Lifetime 모듈 설정

05 Renderer 모듈의 Material에 HitEffect_00_Normal을 등록하고 Order in Layer를 '12'로 설정합니다.

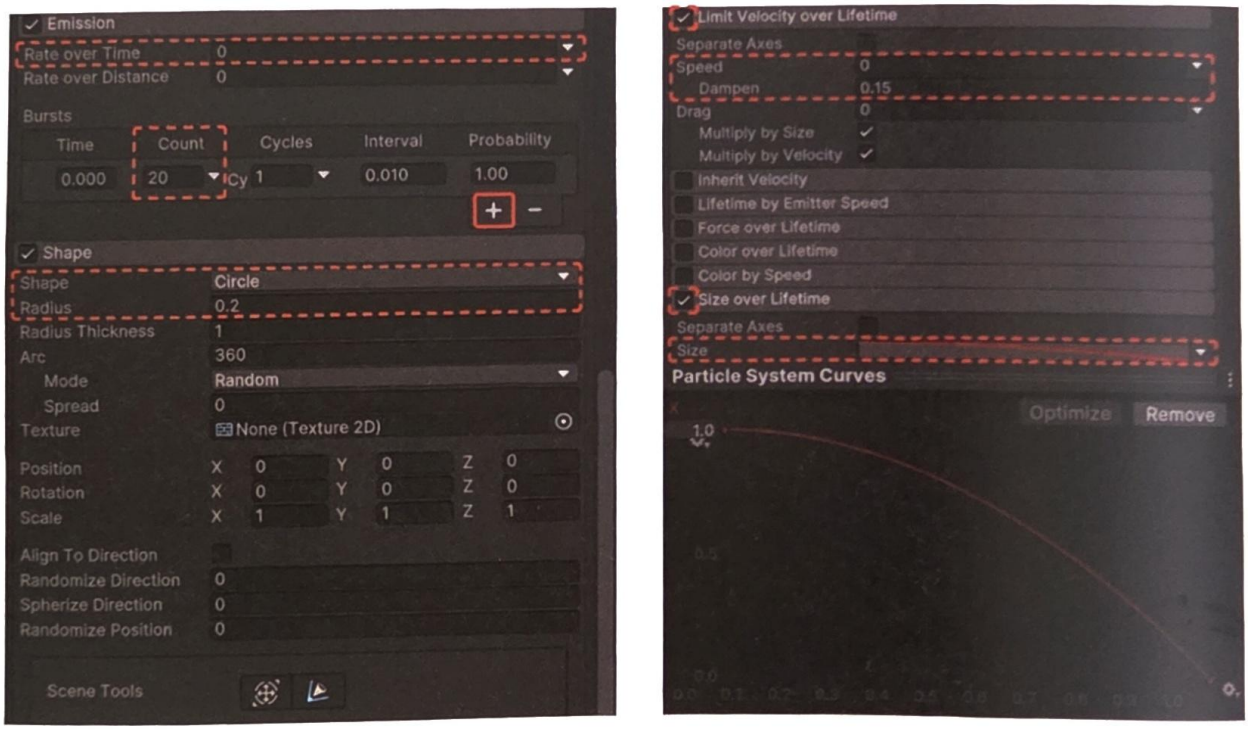


Renderer 모듈 설정

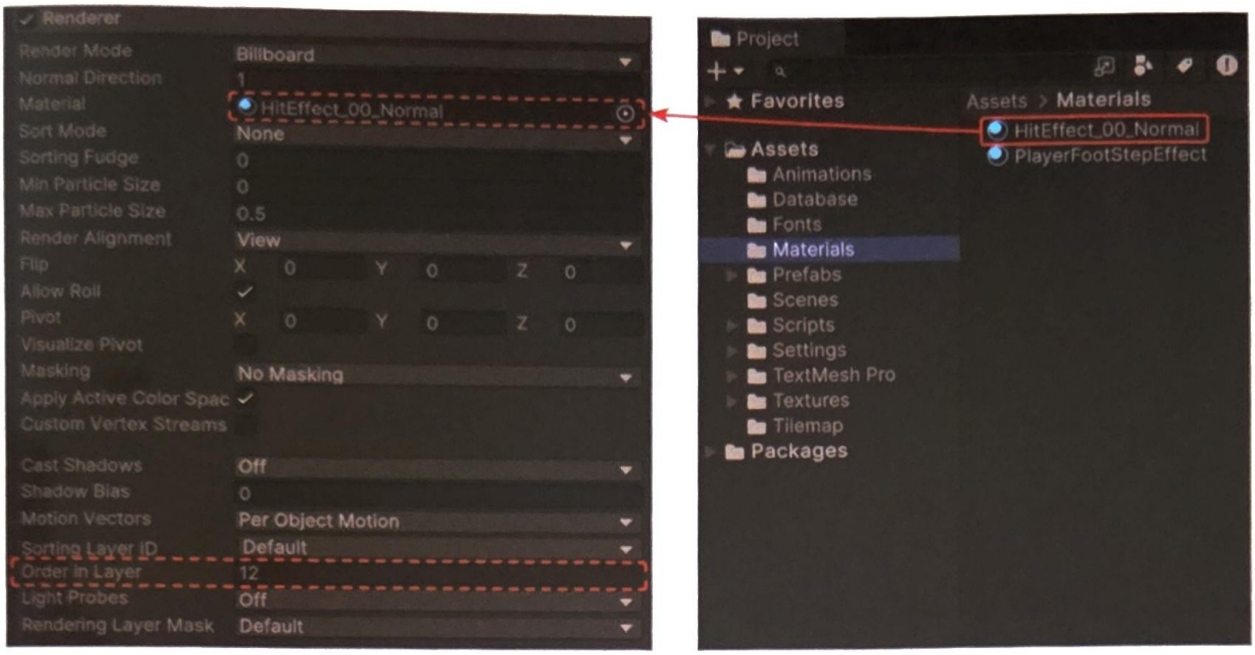
06 [GameObject → Effects → Particle System] 메뉴로 폭죽처럼 주변으로 퍼지는 기본 공격 피격 효과를 생성하고 이름을 'Dust'로 변경한 후 HitEffect_00_Normal 오브젝트의 자식으로 배치합니다. 마찬가지 요령으로 Main 모듈과 Emission, Shape, Limit Velocity over Lifetime, Size over Lifetime, Renderer 모듈의 변수를 그림처럼 설정합니다.



기본 공격 피격 효과 생성하고 설정하기 1

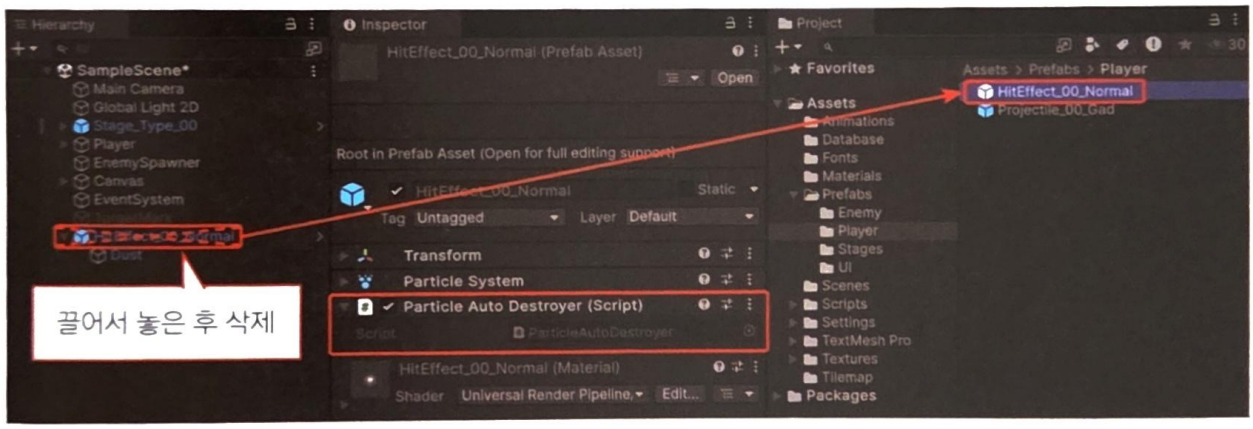


기본 공격 피격 효과 생성하고 설정하기 2



기본 공격 피격 효과 생성하고 설정하기 3

07 HitEffect_00_Normal 오브젝트에 Particle Auto Destroyer 컴포넌트를 추가합니다. HitEffect_00_Normal을 Project View의 Assets 〉 Prefabs 〉 Player 폴더로 끌어서 놓아 프리팹을 생성하고 Hierarchy View에 있는 HitEffect_00_Normal 오브젝트는 삭제합니다.



HitEffect_00_Normal 오브젝트 프리팹 생성하기

08 기본 공격 발사체가 적과 충돌했을 때 피격 효과를 생성하도록 ProjectileGad.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/Projectile/ProjectileGad.cs

코드 05-11 발사체 충돌 시 피격 효과 재생

```
01: using UnityEngine;
02:
03: public class ProjectileGad : MonoBehaviour
04: {
05:     [SerializeField]                                    추가
06:     private Transform hitEffect;
        (... 생략 ...)
28:     private void OnTriggerEnter2D(Collider2D collision)
29:     {
30:         if ( collision.CompareTag("Wall") )
                (... 생략 ...)
34:         else if ( collision.CompareTag("Enemy") &&
35:             collision.TryGetComponent<EntityBase>(out var entity) )
36:         {
37:             if ( entity != target ) return;
38:
39:             if ( hitEffect != null )                    추가
40:             {
41:                 Instantiate(hitEffect, transform.position, Quaternion.identity);
42:             }
43:
44:             entity.TakeDamage(damage);
45:             Destroy(gameObject);
46:         }
47:     }
48: }
```

39~42줄 피격 효과(hitEffect) 오브젝트 변수가 비지 않았다면 Instantiate() 메서드를 호출해 현재 위치에 피격 효과를 생성합니다.

09 마지막으로 Projectile_00_Gad 프리팹 Projectile Gad 컴포넌트의 Hit Effect 변수에 HitEffect_00_Normal 프리팹을 등록합니다.

Projectile_00_Gad 프리팹의 Projectile Gad 컴포넌트 변수 설정하기

결과 화면은 다음 실습 후에 확인합니다.

Do it! 실습 기본 공격 데미지 표시하기

01 Assets 〉 Scripts 〉 UI 폴더에 데미지 텍스트 UI를 제어하는 UIDamageText.cs 스크립트를 생성합니다.

Assets/Scripts/UI/UIDamageText.cs

코드 05-12 기본 공격 데미지 UI 출력 제어

```
01: using UnityEngine;
02: using TMPro;
03:
04: public class UIDamageText : MonoBehaviour
05: {
06:     [SerializeField]
07:     private float arriveTime = 0.5f;
08:     private float percent = 0;
09:     private MovementRigidbody2D movement2D;
10:     private TextMeshPro text;
11:
12:     public void Setup(string text, Color color)
13:     {
14:         movement2D = GetComponent<MovementRigidbody2D>();
```

```
15:         movement2D.MoveTo(new Vector3(Random.Range(-1f, 1f), 1, 0));
16:
17:         this.text = GetComponent<TextMeshPro>();
18:         this.text.text = text;
19:         this.text.color = color;
20:
21:         Destroy(gameObject, arriveTime);
22:     }
23:
24:     private void Update()
25:     {
26:         if ( percent > 1 ) return;
27:
28:         percent += Time.deltaTime / arriveTime;
29:
30:         text.color = new Color(text.color.r, text.color.g, text.color.b,
                1 - percent);
31:     }
32: }
```

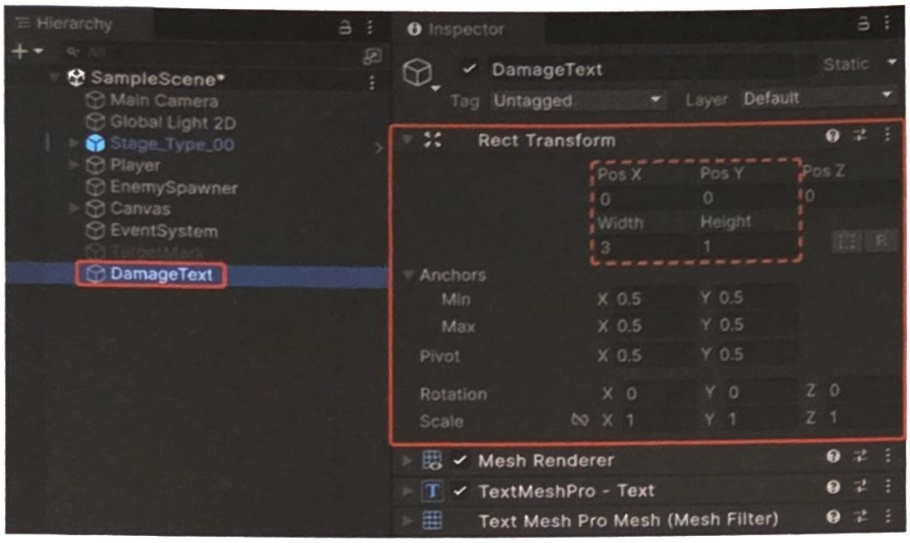
15줄 데미지 Text UI의 이동 방향을 (-1.0~1.0, 1, 0)으로 설정하고 Rigidbody 2D 컴포넌트의 중력을 이용해 UI가 포물선 모양으로 이동하면서 아래로 떨어지도록 설정합니다.

21줄 `arriveTime` 시간 뒤에 오브젝트를 삭제하도록 `Destroy()` 메서드를 호출합니다.

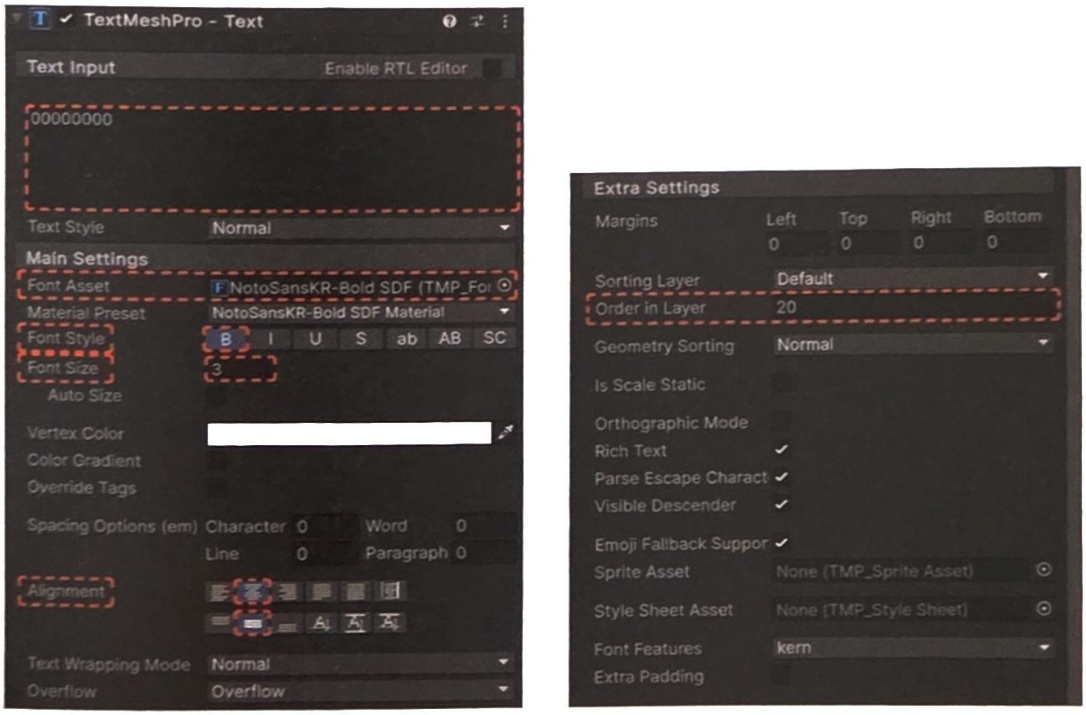
24~31줄 `Update()` 메서드에서 Text UI의 생존 시간 동안 알파값이 서서히 감소해 Text UI가 점점 흐려지도록 설정합니다.

02 [GameObject → UI → Text - TextMeshPro] 메뉴로 기본 공격 데미지 텍스트를 출력하는 오브젝트를 생성하고 이름을 'DamageText'로 변경합니다. Rect Transform 컴포넌트의 Rect를 (0, 0, 3, 1)로 설정합니다.

TextMeshPro - Text 컴포넌트의 Text Input Box 내용을 '00000000'으로 설정하고 Font Asset에 NotoSansKR-Bold SDF 에셋을 등록합니다. Font Style은 굵게를 뜻하는 [B]로, Font Size는 '3'으로, 정렬을 뜻하는 Alignment는 Center | Middle로 설정하고 Extra Settings 옵션을 열어 맨 앞에 표시하도록 Order in Layer를 '20'으로 설정합니다.

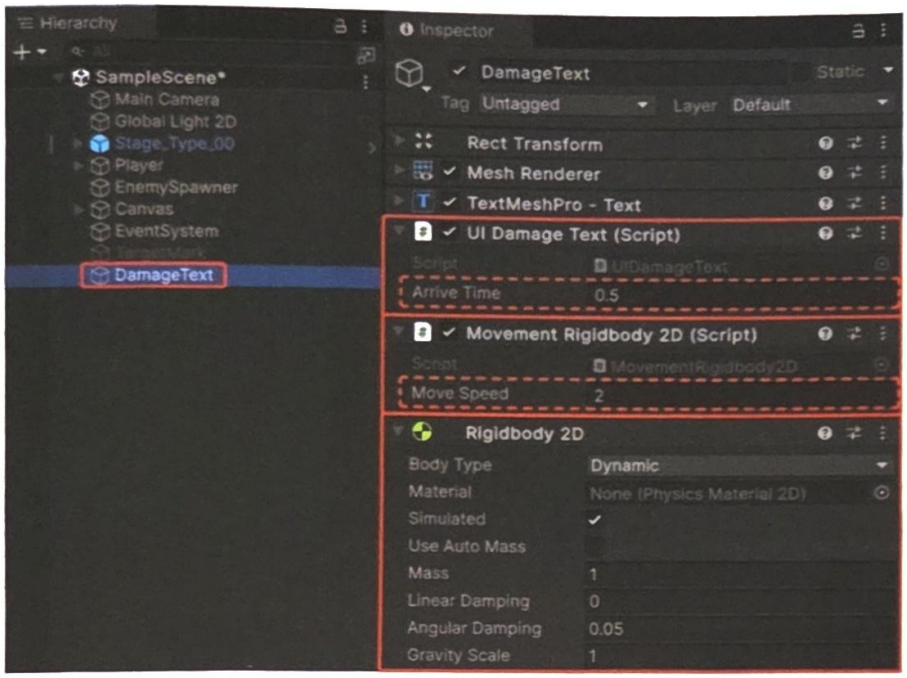


DamageText 오브젝트 Rect Transform 설정하기



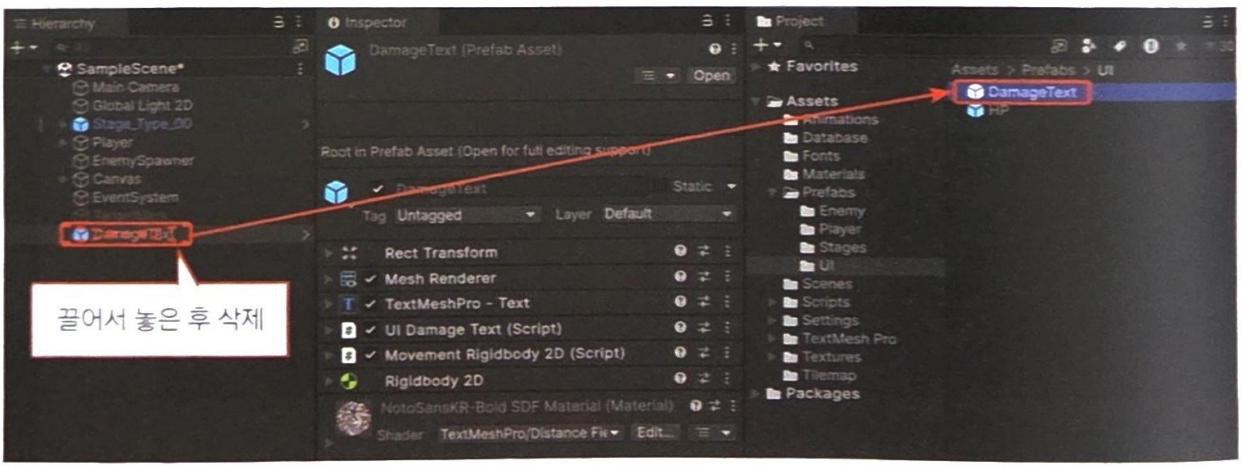
기본 공격 데미지를 출력하는 오브젝트 생성하고 설정하기

03 DamageText 오브젝트에 UI Damage Text, Movement Rigidbody 2D, Rigidbody 2D 컴포넌트를 추가합니다. UI Damage Text 컴포넌트의 Arrive Time은 '0.5'로, Movement Rigidbody 2D 컴포넌트의 Move Speed는 '2'로 설정합니다.



DamageText 오브젝트에 컴포넌트 추가하고 설정하기

04 DamageText 오브젝트를 Project View의 Assets 〉 Prefabs 〉 UI 폴더로 끌어서 놓아 프리팹을 생성하고 Hierarchy View에 있는 DamageText 오브젝트는 삭제합니다.



DamageText 오브젝트 프리팹 생성하기

05 기본 공격 발사체를 생성할 때 크리티컬 공격 여부 정보를 전달하도록 SkillGad.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillGad.cs

코드 05-13 크리티컬 공격 여부 정보 전달

```
01: using UnityEngine;
02:
03: public class SkillGad : MonoBehaviour
04: {
        (... 생략 ...)
18:     public void OnSkill()
19:     {
20:         // 스킬이 사용할 수 있는 상태인지 검사(쿨타임)
21:         if ( Time.time - currentCooldownTime >
                    owner.Stats.GetStat(StatType.CooldownTime).Value )
22:         {
23:             var result = CalculateDamage();
24:             ProjectileGad gad = Instantiate(projectile, spawnPoint.position,
                    Quaternion.identity);
25:             gad.Setup(owner.Target, result.Item1, result.Item2);   ← 수정
26:
27:             currentCooldownTime = Time.time;
28:         }
29:     }
30:
31:     private (float, bool) CalculateDamage()   ← 수정
32:     {
33:         bool isCriticalHit = Random.value <
                owner.Stats.GetStat(StatType.CriticalChance).Value;
34:         float damage = owner.Stats.GetStat(StatType.Damage).Value;
35:
36:         if ( isCriticalHit )
37:             return (damage * owner.Stats.GetStat(StatType.CriticalMultiplier).Value,
                    isCriticalHit);   ← 수정
38:         else
39:             return (damage, isCriticalHit);   ← 수정
40:     }
41: }
```

25줄 CalculateDamage() 메서드에서 공격력(float)과 크리티컬 여부(bool)를 반환받아 ProjectileGad 클래스의 Setup() 메서드 매개변수로 전달합니다.

06 기본 공격 발사체가 적과 충돌했을 때 데미지를 출력하고자 ProjectileGad.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/Projectile/ProjectileGad.cs

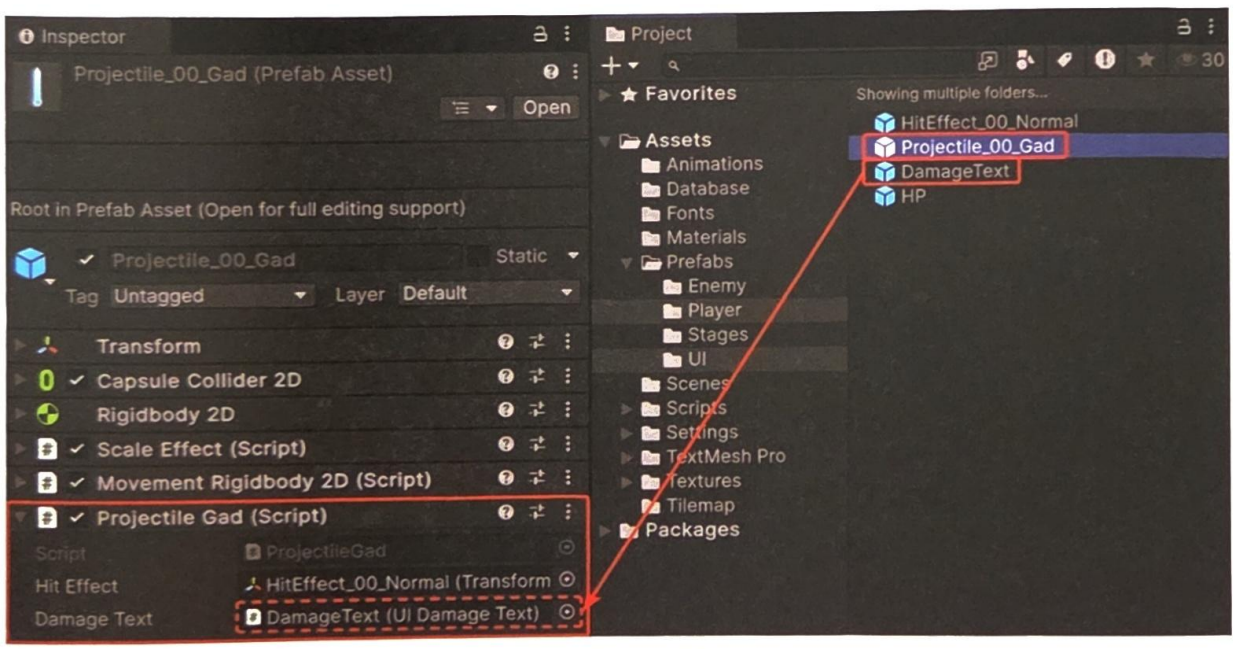
코드 05-14 발사체 충돌 시 데미지 Text UI 출력

```
01: using UnityEngine;
02:
03: public class ProjectileGad : MonoBehaviour
04: {
05:     [SerializeField]
06:     private Transform hitEffect;
07:     [SerializeField]                                    추가
08:     private UIDamageText damageText;
09:
10:     private MovementRigidbody2D movementRigidbody2D;
11:     private ScaleEffect scaleEffect;
12:     private EntityBase target;
13:     private float damage;
14:     private bool isCritical;                            추가
15:
16:     public void Setup(EntityBase target, float damage, bool isCritical=false)   수정
17:     {
18:         movementRigidbody2D = GetComponent<MovementRigidbody2D>();
19:         scaleEffect = GetComponent<ScaleEffect>();
20:         this.target = target;
21:         this.damage = damage;
22:         this.isCritical = isCritical;                   추가
            (... 생략 ...)
30:     }
31:
32:     private void OnTriggerEnter2D(Collider2D collision)
33:     {
34:         if ( collision.CompareTag("Wall") )
                (... 생략 ...)
38:         else if ( collision.CompareTag("Enemy") &&
39:             collision.TryGetComponent<EntityBase>(out var entity) )
40:         {
41:             if ( entity != target ) return;
                (... 생략 ...)
47:             if ( damageText != null )               추가
48:             {
49:                 UIDamageText clone = Instantiate(damageText, transform.position,
                        Quaternion.identity);
```

```
50:                 clone.Setup(damage.ToString("F0"), isCritical ?
                    Color.red : Color.white);
51:             }
52:
53:             entity.TakeDamage(damage);
54:             Destroy(gameObject);
55:         }
56:     }
57: }
```

47~51줄 데미지 Text UI 오브젝트 변수가 비지 않았다면 Instantiate() 메서드를 호출해서 현재 위치에 데미지 Text UI를 생성합니다. Text UI의 색상은 크리티컬 공격이 아니면 하얀색으로, 크리티컬 공격이면 빨간색으로 설정합니다.

07 Projectile_00_Gad 프리팹의 Projectile Gad 컴포넌트 Damage Text 변수에 Damage Text 프리팹을 등록합니다.



Projectile_00_Gad 프리팹의 Projectile Gad 컴포넌트 변수 설정하기

08 플레이 버튼 ▶을 눌러 게임을 실행하면 현재 플레이어의 쿨타임 시간인 2.5초마다 기본 공격 발사체를 생성해 목표를 향해 발사하고 발사체가 충돌했을 때 피격 효과와 데미지 Text UI를 생성합니다.

플레이어 기본 공격 스킬 결과 화면

05-3

버프 스킬 만들기

이 절에서는 캐릭터의 능력을 강화하는 버프Buff 스킬을 구현하고자 스킬을 데이터 형태로 저장하는 SkillTemplate 클래스를 만들고 SkillSystem 클래스에서 스킬 데이터를 불러와 사용하고 관리합니다. 이와 함께 플레이어의 기본 공격 발사체 생성 개수, 발사체 전이 횟수 등을 버프 스킬로 추가하는 기능을 구현합니다.

스킬 클래스 구현하고 버프 구현하기

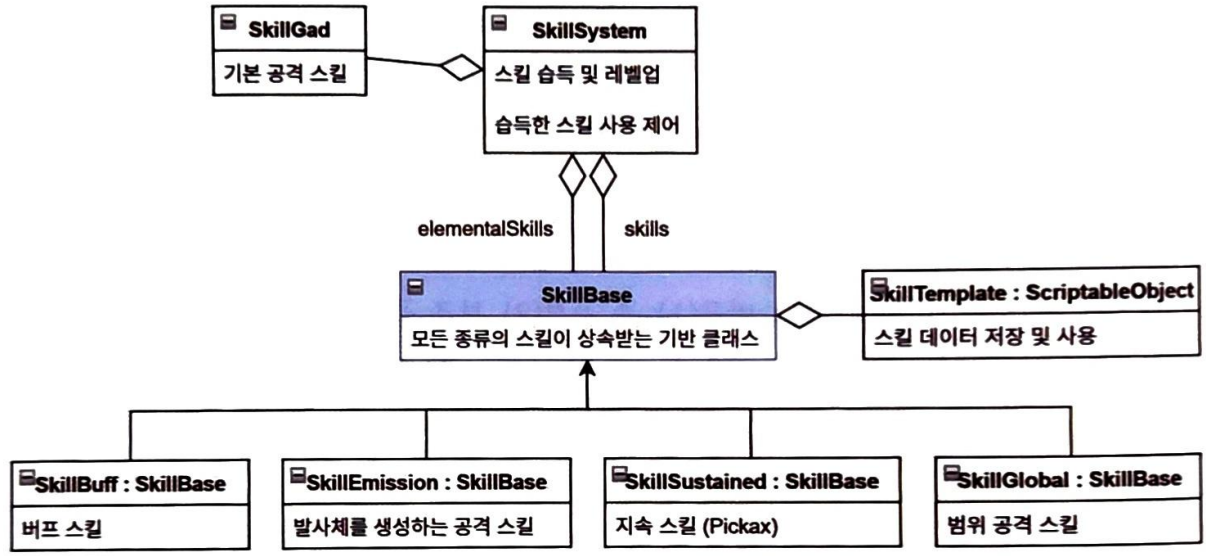
스킬은 속성별로 속성 보너스(1), 버프(1), 주 공격(2), 보조 공격(1), 근접 공격(1) 등 모두 6가지입니다. 속성 보너스 스킬은 해당 속성 스킬 레벨 총합이 3의 배수가 될 때마다 1씩 자동으로 증가하고 나머지 스킬은 플레이어가 스킬 선택 보상을 받을 때마다 선택해서 증가합니다.

속성별로 구분한 6가지 스킬

스킬 분류	얼음(Ice)	불(Fire)	바람(Wind)	빛(Light)	어둠(Dark)
속성 보너스 [Max Lv. 99]	얼음 속성 공격력	불 속성 공격력	바람 속성 공격력	빛 속성 공격력, 체력	어둠 속성 공격력
버프 [Max Lv. 10]	크리티컬 확률, 크리티컬 공격력	기본 공격력	쿨타임 감소, 발사체 전이	회피율	체력 회복량, 발사체 개수
주 공격 1 [Max Lv. 99]	아이스 스피어	파이어 볼	윈드 블레이드	라이트닝	다중 발사체
주 공격 2 [Max Lv. 99]	블리자드	파이어 드래곤	토네이도	라이트닝 스트라이크	암흑
보조 공격 [Max Lv. 99]	(속성) 해머				
근접 공격 [Max Lv. 99]	(속성) 곡괭이				

기본 공격을 포함하여 모든 스킬의 습득, 사용, 레벨업은 SkillSystem에서 제어합니다. 기본 공격 스킬을 구현한 SkillGad를 제외하고 나머지 스킬은 모두 SkillBase를 상속받아 자료형 Type을 통일합니다. 그리고 SkillTemplate을 이용해 스킬 데이터를 저장하고 게임을 실행할 때 불러와 사용합니다.

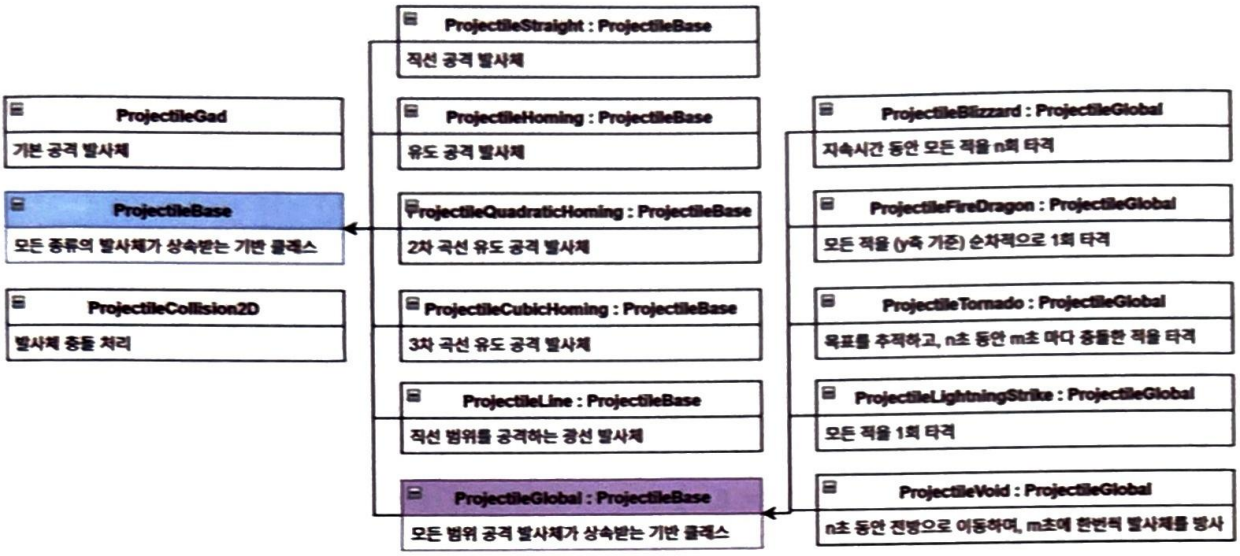
SkillBase를 상속받는 각 스킬은 버프 스킬인 SkillBuff로, 발사체를 생성하는 공격 스킬인 SkillEmission으로, 지속 스킬인 SkillSustained로, 범위 공격 스킬인 SkillGlobal로 구현합니다.



스킬 클래스 관계도

기본 공격을 제외한 모든 스킬의 발사체는 ProjectileBase를 상속받아 자료형을 통일합니다. ProjectileBase를 상속받는 발사체는 각각 사출 형태에 따라 직선 공격 발사체인 ProjectileStraight로, 유도 공격 발사체인 ProjectileHoming으로, 2차 곡선 유도 공격 발사체인 ProjectileQuadraticHoming으로, 3차 곡선 유도 공격 발사체인 ProjectileCubicHoming으로, 직선 범위를 공격하는 광선 발사체인 ProjectileLine으로 구현합니다.

그리고 범위 공격도 발사체를 생성하는 형태로 ProjectileGlobal을 상속받아 공격 형태에 따라 클래스로 구현합니다. 직접 충돌하는 발사체라면 ProjectileCollision2D 클래스로 충돌을 처리합니다.



발사체 클래스 관계도

다음은 버프 스킬 효과에 있는 스탯 증가 중에서 기존 스탯에 발사체 전이 횟수, 체력 회복, 발사체 개수 등의 스탯 정보를 추가한 표입니다.

플레이어 스탯

스탯	변수 이름	기본값	최솟값	최댓값
레벨	Level	1	1	99
경험치	Exp	0	0	99999999
기본 공격력	Damage	25	1	9999
공격 쿨타임	Cooldown Time	2.5	0.5	2.5
크리티컬 확률	Critical Chance	0.05	0	1.0 (1.0=100%)
크리티컬 공격력 계수	Critical Multiplier	2	1	4
(현재, 최대) 체력	HP	100	0	9999
회피	Evasion	0.05	0	1.0 (1.0=100%)
발사체 전이 횟수	Metastasis Count	0	0	3
체력 회복	HP Recovery	0	0	9999
발사체 개수	Projectile Count	1	1	4

Do it! 실습 스킬 에셋 만들기

01 Assets 〉 Scripts 〉 Skill 폴더에 스킬 정보를 에셋으로 저장하는 SkillTemplate.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/SkillTemplate.cs

코드 05-15 스킬 에셋

```
01: using System.Collections.Generic;
02: using UnityEngine;
03:
04: public enum SkillType { Buff = 0, }
05: public enum SkillElement { None = -1, Ice = 100, Fire, Wind, Light, Dark,
        Count = 5 }
06:
07: [CreateAssetMenu(fileName = "NewSkill", menuName = "SkillAsset", order = 0)]
08: public class SkillTemplate : ScriptableObject
09: {
10:     [Header("공통")]
11:     public string skillName;        // 스킬 이름
12:     public SkillType skillType;     // 스킬 타입
13:     public SkillElement element;    // 스킬 속성
14:     public int maxLevel;            // 스킬 최대 레벨
15:     [TextArea(1, 30)]
16:     public string description;      // 스킬 설명
17:     public Sprite disableIcon;      // 스킬 미습득 아이콘
18:     public Sprite enableIcon;       // 스킬 습득 아이콘
19:
20:     [Header("버프 스킬")]
21:     public List<Stat> buffStatList; // 버프 스킬
22: }
```

21줄 버프 스킬에 따라 추가되는 스탯 개수가 다를 수 있으므로 Stat형 리스트 변수를 선언하고 Inspector View에서 해당 버프 스킬에 적용되는 스탯 정보를 설정합니다.

02 새로운 스탯을 StatType 열거형에 추가하고자 Stat.cs 스크립트를 수정합니다.

Assets/Scripts/Entity/Stat.cs

코드 05-16 StatType 열거형에 새로운 스탯 정보 추가

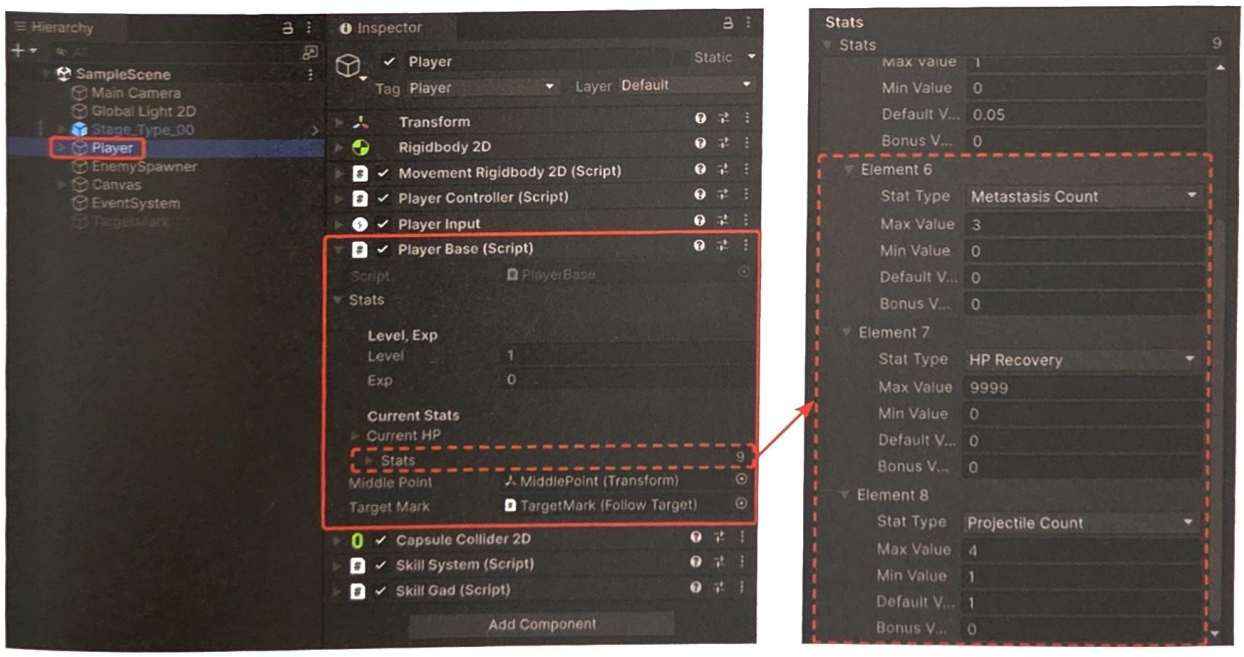
```
01: using UnityEngine;
    (... 생략 ...)
54: public enum StatType { Damage = 0, CooldownTime, CriticalChance,
        CriticalMultiplier, HP, Evasion,
55:     MetastasisCount, HPRecovery, ProjectileCount, }
```

수정

03 Player 오브젝트의 Player Base 컴포넌트에서 Stats 배열 크기를 '9'로 변경하고 다음 표를 참고하여 요소를 추가합니다.

Stats 배열 요소 설정값

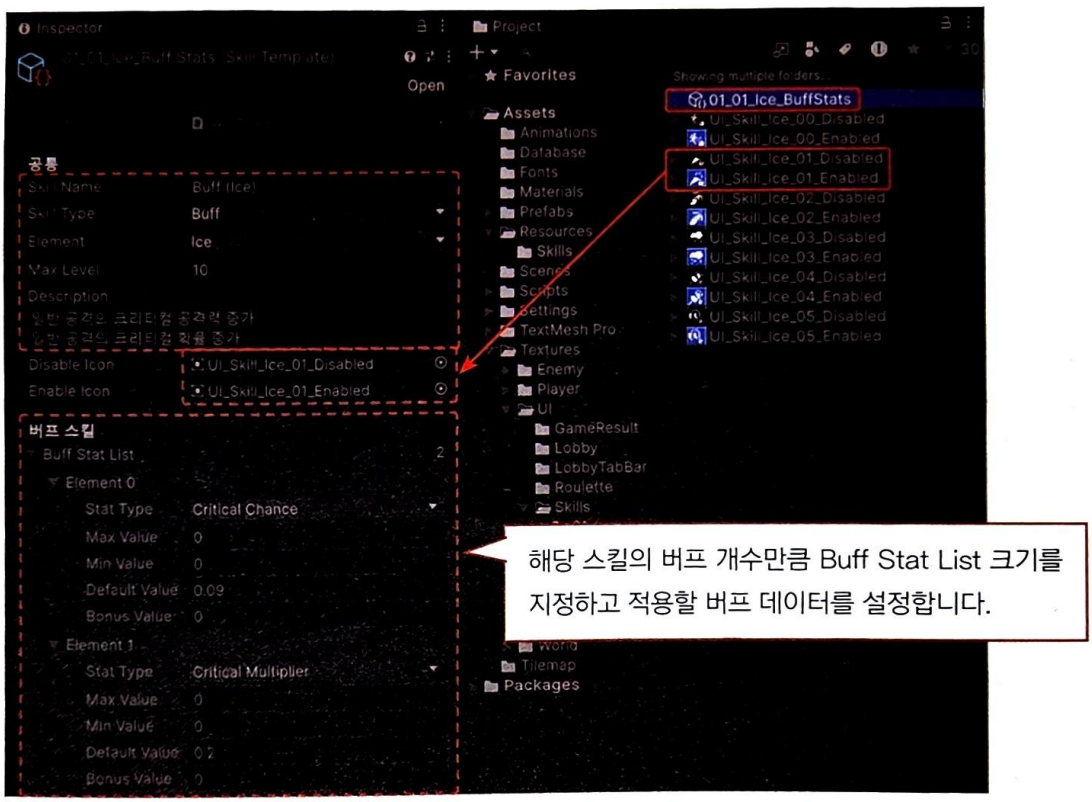
Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
6	Metastasis Count	3	0	0	0
7	HP Recovery	9999	0	0	0
8	Projectile Count	4	1	1	0



Player 오브젝트의 Player Base 컴포넌트에서 Stats 배열 변수 설정하기

04 먼저 Assets 〉 Resources 〉 Skills 폴더를 만듭니다. 그런 다음, 얼음 속성 버프 스킬의 SO(`ScriptableObject`) 데이터를 저장하고자 Project View의 ➕▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '01_01_Ice_BuffStats'로 변경합니다. 얼음 속성 버프 스킬의 Skill Name에 'Buff (Ice)'를 입력하고 Skill Type은 [Buff], Element는 [Ice], Max Level은 '10', 설명과 아이콘은 그림처럼 설정합니다. 그리고 2개의 스탯 버프를 적용하고자 Buff Stat List의 크기를 '2'로 설정합니다.

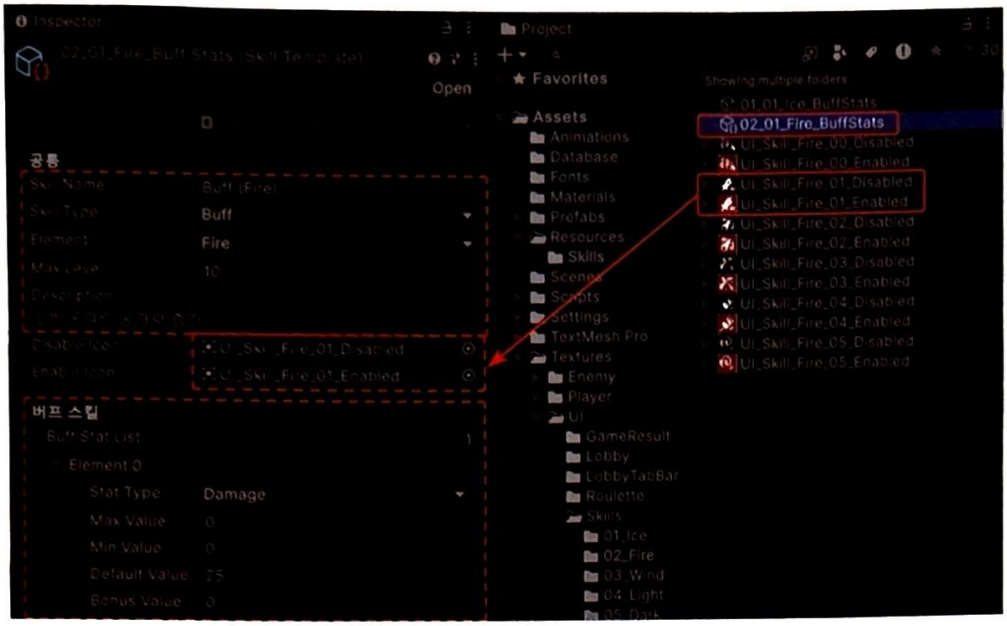
첫 번째 요소(Element 0)의 Stat Type을 [Critical Chance]로, Default Value를 '0.09'로 설정합니다. 두 번째 요소(Element 1)의 Stat Type을 [Critical Multiplier]로, Default Value를 '0.2'로 설정합니다.



얼음 속성 버프 스킬 에셋 생성하고 설정하기

05 불 속성 버프 스킬의 SO 데이터를 저장하고자 앞서와 마찬가지 방법으로 스킬 에셋을 생성하고 이름을 '02_01_Fire_BuffStats'로 변경합니다. 불 속성 버프 스킬의 Skill Name에 'Buff (Fire)'를 입력하고 Skill Type은 [Buff], Element는 [Fire], Max Level은 '10', 설명과 아이콘은 그림처럼 설정합니다. 그리고 1개의 스탯 버프를 적용하고자 Buff Stat List의 크기를 '1'로 설정합니다.

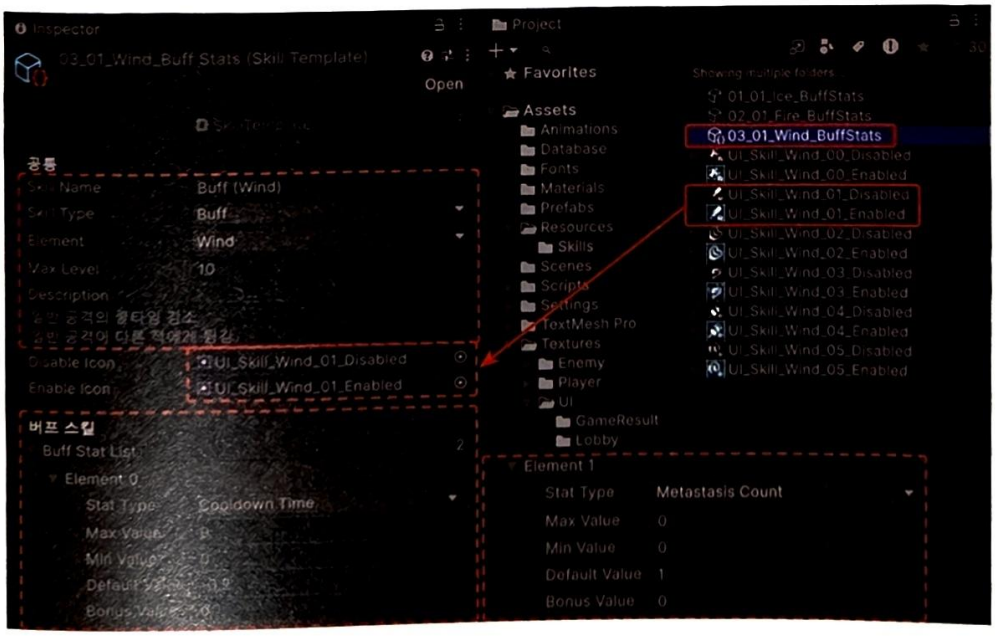
첫 번째 요소(Element 0)의 Stat Type을 [Damage]로, Default Value를 '25'로 설정합니다.



불 속성 버프 스킬 에셋 생성하고 설정하기

06 바람 속성 버프 스킬의 SO 데이터를 저장하고자 스킬 에셋을 한 번 더 생성하고 이름을 '03_01_Wind_BuffStats'로 변경합니다. 바람 속성 버프 스킬의 Skill Name에 'Buff (Wind)'를 입력하고 Skill Type은 [Buff], Element는 [Wind], Max Level은 '10', 설명과 아이콘은 그림처럼 설정합니다. 그리고 2개의 스탯 버프를 적용하고자 Buff Stat List의 크기를 '2'로 설정합니다.

첫 번째 요소(Element 0)의 Stat Type을 [Cooldown Time]으로, 쿨타임이 0.2초씩 줄도록 Default Value를 '-0.2'로 설정합니다. 두 번째 요소(Element 1)의 Stat Type을 [Metastasis Count]로, Default Value를 '1'로 설정합니다.



바람 속성 버프 스킬 에셋 생성하고 설정하기

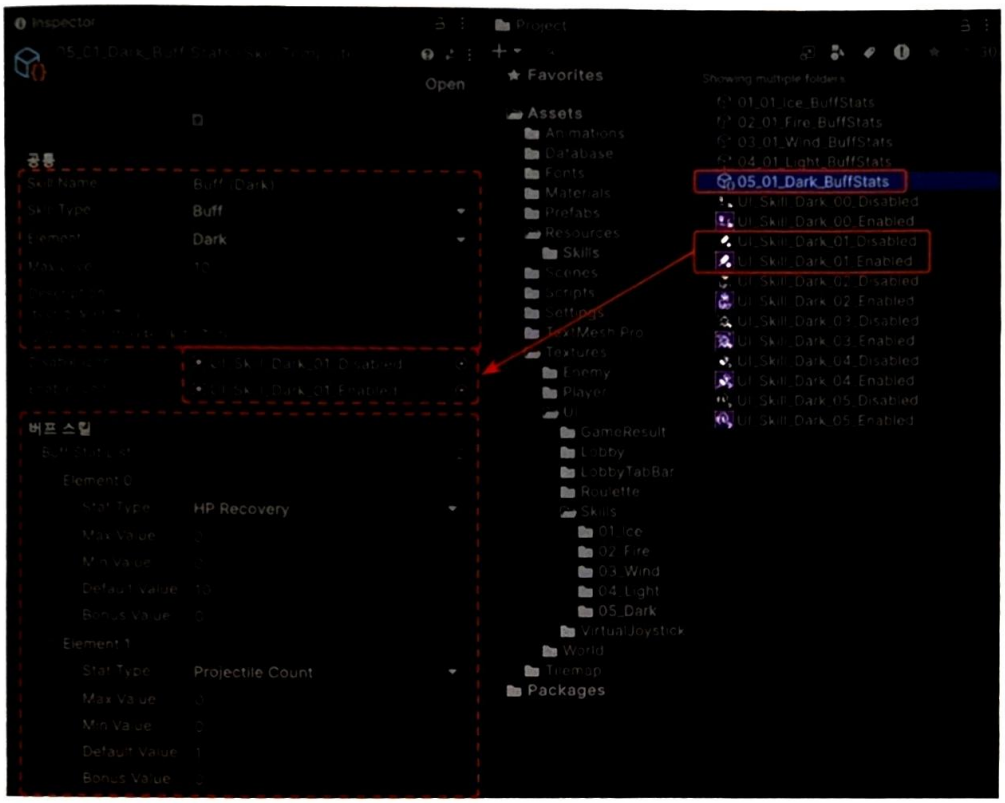
07 빛 속성 버프 스킬의 SO 데이터를 저장하고자 스킬 에셋을 추가로 생성하고 이름을 '04_01_Light_BuffStats'로 변경합니다. 빛 속성 버프 스킬의 Skill Name에 'Buff (Light)'를 입력하고 Skill Type은 [Buff], Element는 [Light], Max Level은 '10', 설명과 아이콘은 그림처럼 설정합니다. 그리고 1개의 스탯 버프가 적용되도록 Buff Stat List의 크기를 '1'로 설정합니다.

첫 번째 요소(Element 0)의 Stat Type을 [Evasion]으로, Default Value를 '0.09'로 설정합니다.

빛 속성 버프 스킬 에셋 생성하고 설정하기

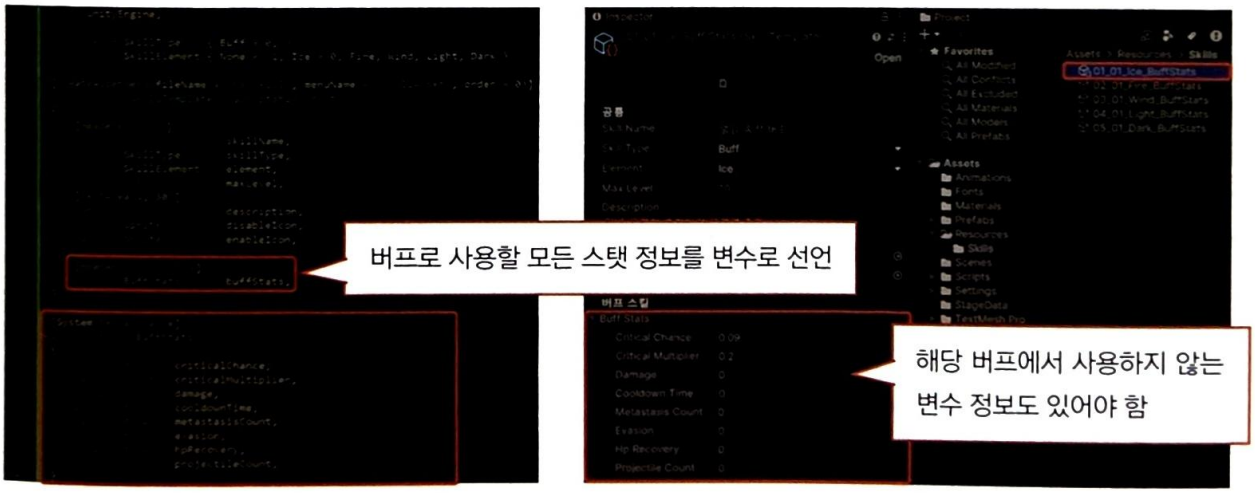
08 어둠 속성 버프 스킬의 SO 데이터를 저장하고자 스킬 에셋을 추가로 생성하고 이름을 '05_01_Dark_BuffStats'로 변경합니다. 어둠 속성 버프 스킬의 Skill Name에 'Buff (Dark)'를 입력하고 Skill Type은 [Buff], Element는 [Dark], Max Level은 '10', 설명과 아이콘은 그림처럼 설정합니다. 그리고 2개의 스탯 버프를 적용하고자 Buff Stat List의 크기를 '2'로 설정합니다.

첫 번째 요소(Element 0)의 Stat Type을 [HP Recovery]로, Default Value를 '10'으로 설정합니다. 두 번째 요소(Element 1)의 Stat Type을 [Projectile Count]로, Default Value를 '1'로 설정합니다.



어둠 속성 버프 스킬 에셋 생성하고 설정하기

Stat 클래스를 정의하지 않고 기존 코드처럼 스탯을 변수로 적용한다면 SkillTemplate 클래스에서 버프로 인해 증가하는 모든 스탯 정보를 변수로 선언해야 하고 스킬 에셋을 생성할 때도 해당 버프에서 사용하지 않는 변수 정보까지 저장해야 합니다.



스탯 클래스를 정의하지 않은 코드

알아 두면 좋아요! 디버깅용 메시지를 출력하고 싶어요

게임을 개발할 때 몇 가지 데이터를 확인하고자 Debug.Log() 메서드를 호출해서 Console View에 출력하곤 합니다. 하지만 유니티는 릴리즈 모드(게임 배포)일 때 Debug.Log() 메서드를 자동으로 삭제하지 않으므로 메모리와 연산 비용을 그대로 소모합니다. 그래서 게임을 릴리즈 할 때는 Debug.Log() 메서드를 삭제해야 하는데, 하나씩 직접 삭제하려면 매우 번거롭습니다. 이에 전처리기를 사용해 #if UNITY_EDITOR ~ #endif처럼 유니티 에디터에서만 호출하도록 메서드를 설정하거나 [Conditional(string text)] 어트리뷰트에서 text를 정의하지 않으면 메서드를 호출하지 않도록 설정합니다.

이 책에서는 Debug.Log()를 호출하는 메서드를 따로 정의하고 어트리뷰트를 이용해서 해당 메서드가 릴리즈 모드일 때는 실행되지 않도록 설정합니다.

Do it! 실습 디버깅용 메시지 출력하기

01 Assets 〉 Scripts 〉 Common 폴더에 Console View에 로그를 출력하는 Logger.cs 스크립트를 생성합니다.

Assets/Scripts/Common/Logger.cs

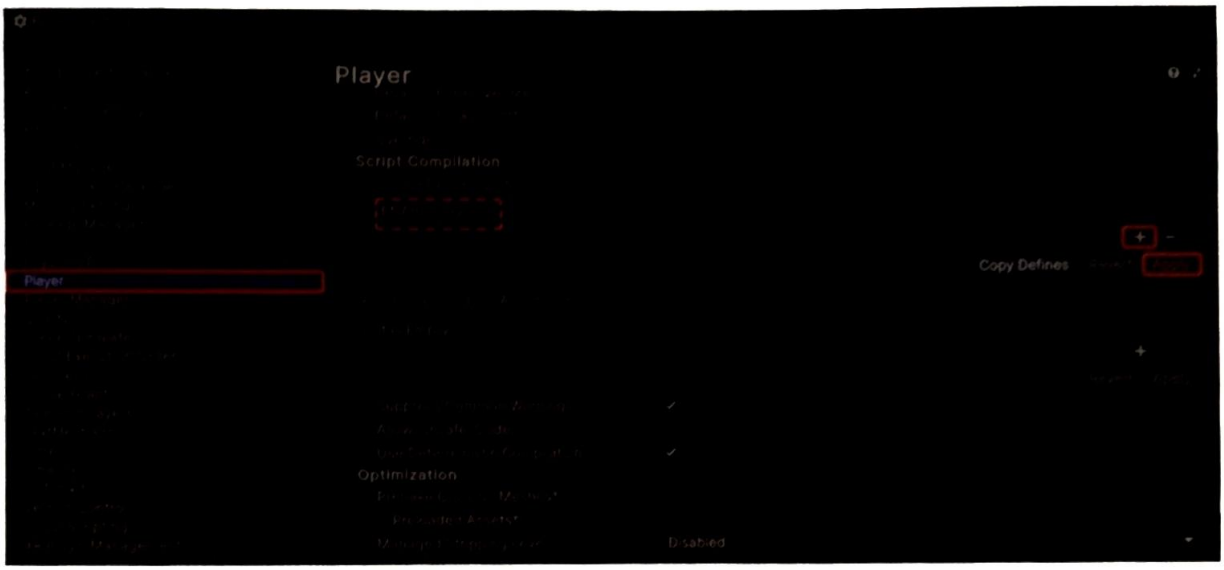
코드 05-17 Console View에 로그 출력

```
01: using UnityEngine;
02:
03: public static class Logger
04: {
05:     [System.Diagnostics.Conditional("ENABLE_LOG")]
06:     public static void Log(string message)
07:     {
08:         Debug.Log(message);
09:     }
10: }
```

05~09줄 유니티에서 제공하는 Debug.Log() 메서드를 호출해서 매개변수 내용을 Console View에 출력하는 Log() 메서드를 정의하고 조건부 어트리뷰트를 추가합니다.

[Conditional(string text)] 어트리뷰트의 text를 정의해야 메서드를 호출합니다. 이름 공간은 System.Diagnostics이고 선언 위치는 메서드 정의 앞입니다. text는 Project Settings View의 [Player] 탭에서 정의합니다.

02 [Edit → Project Settings...] 메뉴로 Project Settings View를 열고 [Player] 탭의 Other Settings에 있는 Scripting Define Symbols 리스트에 'ENABLE_LOG'를 추가하고 〈Apply〉 버튼을 눌러 저장합니다. 게임을 릴리즈(배포)할 때는 ENABLE_LOG 심벌을 삭제해야 `Logger.Log()`를 호출하지 않습니다.



ENABLE_LOG를 추가해 메서드를 호출하도록 설정하기

Do it! 실습 버프 스킬 등록하고 사용하기

01 이 책에서는 스탯을 올리는 Buff, 발사체를 생성해 공격하는 Emission, 근접 보조 공격처럼 지속되는 Sustained, 월드 전체에 전역으로 공격하는 Global 등의 스킬을 만듭니다. Assets 〉 Scripts 〉 Skill 폴더에 이 모든 스킬이 상속받아 사용하는 SkillBase.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/SkillBase.cs

코드 05-18 모든 스킬이 상속받는 기반 클래스

```
01: using UnityEngine;
02:
03: public abstract class SkillBase
04: {
05:     protected SkillTemplate skillTemplate; // 스킬 정보
06:     protected PlayerBase owner;            // 스킬 소유자
07:     protected int currentLevel = 0;        // 현재 스킬 레벨
08:
```

```
09:     // 외부에서 접근할 스킬 정보를 Get 속성으로 정의
10:     public string SkillName => skillTemplate.skillName;
11:     public SkillType SkillType => skillTemplate.skillType;
12:     public SkillElement Element => skillTemplate.element;
13:     public string Description => skillTemplate.description;
14:     public int CurrentLevel => currentLevel;
15:     public bool IsMaxLevel => currentLevel == skillTemplate.maxLevel;
16:
17:     public virtual void Setup(SkillTemplate skillTemplate, PlayerBase owner)
18:     {
19:         this.skillTemplate = skillTemplate;
20:         this.owner = owner;
21:     }
22:
23:     public void TryLevelUp()
24:     {
25:         if ( IsMaxLevel )
26:         {
27:             Logger.Log($"[{SkillName}] 스킬 최고 레벨 도달");
28:             return;
29:         }
30:
31:         currentLevel ++;
32:
33:         OnLevelUp();
34:     }
35:
36:     public abstract void OnLevelUp(); // 스킬 레벨업 시 1회 호출
37:     public abstract void OnSkill();   // 스킬 사용 시 호출
38: }
```

23~34줄 모든 스킬이 레벨업 할 때 호출하는 TryLevelUp() 메서드는 현재 스킬이 최대 레벨인지 검사하고 최대 레벨이 아닐 때만 현재 레벨인 currentLevel을 1 늘리고 OnLevelUp() 메서드를 호출해 레벨업을 처리합니다.

36~37줄 스킬 종류마다 레벨업 할 때와 사용할 때 처리할 내용이 다르므로 OnLevelUp(), OnSkill() 메서드는 추상 메서드로 정의하고 SkillBase 클래스를 상속받는 자식 클래스에서 재정의합니다.

02 같은 폴더에 버프 스킬을 제어하는 SkillBuff.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/SkillBuff.cs

코드 05-19 버프 스킬

```
01: public class SkillBuff : SkillBase
02: {
03:     public override void OnLevelUp()
04:     {
05:         skillTemplate.buffStatList.ForEach(stat =>
06:         {
07:             owner.Stats.GetStat(stat).BonusValue += stat.DefaultValue;
08:         });
09:     }
10:
11:     public override void OnSkill() { }
12: }
```

05~08줄 버프 스킬마다 스킬 에셋을 생성한 것처럼 스킬마다 버프 개수가 다르므로 buffStatList의 모든 요소를 차례대로 검사해 플레이어 stat의 BonusValue에 버프 스킬에 작성한 DefaultValue값을 더합니다.

03 모든 스킬 에셋 정보를 불러와 스킬을 생성하고 제어하고자 SkillSystem.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillSystem.cs

코드 05-20 임의의 스킬 선택 및 레벨업

```
01: using System.Collections.Generic;          ← 추가
02: using System.Linq;
03: using UnityEngine;
04:
05: public class SkillSystem : MonoBehaviour
06: {
        (... 생략 ...)
12:     private PlayerBase owner;
13:
14:     private Dictionary<string, SkillBase> skills =
            new Dictionary<string, SkillBase>();   ← 추가
15:
```

```
16:     private void Awake()
17:     {
18:         owner = GetComponent<PlayerBase>();
19:         skillGad.Setup(owner, skillSpawnPoint);
20:
21:         // Resources/Skills/ 폴더에 있는 모든 스킬 정보를 불러와
                스킬 생성 <파일 이름, SkillTemplate>
22:         var skillDict = Resources.LoadAll<SkillTemplate>("Skills/").ToDictionary(
                item => item.name, item => item);
23:         foreach ( var item in skillDict )
24:         {
25:             SkillBase skill = null;
26:             if ( item.Value.skillType.Equals(SkillType.Buff) )
                    skill = new SkillBuff();
27:
28:             skill.Setup(item.Value, owner);
29:             skills.Add(item.Key, skill);
30:             // 습득한 모든 스킬의 이름, 레벨, 설명 출력 [Debug]
31:             Logger.Log($"[{skill.SkillName}]
                    Lv. {skill.CurrentLevel}\n{skill.Description}");
32:         }
33:     }
34:
35:     private void Update()
36:     {
37:         // 레벨업 가능한 임의 스킬 3개를 선택하고 그중 하나 레벨업 [Debug Test]
38:         if ( UnityEngine.InputSystem.Keyboard.current.digit1Key.wasPressedThisFrame )
                SelectSkill();
39:
40:         // 플레이어의 목표가 없거나 이동 중이면 모든 스킬 사용 불가
41:         if ( owner.Target == null || owner.IsMoved == true ) return;
42:
43:         // 기본 공격 스킬 업데이트
44:         skillGad.OnSkill();
45:     }
46:
47:     public void LevelUp(SkillBase skill)
48:     {
49:         if ( skills.ContainsValue(skill) )
50:         {
51:             skill.TryLevelUp();
52:             Logger.Log($"Level Up [{skill.SkillName}] {skill.Element},
                    Lv. {skill.CurrentLevel}");
53:         }
```

추가

추가

추가

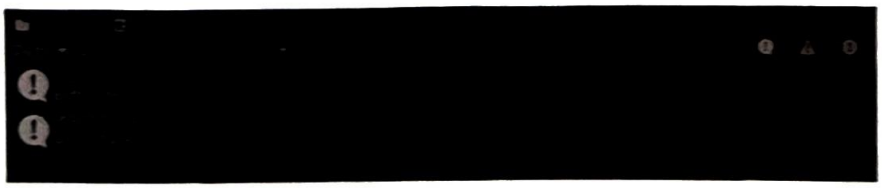
```
54:     }
55:
56:     public void SelectSkill()
57:     {
58:         // 습득 또는 레벨업 할 수 있는 임의의 3개 스킬 선택
59:         var randomSkills = GetRandomSkills(skills, 3);
60:         if ( randomSkills == null )
61:         {
62:             Logger.Log("더 이상 습득할 수 있는 스킬이 없습니다.");
63:             return;
64:         }
65:
66:         // 스킬 선택 UI가 없으므로 임시로 스킬 습득 처리
67:         int index = Random.Range(0, randomSkills.Count);
68:         LevelUp(randomSkills[index]);
69:     }
70:
71:     private List<SkillBase> GetRandomSkills(
            Dictionary<string, SkillBase> skills, int count=3)
72:     {
73:         // 습득할 수 있는 스킬 목록
74:         var values = new List<SkillBase>(
                skills.Values.Where(skill => !skill.IsMaxLevel)).ToList();
75:         var randomSkills = new List<SkillBase>();
76:
77:         count = values.Count == 0 ? 0 : count;
78:
79:         if ( count == 0 ) return null;
80:
81:         for ( int i = 0; i < count; ++ i )
82:         {
83:             int index = Random.Range(0, values.Count);
84:             // index번째 임의의 항목 선택
85:             randomSkills.Add(values[index]);
86:             // 중복을 방지하고자 선택한 항목 제거
87:             values.RemoveAt(index);
88:         }
89:
90:         Logger.Log($"선택 가능한 3개의 스킬\n{randomSkills[0].SkillName}, " +
91:             $"{randomSkills[1].SkillName}, {randomSkills[2].SkillName}");
92:
93:         return randomSkills;
94:     }
95: }
```

21~32줄 Resources.LoadAll<>() 메서드를 호출해 Resources 〉 Skills 폴더에 저장한 SkillTemplate형의 파일 정보를 불러옵니다. 지금은 앞서 만든 5개의 버프 스킬 정보를 불러오며 스킬 개수가 늘어나면 해당 개수만큼 불러옵니다. 또한, RPG 게임처럼 전사, 마법사, 궁수 등 직업별 스킬로 구분한다면 Resources 〉 Skills 〉 Warrior, Resources 〉 Skills 〉 Mage처럼 폴더를 나누어 스킬을 저장하고 직업별 스킬 정보를 불러옵니다. 불러온 스킬 정보는 Dictionary<파일 이름, SkillTemplate>형인 skillDict 변수에 저장하고 foreach() 반복문을 호출해 첫 번째 요소부터 차례대로 탐색합니다. 25줄에서 부모 클래스인 SkillBase형 변수를 선언하고 26줄에서 불러온 스킬 속성이 버프이면 SkillBuff 메모리를 할당합니다.

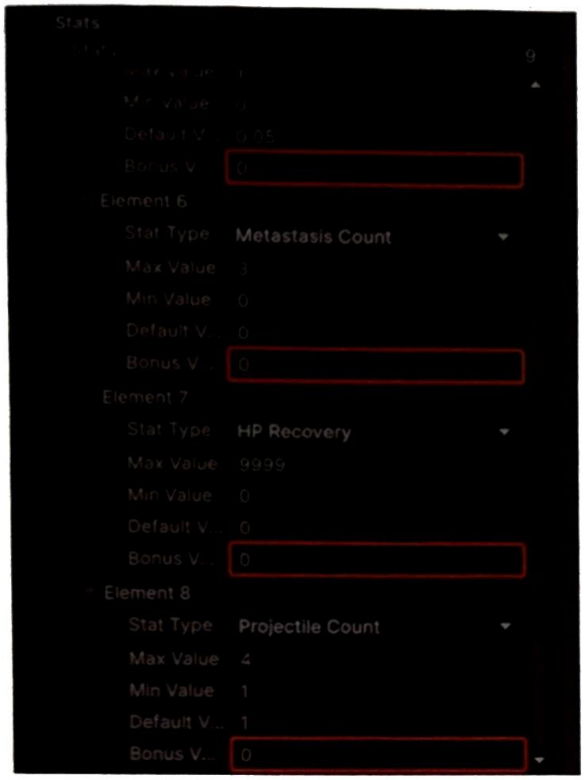
56~69줄 스킬 레벨업 이벤트가 발생했을 때 호출하는 SelectSkill() 메서드는 레벨업 할 수 있는 임의의 스킬 3개를 뽑아 그중에서 플레이어가 선택한 스킬의 레벨을 1 증가시키는 구조인데, 지금은 스킬 선택 UI가 없으므로 67~68줄처럼 스킬 3개 가운데 임의의 스킬 레벨업을 자동으로 수행합니다.

71~94줄 매개변수로 받은 모든 스킬에서 스킬 3개를 선택하는 GetRandomSkills() 메서드는 74줄에서 매개변수로 받은 모든 스킬 중에서 최대 레벨이 아닌 스킬만 values 리스트에 저장하고, 그중 중복되지 않는 3개의 스킬을 선택해 randomSkills 변수에 저장한 후 반환합니다.

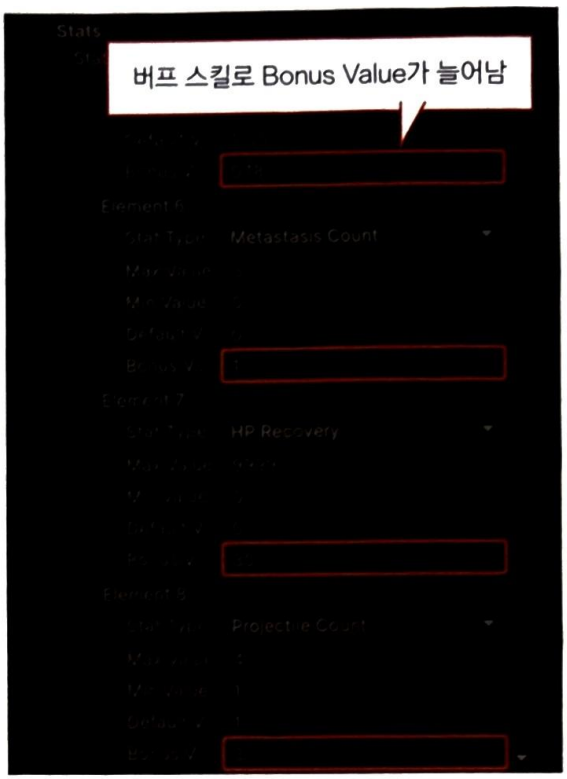
04 플레이 버튼 ▶을 클릭해서 게임을 실행하고 1을 누르면 임의로 선택된 3개의 스킬 정보, 그중에 레벨업이 된 스킬 정보를 Console View에 출력합니다. 이후 Player 오브젝트를 선택하고 Player Base 컴포넌트의 Stats 배열 정보를 확인하면 버프 스킬 때문에 스탯의 Bonus Value가 증가한 것을 알 수 있습니다.



Console View 출력



게임 시작 전의 Stats 배열 정보



게임 시작 후의 Stats 배열 정보

Do it! 실습 공격하는 플레이어 구현하기

01 발사체 개수에 따라 기본 공격 발사체를 연속해서 발사하고자 SkillGad.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillGad.cs

코드 05-21 발사체 개수에 따라 기본 공격 발사체 연속 발사

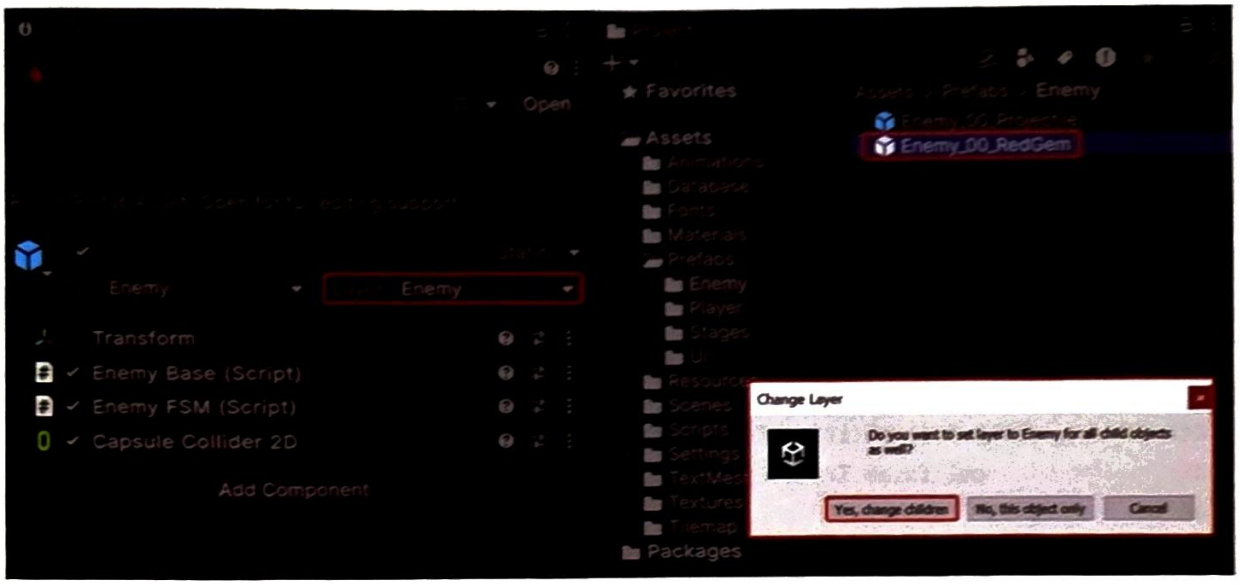
```
01: using System.Collections;                                    추가
02: using UnityEngine;
03:
04: public class SkillGad : MonoBehaviour
05: {
        (... 생략 ...)
11:     private PlayerBase owner;
12:     private WaitForSeconds attackRate = new WaitForSeconds(0.05f);   추가
        (... 생략 ...)
20:     public void OnSkill()
21:     {
22:         // 스킬이 사용할 수 있는 상태인지 검사(쿨타임)
```

```
23:         if ( Time.time - currentCooldownTime >
                owner.Stats.GetStat(StatType.CooldownTime).Value )
24:         {
25:             StartCoroutine(nameof(SpawnProjectile));   ← 수정
26:
27:             currentCooldownTime = Time.time;
28:         }
29:     }
30:
31:     private IEnumerator SpawnProjectile()   ← 추가
32:     {
33:         int projectileCount = 0;
34:         while ( projectileCount <
                (int)owner.Stats.GetStat(StatType.ProjectileCount).Value)
35:         {
36:             var result = CalculateDamage();
37:             ProjectileGad gad = Instantiate(projectile, spawnPoint.position,
                    Quaternion.identity);
38:             gad.Setup(owner.Target, result.Item1, result.Item2);
39:
40:             projectileCount ++;
41:
42:             yield return attackRate;
43:         }
44:     }
        (... 생략 ...)
56: }
```

25줄 발사체 개수(ProjectileCount) 스탯에 따라 기본 공격을 최소 1회에서 최대 4회까지 연속해서 할 수 있습니다. 이를 위해 공격할 수 있는 상태가 되었을 때 SpawnProjectile() 코루틴 메서드를 호출합니다.

31~44줄 SpawnProjectile() 코루틴 메서드 안에서 발사체 개수만큼 기존 발사체 생성 코드를 호출합니다.

02 다음으로, 발사체가 다른 적에게 전이되도록 설정합니다. 전이 대상은 Enemy 레이어를 가진 오브젝트로 설정하고자 아무 오브젝트나 선택한 후 Inspector View에서 Layer 드롭다운 메뉴를 클릭하고 [Add Layer...] 메뉴를 선택합니다. Layers 목록의 User Layer 6에 'Enemy' 레이어를 생성하고 Enemy_00_RedGem 프리팹의 레이어를 [Enemy]로 설정합니다. 이때 Change Layer 창이 활성화되면 〈Yes, change children〉 버튼을 눌러 자식 오브젝트까지 모두 레이어를 변경합니다.



Enemy 레이어 생성하고 설정하기

03 발사체가 다른 적에게 튕기도록 설정하고자 ProjectileGad.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/Projectile/ProjectileGad.cs

코드 05-22 발사체가 주변 적에게 전이

```
01: using UnityEngine;
02:
03: public class ProjectileGad : MonoBehaviour
04: {
05:     [SerializeField]
06:     private Transform hitEffect;
07:     [SerializeField]
08:     private UIDamageText damageText;
09:     [SerializeField]                          추가
10:     private float metaRadius = 4f;
11:
        (... 생략 ...)
```

```
16:     private bool isCritical;
17:     private int metastasisCount;                                    추가
18:     private int targetLayer;
19:
20:     public void Setup(EntityBase owner, EntityBase target,          수정
            float damage, bool isCritical=false)
21:     {
            (... 생략 ...)
26:         this.isCritical = isCritical;
27:         metastasisCount = (int)owner.Stats.GetStat(StatType.MetastasisCount).Value;   추가
28:         targetLayer = 1<<LayerMask.NameToLayer("Enemy");
29:
            (... 생략 ...)
36:     }
37:
38:     private void OnTriggerEnter2D(Collider2D collision)
39:     {
40:         if ( collision.CompareTag("Wall") )
            (... 생략 ...)
44:         else if ( collision.CompareTag("Enemy") &&
45:             collision.TryGetComponent<EntityBase>(out var entity) )
46:         {
                (... 생략 ...)
59:             entity.TakeDamage(damage);
60:
61:             if ( metastasisCount > 0 )                              추가
62:             {
63:                 metastasisCount --;
64:                 FindNextTarget();
65:             }
66:             else Destroy(gameObject);
67:         }
68:     }
69:
70:     private void FindNextTarget()                                   추가
71:     {
72:         Collider2D[] colliders = Physics2D.OverlapCircleAll(
                transform.position, metaRadius, targetLayer);
73:         EntityBase nextTarget = null;
74:
75:         for ( int i = 0; i < colliders.Length; ++ i )
76:         {
77:             if ( colliders[i].CompareTag("Enemy") &&
```

```
78:                 colliders[i].TryGetComponent<EntityBase>(out var entity) &&
                     !entity.Equals(target) )
79:             {
80:                 nextTarget = entity;
81:                 break;
82:             }
83:         }
84:
85:         if ( nextTarget != null )
86:         {
87:             target = nextTarget;
88:             // 발사체를 목표 방향으로 회전
89:             transform.rotation = Utils.RotateToTarget(
                    transform.position, target.MiddlePoint, 90);
90:             // 발사체 이동 방향 설정
91:             movementRigidbody2D.MoveTo(
                    (target.MiddlePoint - transform.position).normalized);
92:         }
93:         else Destroy(gameObject);
94:     }
95: }
```

61~66줄 발사체가 적과 충돌했을 때 발사체 전이 횟수(metastasisCount)가 0보다 크면 전이 횟수를 1 감소한 후 다음 목표를 검색하고 전이 횟수가 0이면 발사체를 삭제합니다.

72줄 다음 전이 목표를 검색하는 FindNextTarget() 메서드에서 반지름이 metaRadius인 원을 생성하고 해당 원에 충돌하는 적 오브젝트를 검출해 colliders 배열에 저장합니다.

75~83줄 반복문에서 colliders 배열의 0번 요소부터 탐색해 태그가 "Enemy"이고 EntityBase 컴포넌트가 있고 방금 공격한 목표(target)와 다르면 해당 entity를 다음 목표인 nextTarget으로 설정합니다.

85~93줄 다음 목표가 있으면 해당 목표로 발사체가 이동하고 다음 목표가 없으면 발사체를 삭제합니다.

04 기본 공격 발사체를 생성할 때 owner 정보를 전달하고자 SkillGad.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillGad.cs

코드 05-23 발사체에 owner 정보 전달

```
    (... 생략 ...)
04: public class SkillGad : MonoBehaviour
05: {
        (... 생략 ...)
31:     private IEnumerator SpawnProjectile()
32:     {
33:         int projectileCount = 0;
34:         while ( projectileCount <
                (int)owner.Stats.GetStat(StatType.ProjectileCount).Value )
35:         {
36:             var result = CalculateDamage();
37:             ProjectileGad gad = Instantiate(projectile, spawnPoint.position,
                    Quaternion.identity);
38:             gad.Setup(owner, owner.Target, result.Item1, result.Item2);   ← 수정
39:
40:             projectileCount ++;
41:
42:             yield return attackRate;
43:         }
44:     }
        (... 생략 ...)
56: }
```

05 플레이어, 적의 회피를 적용하고자 EntityBase.cs 스크립트를 수정합니다.

Assets/Scripts/Entity/EntityBase.cs

코드 05-24 Evasion 스탯 확률로 공격 회피

```
01: using UnityEngine;
02:
03: public class EntityBase : MonoBehaviour
04: {
        (... 생략 ...)
20:     public void TakeDamage(float damage)
21:     {
22:         if ( IsDead ) return;
23:
24:         // Evasion 스탯 확률로 공격 회피   ← 추가
25:         if ( Random.value < Stats.GetStat(StatType.Evasion).Value ) return;
26:
```

```
27:         Stats.CurrentHP.DefaultValue -= damage;
            (... 생략 ...)
33:     }
34: }
```

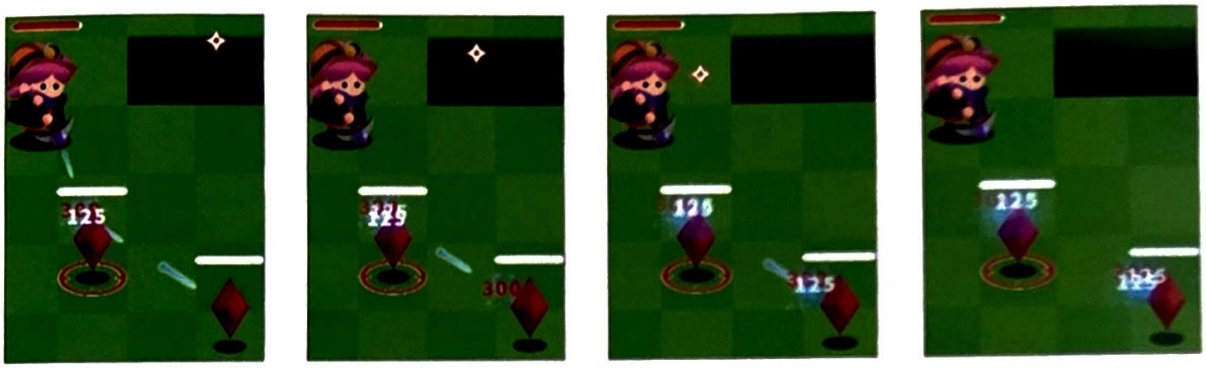
06 플레이어의 체력 회복을 적용하고자 PlayerBase.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerBase.cs

코드 05-25 매 초 HPRecovery 스탯만큼 체력 회복

```
01: using UnityEngine;
02:
03: public class PlayerBase : EntityBase
04: {
        (... 생략 ...)
17:     private void Update()
18:     {
19:         if ( Target == null ) targetMark.gameObject.SetActive(false);
20:
21:         SearchTarget();
22:         Recovery();                                    추가
23:     }
        (... 생략 ...)
47:
48:     private void Recovery()                            추가
59:     {
50:         // 체력 회복
51:         if ( Stats.CurrentHP.DefaultValue < Stats.GetStat(StatType.HP).Value )
52:             Stats.CurrentHP.DefaultValue +=
                    Time.deltaTime * Stats.GetStat(StatType.HPRecovery).Value;
53:         else
54:             Stats.CurrentHP.DefaultValue = Stats.GetStat(StatType.HP).Value;
55:     }
56: }
```

07 플레이 버튼 ▶을 눌러 게임을 실행하면 플레이어와 적의 회피, 플레이어 체력 회복, 기본 공격의 발사체 개수, 발사체 전이 횟수 변화 등 스탯 효과가 적용되는 것을 볼 수 있습니다.



플레이어 스탯에 따라 강화된 기본 공격 스킬

06장

다양한 주 공격 스킬과 보조 공격 스킬 구현하기

이번 장에서는 벽을 통과하고 단일 대상을 공격하는 아이스 스피어와 파이어 볼, 직선 방향으로 이동하며 모든 대상을 공격하는 윈드 블레이드, 직선 방향 광선과 충돌하는 모든 대상을 공격하는 라이트닝, 유도 발사체로 단일 대상을 공격하는 다중 발사체 스킬을 구현합니다. 이에 더해 보조 공격인 해머 스킬도 만들어 봅니다.

학습 목표

1 속성별 발사체와 다중 발사 패턴을 구현할 수 있다.
2 주 공격과 보조 공격을 결합하여 전략적인 전투 시스템을 설계할 수 있다.

06-1 ▶ 다양한 주 공격 발사체 구현하기
06-2 ▶ 보조 공격 스킬 구현하기

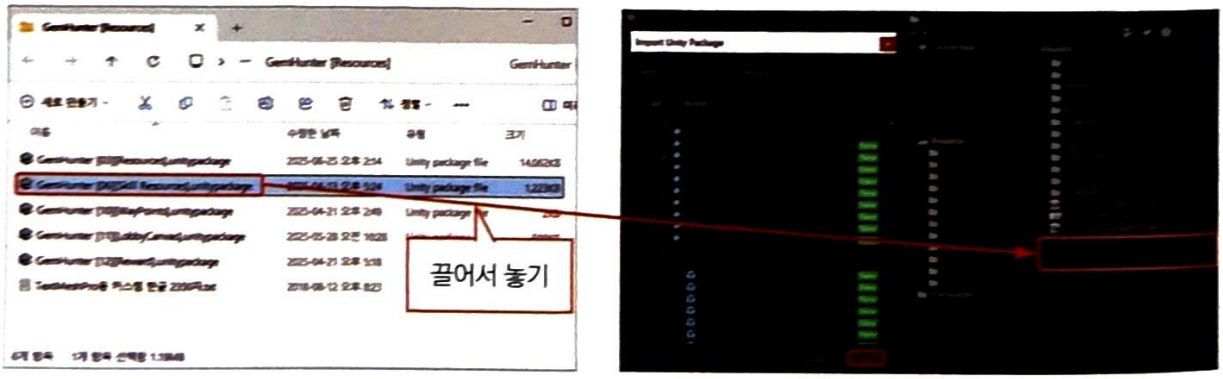
06-1

다양한 주 공격 발사체 구현하기

이 절에서는 5가지 주 공격 스킬을 구현합니다. 벽을 통과하고 단일 대상을 추적해 공격하는 ① 아이스 스피어Ice Spear와 ② 파이어 볼Fire Ball, 벽과 충돌하면 사라지지만 직선 방향으로 이동하며 충돌하는 모든 대상을 공격하는 ③ 윈드 블레이드Wind Blade, 직선 방향으로 광선을 생성해 광선과 충돌하는 모든 대상을 공격하는 ④ 라이트닝Lightning, 벽을 통과하고 10개의 곡선 유도탄으로 단일 대상을 공격하는 ⑤ 다중 발사체Multiple Projectiles 스킬을 만들어 봅시다.

각 속성의 스킬 피격 효과와 스킬 발사체에 사용할 재질인 Material, 프리팹은 미리 만들어 두었으므로 여기서는 이 발사체가 동작하도록 코드만 구현합니다.

먼저 'GemHunter [06][Skill Resources].unitypackage' 파일을 Project View로 끌어서 놓아 사용할 Material과 프리팹을 불러옵니다.



스킬 발사체, 피격 효과 리소스 불러오기

폰트 경고가 나타나거나 한글이 깨진다면 Font Asset을 NotoSansKR-Bold SDF로 설정하세요.

Do it! 실습 아이스 스피어 공격 구현하기

01 Assets 〉 Scripts 〉 Skill 〉 Projectile 폴더에 발사체 충돌을 처리하는 Projectile Collision2D.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/Projectile/ProjectileCollision2D.cs

코드 06-01 공격 스킬 발사체 충돌 처리

```
01: using UnityEngine;
02:
03: public enum DestroyType { None = -1, Collision = 0, }
04:
05: public class ProjectileCollision2D : MonoBehaviour
06: {
07:     [SerializeField]
08:     private Transform hitEffect;
09:     [SerializeField]
10:     private UIDamageText damageText;
11:     [SerializeField]
12:     private DestroyType destroyType = DestroyType.None;
13:     [SerializeField]
14:     private bool isIgnoreWall = false;
15:
16:     private EntityBase target;
17:     private float damage;
18:
19:     public void Setup(EntityBase target, float damage)
20:     {
21:         this.target = target;
22:         this.damage = damage;
23:     }
24:     public void SetTarget(EntityBase target) => this.target = target;
25:
26:     private void OnTriggerEnter2D(Collider2D collision)
27:     {
28:         if ( collision.CompareTag("Wall") && isIgnoreWall == false )
29:         {
30:             Destroy(gameObject);
31:         }
32:         else if ( collision.CompareTag("Enemy") &&
33:                 collision.TryGetComponent<EntityBase>(out var entity) )
34:         {
35:             if ( entity != target ) return;
36:
37:             if ( hitEffect != null )
38:             {
39:                 Instantiate(hitEffect, collision.transform.position,
                        Quaternion.identity);
40:             }
41:             if ( damageText != null )
```

```
42:             {
43:                 UIDamageText clone = Instantiate(damageText,
                                    collision.transform.position, Quaternion.identity);
44:                 clone.Setup(damage.ToString("F0"), Color.white);
45:             }
46:
47:             entity.TakeDamage(damage);
48:
49:             if ( destroyType == DestroyType.Collision )
50:             {
51:                 Destroy(gameObject);
52:             }
53:         }
54:     }
55: }
```

28~31줄 발사체의 종류에 따라 벽을 통과할 수도 있고 벽에 충돌해 사라질 수도 있으므로 발사체와 충돌한 오브젝트의 태그가 "Wall"이고 isIgnoreWall이 false라면 발사체를 삭제합니다. 벽을 통과할 수 있는 발사체라면 isIgnoreWall을 true로 설정합니다.

49~52줄 발사체가 특정 시간 동안 재생되거나 삭제되지 않고 유지될 수도 있으므로 destroyType이 Collision일 때만 발사체를 삭제합니다.

02 이 책에서 만드는 발사체로는 직선으로 이동하는 ProjectileStraight, 적을 추적하는 ProjectileHoming, ProjectileCubicHoming, ProjectileQuadraticHoming, 직선 범위 전체를 공격하는 ProjectileLine, 범위 공격을 제어하는 ProjectileGlobal 등이 있습니다.
같은 폴더에 이 모든 발사체가 상속받는 ProjectileBase.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/Projectile/ProjectileBase.cs

코드 06-02 모든 발사체가 상속받는 기반 클래스

```
01: using UnityEngine;
02:
03: public abstract class ProjectileBase : MonoBehaviour
04: {
05:     protected MovementRigidbody2D movementRigidbody2D;
06:
07:     public virtual void Setup(EntityBase target, float damage)
08:     {
```

```
09:         movementRigidbody2D = GetComponent<MovementRigidbody2D>();
10:
11:         // 발사체 크기를 20%에서 100%로 확대
12:         GetComponent<ScaleEffect>().Play(transform.localScale * 0.2f,
                transform.localScale);
13:         // 적 오브젝트와 충돌 처리
14:         GetComponent<ProjectileCollision2D>().Setup(target, damage);
15:     }
16:
17:     private void Update()
18:     {
19:         Process();
20:     }
21:
22:     public abstract void Process();
23: }
```

17~20줄 Process() 메서드는 프레임마다 호출되도록 Update() 메서드에서 호출합니다.

22줄 각 발사체의 행동을 제어하는 Process() 메서드는 ProjectileBase를 상속받는 자식 클래스에서 정의할 수 있도록 abstract 키워드를 이용하여 추상 메서드로 정의합니다.

03 이번에는 유도 공격 발사체를 제어하는 ProjectileHoming.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/Projectile/ProjectileHoming.cs

코드 06-03 유도 공격 발사체 제어

```
01: public class ProjectileHoming : ProjectileBase
02: {
03:     private EntityBase target;
04:
05:     public override void Setup(EntityBase target, float damage)
06:     {
07:         base.Setup(target, damage);
08:
09:         this.target = target;
10:     }
11:
12:     public override void Process()
13:     {
14:         if ( target == null )
```

```
15:         {
16:             FindTarget();
17:             return;
18:         }
19:
20:         // 발사체를 목표 방향으로 회전
21:         transform.rotation = Utils.RotateToTarget(transform.position,
                target.MiddlePoint, 90);
22:         // 발사체 이동 방향 설정
23:         movementRigidbody2D.MoveTo((target.MiddlePoint -
                transform.position).normalized);
24:     }
25:
26:     private void FindTarget()
27:     {
28:         if ( EnemySpawner.Enemies.Count > 0 )
29:         {
30:             target = EnemySpawner.Enemies[0];
31:             GetComponent<ProjectileCollision2D>().SetTarget(target);
32:         }
33:         else Destroy(gameObject);
34:     }
35: }
```

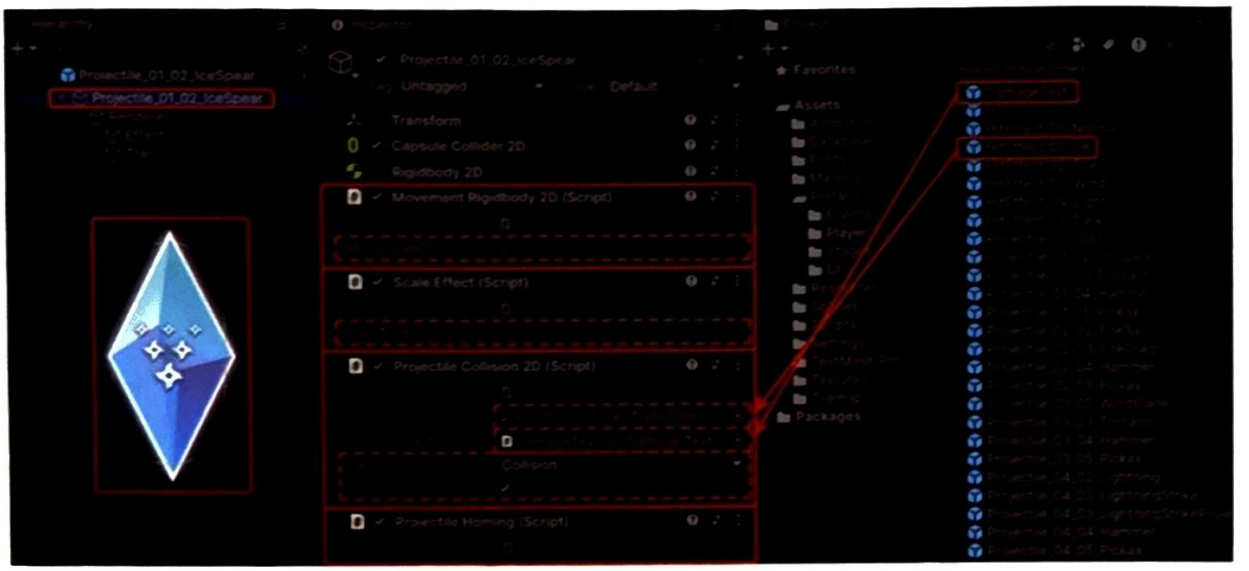
12~24줄 이동하는 목표를 실시간으로 추적하도록 Process() 메서드 내부에서 목표 방향으로 발사체를 회전하고 이동합니다. 다른 공격을 받아 목표가 사망할 수도 있으므로 목표가 사라지면 FindTarget() 메서드를 호출해 새로운 목표를 설정합니다.

26~34줄 새로운 목표는 기본 공격처럼 가까이 있는 적을 설정할 수도 있지만 현재 코드에서는 Enemies 리스트의 [0]번 적으로 설정합니다.

04 Projectile_01_02_IceSpear 프리팹에 Movement Rigidbody 2D, Scale Effect, Projectile Collision 2D, Projectile Homing 컴포넌트를 추가합니다.

Movement Rigidbody 2D 컴포넌트의 Move Speed는 '20'으로, Scale Effect 컴포넌트의 Play Time은 '0.1'로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_01_Ice 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Collision]으로, Is Ignore Wall에 체크하여 true로 설정합니다.

Projectile_01_02_IceSpear 등 발사체 프리팹은 앞서 가져온 패키지 파일 안에 있습니다.



Projectile_01_02_IceSpear 프리팹에 컴포넌트 추가하고 설정하기

05 공격에 필요한 스탯 변수와 발사체 프리팹 변수 정보를 저장하고자 SkillTemplate.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillTemplate.cs

코드 06-04 공격에 필요한 스탯, 발사체 프리팹 변수 선언

```
    (... 생략 ...)
04: public enum SkillType { Buff = 0, Emission, }
05: public enum SkillElement { None = -1, Ice = 100, Fire, Wind, Light, Dark, Count = 5 }
06:
07: [CreateAssetMenu(fileName = "NewSkill", menuName = "SkillAsset", order = 0)]
08: public class SkillTemplate : ScriptableObject
09: {
        (... 생략 ...)
20:     [Header("버프 스킬")]
21:     public List<Stat> buffStatList;     // 버프 스킬
22:
23:     [Header("공격 스킬")]
24:     public List<Stat> attackBaseStats; // 공격 스킬 스탯 정보
25:     public List<Stat> attackBuffStats; // 레벨업 시 추가 스탯
26:     public GameObject projectile;      // 발사체 프리팹
27: }
```

수정

추가

04줄 아이스 스피어처럼 발사체를 생성하는 주 공격 스킬을 구현하고자 **SkillType** 열거형에 **Emission** 상수를 추가합니다.

24줄 **attackBaseStats** 리스트 변수는 해당 공격 스킬을 사용할 때 적용하는 공격력, 쿨타임 등의 스킬 기본 스탯을 나타냅니다.

25줄 **attackBuffStats** 리스트 변수는 공격 스킬이 레벨업 할 때 증가하는 공격력, 쿨타임 등의 스탯입니다.

26줄 스킬을 사용할 때 발사체를 생성한다면 **projectile** 변수에 등록합니다.

06 Assets 〉 Resources 〉 Skills 폴더에서 얼음 속성 주 공격 스킬 1의 SO 데이터를 저장하고자 Project View의 ＋▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '01_02_Ice_IceSpear'로 변경합니다. 아이스 스피어 스킬의 Skill Name은 'Ice Spear'로, Skill Type은 [Emission]으로, Element는 [Ice]로, Max Level은 '99'로 설정한 후 Description에 스킬 설명을 입력하고 아이콘 이미지를 선택합니다. 그리고 Attack Base Stats 크기를 3, Attack Buff Stats 크기를 2로 설정하고 다음 표처럼 값을 입력합니다.

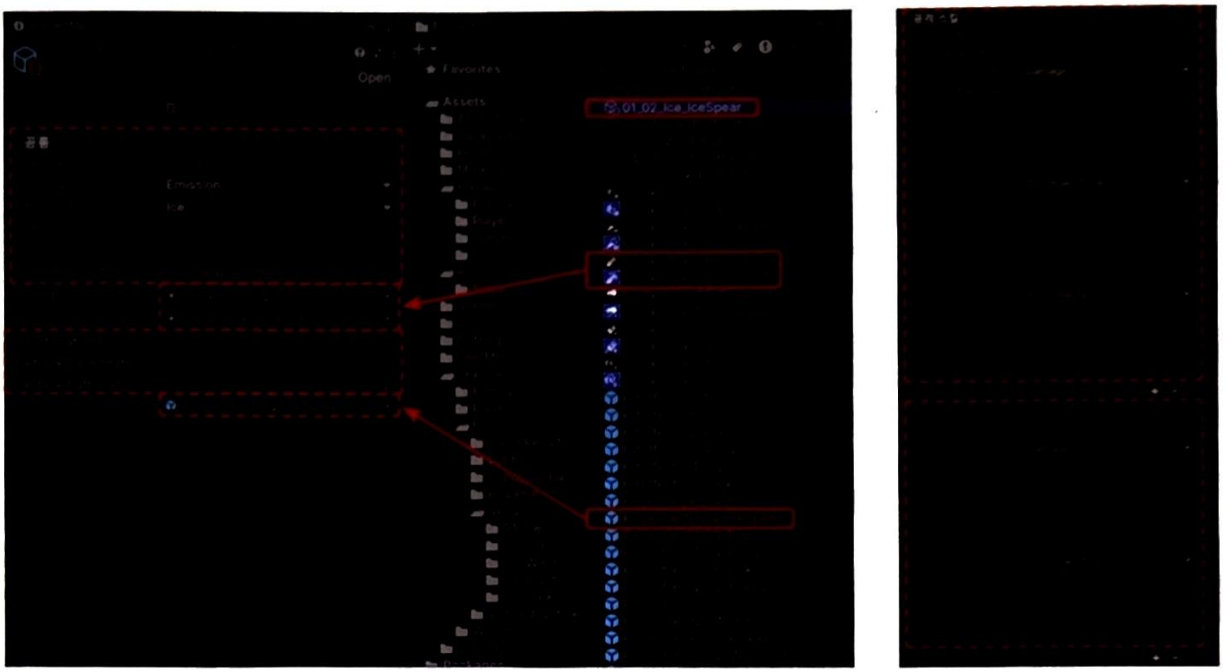
Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	200	200	0
1	Cooldown Time	5	2.5	5	0
2	Projectile Count	1	1	1	0

Attack Buff Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	100	0
1	Cooldown Time	0	0	-0.2	0

마지막으로, 아이스 스피어 발사체로 사용할 Projectile_01_02_IceSpear 프리팹을 Projectile 변수에 등록합니다.



아이스 스피어 스킬 에셋 생성하고 설정하기

07 스탯 정보를 복사할 수 있도록 CopyData() 메서드를 정의하고자 Stat.cs 스크립트를 수정합니다.

Assets/Scripts/Entity/Stat.cs

코드 06-05 스탯 정보를 복사하는 CopyData()

```
01: using UnityEngine;
02:
03: [System.Serializable]
04: public class Stat
05: {
        (... 생략 ...)
19:     [SerializeField]
20:     private float bonusValue; // 아이템, 스킬 등으로 증가하는 추갓값
21:
22:     public void CopyData(Stat newStat)
23:     {
24:         statType = newStat.StatType;
25:         maxValue = newStat.MaxValue;
26:         minValue = newStat.MinValue;
27:         defaultValue = newStat.DefaultValue;
28:         bonusValue = newStat.BonusValue;
29:     }
30:
```

추가

```
31:     public StatType StatType => statType;
32:     public float MaxValue => maxValue;
33:     public float MinValue => minValue;
34:     public float Value => Mathf.Clamp(defaultValue + bonusValue,
            minValue, maxValue);
        (... 생략 ...)
```

추가

08 공격 스탯 추가와 공격 쿨타임을 연산하는 **IsSkillAvailable()** 메서드를 정의하고자 SkillBase.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillBase.cs

코드 06-06 공격 스탯 추가, 공격 쿨타임 연산

```
01: using System.Linq;
02: using UnityEngine;
03:
04: public abstract class SkillBase
05: {
06:     protected SkillTemplate skillTemplate; // 스킬 정보
07:     protected PlayerBase owner;            // 스킬 소유자
08:     protected Transform spawnPoint;        // 스킬 발사 위치
09:     protected int currentLevel = 0;        // 현재 스킬 레벨
10:     protected float currentCooldownTime = 0;
11:     protected bool isSkillAvailable = false;
        (... 생략 ...)
21:     // 공격 스킬 전용(공격력, 쿨타임, 발사체 개수와 같은 스탯)
22:     private Stat[] stats;
23:     public Stat GetStat(Stat stat)
            => stats.FirstOrDefault(s => s.StatType == stat.StatType);
24:     public Stat GetStat(StatType statType)
            => stats.FirstOrDefault(s => s.StatType == statType);
25:
26:     public virtual void Setup(SkillTemplate skillTemplate, PlayerBase owner,
            Transform spawnPoint=null)
27:     {
28:         this.skillTemplate = skillTemplate;
29:         this.owner = owner;
30:         this.spawnPoint = spawnPoint;
31:
32:         // 공격 스킬이라면 필요한 스탯 설정(공격력, 쿨타임, 발사체 개수 등)
33:         if ( SkillType != SkillType.Buff )
```

추가 (08, 08, 10, 11)

추가 (21)

수정 (26)

추가 (30)

```
34:         {
35:             stats = new Stat[skillTemplate.attackBaseStats.Count];
36:             for ( int i = 0; i < stats.Length; ++ i )
37:             {
38:                 stats[i] = new Stat();
39:                 stats[i].CopyData(skillTemplate.attackBaseStats[i]);
40:             }
41:         }
42:     }
      (... 생략 ...)
57:     public void IsSkillAvailable()
58:     {
59:         // 레벨이 0이거나 버프 또는 지속 스킬이라면 사용할 수 있는 상태 아님
60:         if ( CurrentLevel == 0 || SkillType == SkillType.Buff ) return;
61:
62:         if ( Time.time - currentCooldownTime >
                GetStat(StatType.CooldownTime).Value )
63:         {
64:             isSkillAvailable = true;
65:         }
66:     }
67:
68:     public abstract void OnLevelUp(); // 스킬 레벨업 시 1회 호출
69:     public abstract void OnSkill();   // 스킬 사용 시 호출
70: }
```

추가

21~24줄 공격 스킬마다 별도의 공격력, 쿨타임 스탯 정보가 있습니다. EntityStats 클래스에 정의한 플레이어의 스탯 배열, 스탯 접근을 위한 GetStat() 메서드처럼 stats 배열, GetStat() 메서드를 정의합니다.

26~42줄 Setup() 메서드에서 공격 스킬이라면 공격에 필요한 스탯 정보를 SkillTemplate 에셋의 attackBaseStats 리스트로부터 받아 현재 스킬(SkillBase의 stats)에 저장합니다.

57~66줄 스킬의 쿨타임을 계산해서 스킬을 사용할 수 있는 상태인지 검사하고 사용할 수 있다면 isSkillAvailable을 true로 설정합니다.

09 Assets 〉 Scripts 〉 Skill 폴더에 발사체 사출형 스킬을 제어하는 SkillEmission.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/SkillEmission.cs

코드 06-07 발사체 사출형 스킬 제어

```
01: using UnityEngine;
02:
03: public class SkillEmission : SkillBase
04: {
05:     public override void OnLevelUp()
06:     {
07:         // 레벨이 0에서 1로 늘 때는 스탯이 증가하지 않도록 설정
08:         if ( currentLevel <= 1 ) return;
09:
10:         // 공격 스킬 레벨업 시 공격력 등 스탯 갱신
11:         skillTemplate.attackBuffStats.ForEach(stat =>
12:         {
13:             GetStat(stat).BonusValue += stat.DefaultValue;
14:         });
15:     }
16:
17:     public override void OnSkill()
18:     {
19:         // 스킬이 사용할 수 있는 상태인지 검사(쿨타임)
20:         if ( isSkillAvailable == true )
21:         {
22:             GameObject projectile = GameObject.Instantiate(skillTemplate.projectile,
                    spawnPoint.position, Quaternion.identity);
23:             projectile.GetComponent<ProjectileBase>().Setup(owner.Target,
                    GetStat(StatType.Damage).Value);
24:
25:             isSkillAvailable = false;
26:             currentCooldownTime = Time.time;
27:         }
28:     }
29: }
```

05~15줄 해당 스킬이 레벨업 될 때 호출하는 OnLevelUp() 메서드는 SkillTemplate 에셋의 attackBuffStats 리스트로부터 받은 레벨업 스탯 정보를 현재 스킬 스탯의 보너스값에 추가합니다.

17~28줄 스킬을 사용할 때 호출하는 OnSkill() 메서드는 스킬을 사용할 수 있는 상태인지 검사(isSkillAvailable == true)하고 그렇다면 발사체 오브젝트를 생성합니다.

10 발사체 사출형 스킬의 쿨타임을 검사하고 스킬 사용을 제어하고자 SkillSystem.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillSystem.cs

코드 06-08 발사체 사출형 스킬의 쿨타임 검사 및 스킬 사용 제어

```
    (... 생략 ...)
05: public class SkillSystem : MonoBehaviour
06: {
        (... 생략 ...)
16:     private void Awake()
17:     {
            (... 생략 ...)
23:         foreach ( var item in skillDict )
24:         {
25:             SkillBase skill = null;
26:             if ( item.Value.skillType.Equals(SkillType.Buff) )
                    skill = new SkillBuff();
27:             else if ( item.Value.skillType.Equals(SkillType.Emission) )
                    skill = new SkillEmission();                    ← 추가
28:
29:             skill.Setup(item.Value, owner, skillSpawnPoint);    ← 수정
30:             skills.Add(item.Key, skill);
31:             // 습득한 모든 스킬 이름, 레벨, 설명 출력 [Debug]
32:             Logger.Log($"[{skill.SkillName}]
                    Lv. {skill.CurrentLevel}\n{skill.Description}");
33:         }
34:     }
35:
36:     private void Update()
37:     {
38:         // 레벨업 가능한 임의의 스킬 3개를 선택하고 그중 하나를 레벨업 [Debug Test]
39:         if ( UnityEngine.InputSystem.Keyboard.current.digit1Key.wasPressedThisFrame )
                SelectSkill();
40:
41:         // 모든 공격 스킬 업데이트                              ← 추가
42:         foreach ( var item in skills )
43:         {
44:             if ( item.Value.CurrentLevel == 0 ) continue;
45:
46:             item.Value.OnSkill();
47:         }
            (... 생략 ...)
55:         // 모든 공격 스킬 쿨타임 업데이트                       ← 추가
```

```
56:         foreach ( var item in skills )
57:         {
58:             item.Value.IsSkillAvailable();
59:         }
60:     }
        (... 생략 ...)
110: }
```

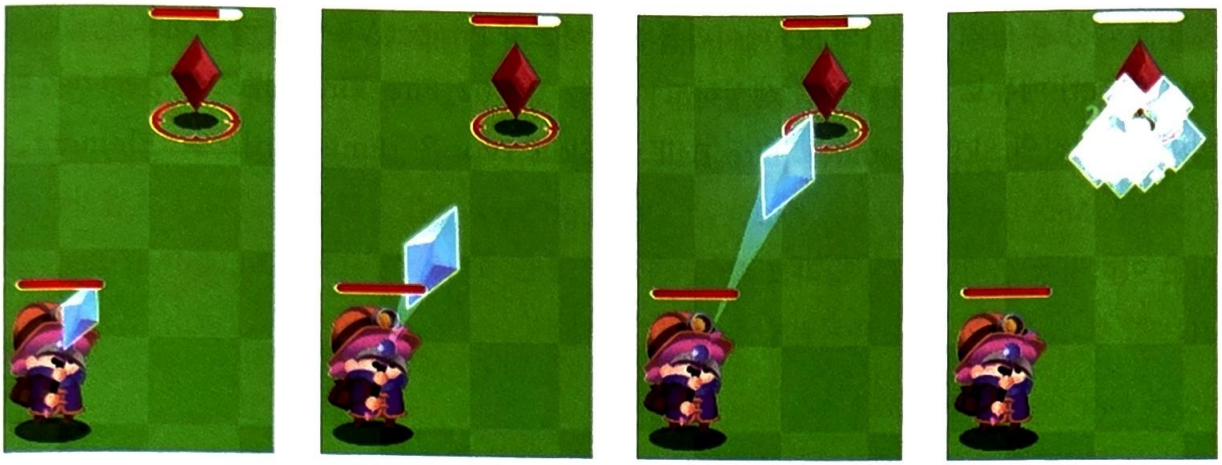
27줄 발사체를 사출하는 SkillEmission 클래스를 추가했으므로 25줄에서 SkillBase형 변수를 선언한 후 해당 스킬이 버프 스킬이면 SkillBuff 메모리를 할당하고 발사체를 이용한 공격 스킬이면 SkillEmission 메모리를 할당합니다.

41~47줄 Update() 메서드에서 레벨이 0이 아닌 모든 스킬의 OnSkill() 메서드를 호출합니다. 버프 스킬은 OnSkill() 메서드 내부가 비었으므로 공격 스킬만 사용할 수 있는 상태(isSkillAvailable == true)이면 스킬을 사용합니다.

55~59줄 플레이어가 이동 중일 때는 스킬을 사용할 수 없으므로 이동 중이 아닐 때만 IsSkillAvailable() 메서드를 호출해 쿨타임 초기화 여부를 검사하고 쿨타임이 된 스킬의 isSkillAvailable 변수를 true로 설정합니다.

이처럼 OnSkill()과 IsSkillAvailable() 메서드로 나눈 데는 이유가 있습니다. 스킬 중에 발사체를 여러 개 생성하는 스킬이 있는데, OnSkill() 메서드를 플레이어가 이동 중일 때만 호출하도록 하면 스킬을 발사하는 도중에 플레이어가 움직이면 해당 스킬이 멈추는 현상이 발생할 수 있기 때문입니다. 그러므로 스킬 사용 메서드 OnSkill()은 플레이어의 이동 여부와 관계없이 호출하고 스킬을 사용할 수 있는 상태인지 검사하는 IsSkillAvailable() 메서드를 플레이어의 이동 여부에 따라 호출합니다.

11 플레이 버튼 ▶ 을 눌러 게임을 실행하고 아이스 스피어 스킬을 배우면 쿨타임이 될 때마다 목표를 향해 발사체를 발사하고 충돌했을 때 피격 효과를 재생합니다.

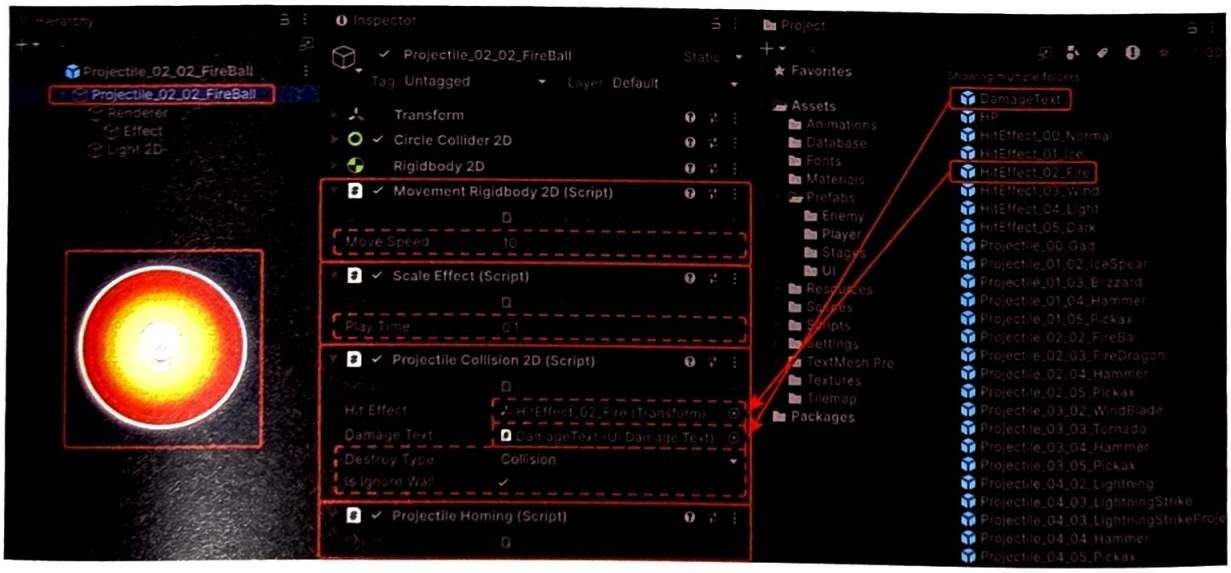


플레이어가 아이스 스피어 스킬로 공격한 결과 화면

Do it! 실습 파이어 볼 공격 구현하기

이어서 같은 방법으로 파이어 볼 공격을 구현합니다.

01 Projectile_02_02_FireBall 프리팹에 Movement Rigidbody 2D, Scale Effect, Projectile Collision 2D, Projectile Homing 컴포넌트를 추가합니다.
Movement Rigidbody 2D 컴포넌트의 Move Speed는 '10'으로, Scale Effect 컴포넌트의 Play Time은 '0.1'로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_02_Fire 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Collision]으로, Is Ignore Wall은 true로 설정합니다.



Projectile_02_02_FireBall 프리팹에 컴포넌트 추가하고 설정하기

02 불 속성 주 공격 스킬 1의 SO 데이터를 저장하고자 Project View의 ➕▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '02_02_Fire_FireBall'로 변경합니다. 파이어 볼 스킬의 Skill Name은 'Fire Ball'로, Skill Type은 [Emission]으로, Element는 [Fire]로, Max Level은 '99'로 설정한 후, Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택합니다. 그리고 Attack Base Stats 크기는 '3'으로, Attack Buff Stats 크기는 '2'로 설정하고 다음 표처럼 값을 입력합니다.

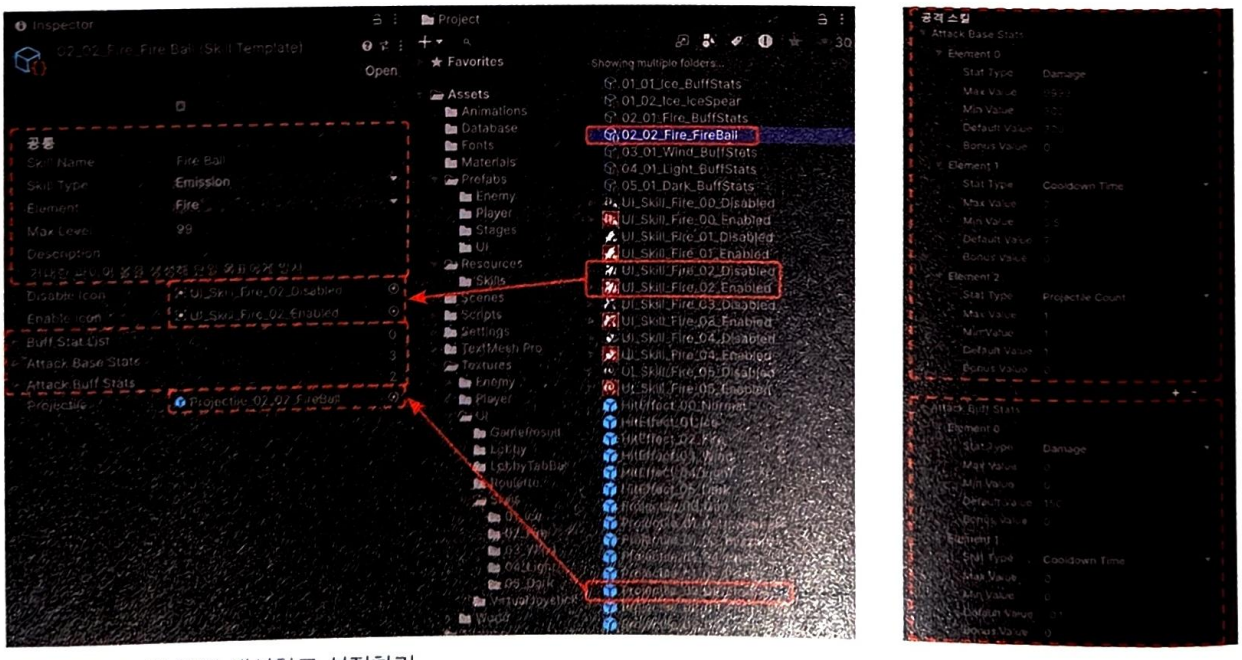
Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	300	300	0
1	Cooldown Time	7	3.5	7	0
2	Projectile Count	1	1	1	0

Attack Buff Stats 리스트 설정값

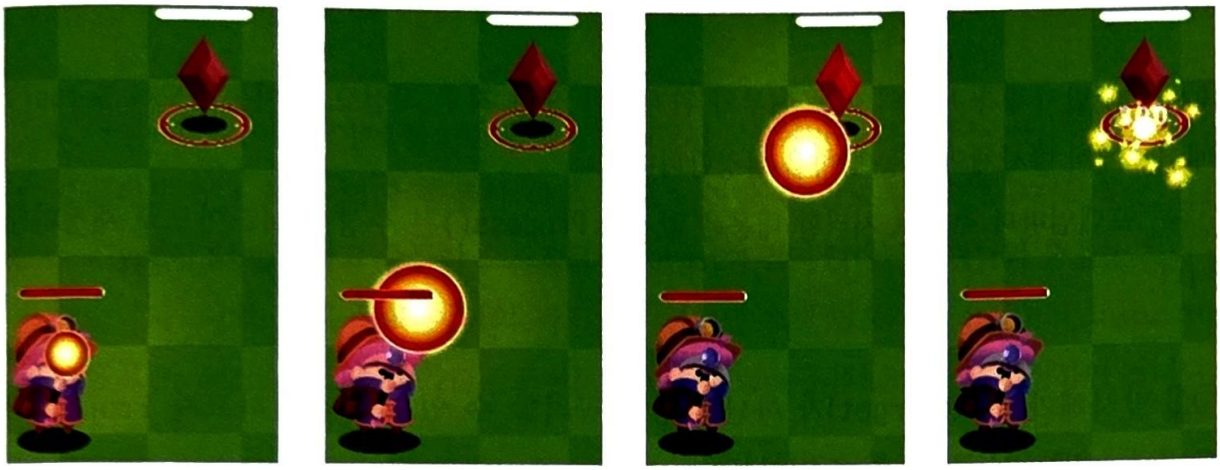
Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	150	0
1	Cooldown Time	0	0	-0.1	0

마지막으로, 파이어 볼 발사체로 사용할 Projectile_02_02_FireBall 프리팹을 Projectile 변수에 등록합니다.



파이어 볼 스킬 에셋 생성하고 설정하기

03 플레이 버튼 ▶을 눌러 게임을 실행하고 파이어 볼 스킬을 배우면 쿨타임이 될 때마다 목표를 향해 발사체를 발사하고 충돌했을 때 피격 효과를 재생합니다.



플레이어가 파이어 볼 스킬로 공격한 결과 화면

Do it! 실습 윈드 블레이드 공격 구현하기

이번에는 모든 대상을 공격하는 윈드 블레이드 공격을 만듭니다.

01 Assets 〉 Scripts 〉 Skill 〉 Projectile 폴더에 직선 공격 발사체를 제어하는 Projectile Straight.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/Projectile/ProjectileStraight.cs

코드 06-09 직선 공격 발사체 제어

```
01: public class ProjectileStraight : ProjectileBase
02: {
03:     public override void Setup(EntityBase target, float damage)
04:     {
05:         base.Setup(target, damage);
06:
07:         // 발사체를 목표 방향으로 회전
08:         transform.rotation = Utils.RotateToTarget(transform.position,
                target.MiddlePoint, 90);
09:         // 발사체 이동 방향 설정
10:         movementRigidbody2D.MoveTo((target.MiddlePoint -
                transform.position).normalized);
11:     }
12:
```

```
13:     public override void Process() { }
14: }
```

07~10줄 발사체가 목표의 이동 여부와 관계없이 생성 직후 직선으로 이동하도록 Setup() 메서드에서 발사체의 이동 방향을 설정합니다.
13줄 프레임마다 추가로 처리할 내용은 없지만 Process()는 추상 메서드이므로 재정의합니다.

02 발사체가 목표(target)가 아닌 대상도 공격할 수 있도록 설정하고자 Projectile Collision2D.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/Projectile/ProjectileCollision2D.cs

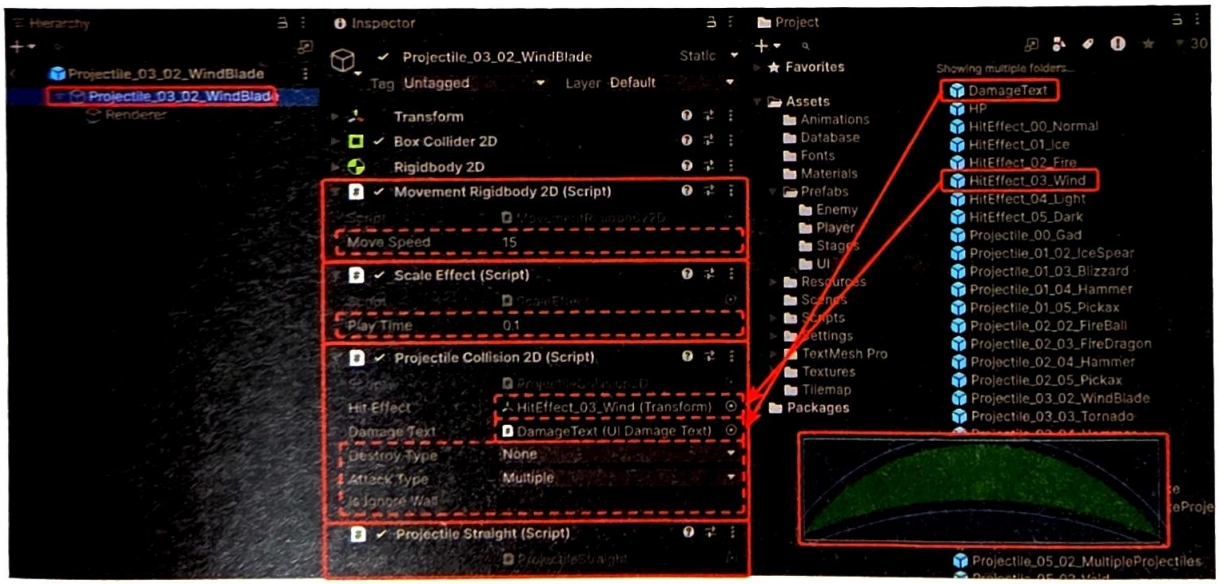
코드 06-10 단일, 다중 공격 설정

```
01: using UnityEngine;
02:
03: public enum DestroyType { None = -1, Collision = 0, }
04: public enum AttackType { Single, Multiple }                    ← 추가
05:
06: public class ProjectileCollision2D : MonoBehaviour
07: {
        (... 생략 ...)
12:     [SerializeField]
13:     private DestroyType destroyType = DestroyType.None;
14:     [SerializeField]                                           ← 추가
15:     private AttackType attackType = AttackType.Single;
16:     [SerializeField]
17:     private bool isIgnoreWall = false;
        (... 생략 ...)
29:     private void OnTriggerEnter2D(Collider2D collision)
30:     {
            (... 생략 ...)
35:         else if ( collision.CompareTag("Enemy") &&
36:             collision.TryGetComponent<EntityBase>(out var entity) )
37:         {
38:             if ( attackType == AttackType.Single && entity != target ) return;   ← 수정
                (... 생략 ...)
56:         }
57:     }
```

```
58: }
```

38줄 아이스 스피어, 파이어 볼처럼 발사체가 지정된 목표에만 타격을 주는 스킬도 있지만 윈드 블레이드처럼 목표 외에 충돌한 다른 적도 타격할 수 있도록 설정할 수도 있습니다. attackType이 Single이고 충돌한 오브젝트와 target이 같지 않다면 빠져나와 충돌을 처리하지 않습니다. 이와 달리 목표 이외의 적도 타격하려면 스킬 발사체를 선택하고 Inspector View에서 Projectile Collision 2D 컴포넌트의 Attack Type을 [Multiple]로 설정하면 됩니다.

03 Projectile_03_02_WindBlade 프리팹에 Movement Rigidbody 2D, Scale Effect, Projectile Collision 2D, Projectile Straight 컴포넌트를 추가합니다.
Movement Rigidbody 2D 컴포넌트의 Move Speed는 '15'로, Scale Effect 컴포넌트의 Play Time은 '0.1'로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_03_Wind 프리팹을, Damage Text 변수에는 Damage Text 프리팹을 등록한 후 Destroy Type은 [None]으로, Attack Type은 [Multiple]로 설정하고 Is Ignore Wall을 false로 설정합니다.



Projectile_03_02_WindBlade 프리팹에 컴포넌트 추가하고 설정하기

04 바람 속성 주 공격 스킬 1의 SO 데이터를 저장하고자 Project View의 +▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '03_02_Wind_WindBlade'로 변경합니다. 윈드 블레이드 스킬의 Skill Name은 'Wind Blade'로, Skill Type은 [Emission]으로, Element는 [Wind]로, Max Level은 '99'로 설정한 후, Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택합니다. 그리고 Attack Base Stats 크기는 '3'으로, Attack Buff Stats 크기는 '2'로 설정하고 다음 표처럼 값을 입력합니다.

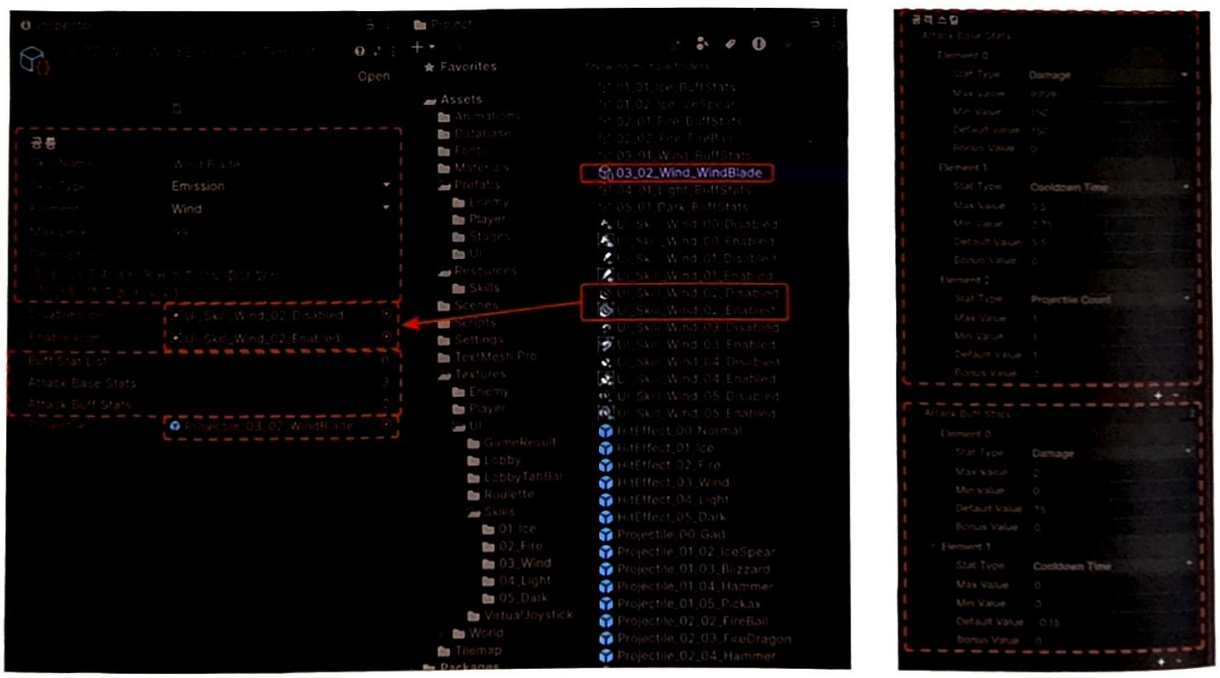
Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	150	150	0
1	Cooldown Time	5.5	2.75	5.5	0
2	Projectile Count	1	1	1	0

Attack Buff Stats 리스트 설정값

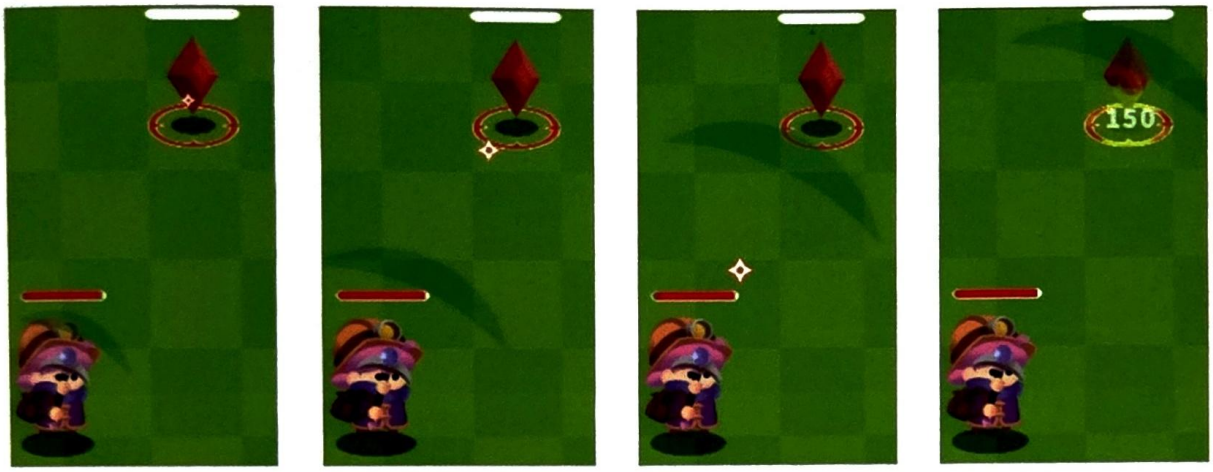
Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	75	0
1	Cooldown Time	0	0	-0.15	0

마지막으로, 윈드 블레이드 발사체로 사용할 Projectile_03_02_WindBlade 프리팹을 Projectile 변수에 등록합니다.



윈드 블레이드 스킬 에셋 생성하고 설정하기

05 플레이 버튼 ▶을 눌러 게임을 실행하고 윈드 블레이드 스킬을 배우면 쿨타임이 될 때마다 목표 방향으로 발사체를 발사하고 충돌하면 피격 효과를 재생합니다.



플레이어가 윈드 블레이드 스킬로 공격한 결과 화면

Do it! 실습 라이트닝 공격 구현하기

다음으로, 광선과 충돌한 모든 대상을 공격하는 라이트닝 공격을 만듭니다.

01 마찬가지로 Assets 〉 Scripts 〉 Skill 〉 Projectile 폴더에 LineRenderer 공격 발사체를 제어하는 ProjectileLine.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/Projectile/ProjectileLine.cs

코드 06-11 LineRenderer를 이용한 광선 공격 발사체 제어

```
01: using UnityEngine;
02:
03: public class ProjectileLine : ProjectileBase
04: {
05:     [SerializeField]
06:     private LineRenderer lineRenderer;
07:     private float currentTime = 1f;
08:     private float duration = 0.5f;
09:
10:     public override void Setup(EntityBase target, float damage)
11:     {
12:         base.Setup(target, damage);
13:
14:         transform.rotation = Utils.RotateToTarget(transform.position,
```

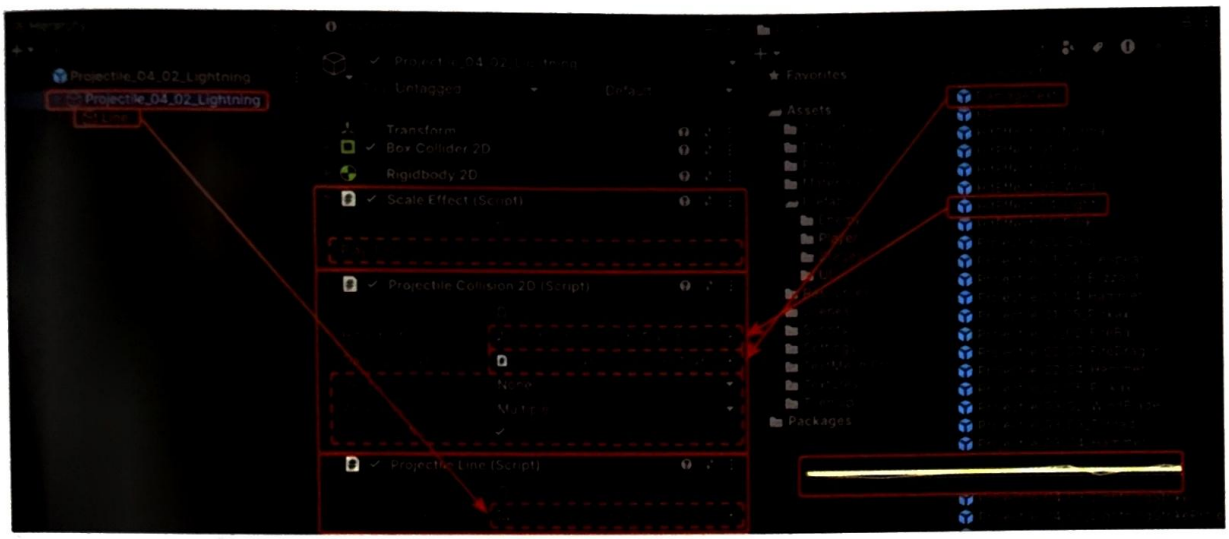
```
                target.MiddlePoint);
15:     }
16:
17:     public override void Process()
18:     {
19:         lineRenderer.material.mainTextureOffset += new Vector2(Time.time, 0);
20:
21:         currentTime -= Time.deltaTime / duration;
22:         lineRenderer.startWidth = currentTime;
23:         lineRenderer.endWidth = currentTime;
24:
25:         if ( currentTime <= 0f ) Destroy(gameObject);
26:     }
27: }
```

19줄 Process() 메서드에서 LightningEffect 재질의 Offset.x값이 계속 증가하여 광선(lineRenderer)에 출력되는 이미지를 x축으로 스크롤 하므로 광선이 앞으로 이동하는 것처럼 보입니다.

21줄 currentTime은 duration 시간 동안 1.0에서 0.0으로 감소합니다.

22~23줄 lineRenderer의 startWidth, endWidth를 currentTime으로 설정하므로 광선의 굵기가 duration 시간 동안 점점 얇아집니다.

02 Projectile_04_02_Lightning 프리팹에 Scale Effect, Projectile Collision 2D, Projectile Line 컴포넌트를 추가합니다. Scale Effect 컴포넌트의 Play Time은 '0.1'로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_04_Light 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [None]으로, Attack Type은 [Multiple]로 설정하고 Is Ignore Wall은 true로 설정합니다. Projectile Line 컴포넌트의 Line Renderer 변수에 Line 오브젝트를 등록합니다.



Projectile_04_02_Lightning 프리팹에 컴포넌트 추가하고 설정하기

03 빛 속성 주 공격 스킬 1의 SO 데이터를 저장하고자 Project View의 ➕▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '04_02_Light_Lightning'으로 변경합니다. 라이트닝 스킬의 Skill Name은 'Lightning'으로, Skill Type은 [Emission]으로, Element는 [Light]로, Max Level은 '99'로 설정한 후, Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택합니다. 그리고 Attack Base Stats 크기는 '3'으로, Attack Buff Stats 크기는 '2'로 설정하고 다음 표처럼 값을 입력합니다.

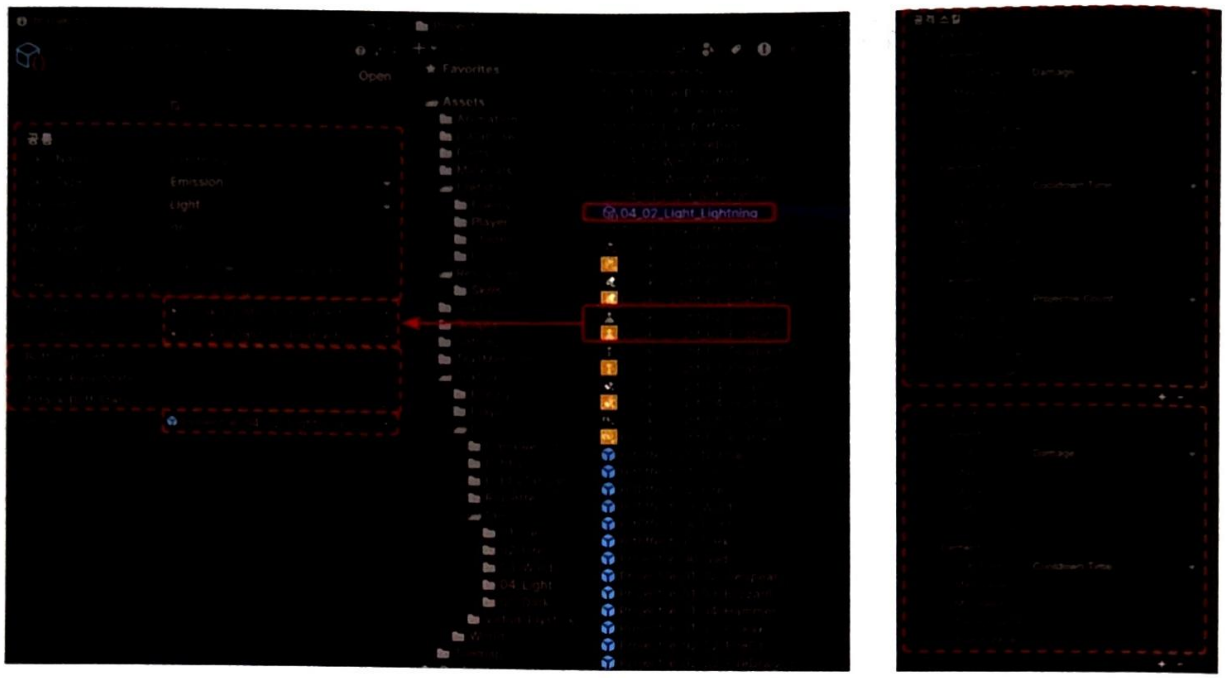
Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	100	100	0
1	Cooldown Time	6	3	6	0
2	Projectile Count	1	1	1	0

Attack Buff Stats 리스트 설정값

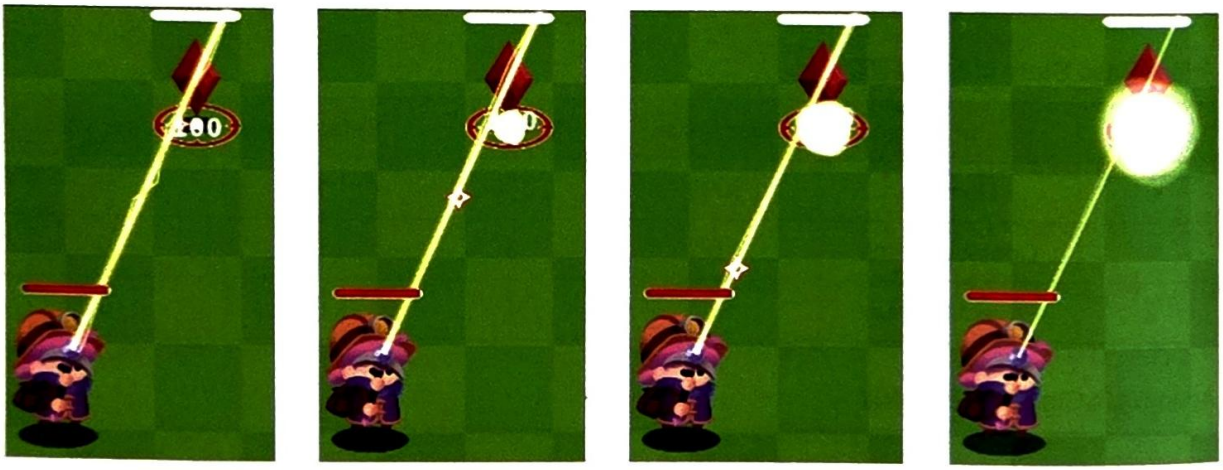
Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	50	0
1	Cooldown Time	0	0	-0.15	0

마지막으로, 라이트닝 발사체로 사용할 Projectile_04_02_Lightning 프리팹을 Projectile 변수에 등록합니다.



라이트닝 스킬 에셋 생성하고 설정하기

04 플레이 버튼 ▶을 눌러 게임을 실행하고 라이트닝 스킬을 배우면 쿨타임이 될 때마다 목표 방향을 지나가는 직선 광선을 생성하고 광선과 충돌한 모든 적에서 피격 효과를 재생합니다.



플레이어가 라이트닝 스킬로 공격한 결과 화면

Do it! 실습 다중 발사체 공격 구현하기

마지막으로, 벽을 통과하여 10개의 곡선 유도탄으로 단일 대상을 공격하는 다중 발사체를 만듭니다.

01 곡선 정보를 추출하도록 메서드를 정의하고자 Utils.cs 스크립트를 수정합니다.

Assets/Scripts/Common/Utils.cs

코드 06-12 곡선 정보 추출 메서드

```
01: using UnityEngine;
02:
03: public static class Utils
04: {
        (... 생략 ...)
25:     public static float GetAngleFromPosition(Vector2 owner, Vector2 target)
26:     {
27:         // 원점에서의 거리, 수평축에서의 각도로 위치를 구하는 극좌표계 이용
28:         // 각도 = arctan(y/x)
29:         // x, y 변윗값 구하기
30:         float dx = target.x - owner.x;
31:         float dy = target.y - owner.y;
32:
33:         // x, y 변윗값을 바탕으로 각도 구하기
34:         // 각도가 radian 단위이므로 Mathf.Rad2Deg를 곱해 각도 단위를 구함
35:         float degree = Mathf.Atan2(dy, dx) * Mathf.Rad2Deg;
36:
37:         //return Mathf.Atan(Mathf.Abs(owner.y-target.y) /
                Mathf.Abs(owner.x-target.x));
38:         return degree;
39:     }
40:
41:     /// <summary>
42:     /// Degree값을 Radian값으로 변환한다.
43:     /// 1도는 "PI/180" radian
44:     /// angle도는 "PI/180 * angle" radian
45:     /// </summary>
46:     public static float DegreeToRadian(float angle)
47:     {
48:         return Mathf.PI * angle / 180;
49:     }
50:
51:     public static Vector2 GetNewPoint(Vector3 start, float angle, float r)
```

추가

```
52:     {
53:         // Degree 각도를 Radian으로 변경
54:         angle = DegreeToRadian(angle);
55:
56:         // 원점을 기준으로 X, Y 좌표를 구하므로 시작 지점 좌표(start)를 더함
57:         Vector2 position = Vector2.zero;
58:         position.x = Mathf.Cos(angle) * r + start.x;
59:         position.y = Mathf.Sin(angle) * r + start.y;
60:
61:         return position;
62:     }
63:
64:     public static Vector2 Lerp(Vector2 a, Vector2 b, float t)
65:     {
66:         return a + (b - a) * t;
67:     }
68:
69:     public static Vector2 QuadraticCurve(Vector2 a, Vector2 b, Vector2 c, float t)
70:     {
71:         Vector2 p1 = Lerp(a, b, t);
72:         Vector2 p2 = Lerp(b, c, t);
73:
74:         return Lerp(p1, p2, t);
75:     }
76:
77:     public static Vector2 CubicCurve(Vector2 a, Vector2 b, Vector2 c,
           Vector2 d, float t)
78:     {
79:         Vector2 p1 = QuadraticCurve(a, b, c, t);
80:         Vector2 p2 = QuadraticCurve(b, c, d, t);
81:
82:         return Lerp(p1, p2, t);
83:     }
84: }
```

25~39줄 내부 동작은 RotateToTarget() 메서드와 거의 비슷합니다. 단, RotateToTarget() 메서드는 오브젝트를 회전시키려고 Quaternion값을 반환하고 이와 달리 GetAngleFrom Position() 메서드는 각도(float형)만 반환합니다.

46~49줄 삼각 함수를 이용한 수학 계산에서는 라디안radian 단위를 사용해야 올바른 값을 출력하므로 DegreeToRadian() 메서드로 각도(degree)를 라디안으로 변환해서 사용합니다.

51~62줄 **GetNewPoint()** 메서드는 단위 원(크기 1)에서 특정 각도에 해당하는 원의 둘레 x, y 위치를 구하는 Cos(θ), Sin(θ) 수식으로 **start** 위치에서 반지름 **r**인 원의 각도 **angle**에 해당하는 원 둘레 위치(**position**)를 구해 반환합니다.

64~67줄 **Lerp()** 메서드는 유니티에서 제공하는 보간 메서드 **Vector2.Lerp()**와 마찬가지로 두 점 a와 b 사이를 보간합니다. 단, **Vector2.Lerp()**는 최댓값과 최솟값을 벗어나지 않도록 처리하지만 **Lerp()** 메서드는 별다른 처리를 하지 않습니다.

69~75줄 **QuadraticCurve()** 메서드는 발사체가 곡선 이동을 하거나 포물선 궤적을 그리며 이동하도록 2차 베지어 곡선 점을 구합니다(C자 곡선).

77~83줄 **CubicCurve()** 메서드는 발사체가 곡선을 2번 그리며 날아가도록 3차 베지어 곡선 점을 구합니다(S자 곡선).

02 매개변수가 4개인 **Setup()** 메서드를 정의하고자 ProjectileBase.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/Projectile/ProjectileBase.cs

코드 06-13 매개변수가 4개인 Setup() 메서드 오버로딩

```
01: using UnityEngine;
02:
03: public abstract class ProjectileBase : MonoBehaviour
04: {
05:     protected MovementRigidbody2D movementRigidbody2D;
        (... 생략 ...)
17:     public virtual void Setup(EntityBase target, float damage, int maxCount,
            int index)                                                          ← 추가
18:     {
19:         Setup(target, damage);
20:     }
        (... 생략 ...)
27:     public abstract void Process();
28: }
```

03 Assets 〉 Scripts 〉 Skill 〉 Projectile 폴더에 곡선 공격 발사체를 제어하는 Projectile CubicHoming.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/Projectile/ProjectileCubicHoming.cs

코드 06-14 곡선 공격 발사체 제어

```
01: using UnityEngine;
02:
03: public class ProjectileCubicHoming : ProjectileBase
04: {
05:     private Vector2 start, end, point1, point2;
06:     private float percent = 0, duration = 1f, r = 2;
07:     private EntityBase target;
08:
09:     public override void Setup(EntityBase target, float damage, int maxCount,
            int index)
10:     {
11:         base.Setup(target, damage);
12:
13:         this.target = target;
14:         start = transform.position;
15:         end = target.MiddlePoint;
16:
17:         float angle = 360 / maxCount * index;
18:         // 현재 플레이어의 회전값을 적용하고자 angle값에 더함
19:         angle += Utils.GetAngleFromPosition(start, end);
20:
21:         point1 = Utils.GetNewPoint(start, angle, r);
22:         point2 = Utils.GetNewPoint(start, angle * -1, r * 2);
23:     }
24:
25:     public override void Process()
26:     {
27:         if ( percent >= 1f ) Destroy(gameObject);
28:         if ( target != null ) end = target.MiddlePoint;
29:
30:         percent += Time.deltaTime / duration;
31:         transform.position = Utils.CubicCurve(start, point1, point2, end, percent);
32:     }
33: }
```

17줄 발사체(maxCount)의 개수에 관계없이 동일한 각도만큼 회전해서 배치하도록 360을 maxCount로 나누고 현재 발사체 순서(index)를 곱하여 angle에 저장합니다.

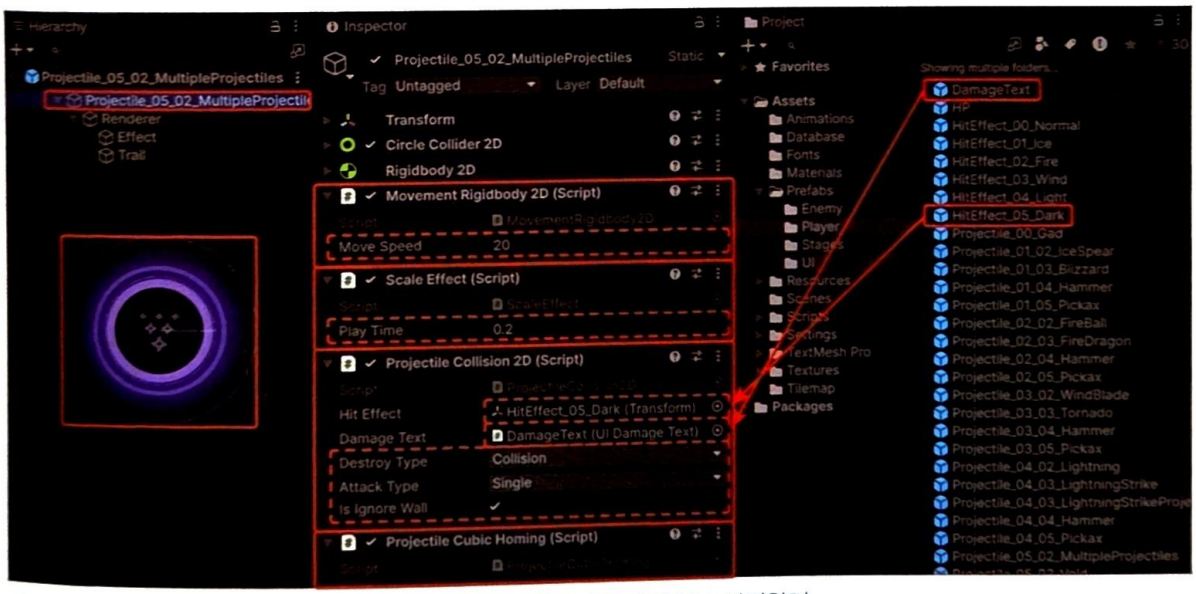
21줄 시작 위치(start)에서 반지름 r만큼 떨어진 angle 각도의 둘레 위치를 구해 point1에 저장합니다.

22줄 시작 위치(start)에서 반지름 r의 2배만큼 떨어진 -angle 각도의 둘레 위치를 구해 point2에 저장합니다. 즉, S자 형태의 곡선을 그리도록 설정합니다.

25~32줄 percent는 0.0부터 1.0까지 프레임마다 Time.deltaTime / duration만큼 증가하므로 duration 시간이 지나면 percent가 1.0이 되어 발사체 오브젝트가 삭제됩니다. 발사체가 추적하는 목표(target)가 다른 공격을 받아 사라지더라도 end 위치를 갱신하지 않으므로 목표가 사라진 마지막 위치로 발사체가 이동해 사라집니다. 발사체는 start 위치에서 end 위치까지 S자 형태의 3차 베지어 곡선을 그리며 duration 시간 동안 날아갑니다.

04 Projectile_05_02_MultipleProjectiles 프리팹에 Movement Rigidbody 2D, Scale Effect, Projectile Collision 2D, Projectile Cubic Homing 컴포넌트를 추가합니다.

Movement Rigidbody 2D 컴포넌트의 Move Speed는 '20', Scale Effect 컴포넌트의 Play Time은 '0.2'로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_05_Dark 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Collision]으로, Attack Type은 [Single]로, Is Ignore Wall은 true로 설정합니다.



Projectile_05_02_MultipleProjectiles 프리팹에 컴포넌트 추가하고 설정하기

05 어둠 속성 주 공격 스킬 1의 SO 데이터를 저장하고자 Project View의 ＋▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '05_02_Dark_MultipleProjectiles'로 변경합니다. 다중 발사체 스킬의 Skill Name은 'Multiple Projectiles'로, Skill Type은 [Emission]으로, Element는 [Dark]로, Max Level은 '99'로 설정한 후, Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택합니다. 그리고 Attack Base Stats 크기는 '3'으로, Attack Buff Stats 크기는 '2'로 설정하고 다음 표처럼 값을 입력합니다.

Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	25	25	0
1	Cooldown Time	6.5	4	6.5	0
2	Projectile Count	10	10	10	0

Attack Buff Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	12.5	0
1	Cooldown Time	0	0	-0.2	0

마지막으로, 다중 발사체로 사용할 Projectile_05_02_MultipleProjectiles 프리팹을 Projectile 변수에 등록합니다.

다중 발사체 스킬 에셋 생성하고 설정하기

06 발사체 사출 스킬을 연속으로 발사할 수 있도록 SkillEmission.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillEmission.cs

코드 06-15 발사체 사출 스킬의 연사 기능

```
01: using UnityEngine;
02:
03: public class SkillEmission : SkillBase
04: {
05:     private float attackRate = 0.05f;                    ← 추가
06:     private float currentAttackRate = 0;
07:     private int currentProjectileCount = 0;
        (... 생략 ...)
21:     public override void OnSkill()
22:     {
23:         // 스킬이 사용할 수 있는 상태인지 검사(쿨타임)
24:         if ( isSkillAvailable == true )
25:         {
26:             int maxCount = (int)GetStat(StatType.ProjectileCount).Value;   ← 수정
27:
28:             // attackRate 주기로 발사체 생성
29:             if ( Time.time - currentAttackRate > attackRate )
30:             {
31:                 GameObject projectile = GameObject.Instantiate(
                    skillTemplate.projectile, spawnPoint.position, Quaternion.identity);
32:
33:                 // ProjectileStraight, ProjectileHoming도 연발은 가능하지만
                       3, 4번째 매개변수는 필요 없으므로 기존과 동일하게 처리
34:                 if ( projectile.TryGetComponent<ProjectileCubicHoming>(out var p) )
35:                     p.Setup(owner.Target, GetStat(StatType.Damage).Value,
                            maxCount, currentProjectileCount);
36:                 else projectile.GetComponent<ProjectileBase>().Setup(
                        owner.Target, GetStat(StatType.Damage).Value);
37:
38:                 currentProjectileCount ++;
39:                 currentAttackRate = Time.time;
40:             }
41:
42:             // ProjectileCount 개수만큼 발사체를 생성한 후 쿨타임 초기화
43:             if ( currentProjectileCount >= maxCount )
44:             {
45:                 isSkillAvailable = false;
46:                 currentProjectileCount = 0;
47:                 currentCooldownTime = Time.time;
```

```
48:                 }
49:             }
50:         }
51: }
```

26줄 해당 스킬의 발사체 개수 정보를 받아 **maxCount** 변수에 저장합니다.

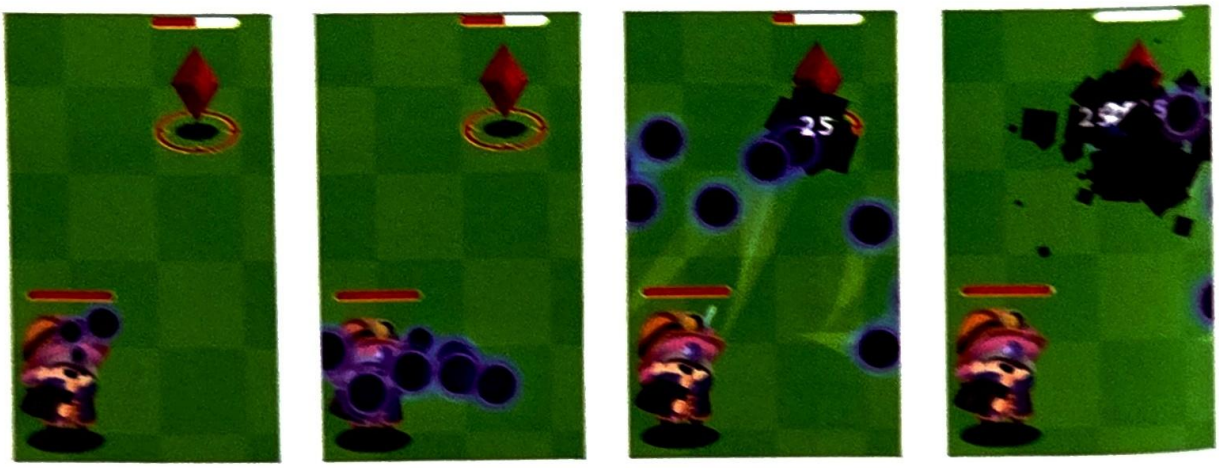
29줄 한 번의 스킬 공격에 발사체가 여러 개라면 **attackRate** 주기가 될 때마다 발사체를 생성합니다.

35줄 곡선 형태로 꺾여서 발사하는 **ProjectileCubicHoming**은 4개의 매개변수를 사용하는 **Setup()** 메서드를 호출합니다.

36줄 직선, 유도 공격은 2개의 매개변수를 사용하는 **Setup()** 메서드를 호출합니다.

42~48줄 한 번의 스킬에서 생성할 수 있는 모든 발사체를 생성하면 스킬 사용을 불가능으로 설정하고 다음 쿨타임까지 기다립니다.

07 플레이 버튼 ▶을 눌러 게임을 실행하고 다중 발사체 스킬을 배우면 쿨타임이 될 때마다 목표를 향해 날아가는 10개의 발사체를 생성하고 발사체와 충돌한 적에서 피격 효과를 재생합니다.



플레이어가 다중 발사체 스킬로 공격한 결과 화면

06-2

보조 공격 스킬 구현하기

이 절에서는 레벨에 따라 최소 2개에서 최대 10개까지 발사체 개수가 증가하고 벽을 통과해 단일 대상에게 곡선 유도 발사체로 공격하는 보조 공격 스킬인 해머 Hammer를 추가해 봅시다.

Do it! 실습 아이스 해머 공격 구현하기

01 주 공격과 마찬가지로 Assets 〉 Scripts 〉 Skill 〉 Projectile 폴더에 곡선 공격 발사체를 제어하는 ProjectileQuadraticHoming.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/Projectile/ProjectileQuadraticHoming.cs

코드 06-16 곡선 공격 발사체 제어

```
01: using UnityEngine;
02:
03: public class ProjectileQuadraticHoming : ProjectileBase
04: {
05:     private Vector2 start, end, point;
06:     private float percent = 0, duration = 1f, r = 5;
07:     private EntityBase target;
08:
09:     public override void Setup(EntityBase target, float damage, int maxCount,
            int index)
10:     {
11:         base.Setup(target, damage);
12:
13:         this.target = target;
14:         start = transform.position;
15:         end = target.MiddlePoint;
16:
17:         float angle = 360 / maxCount * index;
18:         // 현재 플레이어의 회전값을 적용하고자 angle값에 더함
19:         angle += Utils.GetAngleFromPosition(start, end);
20:
```

```
21:         point = Utils.GetNewPoint(start, angle, r);
22:     }
23:
24:     public override void Process()
25:     {
26:         if ( percent >= 1f ) Destroy(gameObject);
27:         if ( target != null ) end = target.MiddlePoint;
28:
29:         percent += Time.deltaTime / duration;
30:         transform.position = Utils.QuadraticCurve(start, point, end, percent);
31:     }
32: }
```

17줄 발사체(maxCount)의 개수에 관계없이 동일한 각도만큼 회전해서 배치하도록 360을 maxCount로 나누고 현재 발사체 순서(index)를 곱한 값을 angle에 저장합니다.

21줄 시작 위치(start)에서 반지름 r만큼 떨어진 angle 각도의 둘레 위치를 구해 point에 저장합니다.

24~31줄 percent는 0.0부터 1.0까지 프레임마다 Time.deltaTime / duration만큼 증가하므로 duration 시간이 지나면 percent가 1.0이 되어 발사체 오브젝트가 삭제됩니다. 발사체가 추적하는 목표(target)가 다른 공격을 받아 사라지더라도 end 위치를 갱신하지 않으므로 목표가 사라진 마지막 위치로 발사체가 이동해 사라집니다. 발사체는 start 위치에서 end 위치까지 C자 형태의 2차 베지어 곡선을 그리며 duration 시간 동안 날아갑니다.

02 이번에는 Assets 〉 Scripts 〉 Common 폴더에 오브젝트 회전을 제어하는 RotateEffect.cs 스크립트를 생성합니다.

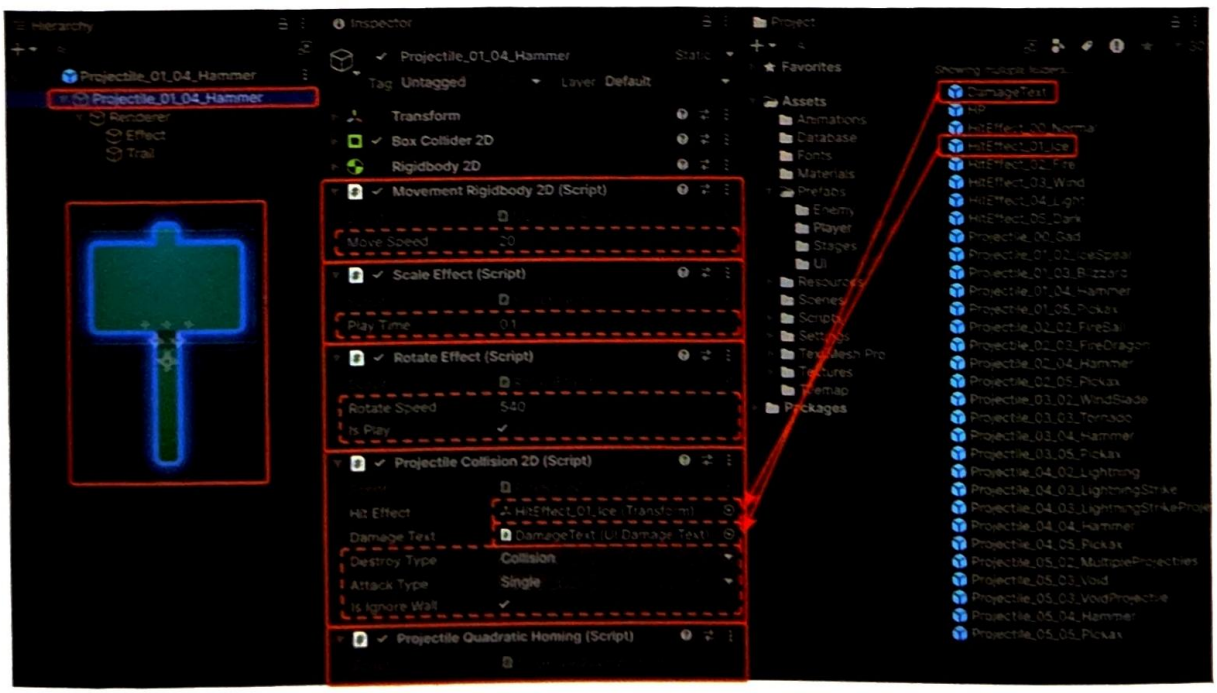
Assets/Scripts/Common/RotateEffect.cs

코드 06-17 오브젝트 회전 제어

```
01: using UnityEngine;
02:
03: public class RotateEffect : MonoBehaviour
04: {
05:     [SerializeField]
06:     private float rotateSpeed = 100;
07:     [SerializeField]
```

```
08:     private bool isPlay = true;
09:
10:     private void Update()
11:     {
12:         if ( !isPlay ) return;
13:
14:         transform.Rotate(Vector3.forward, rotateSpeed * Time.deltaTime);
15:     }
16: }
```

03 Projectile_01_04_Hammer 프리팹에 Movement Rigidbody 2D, Scale Effect, Rotate Effect, Projectile Collision 2D, Projectile Quadratic Homing 컴포넌트를 추가합니다. Movement Rigidbody 2D 컴포넌트의 Move Speed는 '20'으로, Scale Effect 컴포넌트의 Play Time은 '0.1'로 설정합니다. Rotate Effect 컴포넌트의 Rotate Speed는 '540'으로, Is Play는 true로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_01_Ice 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Collision]으로, Attack Type은 [Single]로, Is Ignore Wall은 true로 설정합니다.



Projectile_01_04_Hammer 프리팹에 컴포넌트 추가하고 설정하기

04 얼음 속성 보조 공격 스킬의 SO 데이터를 저장하고자 Project View의 ＋▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '01_04_Ice_Hammer'로 변경합니다. 아이스 해머 스킬의 Skill Name은 'Ice Hammer'로, Skill Type은 [Emission]으로, Element는 [Ice]로, Max Level은 '99'로 설정한 후, Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택합니다. 그리고 Attack Base Stats 크기는 '3'으로, Attack Buff Stats 크기도 '3'으로 설정하고 다음 표처럼 값을 입력합니다.

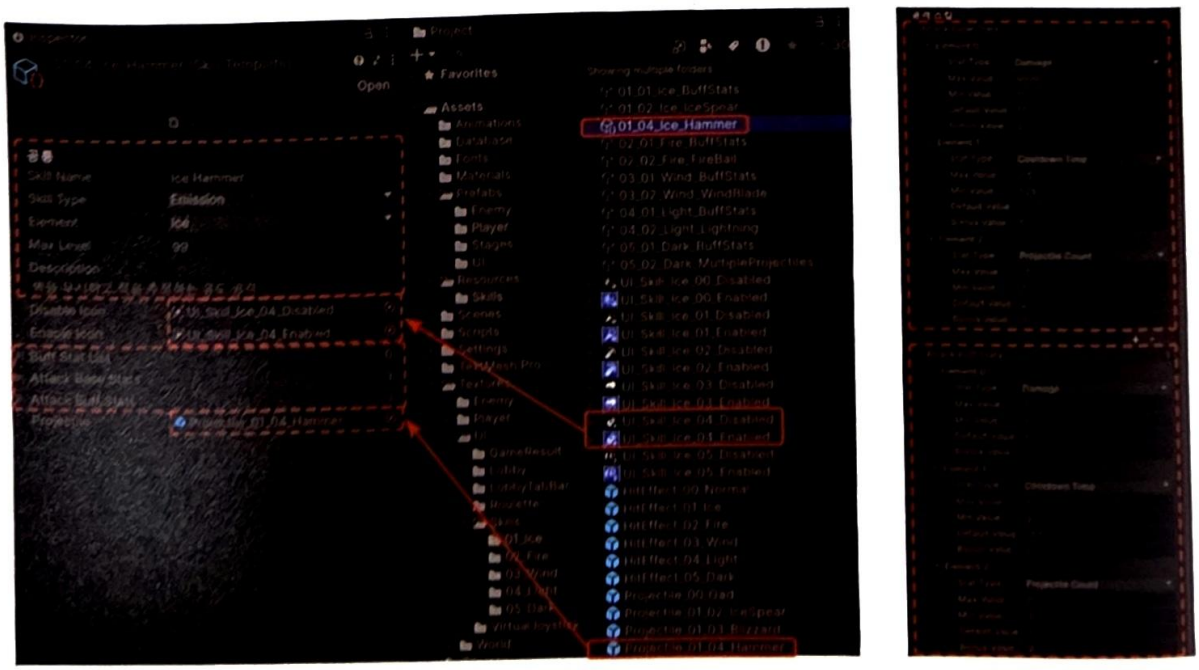
Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	15	15	0
1	Cooldown Time	2.5	1.25	2.5	0
2	Projectile Count	10	2	2	0

Attack Buff Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	5	0
1	Cooldown Time	0	0	-0.1	0
2	Projectile Count	0	0	1	0

마지막으로, 아이스 해머 발사체로 사용할 Projectile_01_04_Hammer 프리팹을 Projectile 변수에 등록합니다.



아이스 해머 스킬 에셋 생성하고 설정하기

05 ProjectileQuadraticHoming 발사체는 매개변수 4개를 사용하는 Setup() 메서드를 호출하므로 SkillEmission.cs 스크립트를 수정합니다.

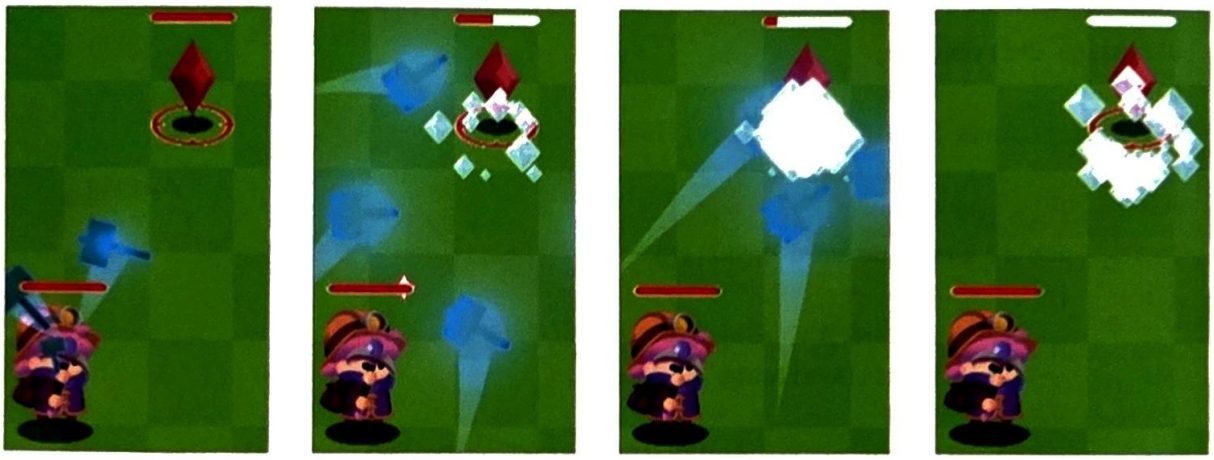
Assets/Scripts/Skill/SkillEmission.cs

코드 06-18 ProjectileQuadraticHoming 발사체 Setup() 메서드 호출

```
using UnityEngine;

public class SkillEmission : SkillBase
{
    (... 생략 ...)
    public override void OnSkill()
    {
        // 스킬이 사용할 수 있는 상태인지 검사(쿨타임)
        if ( isSkillAvailable == true )
        {
            int maxCount = (int)GetStat(StatType.ProjectileCount).Value;

            // attackRate 주기로 발사체 생성
            if ( Time.time - currentAttackRate > attackRate )
            {
                (... 생략 ...)
                // ProjectileStraight, ProjectileHoming도 연발은 가능하지만
                   3, 4번째 매개변수는 필요 없으므로 기존과 동일하게 처리
                if ( projectile.TryGetComponent<ProjectileCubicHoming>(out var p) )
                    p.Setup(owner.Target, GetStat(StatType.Damage).Value,
                                    maxCount, currentProjectileCount);
                else if ( projectile.TryGetComponent<ProjectileQuadraticHoming>(
                    out var p2) )
                    p2.Setup(owner.Target, GetStat(StatType.Damage).Value,
                        maxCount, currentProjectileCount);
                else projectile.GetComponent<ProjectileBase>().Setup(
                    owner.Target, GetStat(StatType.Damage).Value);

                currentProjectileCount ++;
                currentAttackRate = Time.time;
            }
            (... 생략 ...)
        }
    }
}
```

추가

06 플레이 버튼 ▶을 눌러 게임을 실행하고 아이스 해머 스킬을 배우면 쿨타임이 될 때마다 목표를 향해 발사체를 발사하고 충돌하면 피격 효과를 재생합니다. 아이스 해머 스킬은 레벨 1일 때 발사체 2개를 생성하고 레벨업 할 때마다 발사체가 1개씩 증가해서 최대 10개까지 발사할 수 있습니다.

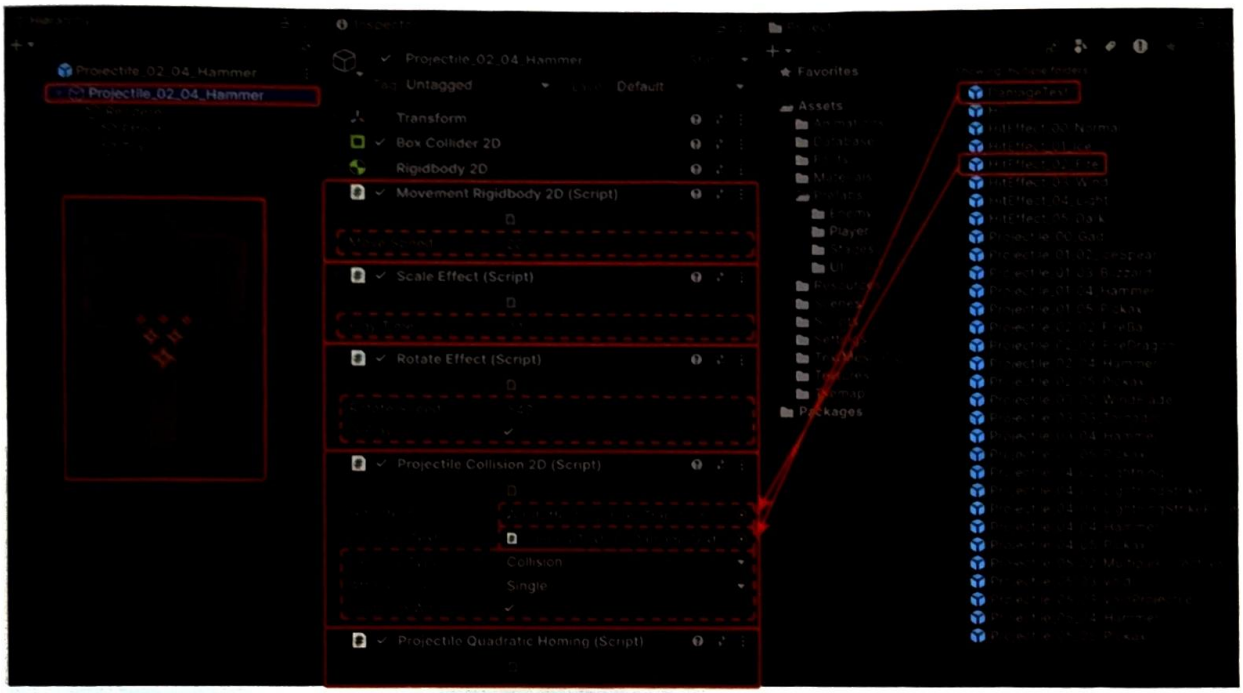


플레이어가 아이스 해머 스킬로 공격한 결과 화면

Do it! 실습 파이어, 윈드, 라이트, 다크 해머 공격 추가하기

아이스 해머 공격을 만드는 과정을 참고하여 파이어, 윈드, 라이트, 다크 등 나머지 4가지 해머 공격을 추가합니다.

01 Projectile_02_04_Hammer 프리팹에 Movement Rigidbody 2D, Scale Effect, Rotate Effect, Projectile Collision 2D, Projectile Quadratic Homing 컴포넌트를 추가합니다. Movement Rigidbody 2D 컴포넌트의 Move Speed는 '20'으로, Scale Effect 컴포넌트의 Play Time은 '0.1'로, Rotate Effect 컴포넌트의 Rotate Speed는 '540'으로, Is Play는 true로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_02_Fire 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Collision]으로, Attack Type은 [Single]로, Is Ignore Wall은 true로 설정합니다.



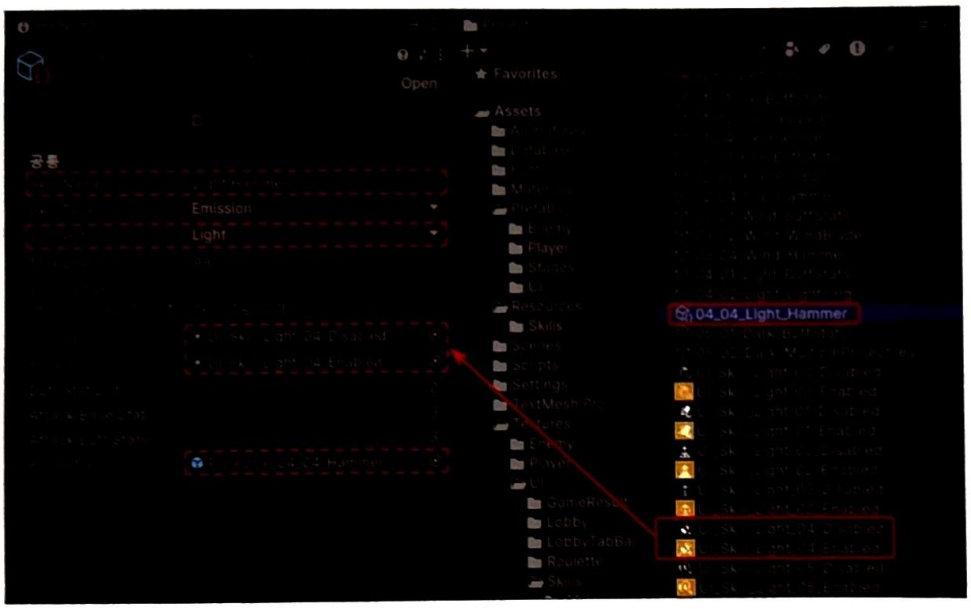
Projectile_02_04_Hammer 프리팹에 컴포넌트 추가하고 설정하기

02 Hammer 스킬은 속성 정보를 제외한 나머지 정보가 모두 같으므로 Assets 〉 Resources 〉 Skills 폴더에 있는 01_04_Ice_Hammer 에셋을 Ctrl+D로 복제한 후 이름을 '02_04_Fire_Hammer'로 변경합니다. Skill Name은 'Fire Hammer'로 설정하고 Element는 [Fire]로 선택합니다.

Disable Icon 변수에는 UI_Skill_Fire_04_Disabled.png를, Enable Icon 변수에는 UI_Skill_Fire_04_Enabled.png를, Projectile 변수에는 Projectile_02_04_Hammer 프리팹을 등록합니다.

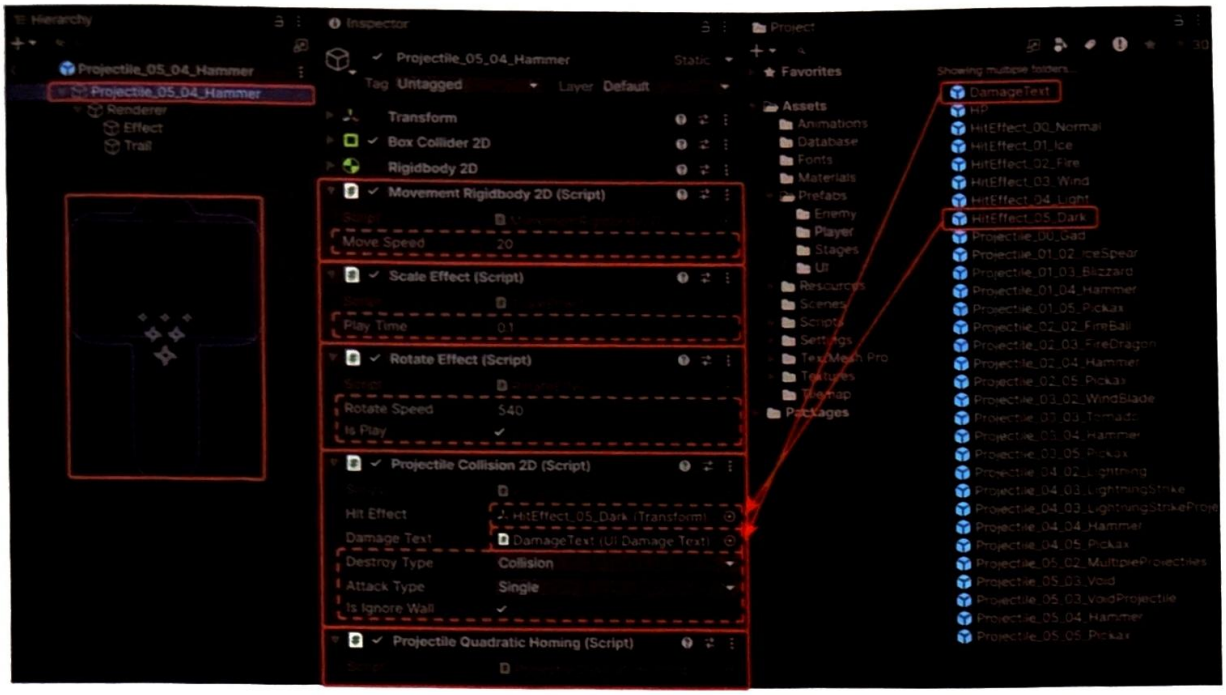
파이어 해머 스킬 에셋 생성하고 설정하기

06 Assets 〉 Resources 〉 Skills 폴더에 있는 01_04_Ice_Hammer 에셋을 Ctrl+D로 복제한 후 이름을 '04_04_Light_Hammer'로 변경합니다. Skill Name은 'Light Hammer'로 설정하고 Element는 [Light]로 선택합니다. Disable Icon 변수에는 UI_Skill_Light_04_Disabled.png를, Enable Icon 변수에는 UI_Skill_Light_04_Enabled.png를, Projectile 변수에는 Projectile_04_04_Hammer 프리팹을 등록합니다.



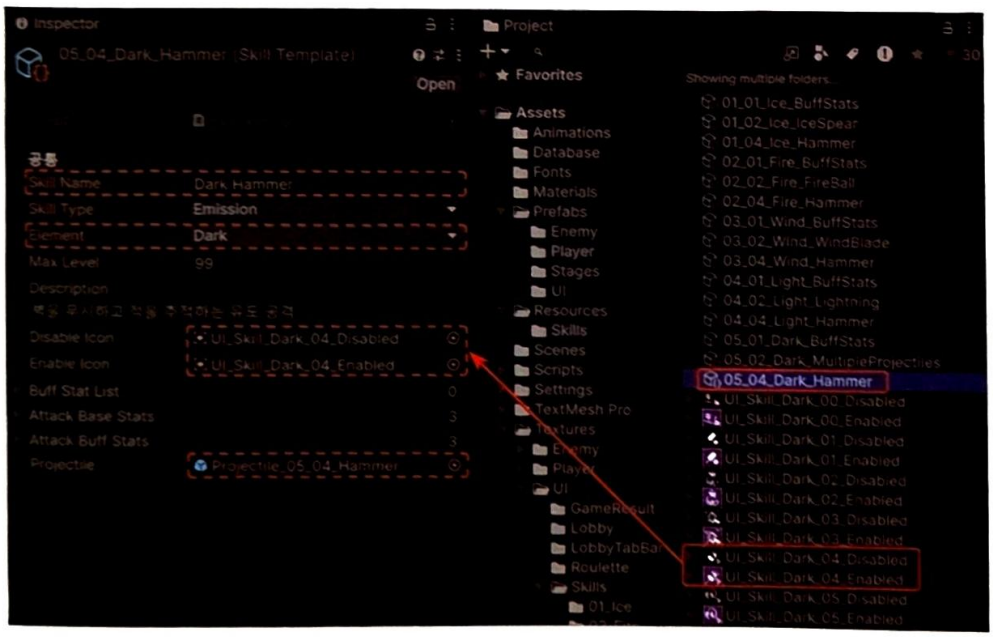
라이트 해머 스킬 에셋 생성하고 설정하기

07 Projectile_05_04_Hammer 프리팹에 Movement Rigidbody 2D, Scale Effect, Rotate Effect, Projectile Collision 2D, Projectile Quadratic Homing 컴포넌트를 추가합니다. Movement Rigidbody 2D 컴포넌트의 Move Speed는 '20'으로, Scale Effect 컴포넌트의 Play Time은 '0.1'로 설정합니다. Rotate Effect 컴포넌트의 Rotate Speed는 '540'으로, Is Play는 true로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_05_Dark 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Collision]으로, Attack Type은 [Single]로, Is Ignore Wall은 true로 설정합니다.



Projectile_05_04_Hammer 프리팹에 컴포넌트 추가하고 설정하기

08 Assets 〉 Resources 〉 Skills 폴더에 있는 01_04_Ice_Hammer 에셋을 Ctrl+D로 복제한 후 이름을 '05_04_Dark_Hammer'로 변경합니다. Skill Name은 'Dark Hammer'로 설정하고 Element는 [Dark]로 선택합니다. Disable Icon 변수에는 UI_Skill_Dark_04_Disabled.png를, Enable Icon 변수에는 UI_Skill_Dark_04_Enabled.png를, Projectile 변수에는 Projectile_05_04_Hammer 프리팹을 등록합니다.



다크 해머 스킬 에셋 생성하고 설정하기

07장

근접 회전 공격과 광역 공격 스킬 구현하기

이 장에서는 플레이어 중심 위치를 기준으로 원 운동하는 근접 공격 곡괭이 스킬을 추가합니다. 그리고 모든 적을 공격하는 블리자드, 모든 적을 차례대로 1회 공격하는 파이어 드래곤, 목표를 추적하고 공격하는 토네이도, 모든 적을 1회 공격하는 라이트닝 스트라이크, 앞으로 이동하며 발사체를 방사하는 암흑 공격 등 넓은 범위를 공격하는 주 공격 스킬 2를 만듭니다.

학습 목표

1 플레이어 중심의 회전 투사체 알고리즘을 구현할 수 있다.
2 다양한 특수 공격 스킬(광역, 추적, 연속 타격 등)을 구현할 수 있다.

07-1 플레이어 주위를 회전하는 곡괭이 구현하기
07-2 전 범위 타격과 유도 공격 패턴 확장하기

07-1 플레이어 주위를 회전하는 곡괭이 구현하기

이 절에서는 한번 생성되면 파괴되지 않고 플레이어의 중심 위치를 기준으로 원운동 하는 곡괭이 Pickax 근접 공격 스킬을 만들어 봅시다.

Do it! 실습 아이스 곡괭이 공격 추가하기

01 근접 공격 스킬에 사용하는 곡괭이 발사체는 파괴되지 않고 계속 유지되므로 **DestroyType** 열거형에 파괴 불가 **Indestructible** 상수를 추가하고자 ProjectileCollision2D.cs 스크립트를 수정합니다.

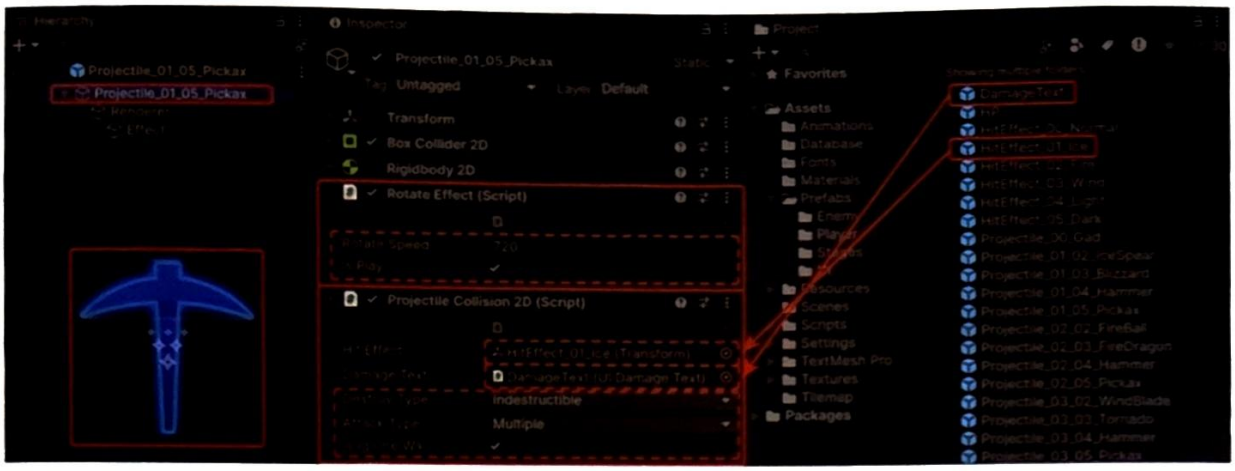
Assets/Scripts/Skill/Projectile/ProjectileCollision2D.cs

코드 07-01 DestroyType 열거형에 파괴 불가 상수 추가

```
01: using UnityEngine;
02:
03: public enum DestroyType { None = -1, Collision = 0, Indestructible, }
04: public enum AttackType { Single, Multiple }
05:
06: public class ProjectileCollision2D : MonoBehaviour
07: {
        (... 생략 ...)
58: }
```

수정

02 Projectile_01_05_Pickax 프리팹에 Rotate Effect, Projectile Collision 2D 컴포넌트를 추가합니다. Rotate Effect 컴포넌트의 Rotate Speed는 '720'으로, Is Play는 true로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_01_Ice 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Indestructible]로, Attack Type은 [Multiple]로, Is Ignore Wall은 true로 설정합니다.



Projectile_01_05_Pickax 프리팹에 컴포넌트 추가하고 설정하기

03 SkillType 열거형에 지속 스킬 Sustained 상수를 추가하고자 SkillTemplate.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillTemplate.cs

코드 07-02 SkillType 열거형에 지속 스킬 상수 추가

```
    (... 생략 ...)
04: public enum SkillType { Buff = 0, Emission, Sustained, }
05: public enum SkillElement { None = -1, Ice = 100, Fire, Wind, Light, Dark,
        Count = 5 }
06:
07: [CreateAssetMenu(fileName = "NewSkill", menuName = "SkillAsset", order = 0)]
08: public class SkillTemplate : ScriptableObject
09: {
        (... 생략 ...)
27: }
```

수정

04 얼음 속성 근접 공격 스킬의 SO 데이터를 저장하고자 Project View의 ＋▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '01_05_Ice_Pickax'로 변경합니다. Skill Name은 'Ice Pickax'로, Skill Type은 [Sustained]로, Element는 [Ice]로, Max Level은 '99'로 설정한 후 Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택합니다. 그리고 Attack Base Stats 크기는 '2'로, Attack Buff Stats 크기는 '1'로 설정하고 다음 표처럼 값을 입력합니다.

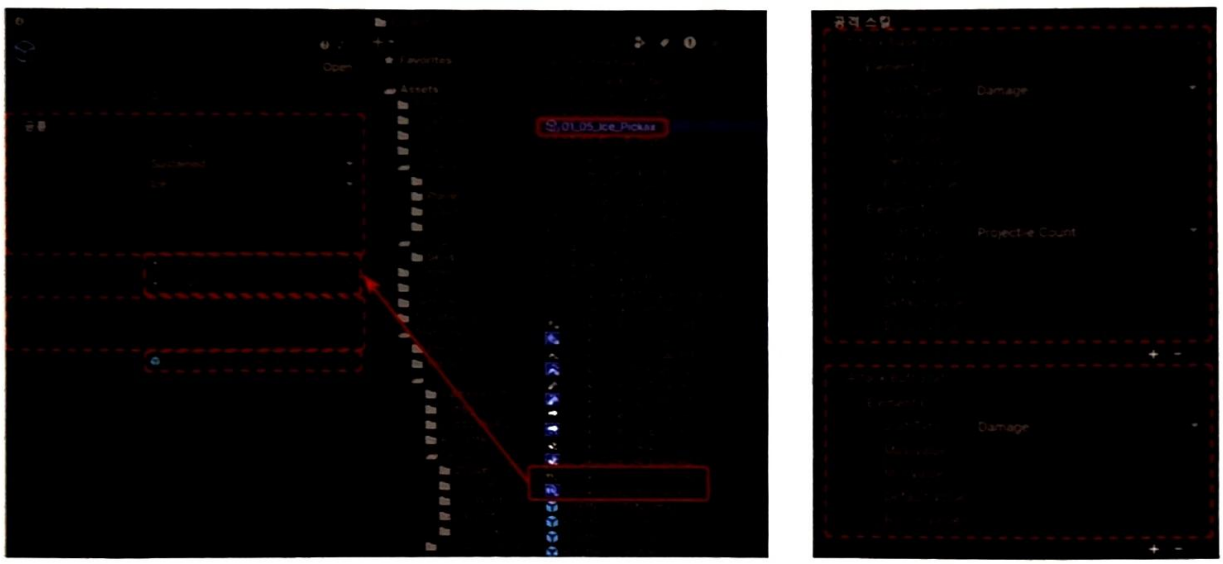
Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	50	50	0
1	Projectile Count	3	3	3	0

Attack Buff Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	50	0

마지막으로, Projectile 변수에 아이스 곡괭이 발사체로 사용할 Projectile_01_05_Pickax 프리팹을 등록합니다.



아이스 곡괭이 스킬 에셋 생성하고 설정하기

05 지속형 스킬이라면 스킬 쿨타임을 계산하지 않도록 설정하고자 SkillBase.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillBase.cs

코드 07-03 지속형 스킬의 스킬 쿨타임 계산 제외

```
    (... 생략 ...)
04: public abstract class SkillBase
05: {
        (... 생략 ...)
```

```
57:     public void IsSkillAvailable()
58:     {
59:         // 레벨이 0이거나 버프 또는 지속 스킬이라면 스킬 사용 가능 상태 아님
60:         if ( CurrentLevel == 0 || SkillType == SkillType.Buff
                || SkillType == SkillType.Sustained ) return;
            (... 생략 ...)
66:     }
67:
68:     public abstract void OnLevelUp(); // 스킬 레벨업 시 1회 호출
69:     public abstract void OnSkill();   // 스킬 사용 시 호출
70: }
```

수정

06 원의 둘레 위치를 구하는 GetPositionFromAngle() 메서드를 정의하고자 Utils.cs 스크립트를 수정합니다.

Assets/Scripts/Common/Utils.cs

코드 07-04 원의 둘레 위치 구하기

```
01: using UnityEngine;
02:
03: public static class Utils
04: {
        (... 생략 ...)
41:     /// <summary>
42:     /// 각도를 기준으로 원의 둘레 위치를 구한다.
43:     /// </summary>
44:     /// <param name="radius">원의 반지름</param>
45:     /// <param name="angle">각도</param>
46:     /// <returns>원의 반지름, 각도에 해당하는 둘레 위치</returns>
47:     public static Vector3 GetPositionFromAngle(float radius, float angle)
48:     {
49:         Vector3 position = Vector3.zero;
50:
51:         angle = DegreeToRadian(angle);
52:
53:         position.x = Mathf.Cos(angle) * radius;
54:         position.y = Mathf.Sin(angle) * radius;
55:
56:         return position;
57:     }
        (... 생략 ...)
102:}
```

추가

47~57줄 GetPositionFromAngle() 메서드는 단위 원(크기 1)에서 특정 각도에 해당하는 원의 둘레 x, y 위치를 구하는 Cos(θ), Sin(θ) 수식을 이용하여 반지름이 radius이고 각도가 angle인 위치(position)를 구해 반환합니다.

즉, 반지름(radius)과 각도(angle)로 표현한 극좌표 위치를 x와 y로 표현한 직교 좌표 위치로 변환합니다.

07 Assets 〉 Scripts 〉 Skill 폴더에 발사체 지속형 스킬을 제어하는 SkillSustained.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/SkillSustained.cs

코드 07-05 발사체 지속형 스킬 제어

```
01: using System.Collections.Generic;
02: using UnityEngine;
03:
04: public class SkillSustained : SkillBase
05: {
06:     private float distanceToPlayer = 2f;
07:     private Transform parent;
08:     private List<GameObject> pickaxs = new List<GameObject>();
09:
10:     public override void Setup(SkillTemplate skillTemplate, PlayerBase owner,
            Transform spawnPoint=null)
11:     {
12:         base.Setup(skillTemplate, owner, spawnPoint);
13:
14:         // 미리 제작한 곡괭이의 부모 오브젝트
15:         parent = GameObject.Find("Pickaxs").transform;
16:     }
17:
18:     public override void OnLevelUp()
19:     {
20:         // 레벨이 0에서 1로 늘 때는 스탯은 증가하지 않고 곡괭이 오브젝트만 생성
21:         if ( currentLevel <= 1 )
22:         {
23:             AddPickax((int)GetStat(StatType.ProjectileCount).Value);
24:
25:             // 현재 활성화된 모든 곡괭이 위치 재설정
26:             int pickaxCount = parent.childCount;
27:             for ( int i = 0; i < pickaxCount; ++ i )
28:             {
29:                 float angle = (360 / pickaxCount) * i;
```

```
30:                 Vector3 position = Utils.GetPositionFromAngle(distanceToPlayer,
                        angle);
31:                 parent.GetChild(i).position = parent.position + position;
32:             }
33:
34:             return;
35:         }
36:
37:         // 공격 스킬 레벨업 시 공격력 등 스탯 갱신
38:         skillTemplate.attackBuffStats.ForEach(stat =>
39:         {
40:             GetStat(stat).BonusValue += stat.DefaultValue;
41:         });
42:
43:         // 모든 곡괭이 공격력 갱신
44:         foreach ( var item in pickaxs )
45:         {
46:             item.GetComponent<ProjectileCollision2D>().Setup(
                    null, GetStat(StatType.Damage).Value);
47:         }
48:     }
49:
50:     private void AddPickax(int count)
51:     {
52:         for ( int i = 0; i < count; ++ i )
53:         {
54:             GameObject clone = GameObject.Instantiate(skillTemplate.projectile,
                        parent);
55:             clone.GetComponent<ProjectileCollision2D>().Setup(
                        null, GetStat(StatType.Damage).Value);
56:             pickaxs.Add(clone);
57:         }
58:     }
59:
60:     public override void OnSkill() { }
61: }
```

근접 공격 스킬은 속성별로 발사체를 최대 3개까지 생성할 수 있고 속성 5개를 모두 습득하면 최대 15개의 발사체를 월드에 출력합니다.

25~32줄 현재 활성화된 모든 발사체가 일정한 간격을 유지하도록 새로운 속성의 근접 공격 스킬을 습득할 때마다 위치를 재설정합니다.

37~41줄 이미 습득한 스킬의 레벨이 증가할 때는 스킬의 스탯 정보를 갱신합니다.

43~47줄 이미 생성된 발사체의 공격력을 갱신합니다.

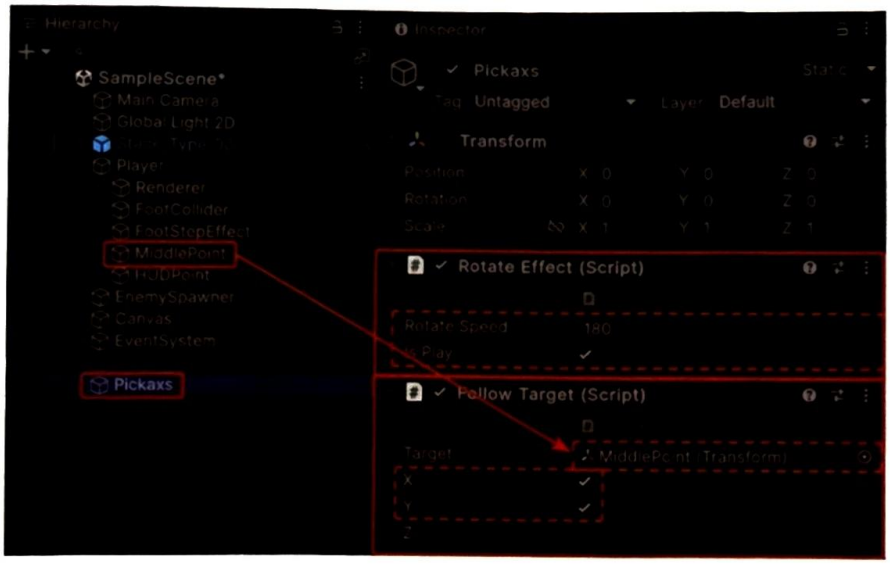
08 지속형 스킬, 즉 **Sustained**일 때는 **SkillSustained** 메모리를 할당하고자 SkillSystem.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillSystem.cs

코드 07-06 지속형 스킬 메모리 할당 처리

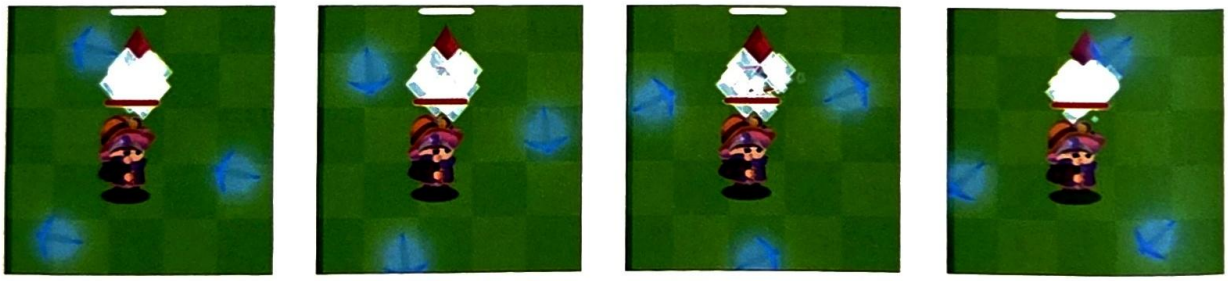
```
    (... 생략 ...)
05: public class SkillSystem : MonoBehaviour
06: {
        (... 생략 ...)
16:     private void Awake()
17:     {
            (... 생략 ...)
23:         foreach ( var item in skillDict )
24:         {
25:             SkillBase skill = null;
26:             if ( item.Value.skillType.Equals(SkillType.Buff) )
                    skill = new SkillBuff();
27:             else if ( item.Value.skillType.Equals(SkillType.Emission) )
                    skill = new SkillEmission();
28:             else if ( item.Value.skillType.Equals(SkillType.Sustained) )   ← 추가
                    skill = new SkillSustained();
                (... 생략 ...)
34:         }
35:     }
        (... 생략 ...)
111:}
```

09 [GameObject → Create Empty] 메뉴로 곡괭이 지속형 발사체를 관리하는 빈 오브젝트를 생성하고 이름을 'Pickaxs'로 변경합니다. Pickaxs 오브젝트에 Rotate Effect, Follow Target 컴포넌트를 추가합니다. Rotate Effect 컴포넌트의 Rotate Speed는 '180'으로, Is Play는 true로 설정합니다. Follow Target 컴포넌트의 Target 변수에는 MiddlePoint 오브젝트를 등록하고 X, Y를 모두 true로 설정합니다.



곡괭이 지속형 발사체를 관리하는 빈 오브젝트 생성하고 설정하기

10 플레이 버튼 ▶을 눌러 게임을 실행하고 아이스 곡괭이 스킬을 배우면 월드에 발사체 3개를 생성하고 플레이어 주변을 원운동 하면서 배회합니다. 곡괭이 스킬은 속성별로 습득할 수 있으며 속성 5개의 곡괭이 스킬을 모두 습득하면 속성별로 3개씩 총 15개의 발사체가 플레이어 주변을 원운동 합니다.

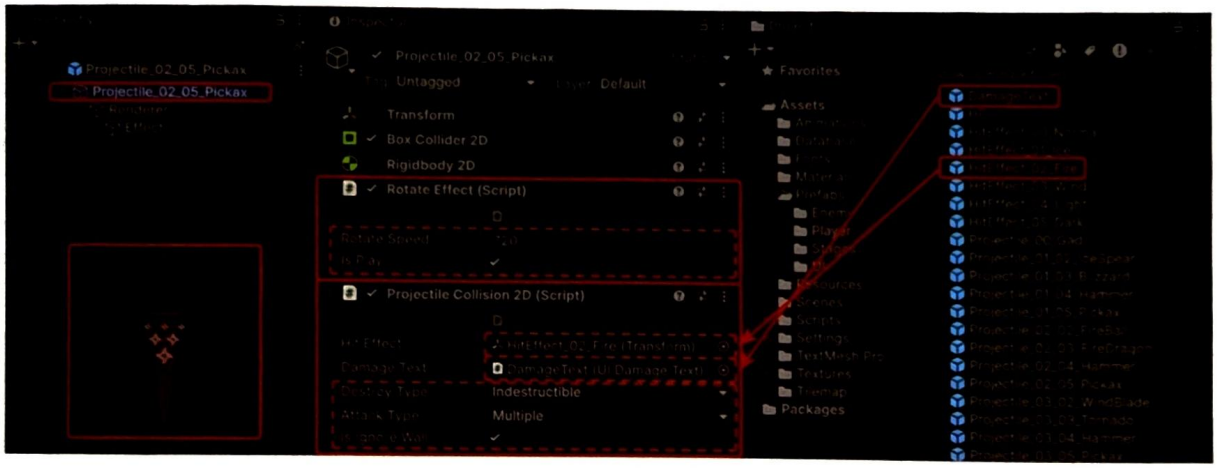


플레이어가 아이스 곡괭이 스킬로 공격한 결과 화면

Do it! 실습 파이어, 윈드, 라이트, 다크 곡괭이 공격 추가하기

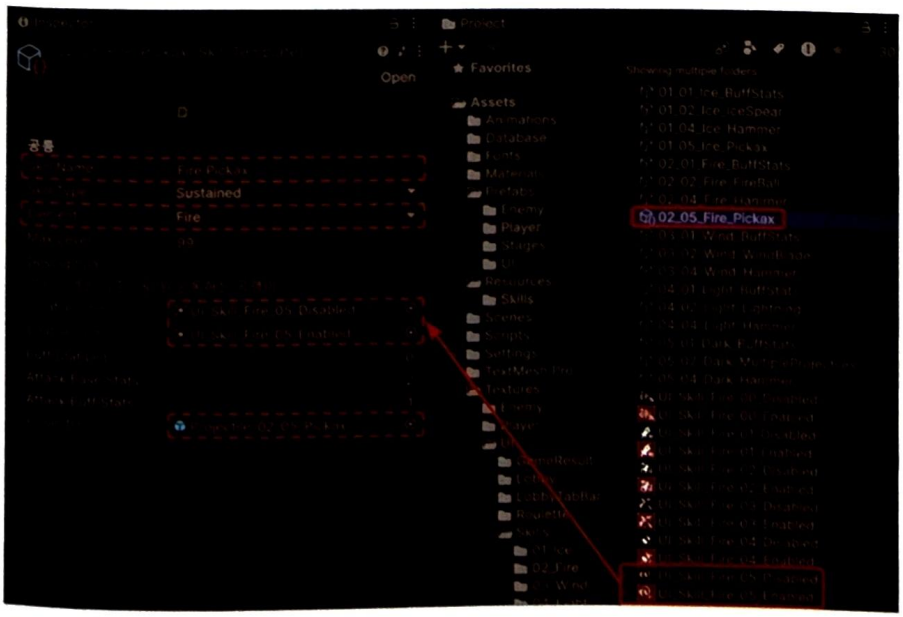
아이스 곡괭이 공격을 만드는 과정을 참고하여 파이어, 윈드, 라이트, 다크 등 나머지 4가지 곡괭이 공격을 추가합니다.

01 Projectile_02_05_Pickax 프리팹에 Rotate Effect, Projectile Collision 2D 컴포넌트를 추가합니다. Rotate Effect 컴포넌트의 Rotate Speed는 '720'으로, Is Play는 true로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_02_Fire 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Indestructible]로, Attack Type은 [Multiple]로, Is Ignore Wall은 true로 설정합니다.



Projectile_02_05_Pickax 프리팹에 컴포넌트 추가하고 설정하기

02 곡괭이 스킬은 속성 정보를 제외한 나머지 정보가 모두 동일하므로 Assets 〉 Resources 〉 Skills 폴더에 있는 01_05_Ice_Pickax 에셋을 Ctrl+D로 복제한 후 이름을 '02_05_Fire_Pickax'로 변경합니다. Skill Name은 'Fire Pickax'로, Element는 [Fire]로 설정합니다. Disable Icon 변수에는 UI_Skill_Fire_05_Disabled.png를, Enable Icon 변수에는 UI_Skill_Fire_05_Enabled.png를, Projectile 변수에는 Projectile_02_05_Pickax 프리팹을 등록합니다.

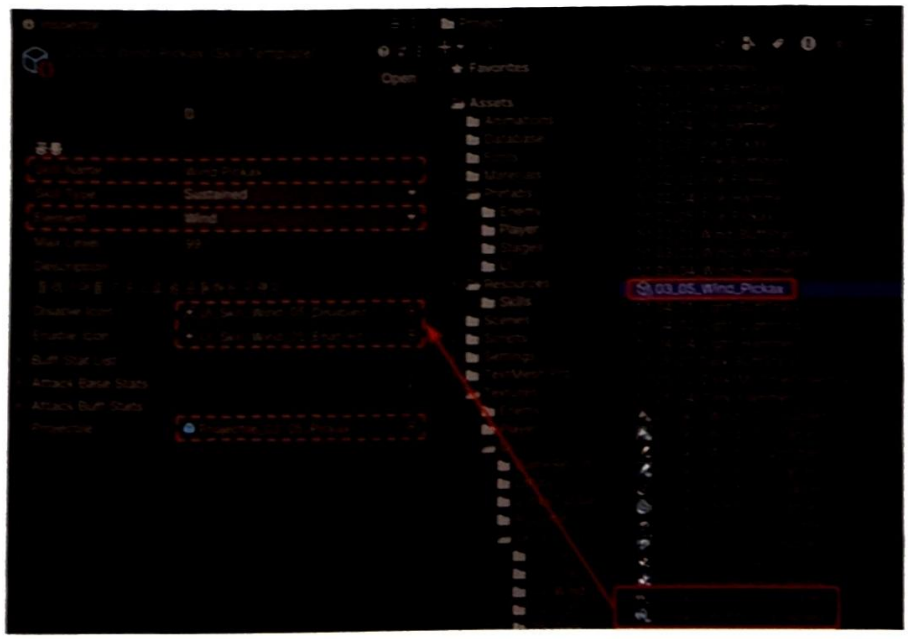


파이어 곡괭이 스킬 에셋 생성하고 설정하기

03 Projectile_03_05_Pickax 프리팹에 Rotate Effect, Projectile Collision 2D 컴포넌트를 추가합니다. Rotate Effect 컴포넌트의 Rotate Speed는 '720'으로, Is Play는 true로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_03_Wind 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Indestructible]로, Attack Type은 [Multiple]로, Is Ignore Wall은 true로 설정합니다.

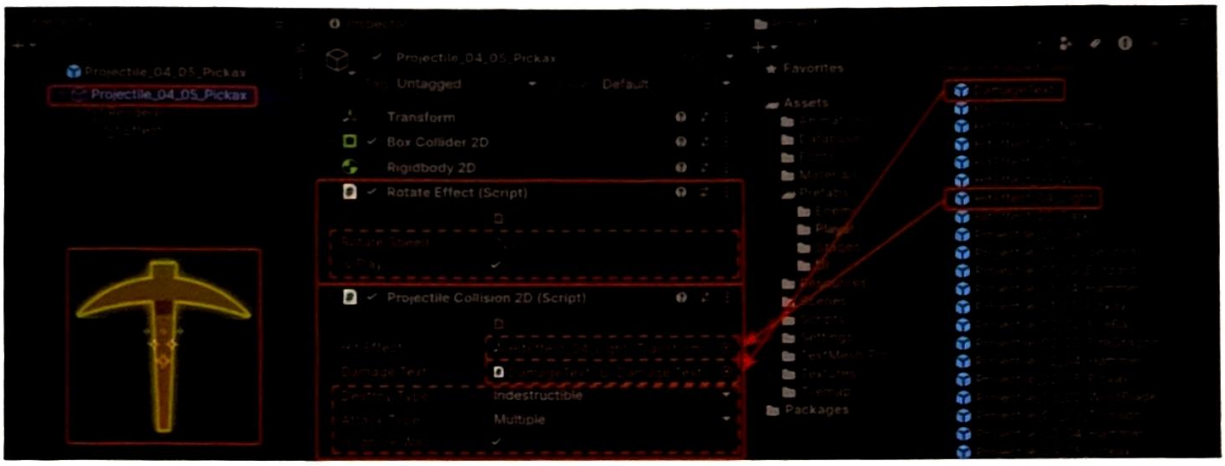
Projectile_03_05_Pickax 프리팹에 컴포넌트 추가하고 설정하기

04 앞서와 마찬가지로 01_05_Ice_Pickax 에셋을 Ctrl+D로 복제한 후 이름을 '03_05_Wind_Pickax'로 변경합니다. Skill Name은 'Wind Pickax'로, Element는 [Wind]로 설정합니다. Disable Icon 변수에는 UI_Skill_Wind_05_Disabled.png를, Enable Icon 변수에는 UI_Skill_Wind_05_Enabled.png를, Projectile 변수에는 Projectile_03_05_Pickax 프리팹을 등록합니다.



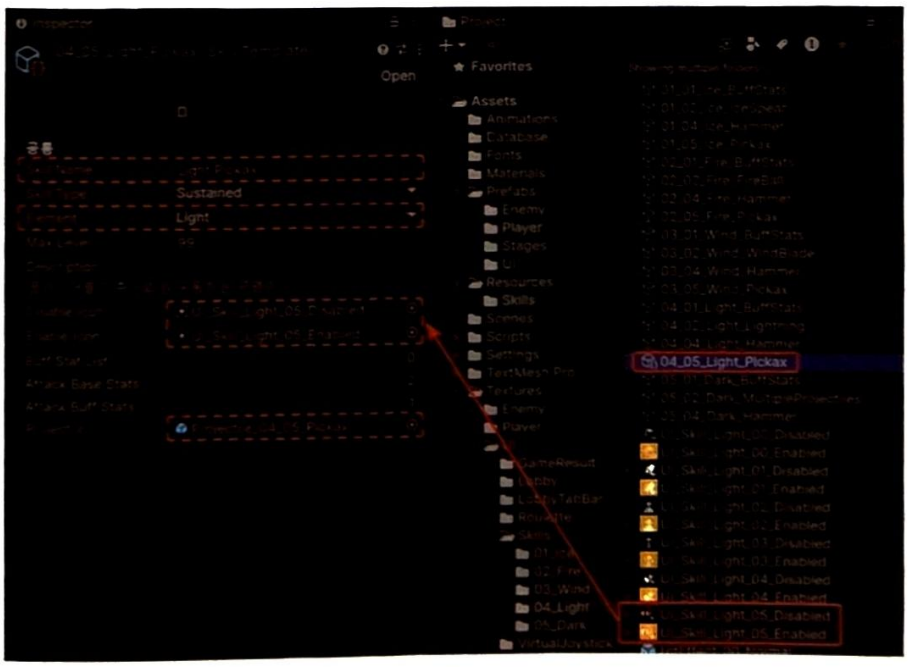
윈드 곡괭이 스킬 에셋 생성하고 설정하기

05 Projectile_04_05_Pickax 프리팹에 Rotate Effect, Projectile Collision 2D 컴포넌트를 추가합니다. Rotate Effect 컴포넌트의 Rotate Speed는 '720'으로, Is Play는 true로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_04_Light 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Indestructible]로, Attack Type은 [Multiple]로, Is Ignore Wall은 true로 설정합니다.



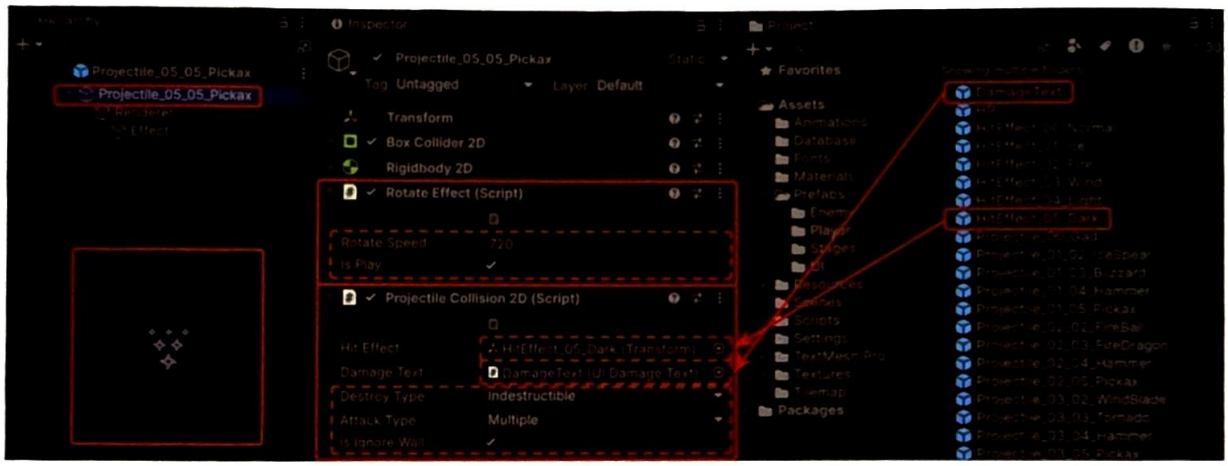
Projectile_04_05_Pickax 프리팹에 컴포넌트 추가하고 설정하기

06 01_05_Ice_Pickax 에셋을 Ctrl+D로 복제한 후 이름을 '04_05_Light_Pickax'로 변경합니다. Skill Name은 'Light Pickax'로, Element는 [Light]로 설정합니다. Disable Icon 변수에는 UI_Skill_Light_05_Disabled.png를, Enable Icon 변수에는 UI_Skill_Light_05_Enabled.png를, Projectile 변수에는 Projectile_04_05_Pickax 프리팹을 등록합니다.



라이트 곡괭이 스킬 에셋 생성하고 설정하기

07 Projectile_05_05_Pickax 프리팹에 Rotate Effect, Projectile Collision 2D 컴포넌트를 추가합니다. Rotate Effect 컴포넌트의 Rotate Speed는 '720'으로, Is Play는 true로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_05_Dark 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Indestructible]로, Attack Type은 [Multiple]로, Is Ignore Wall은 true로 설정합니다.



Projectile_05_05_Pickax 프리팹에 컴포넌트 추가하고 설정하기

08 01_05_Ice_Pickax 에셋을 Ctrl+D로 복제한 후 이름을 '05_05_Dark_Pickax'로 변경합니다. Skill Name은 'Dark Pickax'로, Element는 [Dark]로 설정합니다. Disable Icon 변수에는 UI_Skill_Dark_05_Disabled.png를, Enable Icon 변수에는 UI_Skill_Dark_05_Enabled.png를, Projectile 변수에는 Projectile_05_05_Pickax 프리팹을 등록합니다.

다크 곡괭이 스킬 에셋 생성하고 설정하기

07-2

전 범위 타격과 유도 공격 패턴 확장하기

이 절에서는 5가지 광역 주 공격 스킬을 추가해서 파워업 합니다. 지속 시간 동안 월드에 있는 모든 적을 n회 공격하는 ① 블리자드Blizzard, 월드 아래에서 위로 지나가며 모든 적을 차례대로 1회 공격하는 ② 파이어 드래곤Fire Dragon, 목표를 추적하고 n초 동안 m초 간격으로 충돌한 적을 공격하는 ③ 토네이도Tornado, 월드에 있는 모든 적을 순간적으로 1회 공격하는 ④ 라이트닝 스트라이크Lightning Strike, n초 동안 전방으로 이동하며 m초 간격으로 발사체를 방사하는 ⑤ 암흑Void 공격 스킬을 추가합니다.

Do it! 실습 블리자드 공격 더하기

01 StatType 열거형에 지속형 스킬을 위한 지속 시간, 공격 주기 스탯 상수를 추가하고자 Stat.cs 스크립트를 수정합니다.

Assets/Scripts/Entity/Stat.cs

코드 07-07 지속 시간, 공격 주기 스탯 상수 추가

```
01: using UnityEngine;
    (... 생략 ...)
65: public enum StatType { Damage = 0, CooldownTime, CriticalChance,
        CriticalMultiplier, HP, Evasion,
66:     MetastasisCount, HPRecovery, ProjectileCount, Duration, AttackRate, }
```

수정

02 SkillType 열거형에 광역 스킬 Global 상수를 추가하고자 SkillTemplate.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillTemplate.cs

코드 07-08 SkillType 열거형에 광역 스킬 상수 추가

```
    (... 생략 ...)
04: public enum SkillType { Buff = 0, Emission, Sustained, Global, }
05: public enum SkillElement { None = -1, Ice = 100, Fire, Wind, Light, Dark,
        Count = 5 }
06:
07: [CreateAssetMenu(fileName = "NewSkill", menuName = "SkillAsset", order = 0)]
08: public class SkillTemplate : ScriptableObject
09: {
        (... 생략 ...)
27: }
```

수정

03 외부에서 현재 스킬의 소유주 정보를 알 수 있도록 Get 속성을 정의하고자 SkillBase.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillBase.cs

코드 07-09 소유주 정보 조회를 위한 Get 속성 정의

```
    (... 생략 ...)
04: public abstract class SkillBase
05: {
        (... 생략 ...)
19:     public bool IsMaxLevel => currentLevel == skillTemplate.maxLevel;
20:     public PlayerBase Owner => owner;
        (... 생략 ...)
71: }
```

추가

04 스킬 정보와 공격력을 매개변수로 하는 **Setup()** 메서드를 정의하고자 ProjectileBase.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/Projectile/ProjectileBase.cs

코드 07-10 스킬 정보와 공격력 매개변수를 가지는 Setup() 메서드 오버로딩

```
01: using UnityEngine;
02:
03: public abstract class ProjectileBase : MonoBehaviour
04: {
05:     protected MovementRigidbody2D movementRigidbody2D;
```

```
06:
07:     public virtual void Setup(SkillBase skillBase, float damage) { }
        (... 생략 ...)
30: }
```

추가

05 Assets 〉 Scripts 〉 Skill 〉 Projectile 폴더에 광역 공격 발사체를 제어하는 Projectile Global.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/Projectile/ProjectileGlobal.cs

코드 07-11 광역 공격 발사체 제어

```
01: using UnityEngine;
02:
03: public class ProjectileGlobal : ProjectileBase
04: {
05:     [SerializeField]
06:     private Transform hitEffect;      // 피격 효과
07:     [SerializeField]
08:     private UIDamageText damageText; // 데미지 텍스트 UI
09:
10:     protected SkillBase skillBase;
11:     protected float currentDuration;
12:     protected float currentAttackRate = 0;
13:     protected float damage;
14:
15:     public override void Setup(SkillBase skillBase, float damage)
16:     {
17:         this.skillBase = skillBase;
18:         this.damage = damage;
19:         currentDuration = Time.time;
20:     }
21:
22:     /// <summary>
23:     /// 지속 시간(Duration)이 있는 스킬만 base.Process() 호출한다.
24:     /// </summary>
25:     public override void Process()
26:     {
27:         // 지속 시간이 없는 스킬이 base.Process()를 실행하지 않도록 예외 처리
28:         if ( skillBase.GetStat(StatType.Duration) == null ) return;
29:
30:         // 생성 시점(currentDuration)부터 StatType.Duration 지나면 발사체 삭제
```

```
31:         if ( Time.time - currentDuration >
                skillBase.GetStat(StatType.Duration).Value )
32:         {
33:             Destroy(gameObject);
34:         }
35:     }
36:
37:     protected void TakeDamage(EntityBase entity)
38:     {
39:         if ( hitEffect != null )
40:         {
41:             Instantiate(hitEffect, entity.MiddlePoint, Quaternion.identity);
42:         }
43:         if ( damageText != null )
44:         {
45:             UIDamageText clone = Instantiate(damageText, entity.MiddlePoint,
                    Quaternion.identity);
46:             clone.Setup(damage.ToString("F0"), Color.white);
47:         }
48:
49:         entity.TakeDamage(damage);
50:     }
51: }
```

06 같은 폴더에 블리자드 공격 발사체를 제어하는 ProjectileBlizzard.cs 스크립트를 생성합니다.

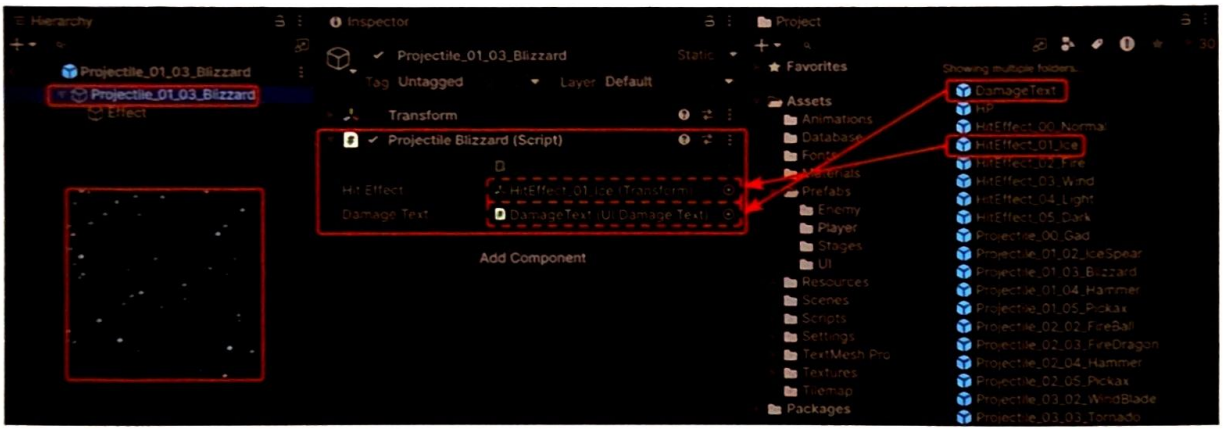
Assets/Scripts/Skill/Projectile/ProjectileBlizzard.cs

코드 07-12 블리자드 공격 발사체 제어

```
01: using UnityEngine;
02:
03: public class ProjectileBlizzard : ProjectileGlobal
04: {
05:     public override void Process()
06:     {
07:         base.Process();
08:
09:         // 눈보라 효과가 화면 전체에 계속 출력되도록 눈보라 위치를
               플레이어 위치와 동일하게 설정
10:         transform.position = skillBase.Owner.transform.position;
11:
```

```
12:         // AttackRate 시간마다 월드에 있는 모든 적 공격
13:         if ( Time.time - currentAttackRate >
                skillBase.GetStat(StatType.AttackRate).Value )
14:         {
15:             for ( int i = 0; i < EnemySpawner.Enemies.Count; ++ i )
16:             {
17:                 if ( EnemySpawner.Enemies[i] == null ) continue;
18:
19:                 TakeDamage(EnemySpawner.Enemies[i]);
20:             }
21:
22:             currentAttackRate = Time.time;
23:         }
24:     }
25: }
```

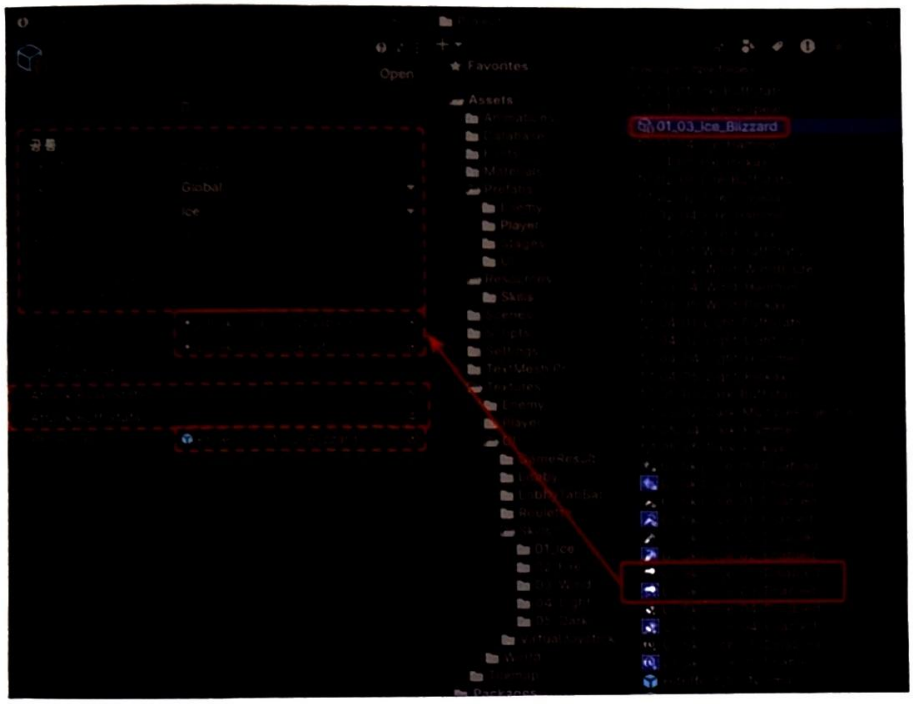
07 Projectile_01_03_Blizzard 프리팹에 Projectile Blizzard 컴포넌트를 추가합니다. Projectile Blizzard 컴포넌트의 Hit Effect 변수에는 HitEffect_01_Ice 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록합니다.



Projectile_01_03_Blizzard 프리팹에 컴포넌트 추가하고 설정하기

08 얼음 속성 주 공격 스킬 2의 SO 데이터를 저장하고자 Project View의 + ▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '01_03_Ice_Blizzard'로 변경합니다. 블리자드 스킬의 Skill Name은 'Blizzard'로, Skill Type은 [Global]로, Element는 [Ice]로, Max Level은 '99'로 설정한 후 Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택하고 나서 Attack Base Stats 크기를 '5', Attack Buff Stats 크기를 '4'로 설정합니다.

마지막으로, 블리자드 발사체로 사용할 Projectile_01_03_Blizzard 프리팹을 Projectile 변수에 등록합니다.



블리자드 스킬 에셋 생성하고 설정하기

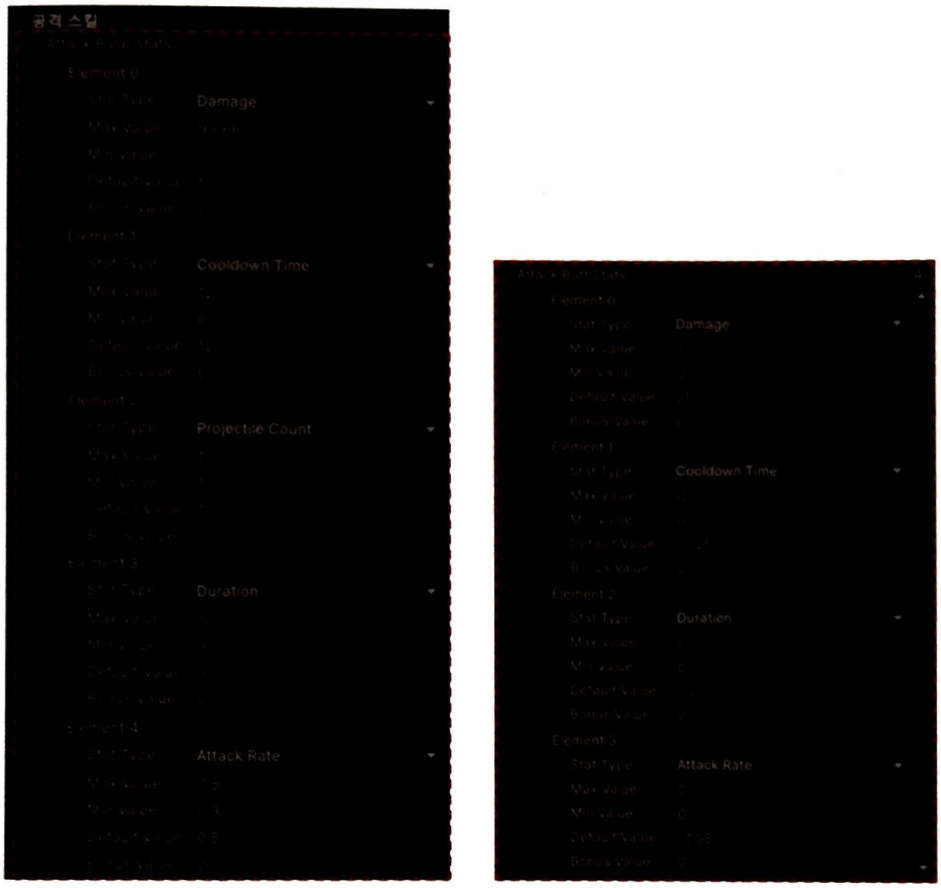
09 Attack Base Stats 리스트와 Attack Buff Stats 리스트에 다음 표처럼 값을 입력합니다.

Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	50	50	0
1	Cooldown Time	12	6	12	0
2	Projectile Count	1	1	1	0
3	Duration	5	3	3	0
4	Attack Rate	0.5	0.3	0.5	0

Attack Buff Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	25	0
1	Cooldown Time	0	0	-0.25	0
2	Duration	0	0	0.2	0
3	Attack Rate	0	0	-0.05	0



블리자드 스킬 에셋의 Attack Base Stats, Attack Buff Stats 스탯 설정

10 Assets 〉 Scripts 〉 Skill 폴더에 광역 공격 스킬을 제어하는 SkillGlobal.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/SkillGlobal.cs

코드 07-13 광역 공격 스킬 제어

```
01: using UnityEngine;
02:
03: public class SkillGlobal : SkillBase
04: {
05:     public override void OnLevelUp()
06:     {
07:         // 레벨이 0에서 1로 늘 때는 스탯이 증가하지 않도록 설정
08:         if ( currentLevel <= 1 ) return;
09:
10:         // 공격 스킬 레벨업 시 공격력 등 스탯 갱신
11:         skillTemplate.attackBuffStats.ForEach(stat =>
12:         {
13:             GetStat(stat).BonusValue += stat.DefaultValue;
```

```
14:         });
15:     }
16:
17:     public override void OnSkill()
18:     {
19:         // 스킬이 사용할 수 있는 상태인지 검사(쿨타임)
20:         if ( isSkillAvailable == true )
21:         {
22:             // Global 스킬 발사체 생성
23:             GameObject projectile = GameObject.Instantiate(skillTemplate.projectile,
                    spawnPoint.position, Quaternion.identity);
24:             projectile.GetComponent<ProjectileGlobal>().
                    Setup(this, GetStat(StatType.Damage).Value);
25:
26:             isSkillAvailable = false;
27:             currentCooldownTime = Time.time;
28:         }
29:     }
30: }
```

11 광역 스킬, 즉 Global일 때는 SkillGlobal 메모리를 할당하고자 SkillSystem.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillSystem.cs

코드 07-14 광역 스킬 메모리 할당 처리

```
    (... 생략 ...)
05: public class SkillSystem : MonoBehaviour
06: {
        (... 생략 ...)
16:     private void Awake()
17:     {
            (... 생략 ...)
23:         foreach ( var item in skillDict )
24:         {
                (... 생략 ...)
28:             else if ( item.Value.skillType.Equals(SkillType.Sustained) )
                    skill = new SkillSustained();
29:             else if ( item.Value.skillType.Equals(SkillType.Global) )   ← 추가
                    skill = new SkillGlobal();
                (... 생략 ...)
```

```
35:         }
36:     }
        (... 생략 ...)
112:}
```

12 플레이 버튼 ▶을 눌러 게임을 실행하고 블리자드 스킬을 배우면 스킬을 사용할 수 있을 때마다 플레이어 위치를 기준으로 눈보라 효과를 재생하고 월드에 있는 모든 적을 공격합니다.

플레이어가 블리자드 스킬로 공격한 결과 화면

Do it! 실습 파이어 드래곤 공격 더하기

01 Assets 〉 Scripts 〉 Skill 〉 Projectile 폴더에 파이어 드래곤 공격 발사체를 제어하는 ProjectileFireDragon.cs 스크립트를 생성합니다.

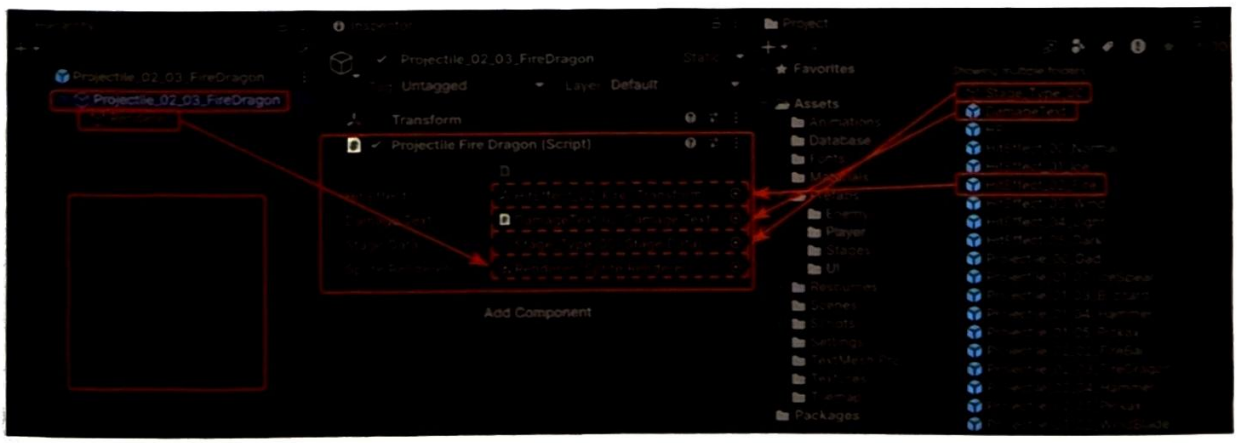
Assets/Scripts/Skill/Projectile/ProjectileFireDragon.cs

코드 07-15 파이어 드래곤 공격 발사체 제어

```
01: using System.Collections.Generic;
02: using UnityEngine;
03:
04: public class ProjectileFireDragon : ProjectileGlobal
05: {
06:     [SerializeField]
07:     private StageData stageData;
08:     [SerializeField]
09:     private SpriteRenderer spriteRenderer;
10:
11:     private float weight = 14f;           // 이미지 크기에 따라 수정
12:     private float distancePerSecond = 20; // 초당 이동 거리
13:     private float moveTime;               // 이동 시간
14:     private float t = 0f;
15:     private Vector3 start, end;
16:     private List<EntityBase> entities;
17:
18:     public override void Setup(SkillBase skillBase, float damage)
19:     {
20:         base.Setup(skillBase, damage);
21:         // 파이어 드래곤 이미지의 시작 위치와 목표 위치 설정
22:         start = new Vector3(0, stageData.CameraLimitMin.y - weight, 0);
23:         end = new Vector3(0, stageData.CameraLimitMax.y + weight, 0);
24:         transform.position = start;
25:         // 이동 시간 계산(맵 크기가 다르더라도 같은 속도로 이동)
26:         moveTime = Vector3.Distance(end, start) / distancePerSecond;
27:         // 현재 월드에 있는 모든 적 목록을 entities에 저장
28:         entities = new List<EntityBase>(EnemySpawner.Enemies);
29:         // entities를 y 위칫값이 낮은 순으로 정렬
30:         entities.Sort((a, b) =>
                a.transform.position.y.CompareTo(b.transform.position.y));
31:     }
32:
33:     public override void Process()
34:     {
35:         if ( t < 1 )
36:         {
37:             t += Time.deltaTime / moveTime;
38:             transform.localPosition = Vector3.Lerp(start, end, t);
39:         }
40:         else
41:         {
```

```
42:             // 필드에 있는 적을 공격하고 있지만 이미지가 목표에 도달했으므로 이미지만 비활성화
43:             spriteRenderer.enabled = false;
44:             // 필드에 있는 모든 적을 공격했으면 파이어 드래곤 제거
45:             if ( entities.Count == 0 ) Destroy(gameObject);
46:         }
47:
48:         // 공격 가능한 적이 없으면 나머지 코드는 처리하지 않음
49:         if ( entities.Count == 0 ) return;
50:
51:         // 현재 entities가 y 위치 기준으로 정렬되었으므로
52:         // entities의 첫 번째 요소에 대해서만 파이어 드래곤 위치와 비교하여 처리
53:         if ( entities[0] == null ) entities.RemoveAt(0);
54:         else
55:         {
56:             if ( entities[0].transform.position.y <= transform.position.y )
57:             {
58:                 TakeDamage(entities[0]); // entities[0] 적을 공격하고
59:                 entities.RemoveAt(0);    // 공격했으면 리스트에서 제외
60:             }
61:         }
62:     }
63: }
```

02 Projectile_02_03_FireDragon 프리팹에 Projectile Fire Dragon 컴포넌트를 추가합니다. Projectile Fire Dragon 컴포넌트의 Hit Effect 변수에는 HitEffect_02_Fire 프리팹을, Damage Text 변수에는 DamageText 프리팹을, Stage Data 변수에는 Stage_Type_00 에셋을, Sprite Renderer 변수에는 Renderer 오브젝트를 등록합니다.



Projectile_02_03_FireDragon 프리팹에 컴포넌트 추가하고 설정하기

03 불 속성 주 공격 스킬 2의 SO 데이터를 저장하고자 Project View의 ➕▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '02_03_Fire_FireDragon'으로 변경합니다. 파이어 드래곤 스킬의 Skill Name은 'Fire Dragon'으로, Skill Type은 [Global]로, Element는 [Fire]로, Max Level은 '99'로 설정한 후 Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택합니다. 그리고 Attack Base Stats 크기는 '3'으로, Attack Buff Stats 크기는 '2'로 설정하고 다음 표처럼 값을 입력합니다.

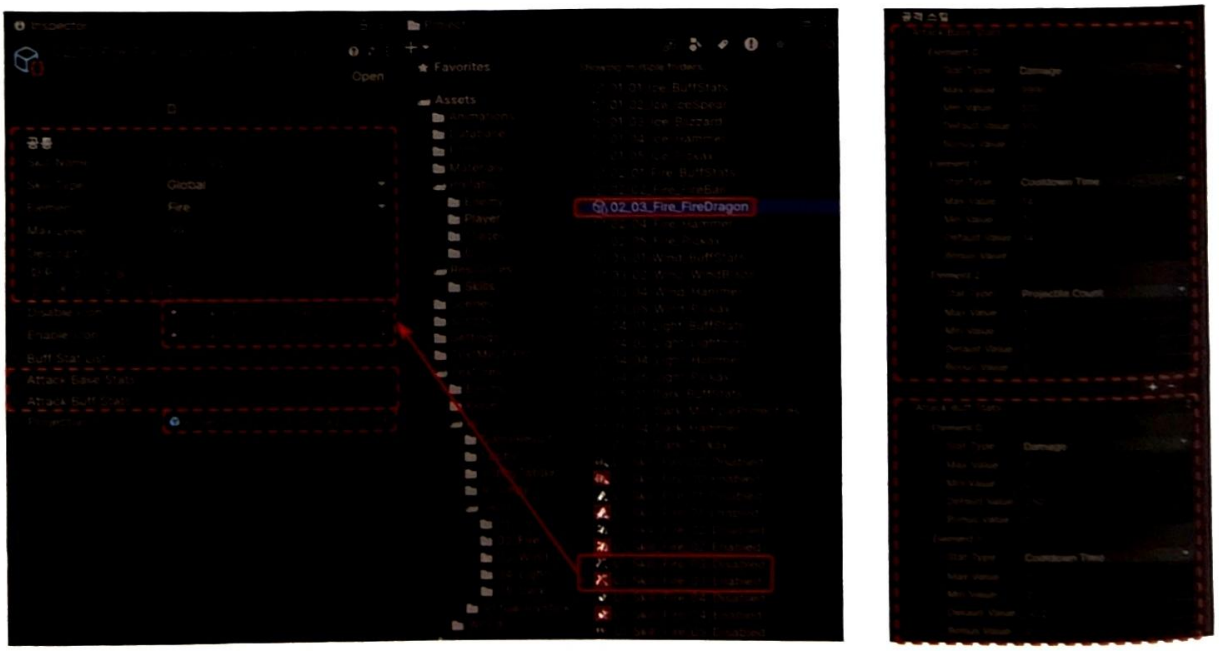
Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	500	500	0
1	Cooldown Time	14	7	14	0
2	Projectile Count	1	1	1	0

Attack Buff Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	250	0
1	Cooldown Time	0	0	-0.2	0

마지막으로, 파이어 드래곤 발사체로 사용할 Projectile_02_03_FireDragon 프리팹을 Projectile 변수에 등록합니다.



파이어 드래곤 스킬 에셋 생성하고 설정하기

04 플레이 버튼 ▶을 눌러 게임을 실행하고 파이어 드래곤 스킬을 배우면 스킬을 사용할 수 있을 때마다 맵 아래쪽에서 위쪽으로 용 그림자가 지나가며 월드에 있는 모든 적을 차례대로 공격합니다.

플레이어가 파이어 드래곤 스킬로 공격한 결과 화면

Do it! 실습 토네이도 공격 더하기

01 Assets 〉 Scripts 〉 Skill 〉 Projectile 폴더에 토네이도 공격 발사체를 제어하는 ProjectileTornado.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/Projectile/ProjectileTornado.cs

코드 07-16 토네이도 공격 발사체 제어

```
01: using UnityEngine;
02:
03: public class ProjectileTornado : ProjectileGlobal
04: {
05:     [SerializeField]
06:     private float attackRadius = 2f; // 토네이도 공격 범위 반지름
07:     [SerializeField]
08:     private LayerMask targetLayer;
09:
```

```
10:     private EntityBase target;
11:
12:     public override void Setup(SkillBase skillBase, float damage)
13:     {
14:         base.Setup(skillBase, damage);
15:         // 토네이도는 필드를 이동하므로 MovementRigidbody2D 컴포넌트 사용
16:         movementRigidbody2D = GetComponent<MovementRigidbody2D>();
17:         // 토네이도의 첫 번째 목표 설정
18:         target = skillBase.Owner.Target;
19:     }
20:
21:     public override void Process()
22:     {
23:         base.Process();
24:
25:         if ( target == null )
26:         {
27:             SearchClosestTarget();
28:             return;
29:         }
30:
31:         // 목표(target) 위치로 이동
32:         movementRigidbody2D.MoveTo((target.MiddlePoint -
                transform.position).normalized);
33:
34:         // AttackRate 시간마다 토네이도 범위(attackRadius)의 모든 적 공격
35:         if ( Time.time - currentAttackRate >
                skillBase.GetStat(StatType.AttackRate).Value )
36:         {
37:             Collider2D[] entities = Physics2D.OverlapCircleAll(transform.position,
                    attackRadius, targetLayer);
38:             for ( int i = 0; i < entities.Length; ++ i )
39:             {
40:                 if ( entities[i].TryGetComponent<EntityBase>(out var entity) )
41:                 {
42:                     TakeDamage(entity);
43:                 }
44:             }
45:
46:             currentAttackRate = Time.time;
47:         }
48:     }
49:
50:     private void SearchClosestTarget()
```

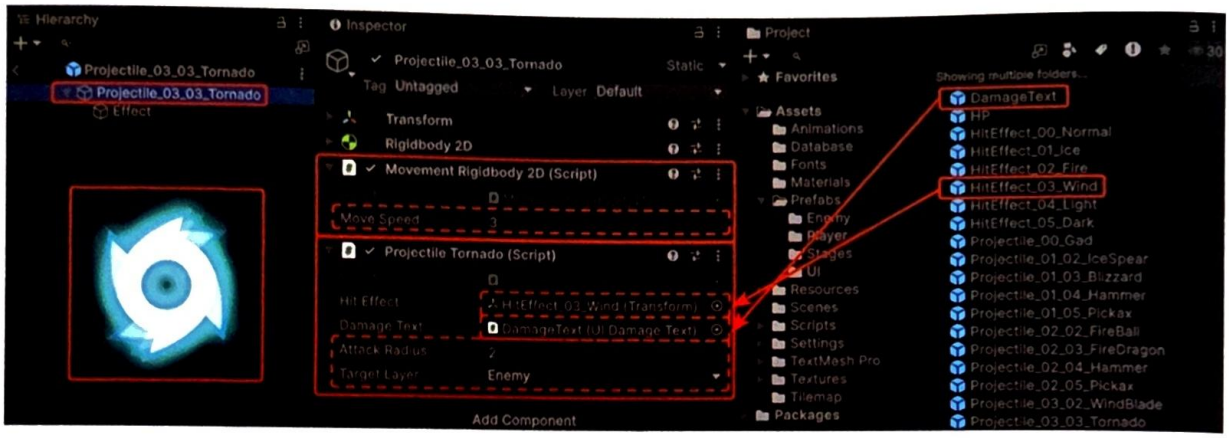
```
51:     {
52:         float closestDistSqr = Mathf.Infinity;
53:         foreach ( var entity in EnemySpawner.Enemies )
54:         {
55:             float distance = (entity.transform.position -
                    transform.position).sqrMagnitude;
56:             if ( distance < closestDistSqr )
57:             {
58:                 closestDistSqr = distance;
59:                 target = entity.GetComponent<EntityBase>();
60:             }
61:         }
62:     }
63: }
```

37줄 토네이도 발사체 위치에서 반지름(attackRadius)이 2인 원을 생성하고 해당 원과 충돌하는 적 오브젝트들의 Collider2D 정보를 entities 배열에 저장합니다.

38~44줄 entities 배열에 저장된 모든 적의 TakeDamage() 메서드를 호출해서 체력을 감소시킵니다.

50~62줄 SearchClosestTarget() 메서드에서 월드에 있는 모든 적(EnemySpawner.Enemies)을 탐색하여 그중에서 가장 가까운 적을 target으로 설정합니다.

02 Projectile_03_03_Tornado 프리팹에 Movement Rigidbody 2D, Projectile Tornado 컴포넌트를 추가합니다. Movement Rigidbody 2D 컴포넌트의 Move Speed는 '3'으로 설정합니다. Projectile Tornado 컴포넌트의 Hit Effect 변수에는 HitEffect_03_Wind 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Attack Radius 변숫값은 '2'로, Target Layer는 [Enemy]로 설정합니다.



Projectile_03_03_Tornado 프리팹에 컴포넌트 추가하고 설정하기

03 바람 속성 주 공격 스킬 2의 SO 데이터를 저장하고자 Project View의 + ▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '03_03_Wind_Tornado'로 변경합니다. 토네이도 스킬의 Skill Name은 'Tornado'로, Skill Type은 [Global]로, Element는 [Wind]로, Max Level은 '99'로 설정한 후 Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택합니다. 그리고 Attack Base Stats 크기는 '5'로, Attack Buff Stats 크기는 '3'으로 설정합니다.

마지막으로, 토네이도 발사체로 사용할 Projectile_03_03_Tornado 프리팹을 Projectile 변수에 등록합니다.

토네이도 스킬 에셋 생성하고 설정하기

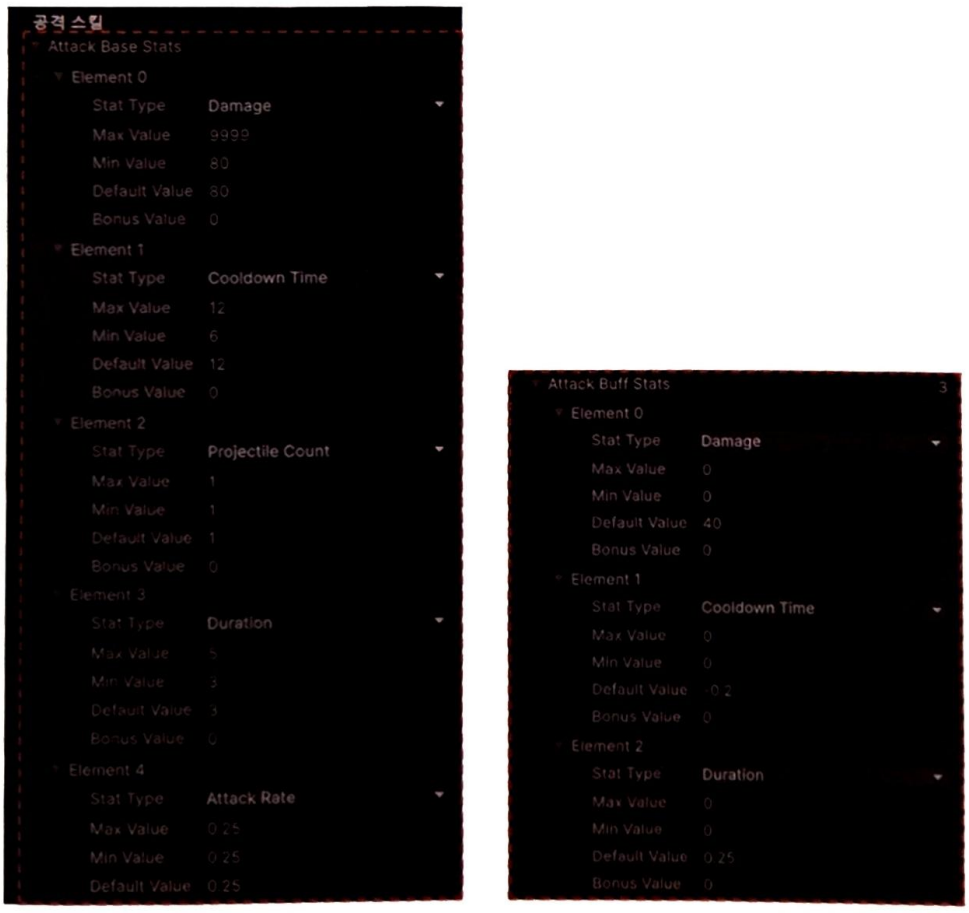
04 Attack Base Stats 리스트와 Attack Buff Stats 리스트에 다음 표처럼 값을 입력합니다.

Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	80	80	0
1	Cooldown Time	12	6	12	0
2	Projectile Count	1	1	1	0
3	Duration	5	3	3	0
4	Attack Rate	0.25	0.25	0.25	0

Attack Buff Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	40	0
1	Cooldown Time	0	0	-0.2	0
2	Duration	0	0	0.25	0



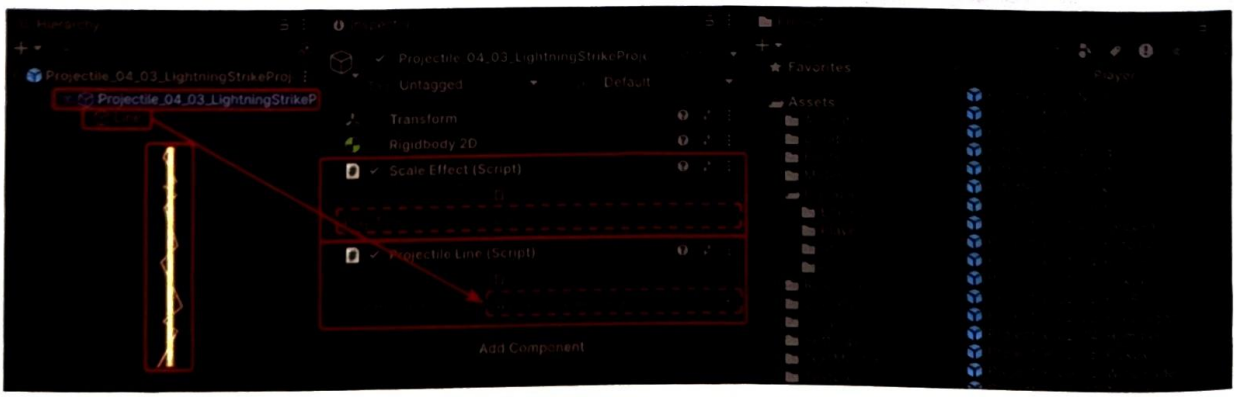
토네이도 스킬 에셋의 Attack Base Stats, Attack Buff Stats 스탯 설정하기

05 플레이 버튼 ▶을 눌러 게임을 실행하고 토네이도 스킬을 배우면 스킬을 사용할 수 있을 때마다 토네이도 발사체를 생성하고 플레이어 위치에서 가장 가까운 적 방향으로 토네이도 발사체가 이동합니다. 토네이도 발사체는 반지름이 2인 원 범위에 있는 모든 적을 공격합니다. 그리고 적을 물리치면 새로운 적을 찾아 이동합니다.

플레이어가 토네이도 스킬로 공격한 결과 화면

Do it! 실습 라이트닝 스트라이크 공격 더하기

01 Projectile_04_03_LightningStrikeProjectile 프리팹에 Scale Effect와 Projectile Line 컴포넌트를 추가합니다. Scale Effect 컴포넌트의 Play Time을 '0.1'로 설정합니다. Projectile Line 컴포넌트의 Line Renderer 변수에 Line 오브젝트를 등록합니다.



Projectile_04_03_LightningStrikeProjectile 프리팹에 컴포넌트 추가하고 설정하기

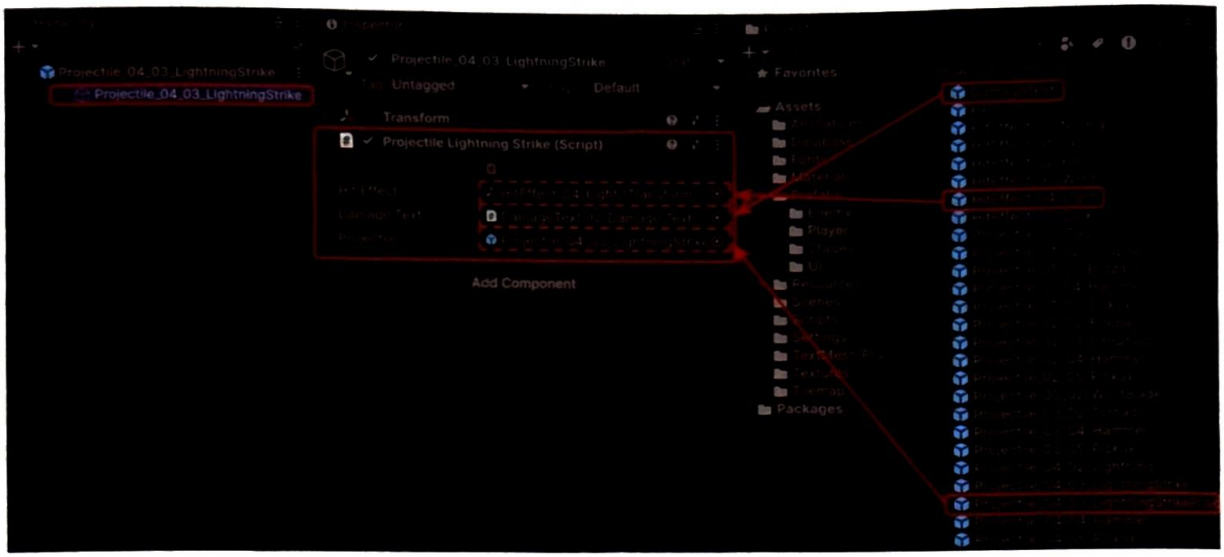
02 Assets 〉 Scripts 〉 Skill 〉 Projectile 폴더에 라이트닝 스트라이크 공격 발사체를 제어하는 ProjectileLightningStrike.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/Projectile/ProjectileLightningStrike.cs

코드 07-17 라이트닝 스트라이크 공격 발사체 제어

```
using UnityEngine;

public class ProjectileLightningStrike : ProjectileGlobal
{
    [SerializeField]
    private GameObject projectile;

    public override void Setup(SkillBase skillBase, float damage)
    {
        base.Setup(skillBase, damage);
    }

    public override void Process()
    {
        for ( int i = 0; i < EnemySpawner.Enemies.Count; ++ i )
        {
            if ( EnemySpawner.Enemies[i] == null ) continue;

            // Enemies[i] 적을 강타하는 번개 효과 생성
            Instantiate(projectile, EnemySpawner.Enemies[i].MiddlePoint,
                Quaternion.identity);
            TakeDamage(EnemySpawner.Enemies[i]);
        }

        Destroy(gameObject);
    }
}
```

03 Projectile_04_03_LightningStrike 프리팹에 Projectile Lightning Strike 컴포넌트를 추가합니다. Projectile Lightning Strike 컴포넌트의 Hit Effect 변수에는 HitEffect_04_Light 프리팹을, Damage Text 변수에는 DamageText 프리팹을, Projectile 변수에는 Projectile_04_03_LightningStrikeProjectile을 등록합니다.



Projectile_04_03_LightningStrike 프리팹에 컴포넌트 추가하고 설정하기

04 빛 속성 주 공격 스킬 2의 SO 데이터를 저장하고자 Project View의 ➕▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '04_03_Light_LightningStrike'로 변경합니다. 라이트닝 스트라이크 스킬의 Skill Name은 'Lightning Strike'로, Skill Type은 [Global]로, Element는 [Light]로, Max Level은 '99'로 설정한 후, Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택합니다. 그리고 Attack Base Stats 크기는 '3'으로, Attack Buff Stats 크기는 '2'로 설정하고 다음 표처럼 값을 입력합니다.

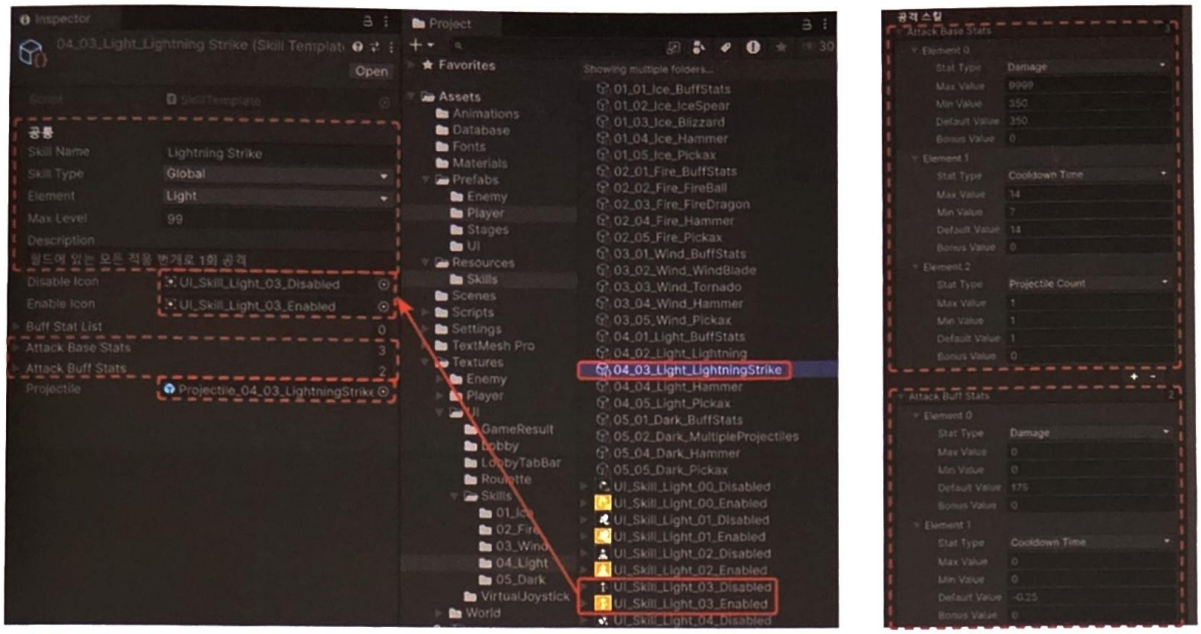
Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	350	350	0
1	Cooldown Time	14	7	14	0
2	Projectile Count	1	1	1	0

Attack Buff Stats 리스트 설정값

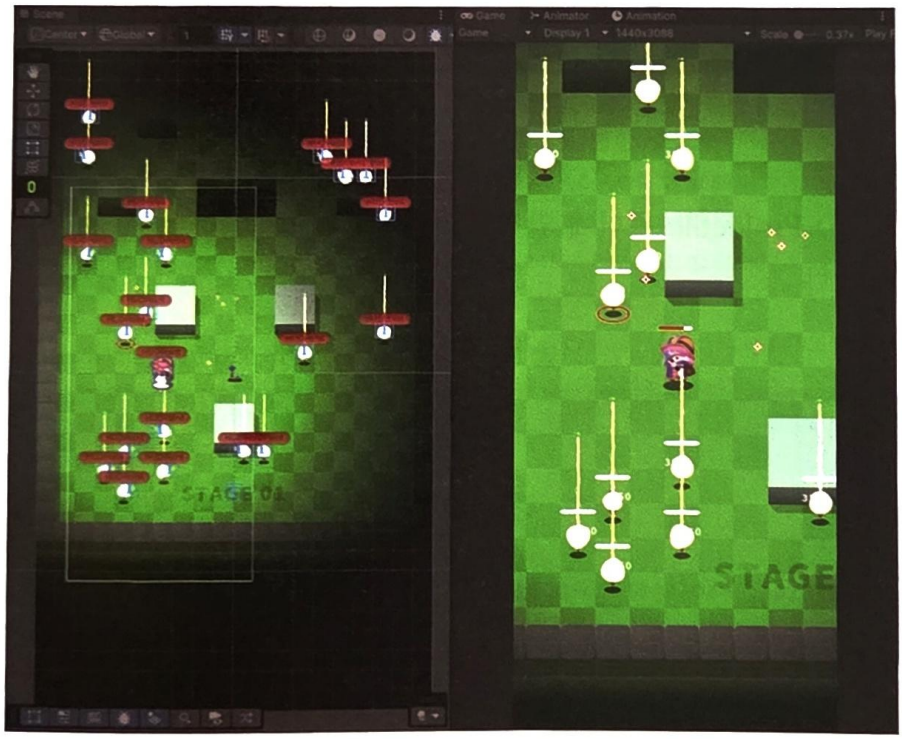
Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	175	0
1	Cooldown Time	0	0	-0.25	0

마지막으로, 라이트닝 스트라이크 발사체로 사용할 Projectile_04_03_LightningStrike 프리팹을 Projectile 변수에 등록합니다.



라이트닝 스트라이크 스킬 에셋 생성하고 설정하기

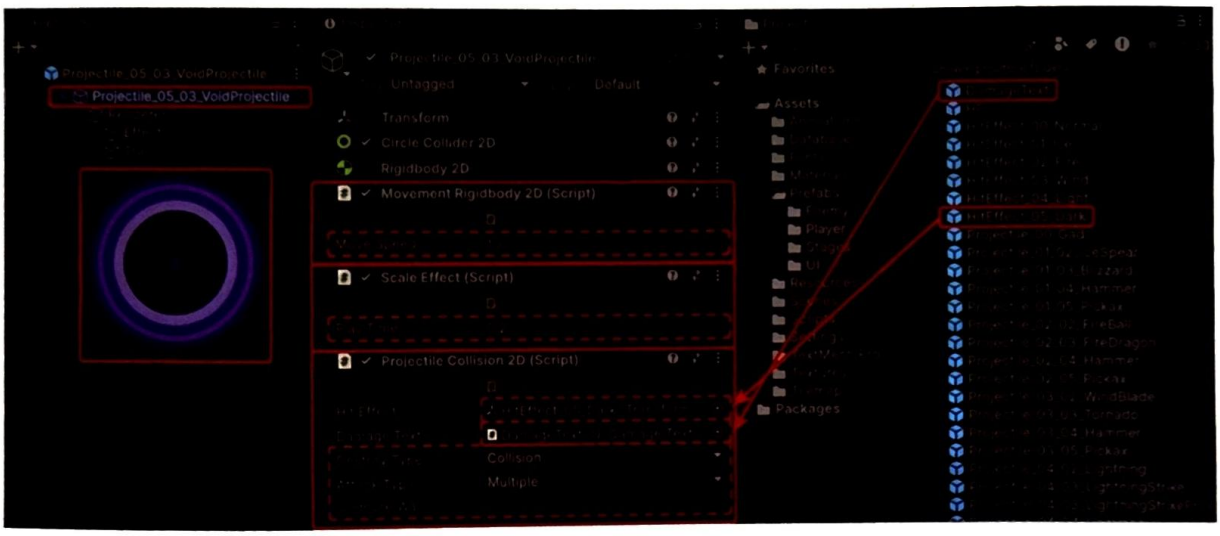
05 플레이 버튼 ▶을 눌러 게임을 실행하고 라이트닝 스트라이크 스킬을 배우면 스킬을 사용할 수 있을 때마다 월드에 있는 모든 적을 공격합니다.



플레이어가 라이트닝 스트라이크 스킬로 공격한 결과 화면

Do it! 실습 암흑 공격 더하기

01 Projectile_05_03_VoidProjectile 프리팹에 Movement Rigidbody 2D, Scale Effect, Projectile Collision 2D 컴포넌트를 추가합니다. Movement Rigidbody 2D 컴포넌트의 Move Speed는 '10'으로, Scale Effect 컴포넌트의 Play Time은 '0.2'로 설정합니다. Projectile Collision 2D 컴포넌트의 Hit Effect 변수에는 HitEffect_05_Dark 프리팹을, Damage Text 변수에는 DamageText 프리팹을 등록한 후 Destroy Type은 [Collision]으로, Attack Type은 [Multiple]로, Is Ignore Wall은 false로 설정합니다.



Projectile_05_03_VoidProjectile 프리팹에 컴포넌트 추가하고 설정하기

02 Assets 〉 Scripts 〉 Skill 〉 Projectile 폴더에 암흑 공격 발사체를 제어하는 Projectile Void.cs 스크립트를 생성합니다.

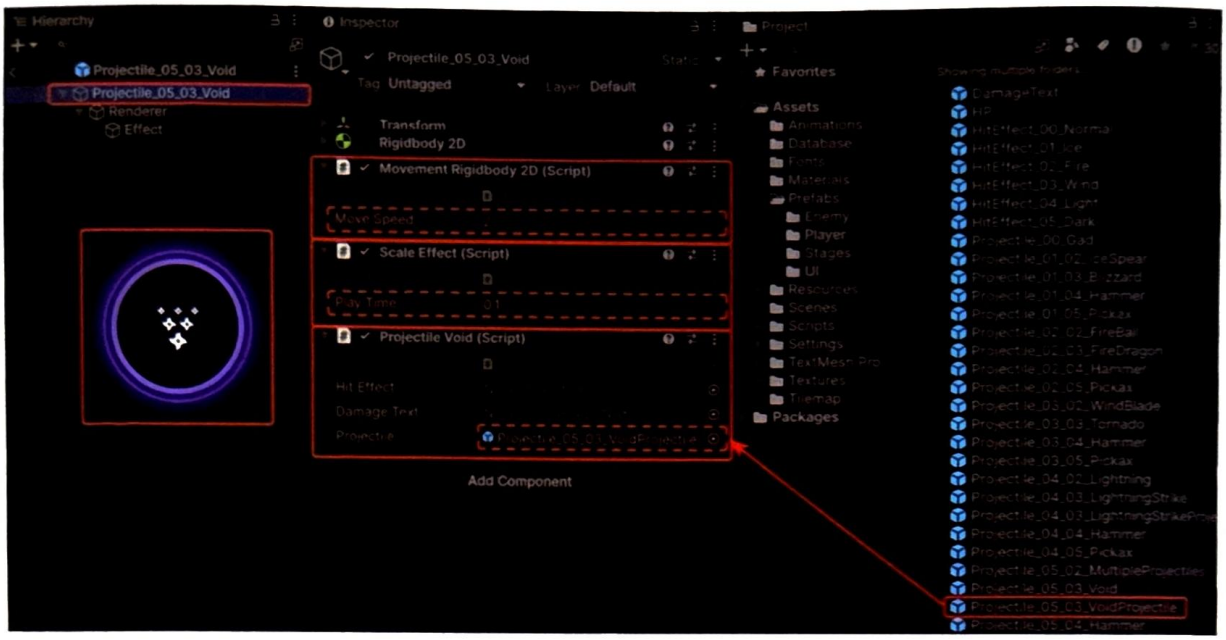
Assets/Scripts/Skill/Projectile/ProjectileVoid.cs

코드 07-18 암흑 공격 발사체 제어

```
01: using UnityEngine;
02:
03: public class ProjectileVoid : ProjectileGlobal
04: {
05:     [SerializeField]
06:     private GameObject projectile;
07:
08:     private float radius = 1f;
09:
10:     public override void Setup(SkillBase skillBase, float damage)
11:     {
```

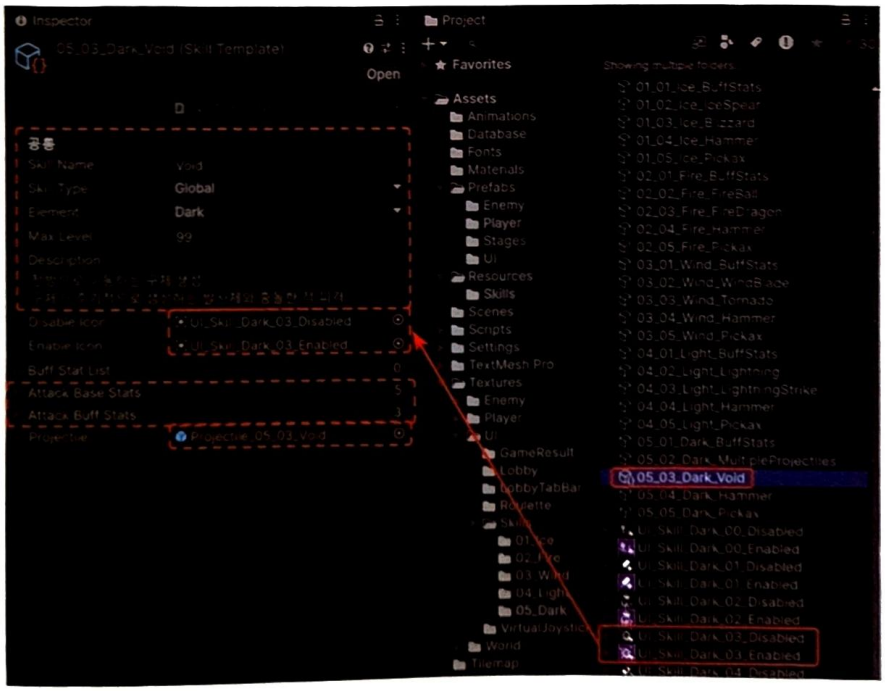
```
12:         base.Setup(skillBase, damage);
13:         // 암흑 공격 본체 이동 방향 설정
14:         Vector3 targetPosition = skillBase.Owner.Target.MiddlePoint;
15:         movementRigidbody2D = GetComponent<MovementRigidbody2D>();
16:         movementRigidbody2D.MoveTo((targetPosition -
                transform.position).normalized);
17:     }
18:
19:     public override void Process()
20:     {
21:         base.Process();
22:
23:         if ( Time.time - currentAttackRate >
                skillBase.GetStat(StatType.AttackRate).Value )
24:         {
25:             for ( int i = 0;
                    i < skillBase.GetStat(StatType.ProjectileCount).Value; ++ i )
26:             {
27:                 // 암흑 공격 발사체 생성과 위치 설정
28:                 float angle =
                        360 / skillBase.GetStat(StatType.ProjectileCount).Value * i;
29:                 Vector3 position = transform.position +
                        Utils.GetPositionFromAngle(radius, angle);
30:                 GameObject clone = Instantiate(projectile, position,
                        Quaternion.identity);
31:                 // 암흑 공격 발사체 이동 방향 설정
32:                 Vector3 direction = (clone.transform.position -
                        transform.position).normalized;
33:                 clone.GetComponent<MovementRigidbody2D>().MoveTo(direction);
34:                 // 암흑 공격 발사체 공격력 설정
35:                 clone.GetComponent<ProjectileCollision2D>().Setup(
                        skillBase.Owner.Target, damage);
36:             }
37:
38:             currentAttackRate = Time.time;
39:         }
40:     }
41: }
```

03 Projectile_05_03_Void 프리팹에 Movement Rigidbody 2D, Scale Effect, Projectile Void 컴포넌트를 추가합니다. Movement Rigidbody 2D 컴포넌트의 Move Speed는 '2'로, Scale Effect 컴포넌트의 Play Time은 '0.1'로 설정합니다. Projectile Void 컴포넌트의 Projectile 변수에 Projectile_05_03_VoidProjectile 프리팹을 등록합니다.



Projectile_05_03_Void 프리팹에 컴포넌트 추가하고 설정하기

04 암흑 속성 주 공격 스킬 2의 SO 데이터를 저장하고자 Project View의 ➕▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '05_03_Dark_Void'로 변경합니다. 암흑 스킬의 Skill Name은 'Void'로, Skill Type은 [Global]로, Element는 [Dark]로, Max Level은 '99'로 설정한 후, Description에 스킬 설명을 입력합니다. 아이콘 이미지를 선택하고 나서 Attack Base Stats 크기를 '5', Attack Buff Stats 크기를 '3'으로 설정합니다. 그리고 암흑 발사체로 사용할 Projectile_05_03_Void 프리팹을 Projectile 변수에 등록합니다.



암흑 스킬 에셋 생성하고 설정하기

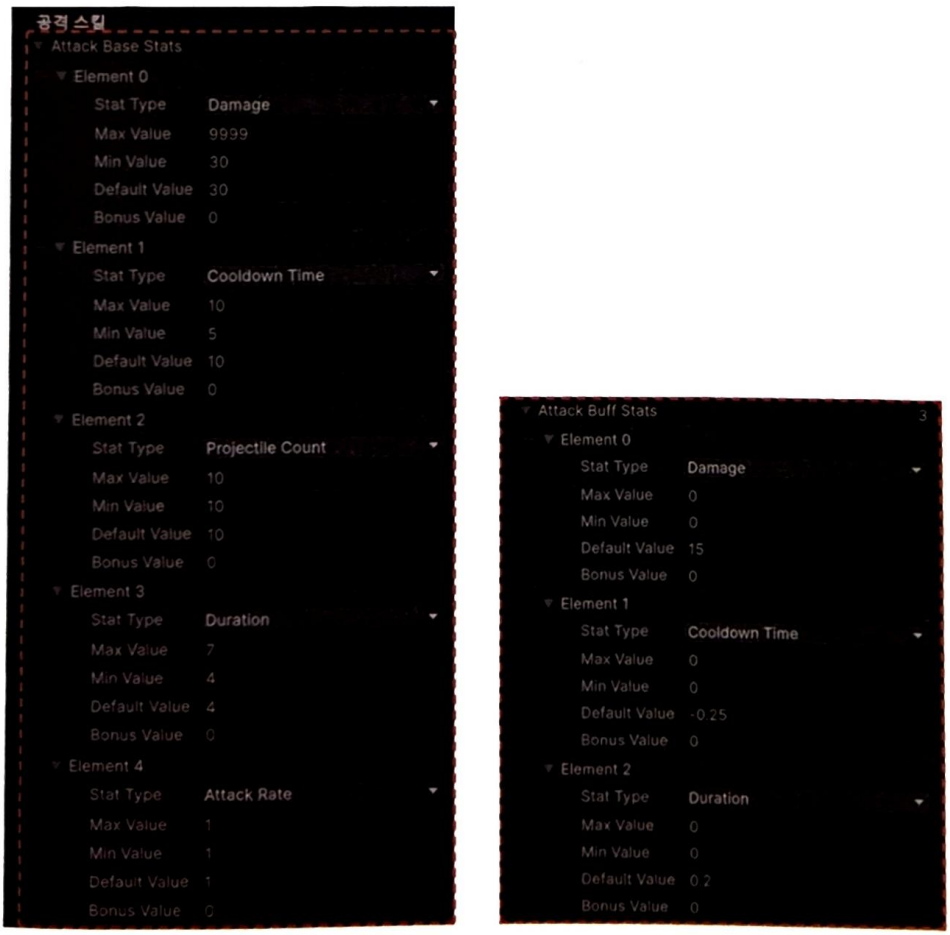
05 Attack Base Stats 리스트와 Attack Buff Stats 리스트를 다음 표처럼 설정합니다.

Attack Base Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	30	30	0
1	Cooldown Time	10	5	10	0
2	Projectile Count	10	10	10	0
3	Duration	7	4	4	0
4	Attack Rate	1	1	1	0

Attack Buff Stats 리스트 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	0	0	15	0
1	Cooldown Time	0	0	-0.25	0
2	Duration	0	0	0.2	0



암흑 스킬 에셋의 Attack Base Stats, Attack Buff Stats 스탯 설정하기

06 플레이 버튼 ▶을 눌러 게임을 실행하고 암흑 스킬을 배우면 스킬을 사용할 수 있을 때마다 암흑 본체를 생성하고 1초 주기로 10개의 암흑 발사체를 원형으로 발사합니다. 암흑 본체는 적을 공격하지 않고 이에서 생성된 암흑 발사체만 적과 충돌했을 때 공격합니다.

플레이어가 암흑 스킬로 공격한 결과 화면

08장

속성 보너스 스킬 구현하고 스킬 UI 제작하기

이번 장에서는 각 속성의 스킬 레벨 총합이 3의 배수가 될 때마다 스킬 레벨이 1씩 자동으로 증가하는 속성 보너스 스킬을 만듭니다. 게임 화면에 속성별 공격/버프 스킬 정보를 출력하는 UI를 추가하고 속성 보너스 스킬은 따로 출력합니다. 스킬 선택 UI는 습득할 수 있는 스킬 3개를 표시하고 그중에 하나를 선택하면 해당 스킬을 습득 또는 레벨업 합니다.

학습 목표

1. 조건 기반 시너지 시스템을 설계하여 스킬 자동 성장 로직을 구현할 수 있다.
2. 동적 리스트를 활용한 스킬 선택 UI를 구현할 수 있다.

08-1 스킬 레벨 합계에 따른 자동 보너스 얻기

08-2 습득 가능한 모든 스킬 목록 UI 구현하기

08-3 무작위로 스킬 배우고 레벨업 하기

08-1 스킬 레벨 합계에 따른 자동 보너스 얻기

이 절에서는 각 속성의 스킬 레벨 총합이 3의 배수가 될 때마다 스킬 레벨이 1씩 자동으로 증가하는 속성 보너스 Elemental Bonus 스킬을 만들어 봅니다.

Do it! 실습 속성 보너스 스탯과 스킬 적용하기

01 StatType 열거형에 속성 보너스를 위한 스탯 상수를 추가하도록 Stat.cs 스크립트를 수정합니다.

Assets/Scripts/Entity/Stat.cs

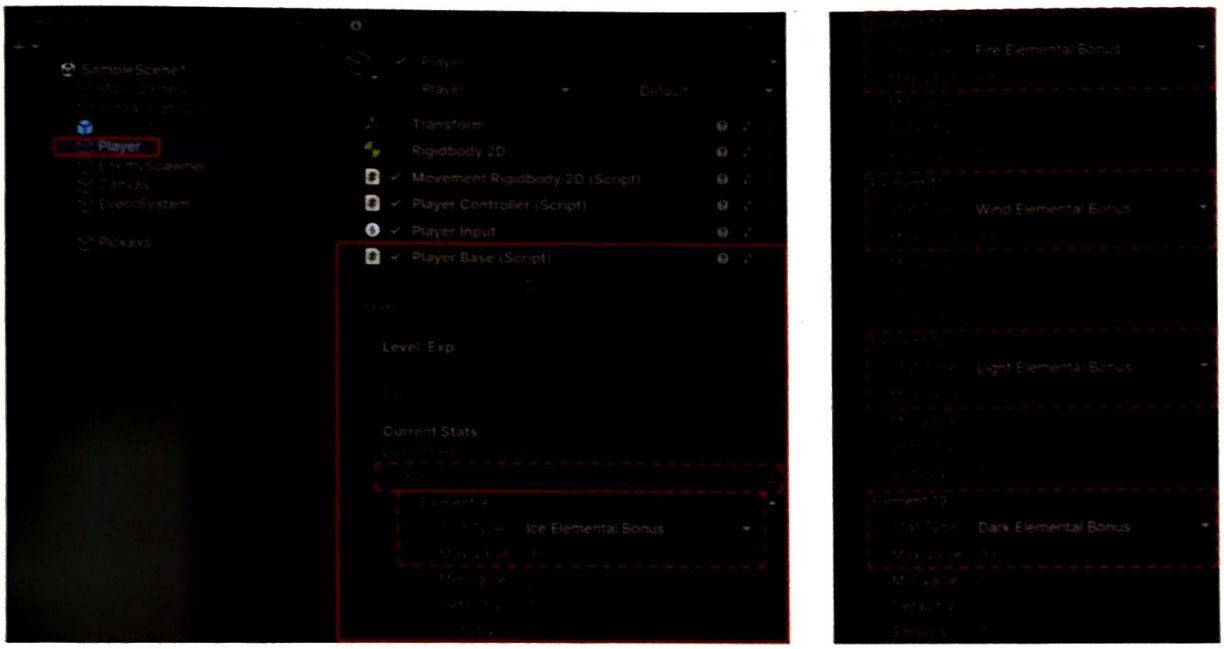
코드 08-01 속성 보너스 스탯 상수 추가

```
01: using UnityEngine;
    (... 생략 ...)
64:
65: public enum StatType { Damage = 0, CooldownTime, CriticalChance,
        CriticalMultiplier, HP, Evasion,
66:     MetastasisCount, HPRecovery, ProjectileCount, Duration, AttackRate,
67:
68:     IceElementalBonus = 100, FireElementalBonus, WindElementalBonus,
69:     LightElementalBonus, DarkElementalBonus }
```

추가

68~69줄 스킬 속성(SkillElement)에 따라 속성 추가 공격력(StatType)을 적용할 때 형 변환을 이용해 코드를 작성할 수 있도록 각 스킬 속성과 관련한 상수를 추가하고 SkillTemplate.cs 스크립트에 정의한 SkillElement의 속성별 값과 같게 설정합니다.

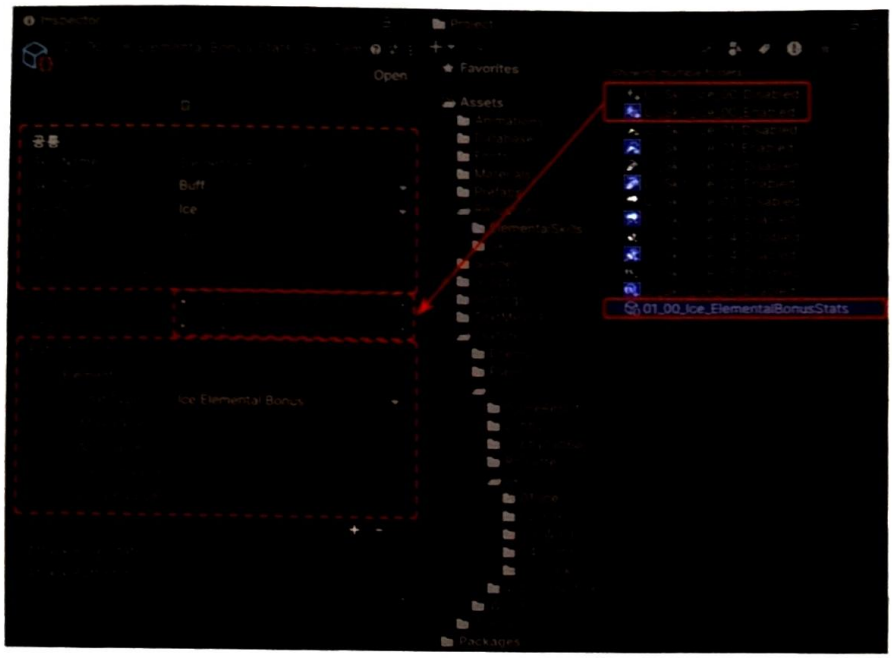
02 플레이어에게 속성 보너스 스탯을 추가하고자 Player 오브젝트의 Player Base 컴포넌트에 있는 Stats 배열의 크기를 9에서 '14'로 변경합니다. 추가한 Element 9~13의 Stat Type을 [각 속성의 Elemental Bonus]로, Max Value를 '99'로 설정합니다.



플레이어의 스탯에 속성 보너스 스탯 추가하기

03 Assets 〉 Resources 〉 ElementalSkills 폴더를 만들고 나서 이곳에 얼음 속성 보너스 스킬의 SO 데이터를 저장하고자 Project View의 +▾ 버튼을 클릭해 [SkillAsset] 메뉴로 스킬 에셋을 생성하고 이름을 '01_00_Ice_ElementalBonusStats'로 변경합니다. Skill Name은 'Elemental Bonus (Ice)'로, Skill Type은 [Buff]로 Element는 [Ice]로, Max Level은 '99'로 설정합니다. Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택한 후 Buff Stat List 크기를 '1'로 설정합니다.

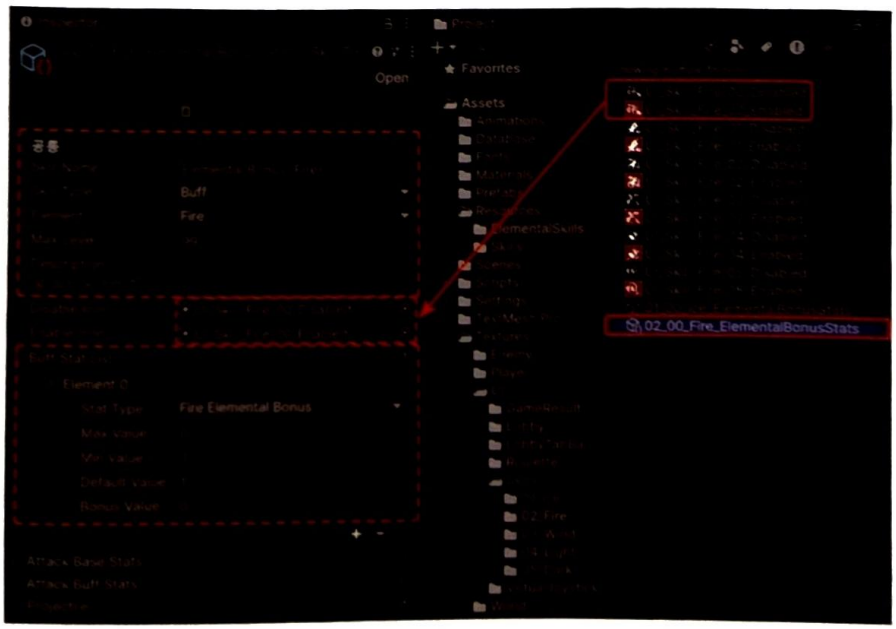
Buff Stat List 리스트에서 첫 번째 요소(Element 0)의 Stat Type은 [Ice Elemental Bonus]로, Default Value는 '0.5'로 설정합니다.



Ice Elemental Bonus 스킬 에셋 생성하고 설정하기

04 불 속성 보너스 스킬의 SO 데이터를 저장하고자 앞서와 마찬가지 방법으로 스킬 에셋을 생성하고 이름을 '02_00_Fire_ElementalBonusStats'로 변경합니다. Skill Name은 'Elemental Bonus (Fire)'로, Skill Type은 [Buff]로, Element는 [Fire]로, Max Level은 '99'로 설정합니다. Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택한 후 Buff Stat List 크기를 '1'로 설정합니다.

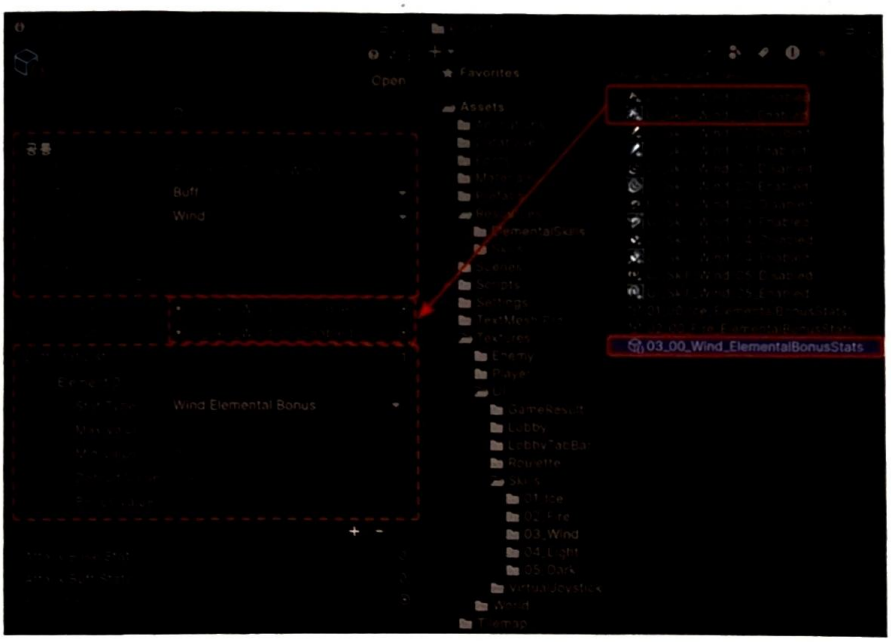
Buff Stat List 리스트에서 첫 번째 요소(Element 0)의 Stat Type은 [Fire Elemental Bonus]로, Default Value는 '1'로 설정합니다.



Fire Elemental Bonus 스킬 에셋 생성하고 설정하기

05 바람 속성 보너스 스킬의 SO 데이터를 저장하고자 스킬 에셋을 추가로 생성하고 이름을 '03_00_Wind_ElementalBonusStats'로 변경합니다. Skill Name은 'Elemental Bonus (Wind)'로, Skill Type은 [Buff]로, Element는 [Wind]로, Max Level은 '99'로 설정합니다. Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 지정한 후 Buff Stat List 크기를 '1'로 설정합니다.

Buff Stat List 리스트에서 첫 번째 요소(Element 0)의 Stat Type은 [Wind Elemental Bonus]로, Default Value는 '0.5'로 설정합니다.



Wind Elemental Bonus 스킬 에셋 생성하고 설정하기

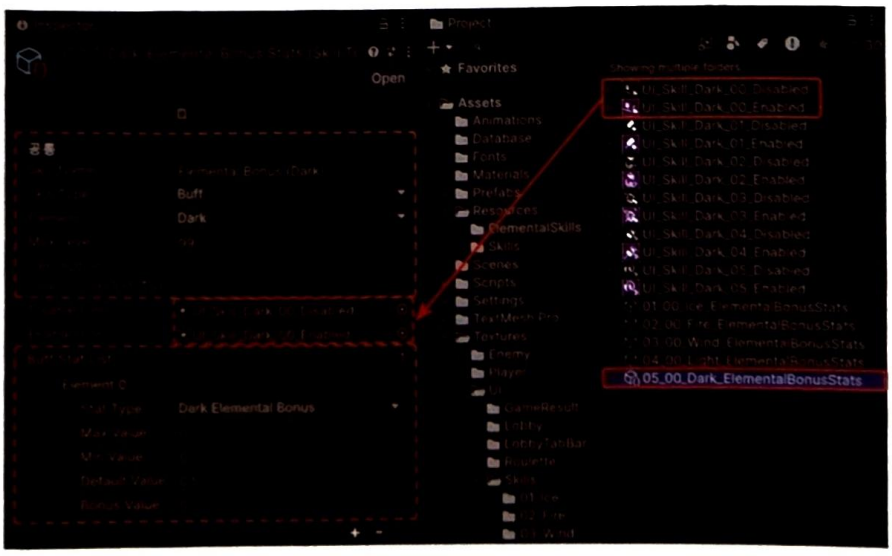
06 빛 속성 보너스 스킬의 SO 데이터를 저장하고자 스킬 에셋을 추가로 생성하고 이름을 '04_00_Light_ElementalBonusStats'로 변경합니다. Skill Name은 'Elemental Bonus (Light)'로, Skill Type은 [Buff]로, Element는 [Light]로, Max Level은 '99'로 설정합니다. Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택한 후 Buff Stat List 크기를 '2'로 설정합니다.

Buff Stat List 리스트에서 첫 번째 요소(Element 0)의 Stat Type은 [Light Elemental Bonus]로, Default Value는 '0.5'로 설정합니다. 두 번째 요소(Element 1)의 Stat Type은 [HP]로, Default Value는 '100'으로 설정합니다.

Light Elemental Bonus 스킬 에셋 생성하고 설정하기

07 어둠 속성 보너스 스킬의 SO 데이터를 저장하고자 스킬 에셋을 추가로 생성하고 이름을 '05_00_Dark_ElementalBonusStats'로 변경합니다. Skill Name은 'Elemental Bonus (Dark)'로, Skill Type은 [Buff]로, Element는 [Dark]로, Max Level은 '99'로 설정합니다. Description에 스킬 설명을 입력하고 그림처럼 아이콘 이미지를 선택한 후 Buff Stat List 크기를 '1'로 설정합니다.

Buff Stat List 리스트에서 첫 번째 요소(Element 0)의 Stat Type은 [Dark Elemental Bonus]로, Default Value는 '0.5'로 설정합니다.



Dark Elemental Bonus 스킬 에셋 생성하고 설정하기

08 속성 보너스 스킬의 레벨업 처리와 레벨 데이터를 관리하고자 SkillSystem.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillSystem.cs

코드 08-02 속성 보너스 스킬의 레벨업과 레벨 데이터 관리

```
    (... 생략 ...)
05: public class SkillSystem : MonoBehaviour
06: {
        (... 생략 ...)
14:     private Dictionary<string, SkillBase> skills =
            new Dictionary<string, SkillBase>();
15:     private Dictionary<SkillElement, int> elementalCounts =          ← 추가
            new Dictionary<SkillElement, int>();
16:     private Dictionary<SkillElement, SkillBase> elementalSkills =
            new Dictionary<SkillElement, SkillBase>();
17:
18:     private void Awake()
19:     {
            (... 생략 ...)
39:         // 속성 보너스 스킬 등록                                      ← 추가
40:         var eSkillDict = Resources.LoadAll<SkillTemplate>("ElementalSkills/").
                ToDictionary(item => item.name, item => item);
41:         foreach ( var item in eSkillDict )
42:         {
43:             SkillBase skill = new SkillBuff();
44:             skill.Setup(item.Value, owner, skillSpawnPoint);
45:
46:             elementalCounts.Add(item.Value.element, 0);     // 속성 스킬 레벨 카운트
47:             elementalSkills.Add(item.Value.element, skill); // 보너스 스킬 SkillBase
48:
49:             Logger.Log($"{item.Value.element}, {item.Value.skillName}");
50:         }
51:     }
        (... 생략 ...)
79:     public void LevelUp(SkillBase skill)
80:     {
81:         if ( skills.ContainsValue(skill) )
82:         {
83:             skill.TryLevelUp();
84:             Logger.Log($"Level Up [{skill.SkillName}]
                    {skill.Element}, Lv. {skill.CurrentLevel}");
85:
86:             // 해당 스킬이 소속된 속성의 총 스킬 레벨 합+1              ← 추가
```

```
87:             elementalCounts[skill.Element] ++;
88:             // 해당 스킬의 속성 보너스 레벨 증가 여부 판단
89:             if ( elementalCounts[skill.Element] % 3 == 0 )
90:             {
91:                 elementalSkills[skill.Element].TryLevelUp();
92:                 Logger.Log($"{skill.Element}
                        Lv. {elementalSkills[skill.Element].CurrentLevel}");
93:             }
94:         }
95:     }
        (... 생략 ...)
136:}
```

15~16줄 보너스 스킬 레벨업을 하고자 각 속성 스킬의 레벨을 세는 elementalCounts와 각 속성 보너스 스킬 정보를 저장하는 elementalSkills 변수를 선언하고 메모리를 할당합니다.

39~50줄 Assets 〉 Resources 〉 ElementalSkills 폴더에 저장된 스킬 에셋 정보를 불러와 초기화합니다.

86~93줄 스킬 레벨업을 할 때 해당 스킬이 소속된 속성의 스킬 레벨을 1 늘리고 스킬 레벨이 3의 배수이면 속성 보너스 스킬을 레벨업 합니다.

Do it! 실습 각 스킬에 공격력 적용하기

이번에는 각 스킬 공격력에 속성 보너스를 적용해 봅시다.

01 스킬의 공격력에 속성 보너스를 추가로 적용하고자 SkillBase.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillBase.cs

코드 08-03 스킬 공격력에 속성 보너스 추가 적용

```
    (... 생략 ...)
04: public abstract class SkillBase
05: {
        (... 생략 ...)
69:     protected float CalculateDamage()        ← 추가
70:     {
71:         float damage = GetStat(StatType.Damage).Value;
```

```
72:         damage += damage * owner.Stats.GetStat((StatType)Element).Value;
73:
74:         return damage;
75:     }
76:
77:     public abstract void OnLevelUp(); // 스킬 레벨업 시 1회 호출
78:     public abstract void OnSkill();   // 스킬 사용 시 호출
79: }
```

72줄 StatType 열거형과 SkillElement 열거형의 속성값을 동일하게 100, 101, 102, 103, 104로 설정했으므로 SkillElement 열거형 변수 Element를 StatType형으로 변환해 코드에서 보듯이 간단하게 사용할 수 있습니다.

참고로, 이를 switch ~ case 문으로 처리하면 다음 예시처럼 코드가 번거로워집니다.

코드 08-04 각 속성 공격력 증가를 switch-case로 처리했을 때

```
protected float CalculateDamage()
{
    float damage = GetStat(StatType.Damage).Value;

    switch ( Element )
    {
        case SkillElement.Ice:
            damage += damage * owner.Stats.GetStat(StatType.IceElementalBonus).Value;
            break;
        case SkillElement.Fire:
            damage += damage * owner.Stats.GetStat(StatType.FireElementalBonus).Value;
            break;
        case SkillElement.Wind:
            damage += damage * owner.Stats.GetStat(StatType.WindElementalBonus).Value;
            break;
        case SkillElement.Light:
            damage += damage * owner.Stats.GetStat(StatType.LightElementalBonus).Value;
            break;
        case SkillElement.Dark:
            damage += damage * owner.Stats.GetStat(StatType.DarkElementalBonus).Value;
            break;
    }
    return damage;
}
```

02 기존의 공격력 스탯 대신 **CalculateDamage()** 메서드를 호출해 Emission 스킬의 공격력을 처리하고자 SkillEmission.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillEmission.cs

코드 08-05 CalculateDamage() 메서드를 이용한 스킬 공격력 처리(Emission)

```
01: using UnityEngine;
02:
03: public class SkillEmission : SkillBase
04: {
        (... 생략 ...)
21:     public override void OnSkill()
22:     {
23:         // 스킬이 사용할 수 있는 상태인지 검사(쿨타임)
24:         if ( isSkillAvailable == true )
25:         {
26:             int maxCount = (int)GetStat(StatType.ProjectileCount).Value;
27:
28:             // attackRate 주기로 발사체 생성
29:             if ( Time.time - currentAttackRate > attackRate )
30:             {
                    (... 생략 ...)
33:                 // ProjectileStraight, ProjectileHoming도 연발은 가능하지만
                       3, 4번째 매개변수는 필요 없으므로 기존과 동일하게 처리
34:                 if ( projectile.TryGetComponent<ProjectileCubicHoming>(out var p) )
35:                     p.Setup(owner.Target, CalculateDamage(), maxCount,
                        currentProjectileCount);                          ← 수정
36:                 else if ( projectile.TryGetComponent<ProjectileQuadraticHoming>(
                        out var p2) )
37:                     p2.Setup(owner.Target, CalculateDamage(), maxCount,
                        currentProjectileCount);                          ← 수정
38:                 else projectile.GetComponent<ProjectileBase>().Setup(
                        owner.Target, CalculateDamage());                 ← 수정
39:
40:                 currentProjectileCount ++;
41:                 currentAttackRate = Time.time;
42:             }
                (... 생략 ...)
51:         }
52:     }
53: }
```

03 기존의 공격력 스탯 대신 CalculateDamage() 메서드를 호출해 Sustained 스킬의 공격력을 처리하고자 SkillSustained.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillSustained.cs

코드 08-06 CalculateDamage() 메서드를 이용한 스킬 공격력 처리(Sustained)

```
    (... 생략 ...)
04: public class SkillSustained : SkillBase
05: {
        (... 생략 ...)
18:     public override void OnLevelUp()
19:     {
            (... 생략 ...)
43:         // 모든 곡괭이의 공격력 갱신
44:         foreach ( var item in pickaxs )
45:         {
46:             item.GetComponent<ProjectileCollision2D>().Setup(null,
                    CalculateDamage());   <-- 수정
47:         }
48:     }
49:
50:     private void AddPickax(int count)
51:     {
52:         for ( int i = 0; i < count; ++ i )
53:         {
54:             GameObject clone =
                    GameObject.Instantiate(skillTemplate.projectile, parent);
55:             clone.GetComponent<ProjectileCollision2D>().Setup(null,
                    CalculateDamage());   <-- 수정
56:             pickaxs.Add(clone);
57:         }
58:     }
59:
60:     public override void OnSkill() { }
61: }
```

04 기존의 공격력 스탯 대신 CalculateDamage() 메서드를 호출해 Global 스킬의 공격력을 처리하고자 SkillGlobal.cs 스크립트를 수정합니다.

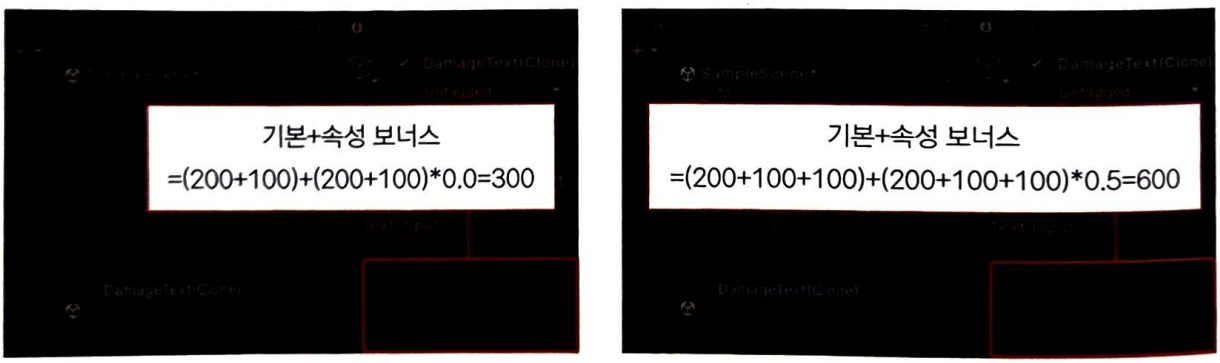
Assets/Scripts/Skill/SkillGlobal.cs

코드 08-07 CalculateDamage() 메서드를 이용한 스킬 공격력 처리(Global)

```
01: using UnityEngine;
02:
03: public class SkillGlobal : SkillBase
04: {
        (... 생략 ...)
17:     public override void OnSkill()
18:     {
19:         // 스킬이 사용할 수 있는 상태인지 검사(쿨타임)
20:         if ( isSkillAvailable == true )
21:         {
22:             // Global 스킬 발사체 생성
23:             GameObject projectile = GameObject.Instantiate(
                    skillTemplate.projectile, spawnPoint.position,
                    Quaternion.identity);
24:             projectile.GetComponent<ProjectileGlobal>().Setup(this,
                    CalculateDamage());          ← 수정
25:
26:             isSkillAvailable = false;
27:             currentCooldownTime = Time.time;
28:         }
29:     }
30: }
```

05 플레이 버튼 ▶을 눌러 게임을 실행하면 스킬을 습득해 특정 속성의 스킬 총합이 3의 배수가 될 때마다 속성 보너스 스킬의 레벨이 1씩 증가하고 속성 보너스 스킬에 의해 스킬의 공격력이 증가하는 것을 볼 수 있습니다.

다음 그림은 아이스 스피어 레벨 2, 아이스 속성 보너스 레벨 0일 때와 아이스 스피어 레벨 3, 아이스 속성 보너스 레벨 1일 때의 공격력을 비교한 것입니다.



속성 공격력을 적용한 결과 화면

08-2

습득 가능한 모든 스킬 목록 UI 구현하기

이 절에서는 모든 스킬을 출력하는 스킬 목록Skill List UI를 만들고 게임 일시 정지 기능을 구현합니다. 스킬 목록 UI는 스킬을 속성별로 정렬해서 표시하고 속성 보너스 스킬은 아래에 별도로 출력합니다. 습득 여부에 따라 스킬 아이콘을 출력하고 스킬 아이콘 오른쪽 위에 스킬 레벨을 출력해 봅시다.

Do it! 실습 스킬 아이콘 오브젝트 만들기

먼저 스킬 목록 UI에서 사용할 아이콘 오브젝트부터 만들어 볼까요?

01 Assets 〉 Scripts 〉 UI 폴더에 스킬 아이콘 UI를 제어하는 UISkillIcon.cs 스크립트를 생성합니다.

Assets/Scripts/UI/UISkillIcon.cs

코드 08-08 스킬 아이콘 UI 제어

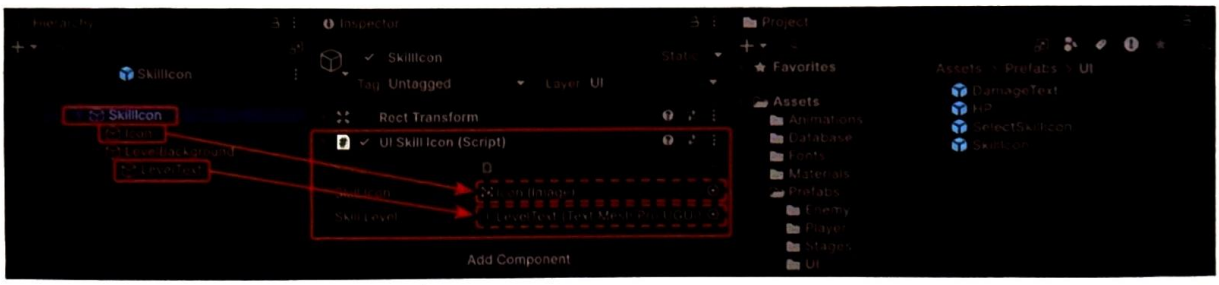
```
01: using UnityEngine;
02: using UnityEngine.UI;
03: using TMPro;
04:
05: public class UISkillIcon : MonoBehaviour
06: {
07:     [SerializeField]
08:     private Image skillIcon;
09:     [SerializeField]
10:     private TextMeshProUGUI skillLevel;
11:
12:     public void Setup(Sprite defaultSprite)
13:     {
14:         skillIcon.sprite = defaultSprite;
15:         skillLevel.text = "-";
```

```
16:     }
17:
18:     public void LevelUp(int currentLevel, Sprite activeSprite)
19:     {
20:         skillIcon.sprite = activeSprite;
21:         skillLevel.text = currentLevel.ToString();
22:     }
23: }
```

12~16줄 스킬 목록에 출력되는 스킬 아이콘은 Setup() 메서드를 호출해 각 스킬의 비활성 아이콘과 레벨 없음("-") 텍스트를 출력합니다.

18~22줄 해당 스킬을 레벨업 하면 호출되는 LevelUp() 메서드는 스킬의 활성 아이콘과 현재 레벨 텍스트를 출력합니다.

02 SkillIcon 프리팹에 UI Skill Icon 컴포넌트를 추가하고 Skill Icon 변수에는 Icon 오브젝트를, Skill Level 변수에는 LevelText 오브젝트를 등록합니다.

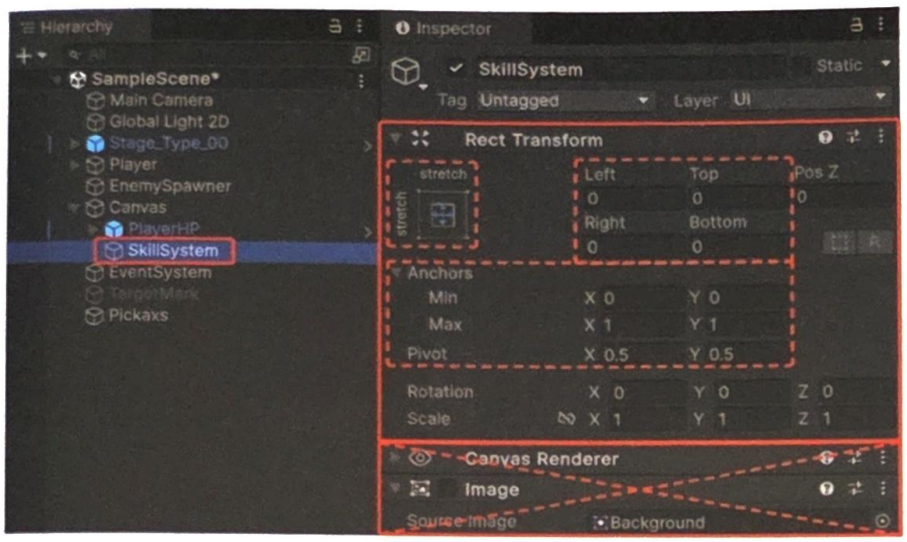


SkillIcon 프리팹에 UI Skill Icon 컴포넌트 추가하고 설정하기

Do it! 실습 스킬 목록 UI 만들기

이번에는 앞서 만든 스킬 아이콘을 표시하는 UI를 만듭니다.

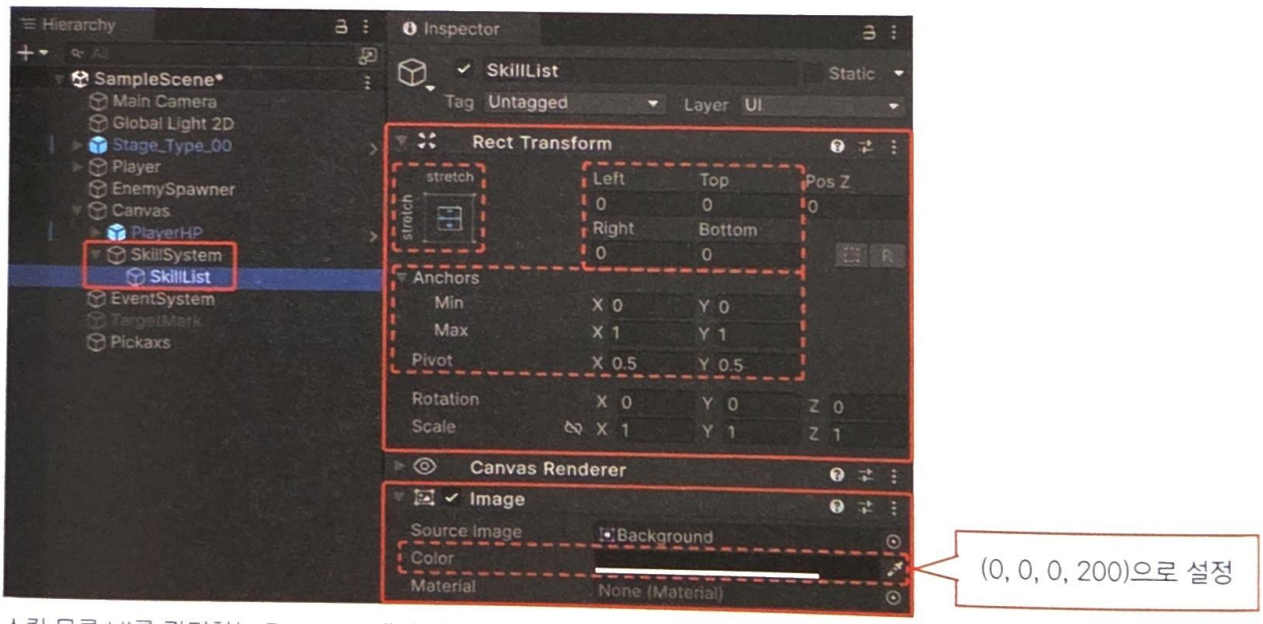
01 [GameObject → UI → Panel] 메뉴로 스킬 목록과 스킬 선택 UI를 관리하는 Panel UI를 Canvas 오브젝트의 자식으로 생성하고 이름을 'SkillSystem'으로 변경합니다.
Rect Transform 컴포넌트의 Anchor Presets은 stretch | stretch로, Rect는 (0, 0, 0, 0)으로 설정합니다. 화면에 이미지를 출력하지 않으므로 Image, Canvas Renderer 순으로 컴포넌트를 삭제합니다.



스킬 목록, 스킬 선택 UI를 관리하는 Panel UI 생성하고 설정하기

02 마찬가지 방법으로 스킬 목록 UI를 관리하는 Panel UI를 생성하고 이름을 'SkillList'로 변경한 후 SkillSystem 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Anchor Presets은 stretch | stretch로, Rect는 (0, 0, 0, 0)으로 설정합니다. Image 컴포넌트의 Color는 (0, 0, 0, 200)으로 설정합니다.



스킬 목록 UI를 관리하는 Panel UI 생성하고 설정하기

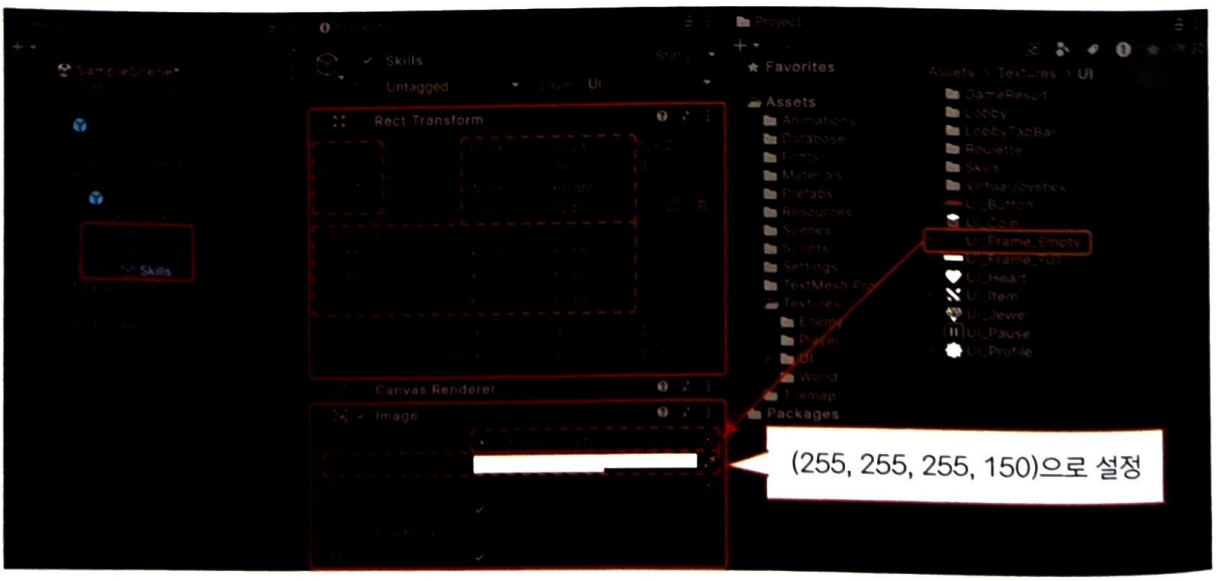
03 이번에는 [GameObject → UI → Text - TextMeshPro] 메뉴로 '스킬 목록' 텍스트를 출력하는 Text UI를 생성하고 이름을 'Theme'로 변경한 후 SkillList 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Anchor Presets은 middle | center로, Rect는 (0, 925, 1240, 150)으로 설정합니다. TextMeshPro - Text 컴포넌트의 Text Input Box에 '스킬 목록'을 입력하고 Font Asset에 NotoSansKR-Bold SDF를 등록합니다. Font Style은 [B]로, Font Size는 '100'으로, Alignment는 Center | Middle로 설정합니다.

'스킬 목록'을 출력하는 Text UI 생성하고 설정하기

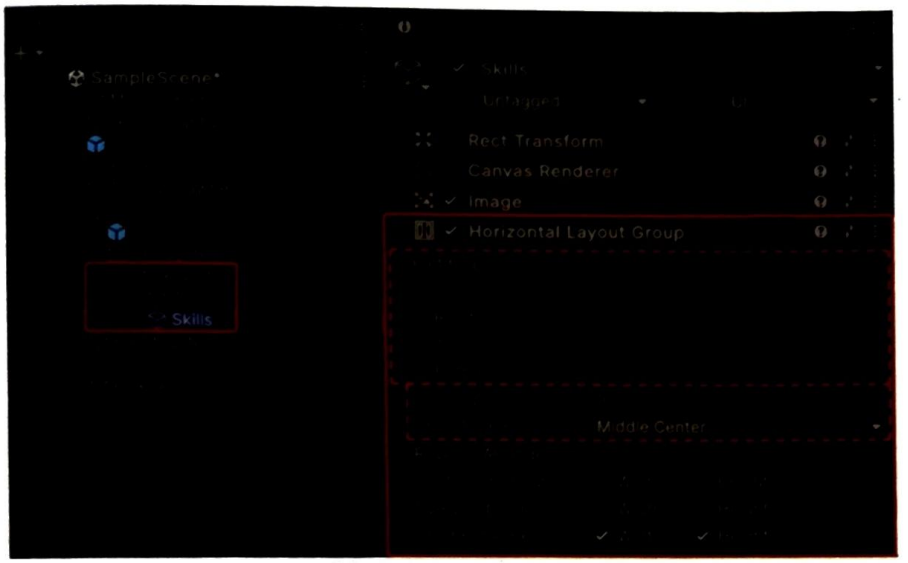
04 [GameObject → UI → Image] 메뉴로 스킬 목록을 관리하는 Image UI를 생성하고 이름을 'Skills'로 변경한 후 SkillList 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Anchor Presets은 middle | center로, Rect는 (0, 180, 1240, 1240)으로 설정합니다. Image 컴포넌트의 Source Image는 UI_Frame_Empty.png 이미지로, Color는 (255, 255, 255, 150)으로 설정합니다.



스킬 목록을 관리하는 Image UI 생성하고 설정하기

05 Skills 오브젝트에 Horizontal Layout Group 컴포넌트를 추가하고 Padding은 (30, 30, 30, 30)으로, Spacing은 '10'으로, Child Alignment는 [Middle Center]로 설정합니다.

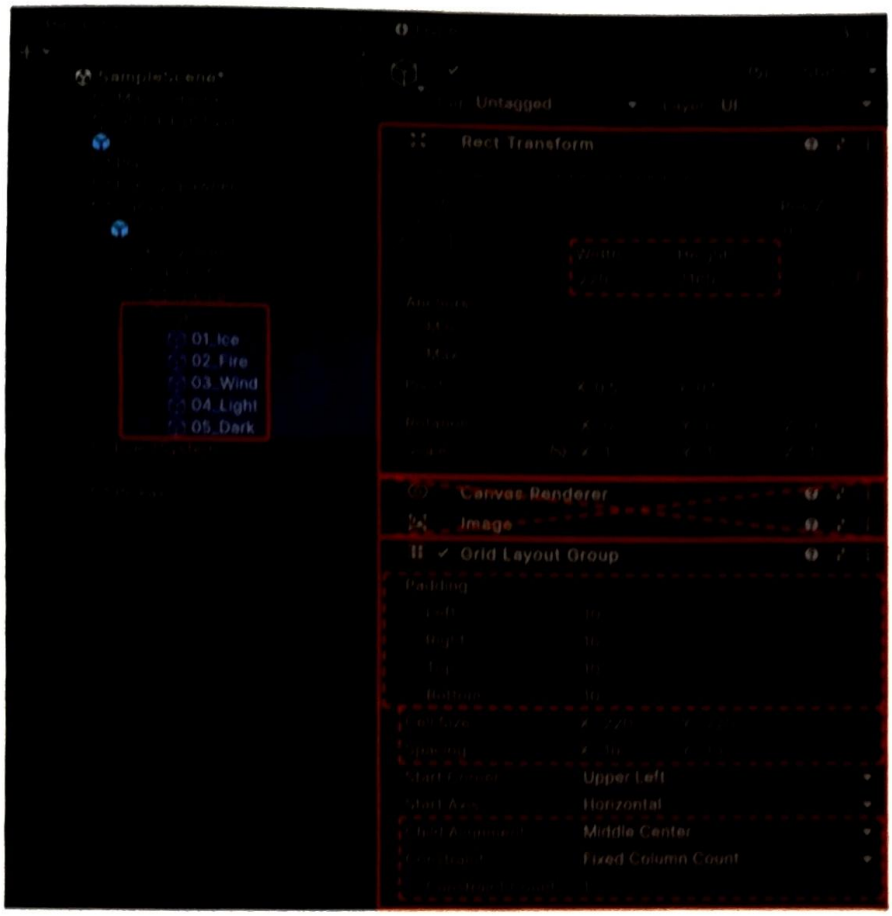


속성별 스킬 목록을 가로로 정렬하고자 Horizontal Layout Group 컴포넌트 추가하고 설정하기

06 [GameObject → UI → Image] 메뉴로 각 속성의 스킬 목록을 관리하는 Image UI를 5개 생성하고 이름을 각각 '01_Ice', '02_Fire', '03_Wind', '04_Light', '05_Dark'로 변경한 후 Skills 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Width/Height는 (220, 1160)으로 설정합니다. 화면에 이미지를 출력하지 않으므로 Image, Canvas Renderer 순으로 컴포넌트를 삭제합니다. 스킬 아이콘의 정렬과 크기를 제어하고자 Grid Layout Group 컴포넌트를 추가하고 Padding은 (10, 10, 10, 10)으로, CellSize는 (220, 220)으로, Spacing은 (10, 10)으로, Child Alignment는 [Middle Center]로, Constraint는 [Fixed Column Count]로, Constraint Count는 '1'로 설정합니다.

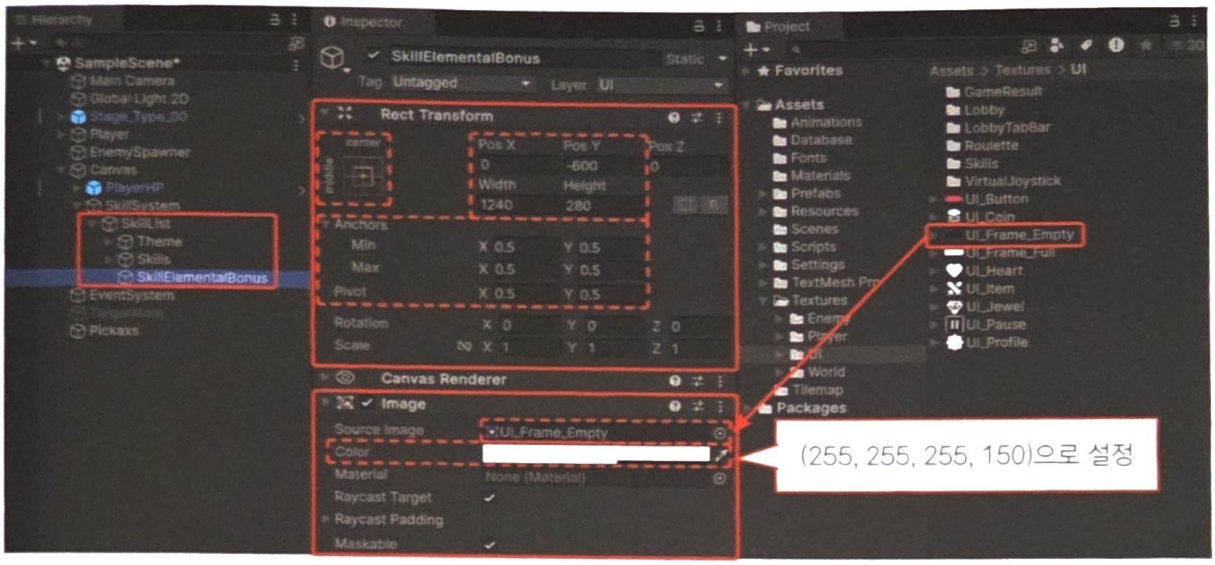
● 오브젝트를 여러 개 선택하고 입력하면 한 번에 같은 값을 설정할 수 있습니다.



각 속성의 스킬 목록을 관리하는 Image UI 생성하고 설정하기

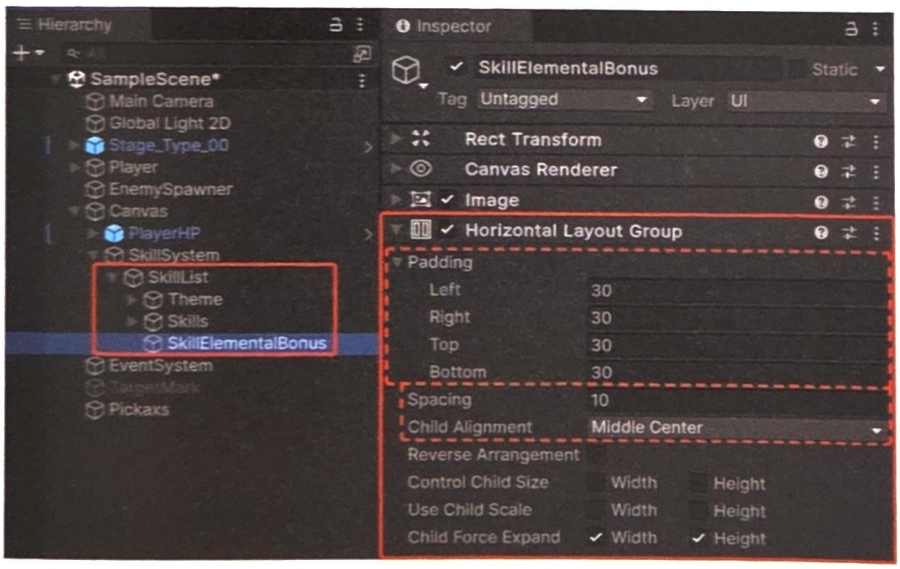
07 [GameObject → UI → Image] 메뉴로 속성 보너스 스킬 목록을 관리하는 Image UI를 생성하고 이름을 'SkillElementalBonus'로 변경한 후 SkillList 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Anchor Presets은 middle | center로, Rect는 (0, -600, 1240, 280)으로 설정합니다. Image 컴포넌트의 Source Image는 UI_Frame_Empty.png 이미지로, Color는 (255, 255, 255, 150)으로 설정합니다.



속성 보너스 스킬 목록을 관리하는 Image UI 생성하고 설정하기

08 SkillElementalBonus 오브젝트에 Horizontal Layout Group 컴포넌트를 추가하고 Padding은 (30, 30, 30, 30)으로, Spacing은 '10'으로, Child Alignment는 [Middle Center]로 설정합니다.



속성 보너스 스킬을 가로로 정렬하고자 Horizontal Layout Group 컴포넌트 추가하고 설정하기

09 외부에서 스킬 활성화 아이콘 이미지 정보를 열람하도록 SkillBase.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillBase.cs

코드 08-09 스킬 활성화 아이콘 이미지 Get 속성 정의

```
    (... 생략 ...)
04: public abstract class SkillBase
05: {
        (... 생략 ...)
20:     public Sprite EnableIcon => skillTemplate.enableIcon;
21:     public PlayerBase Owner => owner;
        (... 생략 ...)
80: }
```

추가

10 Assets 〉 Scripts 〉 UI 폴더에 스킬 목록 UI를 제어하는 UISkillList.cs 스크립트를 생성합니다.

Assets/Scripts/UI/UISkillList.cs

코드 08-10 스킬 목록 UI 제어

```
01: using System.Collections.Generic;
02: using UnityEngine;
03:
04: public class UISkillList : MonoBehaviour
05: {
06:     [SerializeField]
07:     private UISkillIcon skillIconPrefab;
08:     [SerializeField]
09:     private Transform[] skillElementType;
10:     [SerializeField]
11:     private Transform skillElementalBonus;
12:
13:     private Dictionary<string, UISkillIcon> skillIcons;
14:
15:     public void Setup(Dictionary<string, SkillTemplate> skills,
            Dictionary<string, SkillTemplate> elementalBonus)
16:     {
17:         skillIcons = new Dictionary<string, UISkillIcon>();
18:
19:         foreach ( var item in skills )
20:         {
21:             SpawnIcon(item.Value, skillElementType[(int)item.Value.element-100]);
22:         }
23:         foreach ( var item in elementalBonus )
24:         {
```

```
25:                 SpawnIcon(item.Value, skillElementalBonus);
26:             }
27:         }
28:
29:         public void LevelUp(SkillBase skill)
30:         {
31:             if ( skillIcons.ContainsKey(skill.SkillName) )
32:             {
33:                 skillIcons[skill.SkillName].LevelUp(skill.CurrentLevel,
                        skill.EnableIcon);
34:             }
35:         }
36:
37:         private void SpawnIcon(SkillTemplate skill, Transform parent)
38:         {
39:             var clone = Instantiate(skillIconPrefab, parent);
40:             clone.transform.localScale = Vector3.one;
41:             clone.Setup(skill.disableIcon);
42:
43:             skillIcons.Add(skill.skillName, clone);
44:         }
45: }
```

15줄 Setup() 메서드의 첫 번째 매개변수 skills는 버프와 공격 스킬 정보를, 두 번째 매개변수 elementalBonus는 속성 보너스 스킬 정보를 넘겨받습니다.

19~22줄 skills에 저장된 각 속성의 버프/공격 스킬 아이콘을 생성합니다.

23~26줄 elementalBonus에 저장된 각 속성의 보너스 스킬 아이콘을 생성합니다.

29~35줄 스킬 레벨업을 할 때 LevelUp() 메서드를 호출해 해당 스킬의 아이콘, 레벨 정보를 갱신하고 UI에 출력합니다.

37~44줄 SpawnIcon() 메서드는 스킬 아이콘을 생성하고 속성에 따라 부모 오브젝트를 설정합니다.

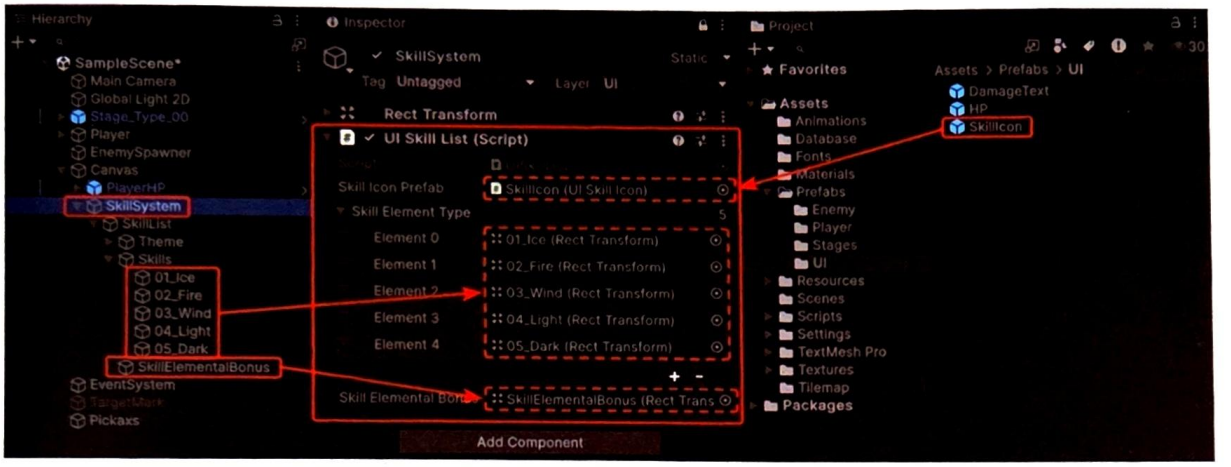
11 스킬 레벨업을 할 때마다 스킬 목록 UI 정보를 갱신하도록 SkillSystem.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillSystem.cs

코드 08-11 스킬 레벨업 시 스킬 목록 UI 정보 갱신

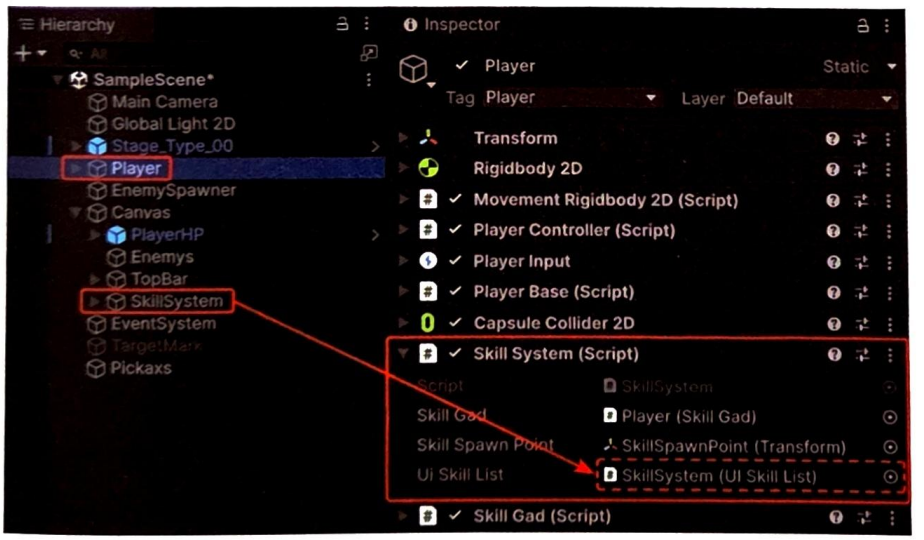
```
    (... 생략 ...)
05: public class SkillSystem : MonoBehaviour
06: {
07:     [SerializeField]
08:     private SkillGad skillGad;
09:     [SerializeField]
10:     private Transform skillSpawnPoint;
11:     [SerializeField]                                    추가
12:     private UISkillList uiSkillList;
        (... 생략 ...)
20:     private void Awake()
21:     {
            (... 생략 ...)
54:         uiSkillList.Setup(skillDict, eSkillDict);       추가
55:     }
        (... 생략 ...)
83:     public void LevelUp(SkillBase skill)
84:     {
85:         if ( skills.ContainsValue(skill) )
86:         {
87:             skill.TryLevelUp();
88:             uiSkillList.LevelUp(skill);                 추가
89:             Logger.Log($"Level Up [{skill.SkillName}]
                    {skill.Element}, Lv. {skill.CurrentLevel}");
90:
91:             // 해당 스킬이 소속된 속성의 총 스킬 레벨 합+1
92:             elementalCounts[skill.Element] ++;
93:             // 해당 스킬의 속성 보너스 레벨 증가 여부 판단
94:             if ( elementalCounts[skill.Element] % 3 == 0 )
95:             {
96:                 elementalSkills[skill.Element].TryLevelUp();
97:                 uiSkillList.LevelUp(elementalSkills[skill.Element]);   추가
98:                 Logger.Log($"{skill.Element}
                        Lv. {elementalSkills[skill.Element].CurrentLevel}");
99:             }
100:        }
101:    }
        (... 생략 ...)
142:}
```

12 SkillSystem 오브젝트에 UI Skill List 컴포넌트를 추가하고 Skill Icon Prefab 변수에 SkillIcon 프리팹을 등록합니다. Skill Element Type 배열의 크기를 '5'로 설정하고 01_Ice, 02_Fire, 03_Wind, 04_Light, 05_Dark 오브젝트를 순서대로 등록합니다. Skill Elemental Bonus 변수에는 SkillElementalBonus 오브젝트를 등록합니다.



SkillSystem 오브젝트에 UI Skill List 컴포넌트 추가하고 설정하기

13 Player 오브젝트 Skill System 컴포넌트의 UI Skill List 변수에 SkillSystem 오브젝트를 등록합니다.

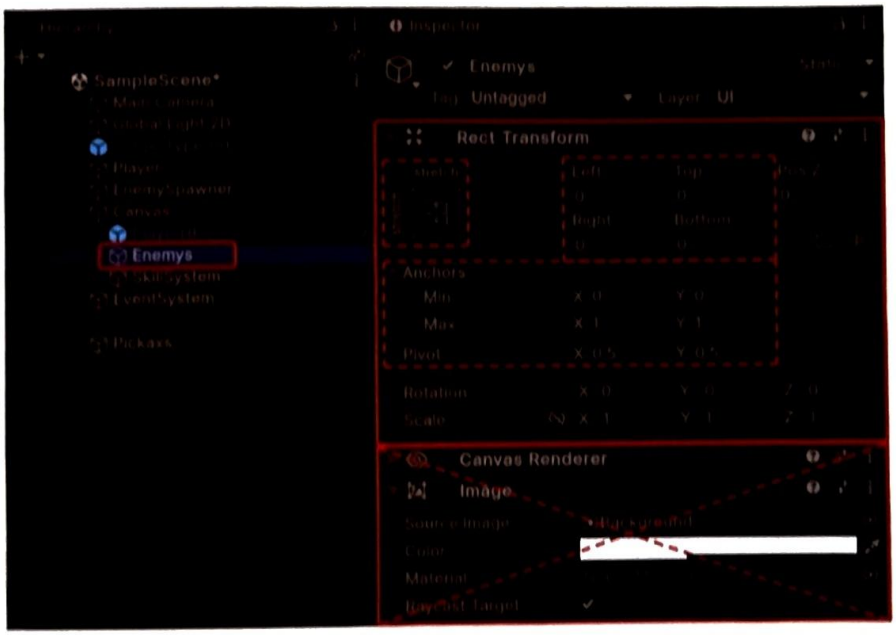


Player 오브젝트의 Skill System 컴포넌트 변수 설정하기

14 지금은 게임을 실행한 이후 생성되는 적 체력 UI를 가장 앞에 출력합니다. UI 출력 순서를 설정하고자 [GameObject → UI → Panel] 메뉴로 적 체력 UI를 관리하는 Panel UI를 생성하고 이름을 'Enemys'로 변경한 후 SkillSystem 오브젝트보다 위에 배치합니다.

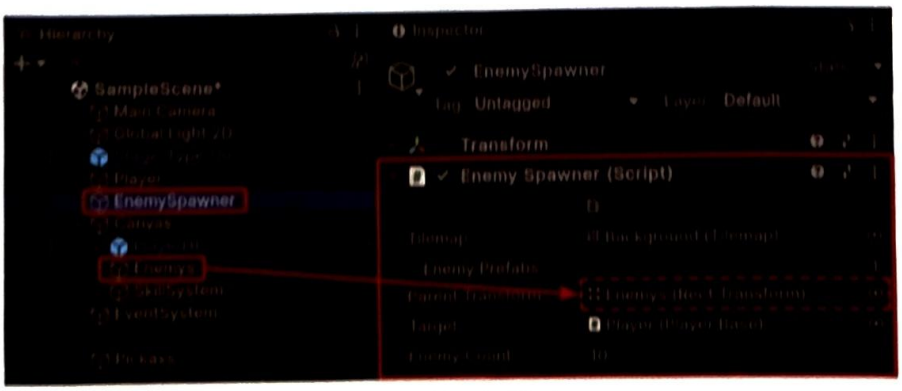
● 유니티로 개발할 때 구분하기 쉽도록 'Enemys'라 표기하기도 합니다.

Rect Transform 컴포넌트의 Anchor Presets은 stretch | stretch로 설정하고 Rect를 (0, 0, 0, 0)으로 설정합니다. 화면에 이미지를 출력하지 않으므로 Image, Canvas Renderer 컴포넌트는 삭제합니다.



UI 출력 순서를 설정하고자 적 체력 UI를 관리하는 Panel UI 생성하고 설정하기

15 EnemySpawner 오브젝트 Enemy Spawner 컴포넌트의 Parent Transform 변수에 Enemys 오브젝트를 등록합니다. 이제 적 체력 UI가 생성되면 Enemys 오브젝트의 자식으로 배치됩니다.



EnemySpawner 오브젝트의 컴포넌트 변수 설정하기

Do it! 실습 게임을 일시 정지하고 스킬 목록 표시하기

지금은 스킬 목록 UI가 계속 활성화된 상태입니다. 게임 일시 정지 기능을 추가한 후 일시 정지 버튼을 눌렀을 때만 스킬 목록을 출력하도록 설정해 봅니다.

01 Assets 〉 Scripts 〉 GameSystem 폴더를 만들고 이곳에 현재 스테이지를 제어하는 GameController.cs 스크립트를 생성합니다.

Assets/Scripts/GameSystem/GameController.cs

코드 08-12 현재 스테이지 제어

```
01: using UnityEngine;
02:
03: public class GameController : MonoBehaviour
04: {
05:     public void SetTimeScale(float scale)
06:     {
07:         Time.timeScale = scale;
08:     }
09: }
```

07줄 Time.timeScale은 시간이 흐르는 척도로, 기본값은 1입니다. 이 값이 0.5이면 기본 속도보다 2배 느려지고 2이면 기본 속도보다 2배 빨라집니다. 그러나 이 값을 0으로 설정하면 멈춥니다.

알아 두면 좋아요! Time.timeScale을 이용하여 시간 멈춤을 구현하려면?

Time.timeScale을 0으로 설정하면 Update() 메서드의 실행은 멈추지 않지만 Time.deltaTime을 곱해서 사용하는 코드, 애니메이션 재생, FixedUpdate(), WaitForSeconds()를 포함한 코루틴 등은 멈춥니다.

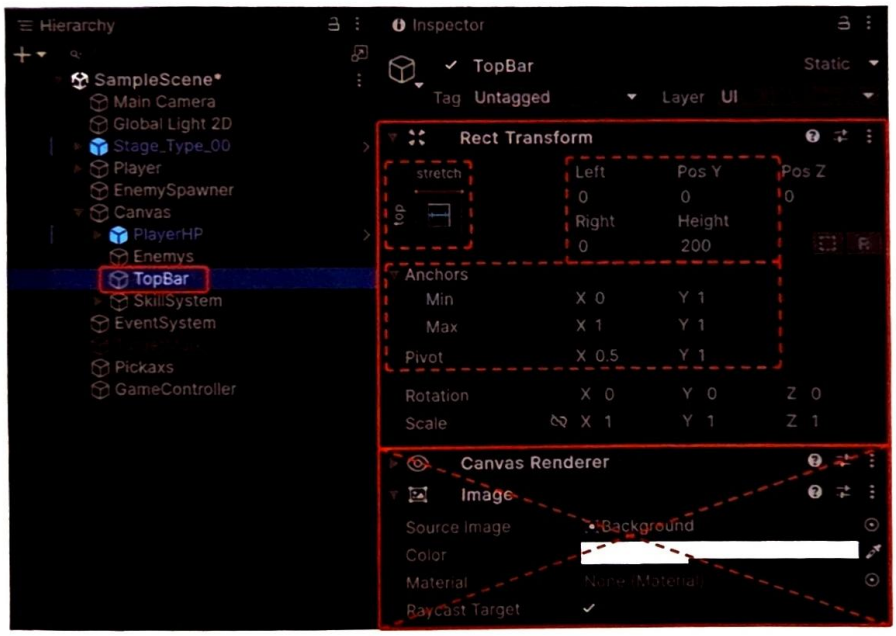
참고로, 이 책에서는 사운드를 사용하지 않았지만 이 방법으로 사운드를 멈출 수는 없습니다. 사운드 재생을 멈추려면 AudioListener.pause = true; 코드를 이용하여 따로 멈춰야 합니다.

02 [GameObject → Create Empty] 메뉴로 빈 오브젝트를 생성하고 이름을 'Game Controller'로 변경한 후 Game Controller 컴포넌트를 추가합니다.

GameController 오브젝트에 Game Controller 컴포넌트 추가하기

03 [GameObject → UI → Panel] 메뉴로 일시 정지 버튼처럼 상단 UI를 관리하는 Panel UI를 생성하고 이름을 'TopBar'로 변경합니다.

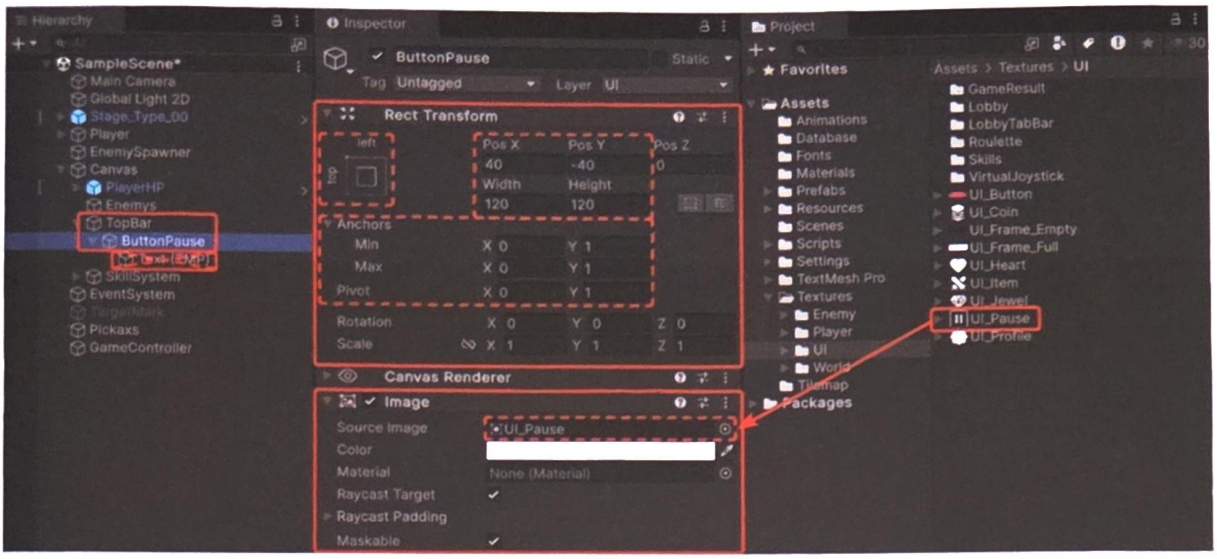
Rect Transform 컴포넌트의 Anchor Presets은 top | stretch로, Rect는 (0, 0, 0, 200)으로 설정합니다. 이미지를 출력하지 않으므로 Image, CanvasRenderer 컴포넌트는 삭제합니다.



정지 버튼과 같은 상단 UI를 관리하는 Panel UI 생성하고 설정하기

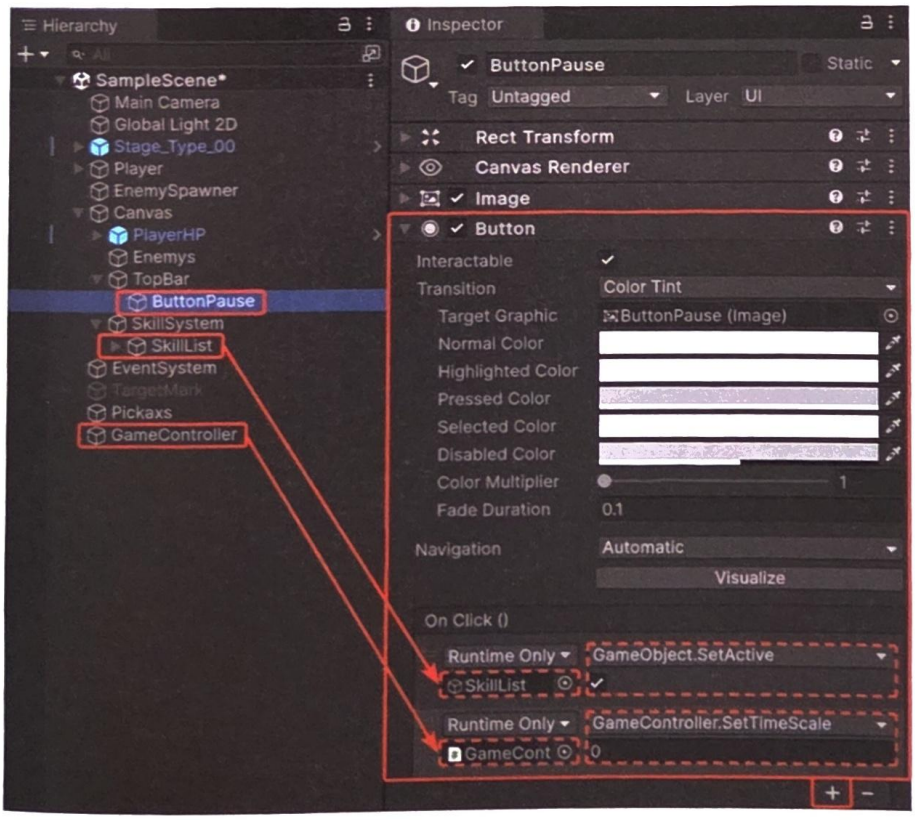
04 [GameObject → UI → Button - TextMeshPro] 메뉴로 일시 정지 Button UI를 생성하고 이름을 'ButtonPause'로 변경한 후 TopBar 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Anchor Presets은 top | left로, Rect는 (40, -40, 120, 120)으로 설정합니다. Image 컴포넌트의 Source Image는 UI_Pause.png 이미지로 설정합니다. ButtonPause 오브젝트의 자식인 Text (TMP) 오브젝트는 삭제합니다.



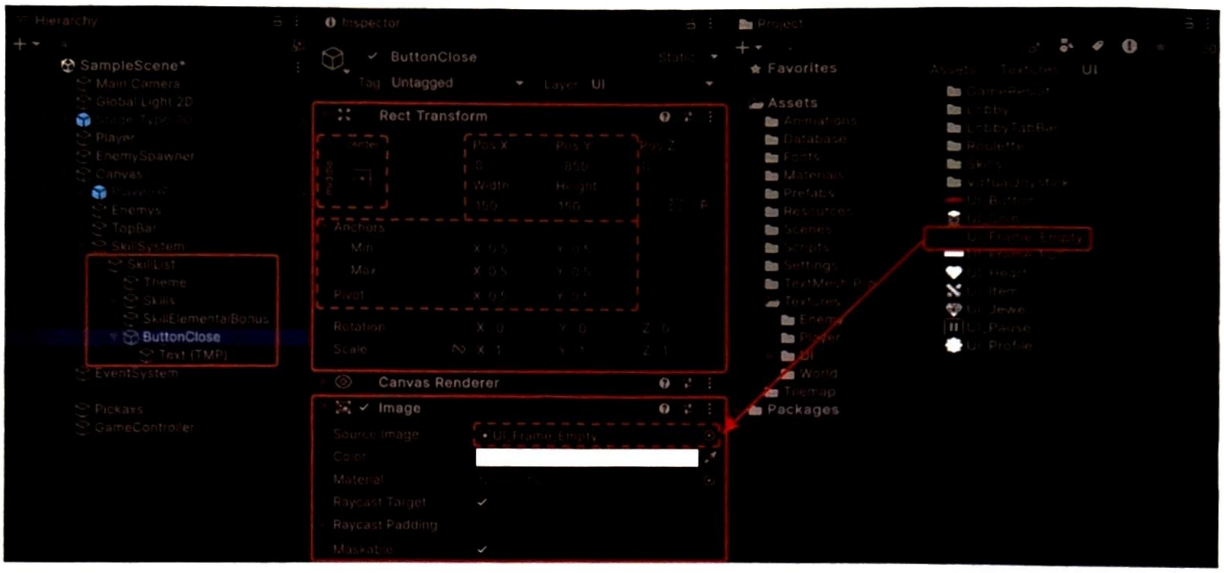
일시 정지 Button UI 생성하고 설정하기

05 ButtonPause 버튼을 눌렀을 때 게임을 일시 정지하고 스킬 목록 UI를 활성화하도록 ButtonPause 오브젝트의 Button 컴포넌트 On Click () 리스트에 요소를 2개 추가합니다. 첫 번째 요소에는 SkillList 오브젝트의 Game Object 컴포넌트 SetActive(true) 메서드를, 두 번째 요소에는 GameController 오브젝트의 Game Controller 컴포넌트 SetTimeScale(0) 메서드를 등록합니다.



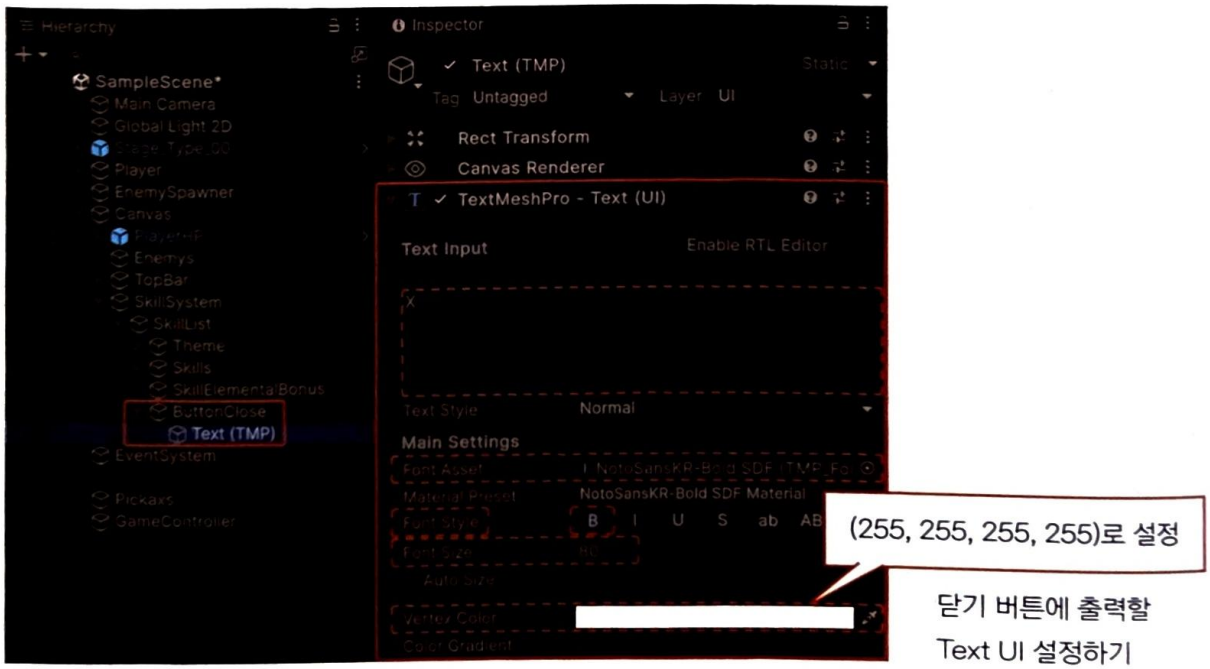
일시 정지 버튼에 On Click () 이벤트 메서드 등록하기

06 [GameObject → UI → Button - TextMeshPro] 메뉴로 닫기 Button UI를 생성하고 이름을 'ButtonClose'로 변경한 후 SkillList 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 middle | center로, Rect는 (0, -850, 150, 150)으로 설정합니다. Image 컴포넌트의 Source Image는 UI_Frame_Empty.png 이미지로 설정합니다.



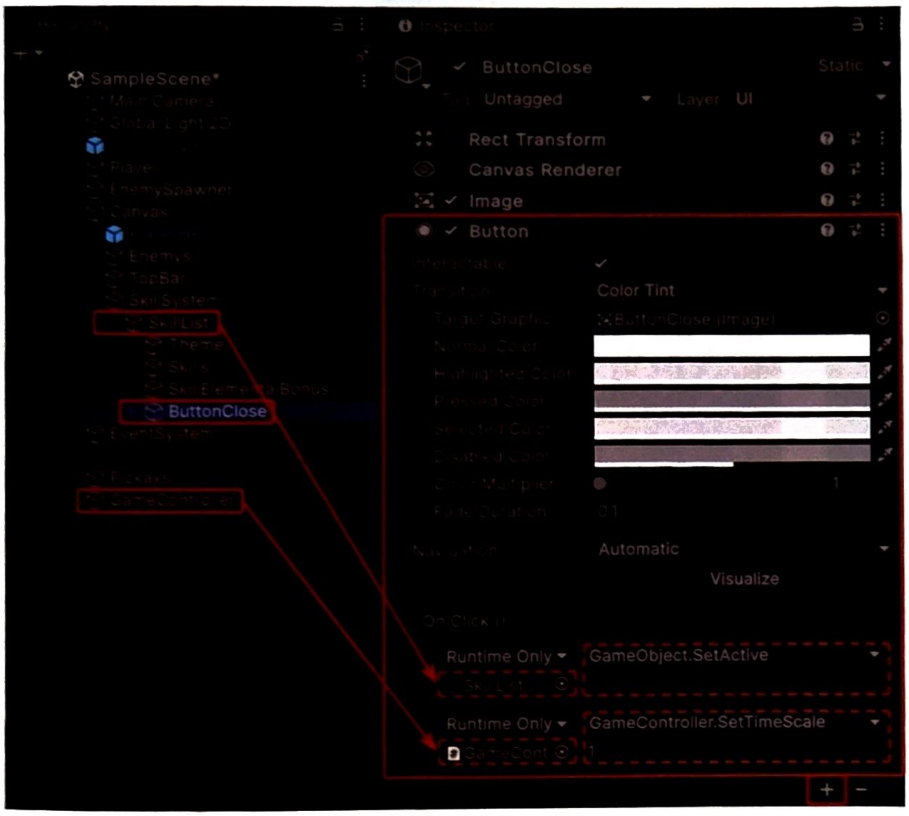
닫기 Button UI 생성하고 설정하기

07 ButtonClose 오브젝트의 자식인 Text (TMP) 오브젝트에서 TextMeshPro - Text 컴포넌트의 Text Input Box에 'X'를 입력하고 Font Asset에 NotoSansKR-Bold SDF를 등록합니다. Font Style은 [B]로, Font Size는 '80'으로, 폰트 색상인 Vertex Color는 (255, 255, 255, 255)로 설정합니다.



닫기 버튼에 출력할
Text UI 설정하기

08 ButtonClose 버튼을 눌렀을 때 게임을 재개하고 스킬 목록 UI를 비활성화하도록 ButtonClose 오브젝트의 Button 컴포넌트 On Click () 리스트에 요소를 2개 추가합니다. 첫 번째 요소에는 SkillList 오브젝트의 GameObject 컴포넌트 SetActive(false) 메서드를, 두 번째 요소에는 GameController 오브젝트의 GameController 컴포넌트 SetTimeScale(1) 메서드를 등록합니다.

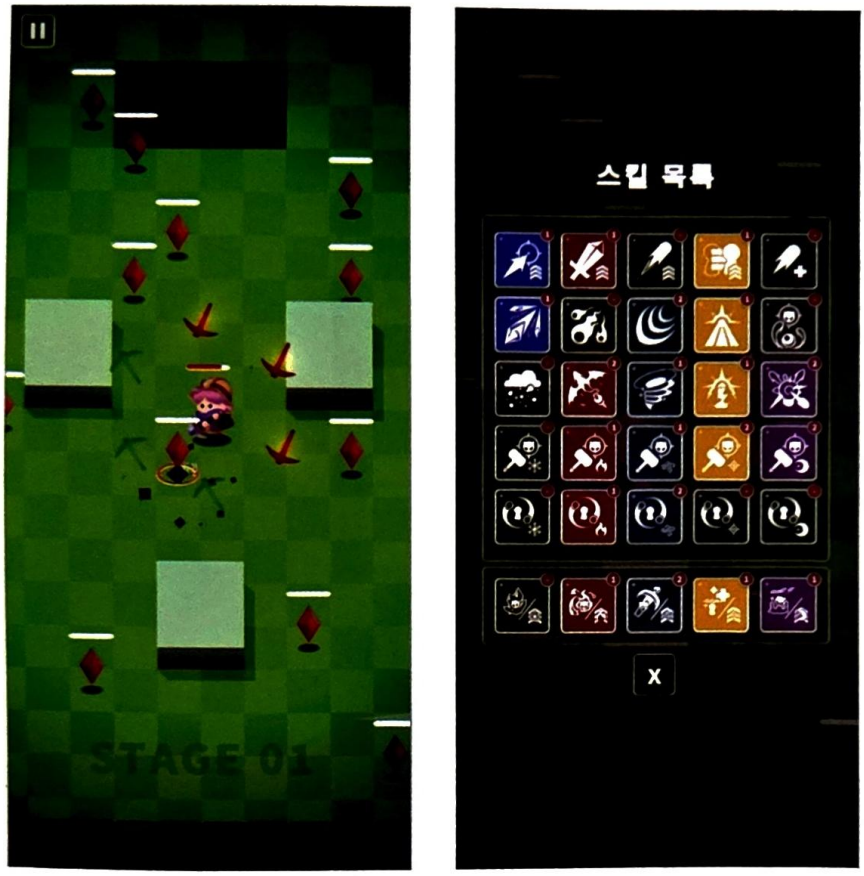


닫기 버튼에 On Click () 이벤트 메서드 등록하기

09 일시 정지 버튼을 눌렀을 때만 스킬 목록을 출력하므로 SkillList 오브젝트는 비활성화하여 평소에는 숨깁니다.

SkillList 오브젝트 비활성화하기

10 플레이 버튼 ▶을 눌러 게임을 실행하면 일시 정지 버튼을 눌렀을 때 게임이 잠시 멈추고 스킬 목록 UI를 활성화합니다. UI 아래에 있는 닫기 버튼 X을 클릭하면 스킬 목록을 닫고 게임을 다시 시작합니다.



스킬 목록 UI 결과 화면

08-3

무작위로 스킬 배우고 레벨업 하기

이 절에서는 스킬 선택 UI를 만듭니다. 스킬을 습득할 수 있을 때 선택 UI를 출력하고 해당하는 스킬 3가지를 표시합니다. 스킬 3가지 중에 하나를 선택하면 해당 스킬을 습득 또는 레벨업 하고 스킬 선택 팝업 창이 비활성화되도록 구현해 봅니다.

Do it! 실습 스킬 선택 아이콘 UI 추가하기

스킬 선택 UI에서 사용할 아이콘부터 추가합니다.

01 Assets 〉 Scripts 〉 UI 폴더에 스킬 선택 아이콘 UI를 제어하는 UISelectSkillIcon.cs 스크립트를 생성합니다.

Assets/Scripts/UI/UISelectSkillIcon.cs

코드 08-13 스킬 선택 아이콘 UI 제어

```
01: using UnityEngine;
02: using UnityEngine.EventSystems;
03: using UnityEngine.UI;
04: using TMPro;
05:
06: public class UISelectSkillIcon : MonoBehaviour, IPointerClickHandler
07: {
08:     [SerializeField]
09:     private Image skillIcon;
10:     [SerializeField]
11:     private TextMeshProUGUI skillName;
12:     [SerializeField]
13:     private TextMeshProUGUI skillDescription;
14:
15:     private SkillSystem skillSystem;
16:     private SkillBase skillBase;
17:
```

```
18:     public void Setup(SkillSystem skillSystem, SkillBase skillBase)
19:     {
20:         this.skillSystem = skillSystem;
21:         this.skillBase = skillBase;
22:         skillIcon.sprite = skillBase.EnableIcon;
23:         skillName.text = skillBase.SkillName;
24:         skillDescription.text = skillBase.Description;
25:     }
26:
27:     public void OnPointerClick(PointerEventData eventData)
28:     {
29:         Logger.Log($"{skillBase.SkillName} 스킬 선택");
30:     }
31: }
```

06줄 IPointerClickHandler 인터페이스를 상속하면 이 스크립트를 컴포넌트로 가진 UI 오브젝트를 클릭했을 때 OnPointerClick() 메서드를 호출합니다.

27~30줄 UI 오브젝트 클릭으로 호출하는 OnPointerClick() 메서드를 정의합니다. 지금은 클릭한 스킬 이름인 SkillName을 출력합니다.

02 SelectSkillIcon 프리팹에 UI Select Skill Icon 컴포넌트를 추가하고 Skill Icon 변수에는 Icon 오브젝트를, Skill Name 변수에는 Name 오브젝트를, Skill Description 변수에는 Description 오브젝트를 등록합니다.

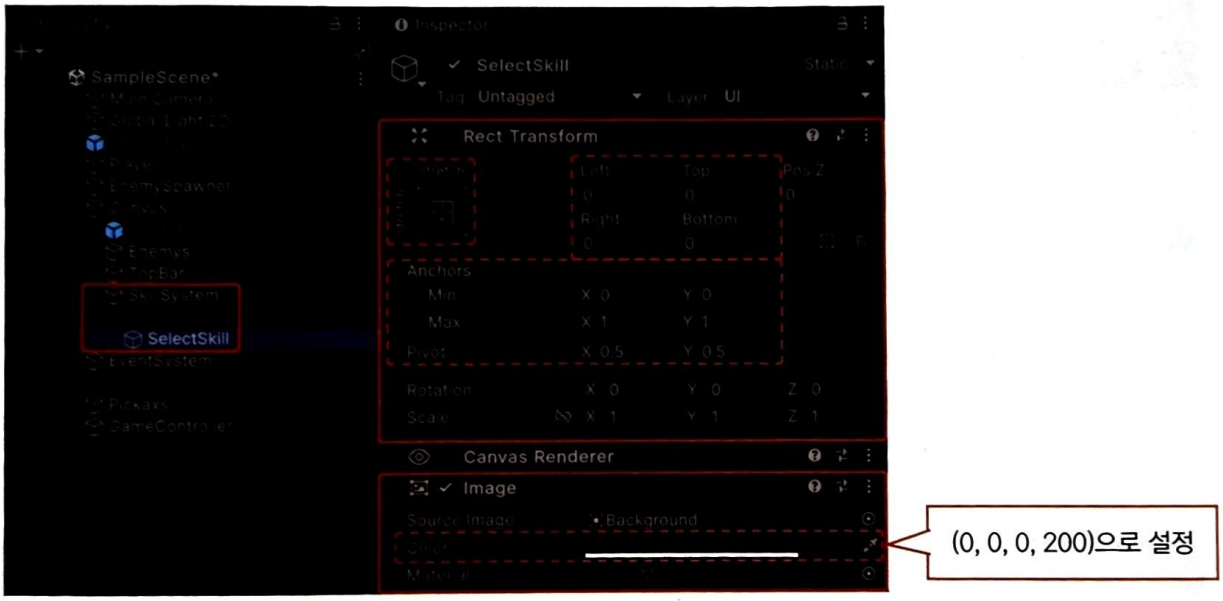
SelectSkillIcon 프리팹에 UI Select Skill Icon 컴포넌트 추가하고 설정하기

Do it! 실습 스킬 선택 UI 구현하기

이번에는 앞서 추가한 아이콘을 이용하여 스킬 선택 UI를 구성합니다.

01 [GameObject → UI → Panel] 메뉴로 스킬 선택 UI를 관리하는 Panel UI를 생성하고 이름을 'SelectSkill'로 변경한 후 SkillSystem 오브젝트의 자식으로 배치합니다.

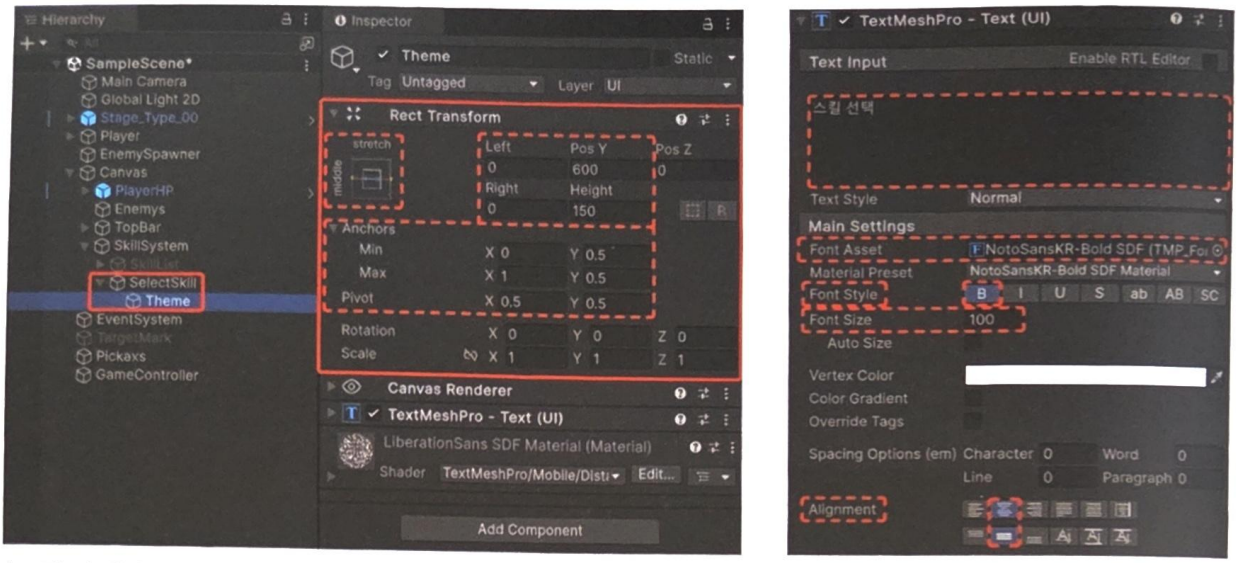
Rect Transform 컴포넌트의 Anchor Presets은 stretch | stretch로, Rect는 (0, 0, 0, 0)으로 설정합니다. Image 컴포넌트의 Color는 (0, 0, 0, 200)으로 설정합니다.



스킬 선택 UI를 관리하는 Panel UI 생성하고 설정하기

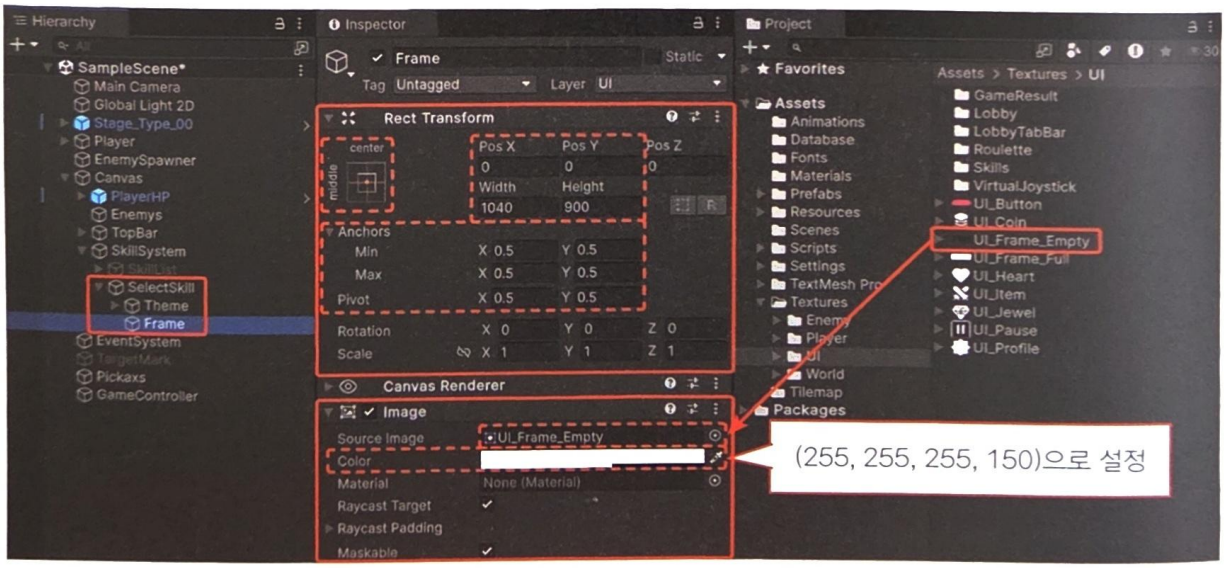
02 [GameObject → UI → Text - TextMeshPro] 메뉴로 '스킬 선택' 텍스트를 출력하는 Text UI를 생성하고 이름을 'Theme'로 변경한 후 SelectSkill 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Anchor Presets은 middle | stretch로, Rect는 (0, 600, 0, 150)으로 설정합니다. TextMeshPro - Text 컴포넌트의 Text Input Box에 '스킬 선택'을 입력하고 Font Asset에 NotoSansKR-Bold SDF를 등록합니다. Font Style은 [B]로, Font Size는 '100'으로, Alignment는 Center | Middle로 설정합니다.



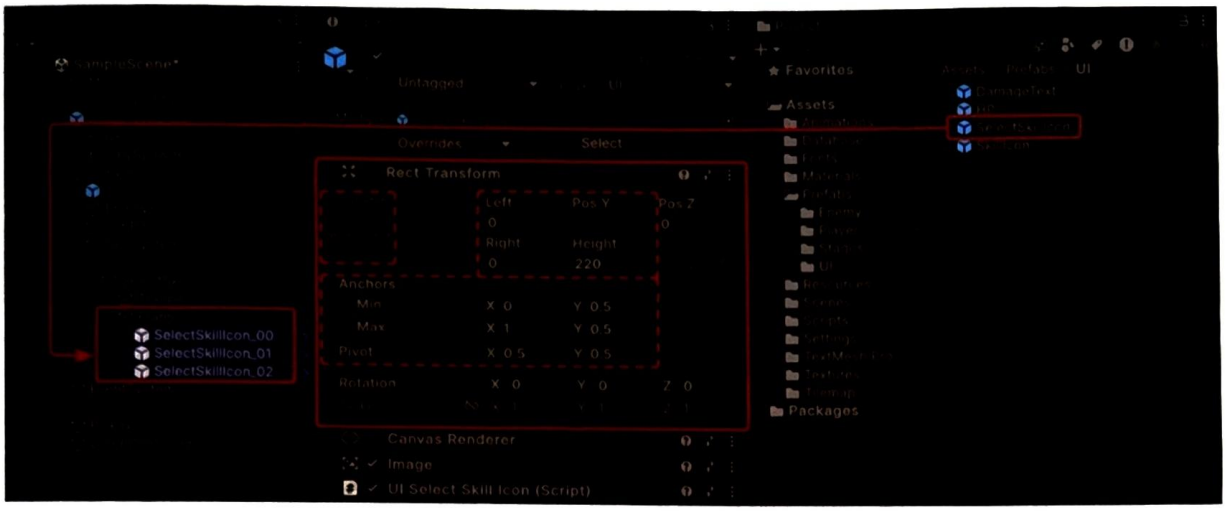
'스킬 선택'을 출력하는 Text UI 생성하고 설정하기

03 [GameObject → UI → Image] 메뉴로 선택할 수 있는 스킬 목록을 관리하는 Image UI를 생성하고 이름을 'Frame'으로 변경한 후 SelectSkill 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 middle | center로, Rect는 (0, 0, 1040, 900)으로 설정합니다. Image 컴포넌트의 Source Image는 UI_Frame_Empty.png 이미지로, Color는 (255, 255, 255, 150)으로 설정합니다.



선택할 수 있는 스킬 목록을 관리하는 Image UI 생성하고 설정하기

04 선택할 수 있는 스킬 아이콘을 배치하고자 SelectSkillIcon 프리팹을 Frame 오브젝트의 자식으로 3개 배치한 후 이름은 각각 'SelectSkillIcon_00', 'SelectSkillIcon_01', 'SelectSkillIcon_02'로, Pos Y는 각각 '270', '0', '-270'으로 설정합니다.



선택할 수 있는 스킬 아이콘 배치하고 설정하기

05 Assets 〉 Scripts 〉 UI 폴더에 스킬 선택 UI를 제어하는 UISelectSkill.cs 스크립트를 생성합니다.

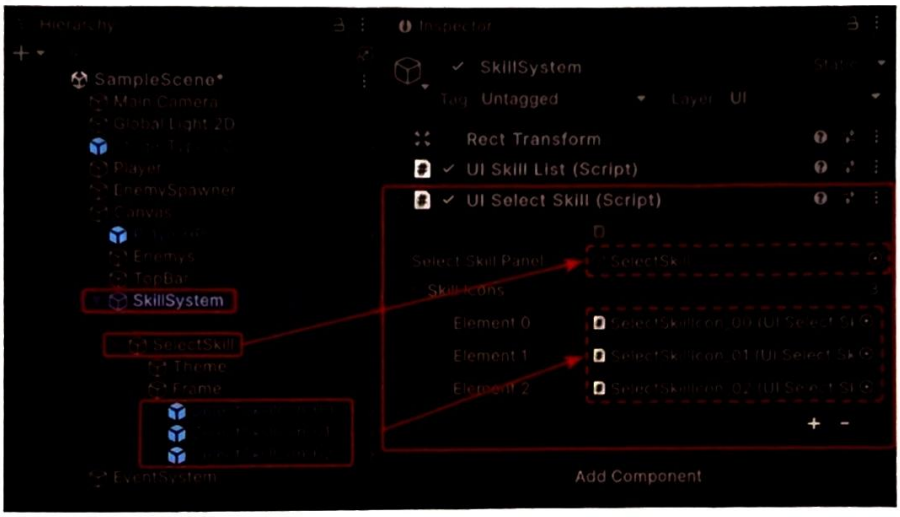
Assets/Scripts/UI/UISelectSkill.cs

코드 08-14 스킬 선택 UI 제어

```
using UnityEngine;

public class UISelectSkill : MonoBehaviour
{
    [SerializeField]
    private GameObject selectSkillPanel;
    [SerializeField]
    private UISelectSkillIcon[] skillIcons;

    public void StartSelectSkillUI(SkillSystem system, SkillBase[] skills)
    {
        selectSkillPanel.SetActive(true);

        for ( int i = 0; i < skills.Length; ++ i )
        {
            skillIcons[i].Setup(system, skills[i]);
        }
```

```
18:     }
19:
20:     public void EndSelectSkillUI()
21:     {
22:         selectSkillPanel.SetActive(false);
23:     }
24: }
```

06 SkillSystem 오브젝트에 UI Select Skill 컴포넌트를 추가하고 Select Skill Panel 변수에 SelectSkill 오브젝트를 등록합니다. SkillIcons 배열의 크기를 '3'으로 설정하고 SelectSkillIcon_00, 01, 02 오브젝트를 순서대로 등록합니다.



SkillSystem 오브젝트에 UI Select Skill 컴포넌트 추가하고 설정하기

07 스킬 선택 UI 활성화와 선택한 스킬 레벨 증가를 처리하도록 SkillSystem.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillSystem.cs

코드 08-15 스킬 선택 UI 활성화 및 선택한 스킬 레벨 증가 처리

```
    (... 생략 ...)
05: public class SkillSystem : MonoBehaviour
06: {
        (... 생략 ...)
12:     private UISkillList uiSkillList;
13:     [SerializeField]
14:     private UISelectSkill uiSelectSkill;
```

추가

```
15:     [SerializeField]
16:     private GameController gameController;
        (... 생략 ...)
61:     private void Update()
62:     {
63:         // 레벨업 가능한 임의의 스킬 3개를 선택하고 그중 하나를 레벨업 [Debug Test]
64:         if ( UnityEngine.InputSystem.Keyboard.current.digit1Key.wasPressedThisFrame )
                StartSelectSkill();                                          ← 수정
            (... 생략 ...)
85:     }
        (... 생략 ...)
107:    public void StartSelectSkill()                                       ← 수정
108:    {
109:        // 스킬 선택 중에는 일시 정지                                    ← 추가
110:        gameController.SetTimeScale(0);
111:
112:        // 습득 또는 레벨업할 수 있는 임의의 3개 스킬 선택
113:        var randomSkills = GetRandomSkills(skills, 3);
            (... 생략 ...)
120:        /*// 스킬 선택 UI가 없으므로 임시로 스킬 습득 처리
121:        int index = Random.Range(0, randomSkills.Count);
122:        LevelUp(randomSkills[index]);*/                                  ← 삭제
123:        // 획득 가능한 3개의 스킬 정보를 UI에 출력                       ← 추가
124:        uiSelectSkill.StartSelectSkillUI(this, randomSkills.ToArray());
125:    }
126:
127:    public void EndSelectSkill(SkillBase skill)                          ← 추가
128:    {
129:        LevelUp(skill);                     // {skill} 스킬 레벨업
130:        uiSelectSkill.EndSelectSkillUI();   // 스킬 선택 UI 비활성화
131:        gameController.SetTimeScale(1);     // 게임 다시 시작
132:    }
        (... 생략 ...)
158:}
```

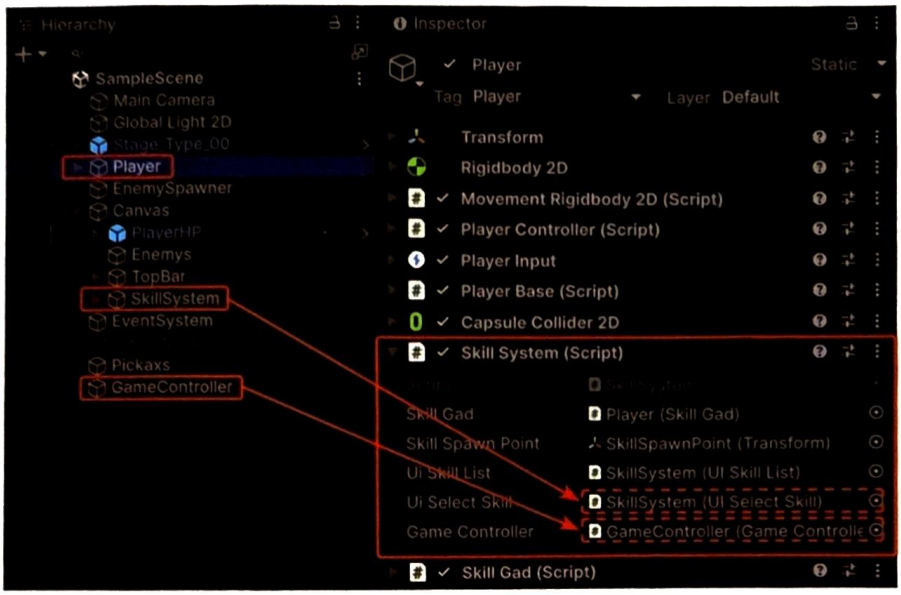
107줄 SelectSkill() 메서드 이름을 StartSelectSkill()로 변경합니다.

109~113줄 숫자 1을 눌러 StartSelectSkill() 메서드를 호출하면 게임을 일시 정지하고 임의의 스킬 3개를 선택합니다.

123~124줄 선택한 스킬 정보를 매개변수로 하여 스킬 선택 UI를 출력합니다.

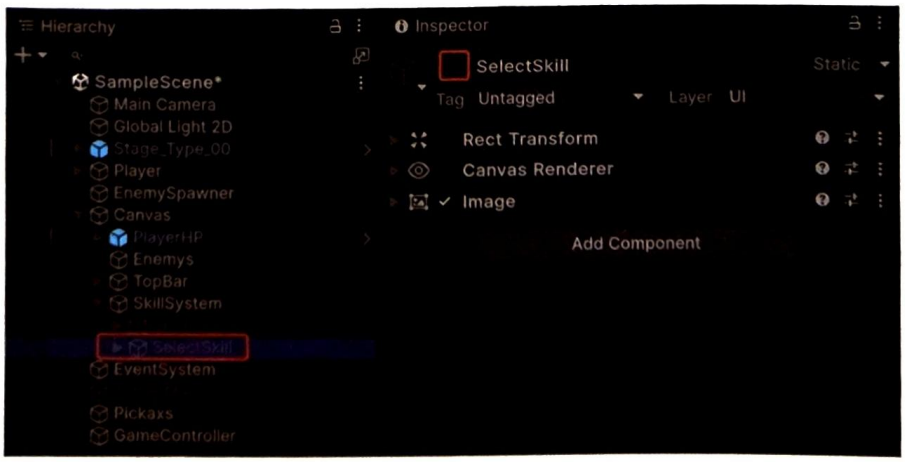
127~132줄 스킬을 선택하면 UISelectSkillIcon 클래스에서 EndSelectSkill() 메서드를 호출해 선택한 스킬의 레벨업과 함께 스킬 선택 UI를 닫고 게임을 다시 시작합니다.

08 Player 오브젝트의 Skill System 컴포넌트에서 UI Select Skill 변수에는 SkillSystem 오브젝트를, Game Controller 변수에는 GameController 오브젝트를 등록합니다.



Player 오브젝트의 Skill System 컴포넌트 변수 설정하기

09 SkillList와 마찬가지로 평소에는 숨겼다가 스킬을 배울 수 있을 때 스킬 선택 UI가 활성화되도록 SelectSkill 오브젝트는 비활성화합니다.



SelectSkill 오브젝트 비활성화

10 `SkillSystem.EndSelectSkill()` 메서드를 호출해 선택한 스킬 레벨을 늘리도록 UISelectSkillIcon.cs 스크립트를 수정합니다.

Assets/Scripts/UI/UISelectSkillIcon.cs

코드 08-16 선택한 스킬 레벨 증가

```
    (... 생략 ...)
06: public class UISelectSkillIcon : MonoBehaviour, IPointerClickHandler
07: {
        (... 생략 ...)
26:
27:     public void OnPointerClick(PointerEventData eventData)
28:     {
29:         skillSystem.EndSelectSkill(skillBase);
30:     }
31: }
```

수정

11 플레이 버튼 ▶을 눌러 게임을 실행하면 아직은 플레이어 레벨업과 같은 시스템이 없으므로 숫자 1을 눌러 스킬 선택 UI를 활성화하고 게임을 일시 정지합니다. 스킬 3개 중에서 하나를 선택하면 해당 스킬 레벨이 1 증가합니다. 그러고 나서 스킬 선택 UI를 닫고 게임을 다시 시작합니다.

스킬 선택 UI 결과 화면

셋째마당

게임 시스템과 적 AI 구현하기

이번 마당에서는 게임의 완성도를 결정짓는 '성장·재화 시스템'과 '지능형 게임 AI'를 구현합니다. 이를 통해 캐릭터 성장을 시각화하는 레벨업 시스템과 많은 오브젝트를 성능 저하 없이 관리하는 메모리 풀 최적화 기법을 배웁니다. 또한, NavMesh2D와 행동 트리를 결합하여 적이 스스로 상황을 판단하고 행동하도록 하는 게임 인공지능을 구축합니다.

09장 레벨업 시스템과 자원 최적화하기

10장 똑똑한 적 AI — 행동 트리와 스마트 경로 탐색하기

09장

레벨업 시스템과 자원 최적화하기

이번 장에서는 플레이어 레벨업과 적이 남긴 보석을 저장하는 기능을 구현합니다. 적을 물리쳤을 때 경험치가 증가하고 최대로 채워지면 레벨업 합니다. 보석은 다량 생성되므로 최적화하고자 오브젝트를 삭제하지 않고 리스트 형태로 보관한 후 활성화/비활성화로 관리하는 메모리 풀 기능을 이용합니다.

학습 목표

1. ScriptableObject 기반의 경험치 및 레벨업 시스템을 구현할 수 있다.
2. 메모리 풀을 활용하여 런타임 성능을 최적화할 수 있다.

09-1 경험치 테이블 설계하고 레벨업 하기

09-2 메모리 풀을 활용하여 보석 생성하고 최적화하기

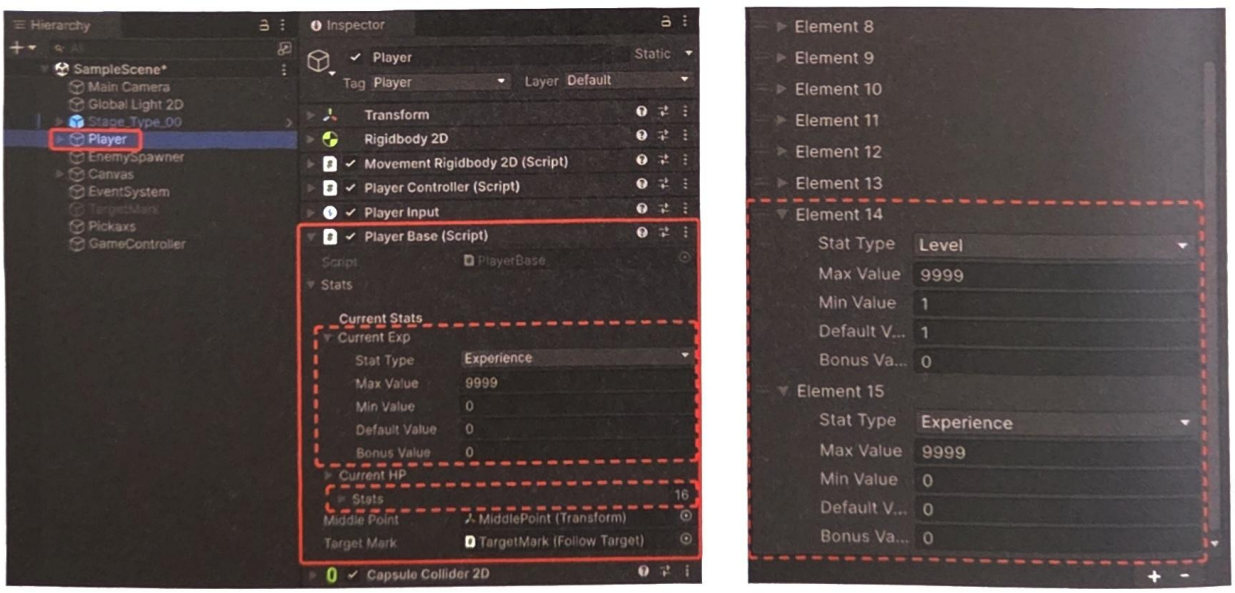
```
17:         // 기본 체력은 DefaultValue에 할당하므로 추가 체력(BonusValue)만 설정
18:         Stats.GetStat(StatType.HP).BonusValue =
                50 * (Stats.GetStat(StatType.Level).Value - 1);
19:
20:         base.Setup();
21:     }
        (... 생략 ...)
30: }
```

수정

04 Player 오브젝트의 Player Base 컴포넌트에 있는 Stats 옵션의 Current Exp 변숫값을 설정합니다. Stat Type을 [Experience]로, Max Value를 '9999'로 설정합니다. Stats 배열의 크기를 '16'으로 수정하고 다음 표를 참고하여 추가한 요소에 값을 입력합니다.

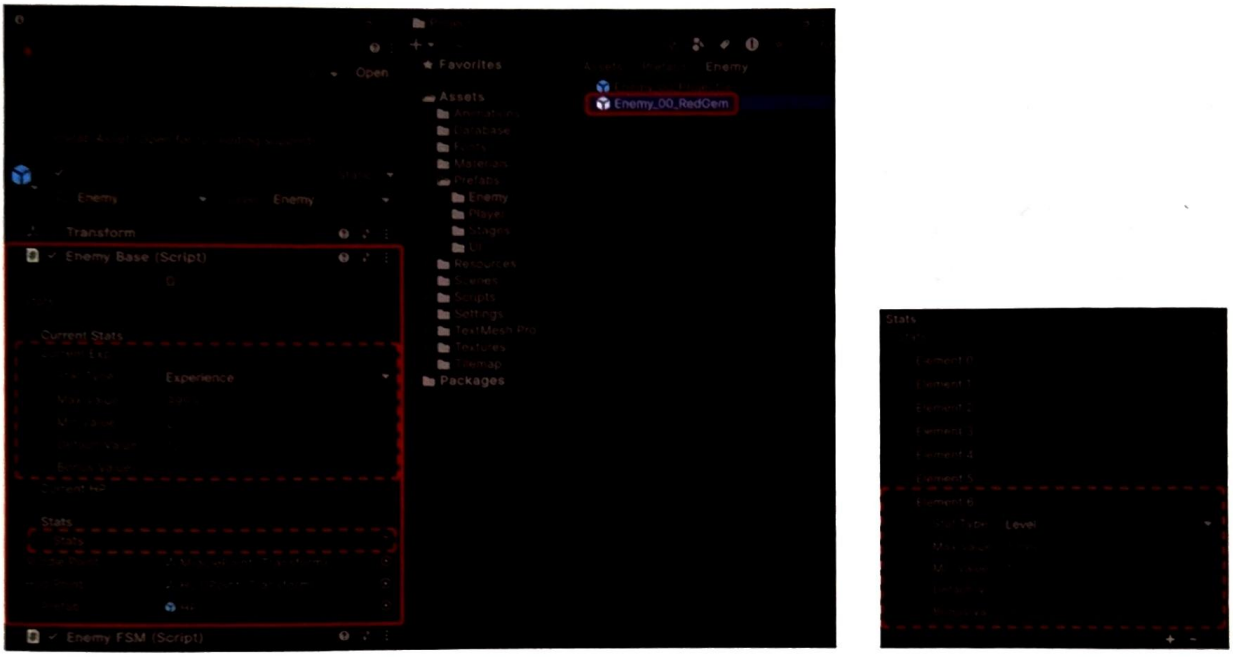
Stats 배열 요소 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
14	Level	9999	1	1	0
15	Experience	9999	0	0	0



플레이어의 스탯에 레벨, 경험치 스탯 추가하고 설정하기

05 Enemy_00_RedGem 프리팹의 Enemy Base 컴포넌트에서 Stats 옵션의 Current Exp 변숫값을 설정합니다. Stat Type은 [Experience]로, Max Value는 '9999'로, Default Value는 '10'으로 설정합니다. 적은 경험치를 획득해 레벨업 하지 않으므로 적을 물리쳤을 때 플레이어에게 Current Exp의 Default Value만큼 전달하는 용도로 사용합니다.
Stats 배열의 크기를 '7'로 설정합니다. Element 6의 Stat Type은 [Level]로, Max Value는 '9999'로, MinValue와 Default Value는 각각 '1'로 설정합니다.



적(Enemy_00_RedGem)의 스탯에 레벨, 경험치 스탯 추가하고 설정하기

Do it! 실습 플레이어 레벨 관리하기

01 Assets 〉 Scripts 〉 Database 폴더에 플레이어 레벨별 경험치 정보를 관리하는 LevelData.cs 스크립트를 생성합니다.

Assets/Scripts/Database/LevelData.cs

코드 09-04 플레이어 레벨별 최대 경험치 정보 관리

```
01: using UnityEngine;
02:
03: [CreateAssetMenu]
04: public class LevelData : ScriptableObject
05: {
```

```
06:     [SerializeField]
07:     private int maxLevel;
08:     [SerializeField]
09:     private float[] maxExperience;
10:
11:     public int MaxLevel => maxLevel;
12:     public float[] MaxExperience => maxExperience;
13: }
```

02 인게임에서 플레이어 레벨별로 필요한 경험치를 설정하고자 Project View의 ＋▾ 버튼을 클릭합니다. [Level Data] 메뉴를 클릭해 Assets 〉 Database 폴더에 레벨 데이터 에셋을 생성하고 이름을 'InGameLevel_00_StageMode'로 변경합니다.

Max Level은 '9999'로, Max Experience의 배열 크기는 '10'으로 설정합니다. 이 배열은 차례대로 '40', '100', '200', '250', '400', '500', '600', '700', '800', '1000'으로 설정합니다. 현재 최대 경험치를 10개만 설정했기 때문에 코드에서 레벨 10 이후의 최대 경험치는 마지막 요소인 1000으로 설정되도록 합니다.

플레이어 레벨별로 필요한 경험치 설정하기

03 스킬 선택 팝업 창의 활성 여부를 저장하는 속성을 정의하고 설정하도록 SkillSystem.cs 스크립트를 수정합니다.

Assets/Scripts/Skill/SkillSystem.cs

코드 09-05 스킬 선택 팝업 창의 활성 여부를 저장하는 속성

```
    (... 생략 ...)
05: public class SkillSystem : MonoBehaviour
06: {
        (... 생략 ...)
24:     public bool IsSelectSkill { get; private set; } = false;   // 추가
        (... 생략 ...)
109:    public void StartSelectSkill()
110:    {
111:        IsSelectSkill = true;   // 추가
            (... 생략 ...)
125:    }
126:
127:    public void EndSelectSkill(SkillBase skill)
128:    {
            (... 생략 ...)
132:        IsSelectSkill = false;   // 추가
133:    }
        (... 생략 ...)
159: }
```

04 적을 물리치면 플레이어 경험치가 증가하고 최대 경험치가 되면 레벨업 하도록 PlayerBase.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerBase.cs

코드 09-06 플레이어 경험치 증가 및 레벨업 처리

```
01: using UnityEngine;
02:
03: public class PlayerBase : EntityBase
04: {
05:     [SerializeField]
06:     private FollowTarget targetMark;
07:     [SerializeField]   // 추가
08:     private LevelData levelData;      // 인게임 레벨별 경험치 테이블 정보
09:     [SerializeField]
10:     private SkillSystem skillSystem; // 레벨업 할 때 스킬을 배울 수 있도록
11:
12:     private float expAmount = 2f;     // 매 프레임 흡수하는 경험치 양
13:
14:     // 현재 플레이어가 이동 중인지
```

```
15:     // 스킬 사용 등 여러 곳에서 필요하므로 public 속성으로 정의
16:     public bool IsMoved { get; set; } = false;
17:     // 적을 물리치고 축적한 경험치                          추가
18:     public float AccumulationExp { get; set; } = 0f;
19:
20:     private void Awake()
21:     {
22:         base.Setup();
23:
24:         Stats.CurrentExp.DefaultValue = 0f;                  추가
25:         Stats.CurrentExp.OnValueChanged += IsLevelUP;
26:         Stats.GetStat(StatType.Experience).DefaultValue =
                levelData.MaxExperience[0];
27:     }
28:
29:     private void Update()
30:     {
31:         if ( Target == null ) targetMark.gameObject.SetActive(false);
32:
33:         SearchTarget();
34:         Recovery();
35:         UpdateEXP();                                         추가
36:     }
        (... 생략 ...)
70:     private void UpdateEXP()                                 추가
71:     {
72:         if ( Mathf.Approximately(AccumulationExp, 0f) ||
                skillSystem.IsSelectSkill == true ) return;
73:
74:         float getEXP = AccumulationExp > expAmount ? expAmount : AccumulationExp;
75:         AccumulationExp -= getEXP;                  // 축적 경험치에서 getEXP만큼 소모
76:         Stats.CurrentExp.DefaultValue += getEXP; // 경험치를 getEXP만큼 증가
77:     }
78:
79:     private void IsLevelUP(Stat stat, float prev, float current)
80:     {
81:         // 경험치가 최대가 아니면 return
82:         if ( !Mathf.Approximately(Stats.CurrentExp.Value,
                Stats.GetStat(StatType.Experience).Value) ) return;
83:
84:         // 플레이어 레벨업(현재는 최대 레벨일 때 UI를 출력하거나 하지 않음)
85:         Stats.GetStat(StatType.Level).DefaultValue ++;
86:
```

```
87:         // 현재 경험치 설정(레벨업에 사용한 양만큼 감소)
88:         Stats.CurrentExp.DefaultValue -= Stats.GetStat(StatType.Experience).Value;
89:
90:         // 최대 경험치 설정
91:         if ( Stats.GetStat(StatType.Level).Value < levelData.MaxExperience.Length )
92:             Stats.GetStat(StatType.Experience).DefaultValue =
                    levelData.MaxExperience[(int)Stats.GetStat(StatType.Level).Value-1];
93:         else
94:             Stats.GetStat(StatType.Experience).DefaultValue =
                    levelData.MaxExperience[levelData.MaxExperience.Length-1];
95:
96:         // 레벨업 할 때 스킬을 선택할 수 있도록 스킬 선택 팝업 창 출력
97:         skillSystem.StartSelectSkill();
98:     }
99: }
```

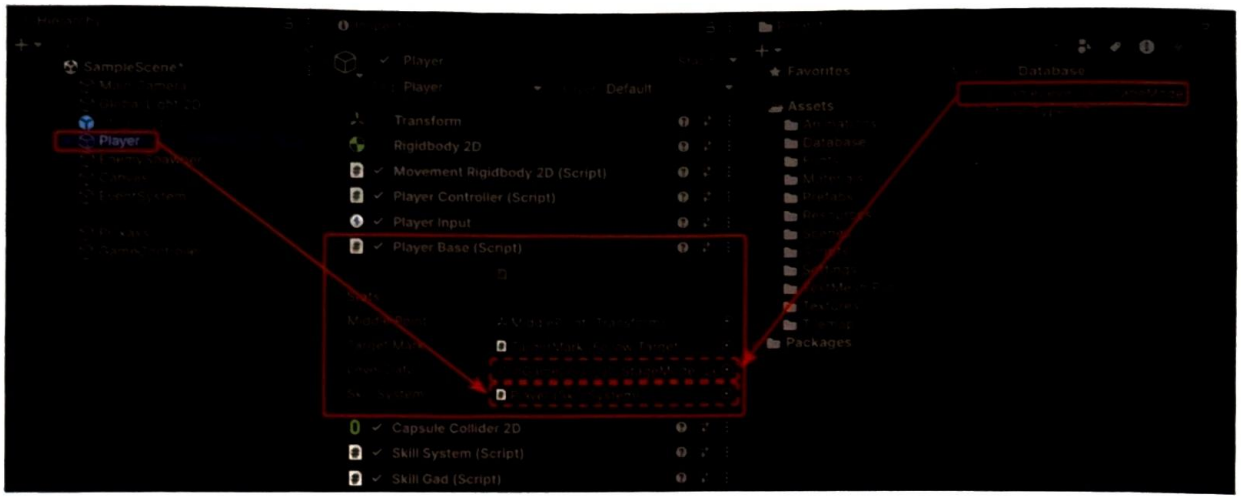
경험치 게이지가 점점 채워지도록 적을 물리쳤을 때 플레이어 경험치를 바로 늘리지 않고 별도로 AccumulationExp 속성에 경험치를 축적해 프레임마다 원하는 수치(expAmount)만큼 경험치가 채워지도록 합니다.

25줄 Stats.CurrentExp가 현재 경험치 스탯이므로 현재 경험치 값이 바뀔 때마다 IsLevelUP() 메서드를 호출하도록 등록합니다.

70~77줄 UpdateEXP() 메서드에서 축적된 경험치가 0이거나 현재 스킬을 선택 중이면 경험치가 채워지지 않도록 메서드를 빠져나옵니다. 프레임마다 getEXP만큼 축적된 경험치를 줄이고 플레이어의 현재 경험치를 늘립니다.

79~98줄 IsLevelUP() 메서드에서 현재 경험치가 최대가 아니면 레벨업이 아니므로 return으로 메서드를 종료합니다. 레벨업 할 수 있으면 현재 레벨을 1 늘리고 현재 경험치에서 레벨업에 사용한 최대 경험치의 양을 줄입니다. 현재 레벨에 맞는 레벨 테이블의 최대 경험치 정보를 불러오고 현재 레벨이 테이블에 설정된 최대 경험치 개수보다 크면 마지막 테이블값을 최대 경험치로 사용합니다. 레벨업을 했으므로 스킬 선택 팝업 창을 활성화합니다.

05 Player 오브젝트의 Player Base 컴포넌트에서 Level Data 변수에는 InGameLevel_00_StageMode 에셋을, SkillSystem 변수에는 Player 오브젝트를 등록합니다.



Player 오브젝트의 Player Base 컴포넌트 변수 설정하기

Do it! 실습 적을 물리쳤을 때 처리하기

01 적을 물리쳤을 때 호출하는 `Deactivate()` 메서드를 정의하고자 EnemySpawner.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemySpawner.cs

코드 09-07 적을 물리쳤을 때 호출하는 Deactivate() 메서드 정의

```
    (... 생략 ...)
05: public class EnemySpawner : MonoBehaviour
06: {
        (... 생략 ...)
69:     public void Deactivate(EntityBase enemy)
70:     {
71:         Enemies.Remove(enemy);
72:         Destroy(enemy.gameObject);
73:     }
74: }
```

추가

02 오브젝트가 사라질 때 호출하는 `OnDie()` 추상 메서드를 정의하고 호출하도록 EntityBase.cs 스크립트를 수정합니다.

Assets/Scripts/Entity/EntityBase.cs

코드 09-08 추상 메서드 OnDie() 정의

```
01: using UnityEngine;
02:
03: public abstract class EntityBase : MonoBehaviour                    수정
04: {
        (... 생략 ...)
20:     public void TakeDamage(float damage)
21:     {
            (... 생략 ...)
29:         if ( Mathf.Approximately(Stats.CurrentHP.DefaultValue, 0f) )
30:         {
31:             OnDie(); // Entity 사망 처리                            추가
32:         }
33:     }
34:
35:     protected abstract void OnDie();                                 추가
36: }
```

03 플레이어가 죽었을 때 호출하는 OnDie() 메서드를 재정의하도록 PlayerBase.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerBase.cs

코드 09-09 OnDie() 메서드 재정의

```
01: using UnityEngine;
02:
03: public class PlayerBase : EntityBase
04: {
        (... 생략 ...)
38:     protected override void OnDie()                                  추가
39:     {
40:         Logger.Log("플레이어 사망 처리");
41:     }
        (... 생략 ...)
104: }
```

04 적이 죽었을 때 호출하는 OnDie() 메서드를 재정의하도록 EnemyBase.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemyBase.cs

코드 09-10 OnDie() 메서드 재정의

```
01: using UnityEngine;
02:
03: public class EnemyBase : EntityBase
04: {
05:     [SerializeField]
06:     private Transform hudPoint;
07:     [SerializeField]
08:     private GameObject uiPrefab;
09:
10:     private EnemySpawner enemySpawner;                          추가
        (... 생략 ...)
25:     public void Initialize(EnemySpawner enemySpawner, Transform parent)   수정
26:     {
27:         this.enemySpawner = enemySpawner;                       추가
            (... 생략 ...)
33:     }
34:
35:     protected override void OnDie()                             추가
36:     {
37:         // 적은 레벨업 하지 않으므로 적 경험치 스탯만큼 플레이어 경험치 증가
38:         (Target as PlayerBase).AccumulationExp += Stats.CurrentExp.Value;
39:         // 적 본인(this) 사망 처리
40:         enemySpawner.Deactivate(this);
41:     }
42: }
```

05 적을 생성할 때 EnemyBase에 EnemySpawner 정보를 전달하고자 EnemySpawner.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemySpawner.cs

코드 09-11 적 자동 생성 스크립트

```
    (... 생략 ...)
05: public class EnemySpawner : MonoBehaviour
06: {
        (... 생략 ...)
23:     private void Awake()
24:     {
            (... 생략 ...)
```

```
37:             clone.GetComponent<EnemyBase>().Initialize(this, parentTransform);
38:             clone.GetComponent<EnemyFSM>().Setup(target);
39:
40:             Enemies.Add(clone.GetComponent<EntityBase>());
41:         }
42:     }
      (... 생략 ...)
74: }
```

수정

Do it! 실습 플레이어 레벨과 경험치 출력하기

01 Assets 〉 Scripts 〉 UI 폴더에 플레이어 레벨, 경험치와 같은 인게임 정보를 출력하는 UIPlayerData.cs 스크립트를 생성합니다.

Assets/Scripts/UI/UIPlayerData.cs

코드 09-12 플레이어 인게임 정보 출력

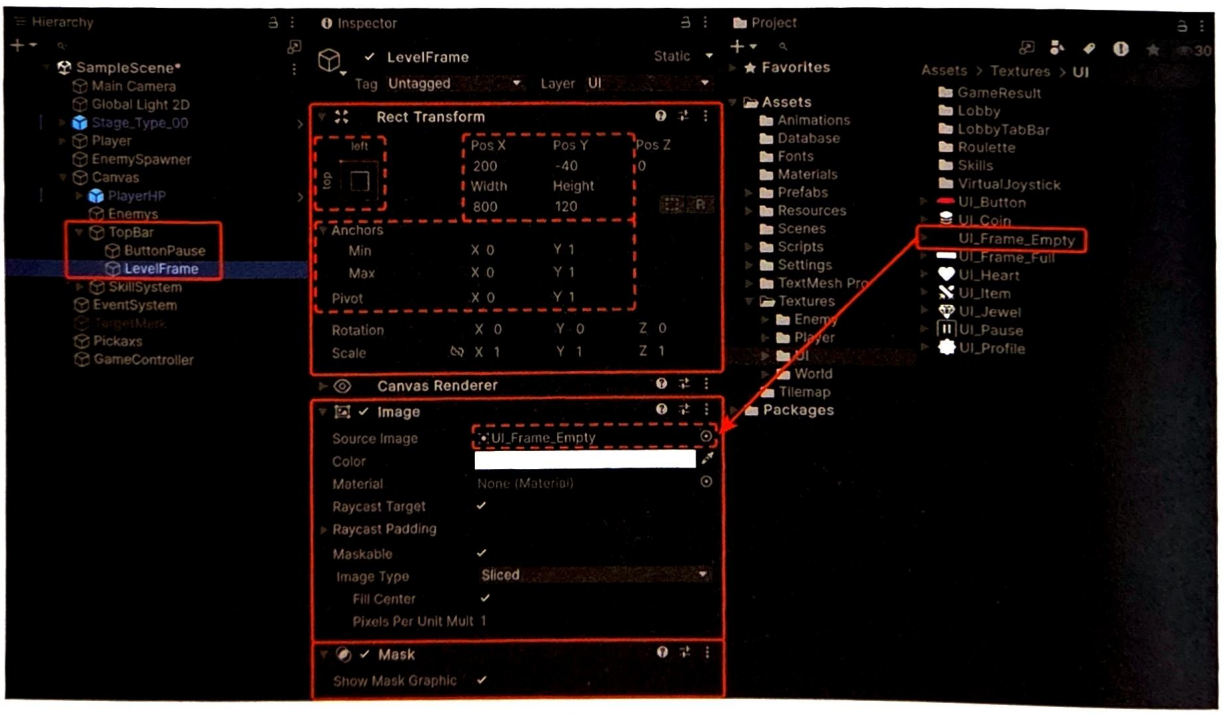
```
01: using UnityEngine;
02: using UnityEngine.UI;
03: using TMPro;
04:
05: public class UIPlayerData : MonoBehaviour
06: {
07:     [SerializeField]
08:     private TextMeshProUGUI textLevel;
09:     [SerializeField]
10:     private Image fillGaugeEXP;
11:     [SerializeField]
12:     private PlayerBase entity;
13:
14:     private void Awake()
15:     {
16:         entity.Stats.CurrentExp.OnValueChanged += UpdateEXP;
17:     }
18:
19:     private void UpdateEXP(Stat stat, float prev, float current)
20:     {
21:         textLevel.text = $"Lv.{entity.Stats.GetStat(StatType.Level).Value}";
22:         fillGaugeEXP.fillAmount = entity.Stats.CurrentExp.Value /
                entity.Stats.GetStat(StatType.Experience).Value;
```

```
23:     }
24: }
```

16줄 플레이어의 현재 경험치 스탯인 CurrentExp가 바뀔 때 UpdateEXP() 메서드를 호출하도록 등록합니다.

19~23줄 UpdateEXP() 메서드는 플레이어의 현재 레벨을 텍스트 UI에 출력하고 경험치를 게이지 UI에 출력합니다.

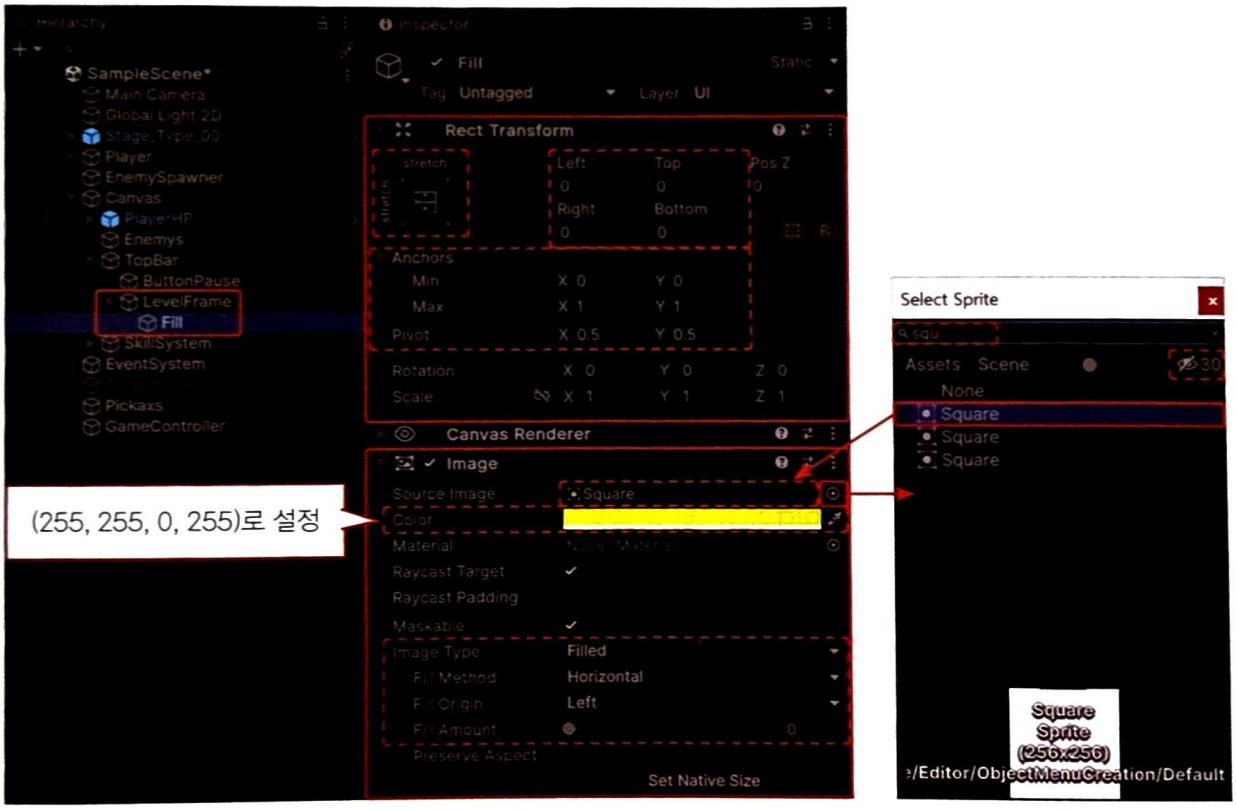
02 [GameObject → UI → Image] 메뉴로 레벨과 경험치 UI를 관리하는 Image UI를 생성하고 이름을 'LevelFrame'으로 변경한 후 TopBar 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 top | left로 설정하고 Rect는 (200, -40, 800, 120)으로 설정합니다. Image 컴포넌트의 Source Image는 UI_Frame_Empty.png 이미지로 설정한 후 자식 오브젝트의 출력 범위를 설정하고자 Mask 컴포넌트를 추가합니다.



레벨과 경험치 UI를 관리하는 Image UI 생성하고 설정하기

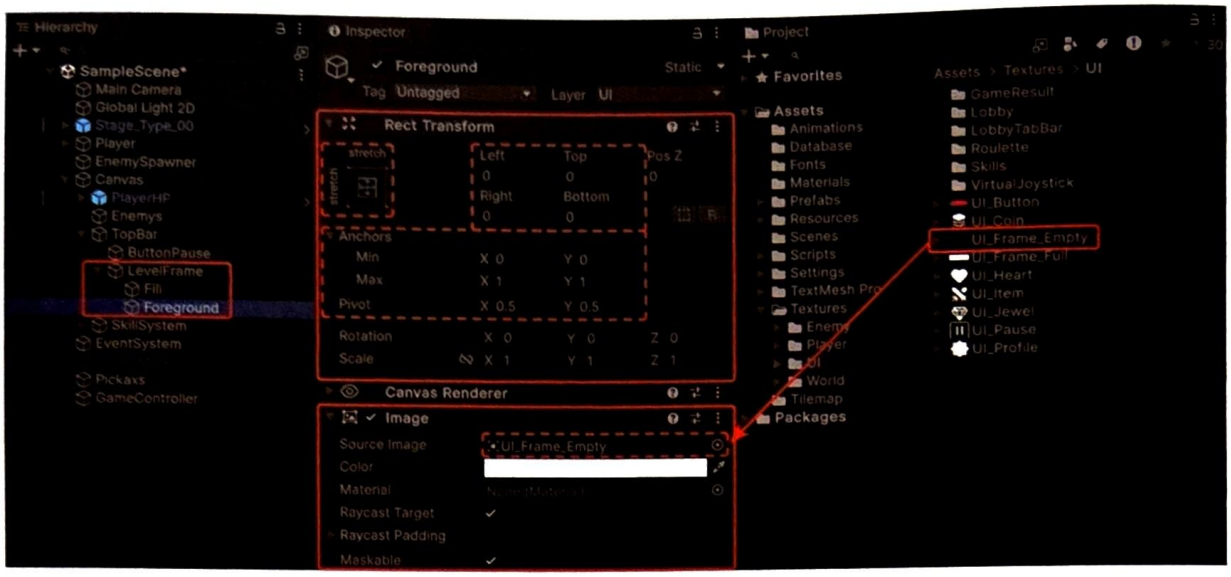
03 [GameObject → UI → Image] 메뉴로 경험치 게이지 UI를 출력하는 Image UI를 생성하고 이름을 'Fill'로 변경한 후 LevelFrame 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 stretch | stretch로, Rect는 (0, 0, 0, 0)으로 설정합니다. Image 컴포넌트의 Source Image는 Square.png 이미지로, Color는 (255, 255, 0, 255)로 설정합니다. Image Type은 [Filled]로, Fill Method는 [Horizontal]로, Fill Origin은 [Left]로, Fill Amount는 '0'으로 설정합니다.

Square.png는 유니티에서 기본으로 제공하는 이미지입니다.



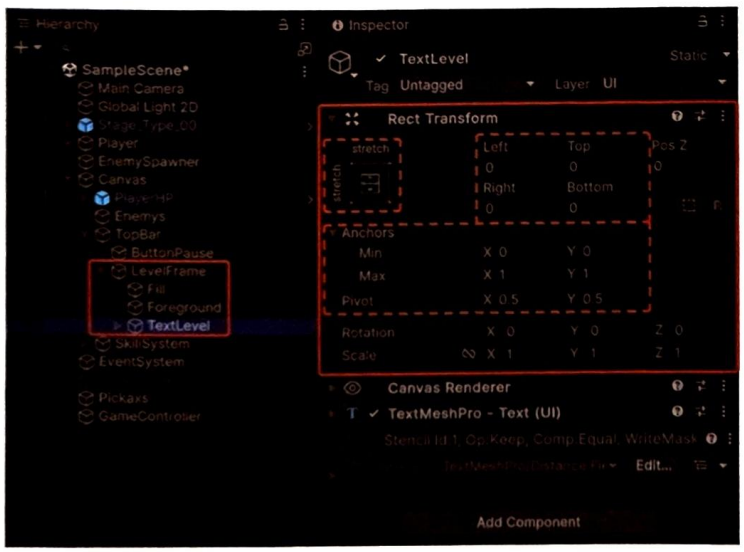
경험치 게이지 UI를 출력하는 Image UI 생성하고 설정하기

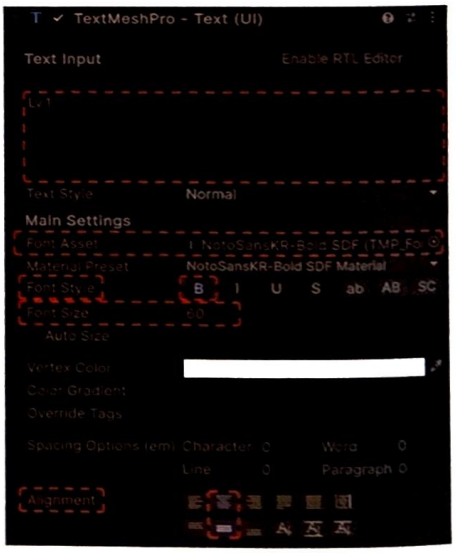
04 [GameObject → UI → Image] 메뉴로 경험치 게이지 외곽 UI를 출력하는 Image UI를 생성하고 이름을 'Foreground'로 변경한 후 LevelFrame 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 stretch | stretch로, Rect는 (0, 0, 0, 0)으로 설정합니다. Image 컴포넌트의 Source Image는 UI_Frame_Empty.png 이미지로 설정합니다.



경험치 게이지 외곽 UI를 출력하는 Image UI 생성하고 설정하기

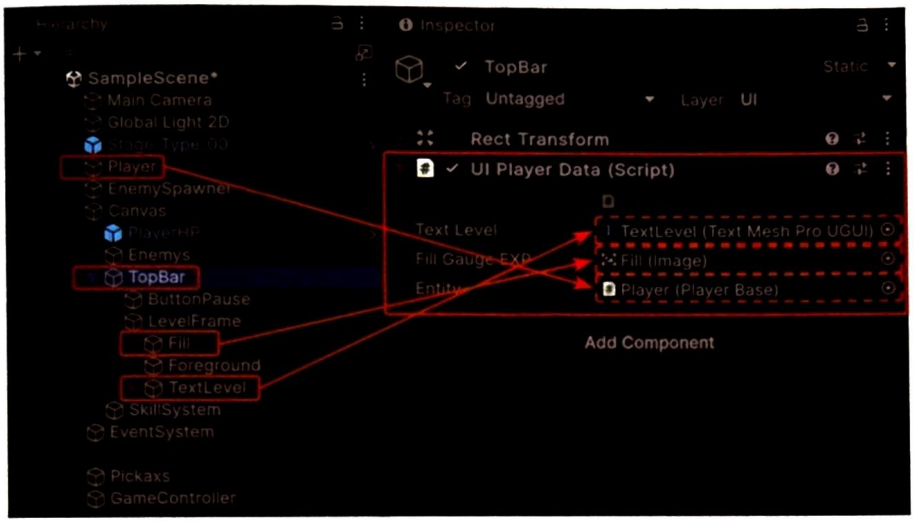
05 [GameObject → UI → Text - TextMeshPro] 메뉴로 레벨을 출력하는 Text UI를 생성하고 이름을 'TextLevel'로 변경한 후 LevelFrame 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 stretch | stretch로, Rect는 (0, 0, 0, 0)으로 설정합니다. TextMeshPro - Text 컴포넌트의 Text Input Box에 'Lv.1'을 입력하고 Font Asset에 NotoSansKR-Bold SDF를 등록합니다. Font Style은 굵게 [B]로, Font Size는 '60'으로, Alignment는 Center | Middle로 설정합니다.





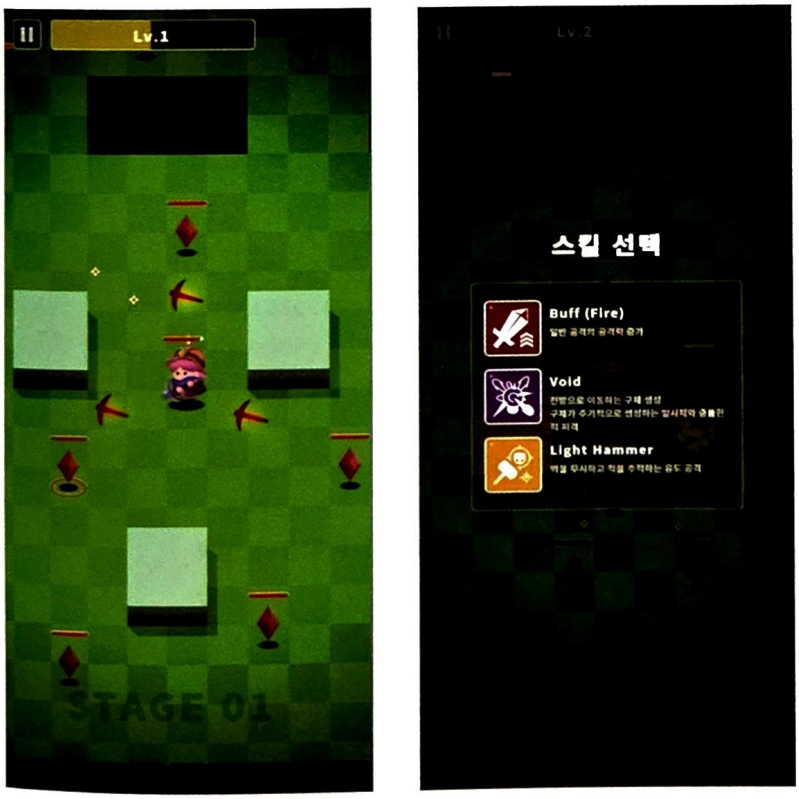
레벨을 출력하는 Text UI 생성하고 설정하기

06 TopBar 오브젝트에 UI Player Data 컴포넌트를 추가하고 Text Level 변수에는 TextLevel 오브젝트를, Fill Gauge EXP 변수에는 Fill 오브젝트를, Entity 변수에는 Player 오브젝트를 등록합니다.



TopBar 오브젝트에 UI Player Data 컴포넌트 추가하고 설정하기

07 플레이 버튼 ▶ 을 눌러 게임을 실행하고 적을 물리치면 플레이어가 경험치를 획득하고, 경험치가 최대로 채워지면 레벨업과 함께 스킬을 배울 수 있는 스킬 선택 팝업 창을 활성화합니다.



인게임 레벨 시스템 결과 화면

09-2

메모리 풀을 활용하여 보석 생성하고 최적화하기

이 절에서는 게임에서 사용할 수 있는 **재화**Currency 시스템의 하나인 보석을 구현합니다. 보석은 적을 물리칠 때마다 다량 생성되므로 최적화하고자 오브젝트를 삭제하지 않고 리스트 형태로 보관한 후 활성화/비활성화로 관리하는 **메모리 풀**Memory Pool 기능을 이용합니다.

적을 물리치면 보석 효과가 생성되고 보석 UI 위치까지 곡선 이동하며 서서히 사라지도록 만들어 봅시다.

Do it! 실습 메모리 풀과 페이드 효과 준비하기

01 먼저 오브젝트를 관리하는 메모리 풀 기능을 구현합니다. Assets 〉 Scripts 〉 Common 폴더에 오브젝트를 삭제하지 않고 리스트로 관리하는 MemoryPool.cs 스크립트를 생성합니다.

Assets/Scripts/Common/MemoryPool.cs

코드 09-13 오브젝트를 관리하는 메모리 풀

```
01: using UnityEngine;
02: using System.Collections.Generic;
03:
04: public class MemoryPool
05: {
06:     // 메모리 풀로 관리하는 오브젝트 정보
07:     private class PoolItem
08:     {
09:         public GameObject gameObject; // 화면에 보이는 실제 게임오브젝트
10:         private bool isActive;        // gameObject 활성화/비활성화 정보
11:
12:         public bool IsActive
13:         {
14:             set
15:             {
```